EXERCISE WORKBOOK

Advanced Rust

Exercises, labs, and review questions for all 56 chapters — the companion workbook.

O. Iakushkin

Contents

Part I · Getting Your Bearings in Rust

1 Why Rust Feels Different 5
2 The Rust Mental Model 8
3 Project Structure and Tooling 11

Part II · Memory, Ownership, and Data Layout

4 Ownership, Borrowing, and Lifetimes 15
5 Pointers, References, and Ownership Inside Structs 18
6 Pointers, References, and Ownership Inside Vectors 21
7 Copying Data vs Cloning Data 24
8 Undefined Behavior and Unsafe Rust 27
9 Smart Pointers and Pinning 30
10 Arrays, Slices, and Vectors 34
11 Hash Maps and Sets 37
12 Matrices and Multidimensional Data 41
13 Arena Allocation and Region-Based Memory 45

Part III · Designing Types and Abstractions

14 Interfaces in Rust: Traits 50
15 OOP Models in Rust 54
16 Domain-Driven Design in Rust 58
17 Refactoring Toward Idiomatic Rust 63
18 Generics Instead of Templates 66
19 Serialization and Data Contracts 70
20 Metaprogramming 74
21 Reflection and Type Introspection 78

Part IV · Concurrency, Async, and Systems IO

22 Multithreading in Rust 83
23 Synchronization Primitives 87
24 Coroutines, Futures, and Async Rust 90
25 Tokio 94
26 Task Libraries and Parallel Execution 98
27 IO Tricks and Systems Programming Patterns 102

Part V · Integration, Distributed Systems, and HPC

28 C++ Integration 107
29 JavaScript and C++ Integration for WASM 111
30 AMQP and Message Brokers 115
31 Distributed Task Execution 119
32 MPI and High-Performance Computing 123
33 Performance-Oriented Rust 127
34 Memory Profiling 131
35 Performance Profiling 135
36 Distributed Tasks Profiling 139
37 CUDA and GPU Acceleration 142
38 Merkle Tree Games and Challenges 146
39 Graph Search Games 150
40 Matrix Optimization Games 154

Part VI · Engineering Systems for Production

41 Error Handling in Large Systems 159

42 Testing Advanced Rust Systems 163
43 Observability 167
44 Packaging and Deployment 171
45 Capstone: A Distributed Rust System 175

Part VII · Networked Services and Secure Boundaries

46 FastAPI-Style Web Apps, Swagger, and OpenAPI Codegen 179
47 gRPC Services with Protobuf and Service API Codegen 182
48 WebSockets and Long-Lived Connections 186
49 HTTPS, TLS, and Secure Service Boundaries 190
50 libp2p and Peer-to-Peer Rust Systems 194

Part VIII · Frontier Specializations

51 Zero-Knowledge Proofs for Rust Engineers 199
52 ZoKrates Workflows and Ethereum Verifiers 203
53 EZKL, Verifiable LLM Inference, and GPU-Aware ZKML 207
54 ONNX Model Inference in Rust 211
55 Smart Contracts in Rust: Solana, Sei, and Beyond 215
56 no-std Rust and Constrained Runtime Derivatives 219

PART I

Getting Your Bearings in Rust

Recalibrate your mental model and stand up a working toolchain before you fight the borrow checker.

1 Why Rust Feels Different
2 The Rust Mental Model
3 Project Structure and Tooling

CHAPTER 1 · WHY RUST FEELS DIFFERENT

Why Rust Feels Different

Exercises, labs, and review questions for this chapter.

Progressive drills on diagnostics, tradeoffs, and Rust-first design

EXERCISE 1 WARM-UP COMPREHENSION

Translate a familiar lifecycle pattern into Rust

OBJECTIVE

Map one resource-management idea from C++, C#, or Go to idiomatic Rust ownership and cleanup semantics.

STARTER PROMPT

Choose one familiar lifecycle pattern and restate it as a Rust ownership design for a resource owner named `Session` or `LogWriter`.

- Start from a single owner that is responsible for cleanup.
- Decide which methods should take `&self`, which should take `&mut self`, and which should consume `self`.
- Explain where deterministic cleanup happens and why it does not depend on a garbage collector or inheritance.

ACCEPTANCE CRITERIA

- Your design identifies a single owner responsible for cleanup.
- You explain why borrowed methods do or do not allow mutation.
- Cleanup is deterministic and does not rely on a garbage collector or inheritance.

HINTS

- Think in terms of a struct plus methods, not a class hierarchy.
- If cleanup must always happen, describe the role of Drop even if you do not fully implement it yet.

EXERCISE 2 CODE READING

Find the zero-cost part and the real runtime cost

OBJECTIVE

Identify which parts of a Rust abstraction are compile-time structure and which parts still do real work at runtime.

STARTER PROMPT

Study an iterator-based function that filters high-priority jobs from a slice, then annotate which parts are compile-time abstraction structure and which parts still cost CPU cycles.

- Call out which parts are likely monomorphized or optimized as abstraction structure.
- List the runtime work that still exists: iteration, branching, cache behavior, and any downstream materialization cost.
- Explain where dynamic dispatch would appear if the return type changed to a trait object.

ACCEPTANCE CRITERIA

- You identify iterator adapters and impl Iterator as abstraction mechanisms that do not imply dynamic dispatch by default.
- You name the runtime work that still exists: iteration, branching, cache behavior, and any downstream allocation if materialized.
- You explain that a trait object would opt into dynamic dispatch.

HINTS

- Separate 'how the code is expressed' from 'what the machine still has to do.'
- Look for where values are actually stored, allocated, or compared.

EXERCISE 3 IMPLEMENTATION

Make fallibility explicit instead of implicit

OBJECTIVE

Replace a sentinel-value style API with Rust's explicit Option and Result modeling.

STARTER PROMPT

Implement `parse_limit(input: Option<&str>) -> Result<usize, &'static str>` so missing input uses the default value `100`, `"0"` is invalid, non-numeric input is invalid, and valid positive integers return `Ok(limit)`.

- Do not use `panic!`, `unwrap`, or sentinel values such as `-1`.
- Keep the failure mode explicit in the type system so callers cannot ignore it accidentally.

ACCEPTANCE CRITERIA

- The function returns Result<usize, &'static str> exactly.
- Missing input becomes the default value without error.
- Invalid input is represented as an Err, not as a magic number or log-only failure.

HINTS

- Handle Option first, then parse the string branch.
- The type system should make invalid states harder to ignore at call sites.

EXERCISE 4 DEBUGGING OR REFACTORING

Interpret three compiler diagnostics and propose repairs

OBJECTIVE

Practice reading Rust diagnostics as ownership and concurrency design feedback.

STARTER PROMPT

For each diagnostic, explain the rule Rust is enforcing, sketch the likely shape of the buggy code, and propose one or two valid repair strategies.

- E0382: borrow of moved value: request
- E0499: cannot borrow buffer as mutable more than once at a time
- E0277: Rc<String> cannot be sent between threads safely

ACCEPTANCE CRITERIA

- Your explanation of E0382 mentions ownership transfer and alternatives such as borrowing, returning ownership, or intentional cloning.
- Your explanation of E0499 mentions exclusive mutable access and proposes scope shortening or data-structure redesign.
- Your explanation of E0277 mentions thread-safety trait bounds and distinguishes Rc from thread-safe sharing approaches such as Arc.

HINTS

- Focus on the violated rule first, then the syntax.
- A good repair changes ownership shape before reaching for shared mutable state.

EXERCISE 5 DESIGN OR PRODUCTION SCENARIO

Design a worker pipeline with explicit ownership boundaries

OBJECTIVE

Choose Rust-native boundaries for parsing, validation, dispatch, and cross-thread communication in a production-style service.

STARTER PROMPT

You are designing an ingestion service with the flow `socket bytes -> parse -> validate -> route to workers -> emit metrics`.

- At what boundary do bytes become owned domain values?
- What data is borrowed only locally?
- What types are allowed to cross thread boundaries?
- Where do you use `Result` or enums to represent failure and state?

ACCEPTANCE CRITERIA

- You place ownership boundaries before cross-thread or queued work.
- You avoid defaulting to a single global `Arc<Mutex<HashMap<...>>>` as the first design move.
- You make error handling explicit with Result, enums, or both.
- You justify at least one tradeoff involving performance, safety, or maintainability.

HINTS

- Prefer message passing of owned work items over broad shared mutability.
- If you need sharing, explain why the sharing is semantically real rather than just convenient.

Runnable lab

Tip: keep the type as Result<usize, &'static str>, handle None first, then validate and parse the Some branch.

Example 1-L. `parse_limit_lab.rs` — *Runnable lab · Exercise 3*

```rust
fn parse_limit(input: Option<&str>) -> Result<usize, &'static str> {
    match input {
        None => Ok(100),
        Some(raw) => match raw.parse::<usize>() {
            Ok(0) => Err("limit must be greater than 0"),
            Ok(limit) => Ok(limit),
            Err(_) => Err("invalid number"),
        },
    }
}

fn main() {
    println!("default = {:?}", parse_limit(None));
    println!("zero = {:?}", parse_limit(Some("0")));
    println!("value = {:?}", parse_limit(Some("25")));
}
```

OUTPUT

```
default = Ok(100)
zero = Err("limit must be greater than 0")
value = Ok(25)
```

Review questions

- Why does Rust often feel harder at API boundaries than inside small local functions?
- What is the practical difference between a generic function using traits and a trait object?
- Name one category of bug Rust usually catches at compile time and one category it cannot remove from runtime reality.
- Why is 'just clone it' sometimes correct and sometimes a design smell?
- How would you explain `Send` and `Sync` to a teammate coming from Go or C#?

CHAPTER 2 · THE RUST MENTAL MODEL

The Rust Mental Model

Exercises, labs, and review questions for this chapter.

Predict ownership behavior, reason about allocation, and practice expression-oriented Rust

EXERCISE 1 WARM-UP COMPREHENSION

Predict the ownership timeline

OBJECTIVE

Practice narrating which binding owns a value after each move and where `Drop` is guaranteed to run.

STARTER PROMPT

Read a short ownership-transfer sequence with `service`, `alias`, and `consume`, then narrate the owner after each move in plain English.

- Which binding owns the string after each line?
- At what scope does the string get dropped?
- Which extra line could you add that would fail to compile, and why?

ACCEPTANCE CRITERIA

- You explain that ownership moves from `service` to `alias`, then from `alias` into `consume`.
- You identify the end of `consume` as the drop point for the owned `String`.
- Your failing-line example correctly refers to using a moved binding after ownership transfer.

HINTS

- For non-`Copy` types, assignment normally transfers ownership.
- Describe ownership one scope at a time instead of thinking about hidden object identity.

EXERCISE 2 CODE READING

Map stack and heap storage precisely

OBJECTIVE

Explain which parts of a composite value are inline and which parts own heap allocations.

STARTER PROMPT

Analyze a `Batch` struct that stores a fixed array plus a `Vec<String>`, then write a short note about which pieces are inline and which pieces own heap allocations.

- Which parts of `Batch` are stored inline in the local value?
- What heap allocations exist?
- Which value owns each heap allocation?

ACCEPTANCE CRITERIA

- You identify the fixed-size array as inline in the `Batch` value.
- You explain that `Vec<String>` contains inline metadata while its element buffer is heap allocated.
- You note that each `String` element owns its own heap-backed bytes.

HINTS

- Separate the outer struct layout from the storage used by each field's owned data.
- A `Vec<T>` is a fixed-size handle that owns a separate buffer.

EXERCISE 3 IMPLEMENTATION

Refactor imperative mutation into expression-oriented Rust

OBJECTIVE

Rewrite a function so temporary state stays narrow and the final result is produced by expressions.

STARTER PROMPT

Refactor `classify(depth: usize) -> (&'static str, usize)` so the tuple comes directly from an expression instead of a wide mutable temporary.

- Keep the return type the same.
- Prefer `if` and block expressions.
- Do not introduce heap allocation or cloning.

ACCEPTANCE CRITERIA

- Your final version avoids the wide `mut scaled` binding.
- The result is still `(&'static str, usize)`.
- The control flow is expression-oriented rather than early-return driven everywhere.

HINTS

- An `if` expression can return a tuple.
- A short inner block can compute a value without widening mutable scope.

EXERCISE 4 DEBUGGING OR REFACTORING

Explain the lifetime bug instead of fighting the annotation

OBJECTIVE

Diagnose why returning a reference to local data fails and propose correct repairs.

STARTER PROMPT

Diagnose a function that tries to return the first line of a local `String` by reference, then explain why that borrow cannot outlive the owner.

- What rule Rust is enforcing
- Why adding a random lifetime annotation does not solve it
- Two valid repairs with different tradeoffs

ACCEPTANCE CRITERIA

- You explain that the returned reference would outlive the local `String` owner.
- You state that lifetimes describe valid borrowing relationships and do not extend object lifetime.
- You propose at least two real repairs, such as returning an owned `String` or borrowing from caller-provided input.

HINTS

- Ask who owns `text` and when that owner is dropped.
- A repair is valid only if the referenced data outlives the returned reference.

EXERCISE 5 DESIGN OR PRODUCTION SCENARIO

Choose owned versus borrowed data at a queue boundary

OBJECTIVE

Make an ownership plan for a realistic parser-to-worker pipeline.

STARTER PROMPT

You are designing an ingest path with the flow `&[u8] request bytes -> parse headers -> build Event -> send to worker queue -> persist -> emit metrics`.

- Which data stays borrowed only inside parsing?
- Which data becomes owned inside `Event` before the queue boundary?
- Where is cloning acceptable, and where is it a smell?
- What cleanup should rely on ordinary scope exit or `Drop`?

ACCEPTANCE CRITERIA

- You keep borrowed data local to parsing or validation where possible.
- You convert data that crosses the queue boundary into owned values.
- You justify cloning as an explicit economic choice rather than a default response.
- You describe cleanup in terms of ownership and scope, not hidden runtime behavior.

HINTS

- A worker queue is usually an ownership boundary.
- Try to name the owner at each subsystem edge.

Runnable lab

Example 2-L. `classify_lab.rs` — *Runnable lab · Exercise 3*

```rust
fn classify(depth: usize) -> (&'static str, usize) {
    let mut scaled = 0;

    if depth > 1000 {
        scaled = depth / 2;
        return ("hot", scaled);
    }

    scaled = depth + 50;
    ("steady", scaled)
}

fn main() {
    let (status_a, value_a) = classify(120);
    let (status_b, value_b) = classify(1200);
    println!("{} {}", status_a, value_a);
    println!("{} {}", status_b, value_b);
}
```

OUTPUT

```
steady 170
hot 600
```

Review questions

- What is the difference between a value and a binding in Rust?
- Where do `Vec<T>` metadata and `Vec<T>` elements usually live?
- What does a lifetime annotation constrain, and what does it not do?
- Why can block expressions reduce mutation pressure in production code?
- When is `Drop` useful, and what kinds of logic should usually stay out of it?

CHAPTER 3 · PROJECT STRUCTURE AND TOOLING

Project Structure and Tooling

Exercises, labs, and review questions for this chapter.

Design workspace layout, feature matrices, dependency policy, and the command pack for CI

EXERCISE 1 WARM-UP COMPREHENSION

Name the unit before you design it

OBJECTIVE

Practice distinguishing package, crate, module, workspace, and public API surface without hand-waving.

STARTER PROMPT

You inherit a repository with an API service, a worker, shared domain models, and a CLI. Classify which parts should be packages, which should be crates, and which should remain modules inside one crate.

- Which boundary is only organizational and therefore should stay a module?
- Which boundary justifies a separate crate because of dependency or API isolation?
- Which boundary belongs at workspace scope rather than crate scope?

ACCEPTANCE CRITERIA

- You distinguish package, crate, module, and workspace correctly.
- You justify at least one decision by dependency isolation rather than folder preference.
- You identify at least one case where a module is cheaper than a new crate.

HINTS

- Ask what the compiler builds, what Cargo manages, and what callers should be allowed to import.
- A directory boundary is not automatically a crate boundary.

EXERCISE 2 CODE READING

Read the public surface from a module tree

OBJECTIVE

Infer what is public, what remains internal, and where refactoring freedom still exists.

STARTER PROMPT

Given a crate root that re-exports `pub use domain::OrderId;` and `pub use service::OrderService;`, with adapters hidden under internal modules, explain what downstream callers can rely on and what they should not know about.

- What path would a downstream crate import?
- Which internal module names may change without breaking callers?
- Why is `pub(crate)` often a better first choice than `pub`?

ACCEPTANCE CRITERIA

- You identify the stable import path at the crate root.
- You explain how re-exporting hides internal layout and preserves refactoring room.
- You describe the maintenance cost of widening visibility too early.

HINTS

- Think about what another crate sees, not what the current file sees.
- A public path is an API promise even if it felt temporary when written.

EXERCISE 3 IMPLEMENTATION

Create a workspace layout for a multi-crate product

OBJECTIVE

Design a realistic Rust workspace for a product with multiple binaries and shared domain logic.

STARTER PROMPT

Sketch the top-level layout for a system with `api`, `worker`, `domain`, and `adapters` components. Use a workspace root plus packages and explain which crates own which dependencies.

- Which package should contain the shared domain types?
- Which packages should depend on network or database crates?
- Which package should remain free of infrastructure dependencies?

ACCEPTANCE CRITERIA

- Your layout includes a workspace root and multiple packages.
- The domain crate stays isolated from infrastructure dependencies.
- The API and worker binaries depend inward rather than the core depending outward.
- You explain one tradeoff involving build times, API stability, or dependency hygiene.

HINTS

- If a crate can stay free of HTTP and database types, that is usually a good sign.
- Think in dependency direction, not just file organization.

EXERCISE 4 DEBUGGING OR REFACTORING

Repair a feature flag matrix without turning Cargo into a mode engine

OBJECTIVE

Fix a design that uses features for mutually exclusive operational modes.

STARTER PROMPT

A crate has features `sqlite`, `postgres`, and `memory`, and the code assumes exactly one is enabled. Refactor the design so additive features remain sane and invalid combinations are handled explicitly.

- Which parts should stay compile-time options?
- Which parts belong in runtime configuration instead?
- How would you reject impossible combinations early?

ACCEPTANCE CRITERIA

- You explain why mutually exclusive operational modes are awkward as Cargo features.
- You preserve additive capability where it makes sense.
- You propose an explicit validation strategy for impossible combinations.

HINTS

- Cargo resolves one feature set for the build graph, not per request or per environment.
- A runtime enum is often cleaner than a compile-time feature war.

EXERCISE 5 DESIGN OR PRODUCTION SCENARIO

Choose a dependency policy for shared crates

OBJECTIVE

Decide which crates may depend on infrastructure libraries and which crates must remain clean.

STARTER PROMPT

Your domain crate currently imports an HTTP status type, a database error type, and a tracing crate because it was faster during prototyping. Redesign the dependency policy for long-term maintainability.

- Which dependencies move outward into adapter crates?
- Which types should be translated at the boundary instead of leaking inward?
- Where would you allow a shared utility crate, and where would you avoid it?

ACCEPTANCE CRITERIA

- You move protocol and storage dependencies out of the core domain crate.
- You explain at least one translation boundary where infrastructure types should be converted.
- You justify any remaining shared utility dependency rather than using it as a default bucket.

HINTS

- Core crates should know business concepts first and infrastructure concepts last.
- A shared crate is only helpful when it removes duplication without becoming a junk drawer.

EXERCISE 6 DESIGN OR PRODUCTION SCENARIO

Write the CI command pack for the chapter examples

OBJECTIVE

Define the baseline commands a serious Rust workspace should run for formatting, linting, testing, and benchmarking.

STARTER PROMPT

Pretend the chapter examples live inside a real Cargo workspace. Write the command sequence you would put into CI and explain what each command protects against.

- Which command gives the fastest structural feedback?
- Which command enforces formatting consistency?
- Which command should fail on lint warnings?
- How will benchmarking fit in without pretending every repository already has a benchmark harness?

ACCEPTANCE CRITERIA

- Your command pack includes check, test, formatting, and linting.
- You mention benchmark execution conditionally rather than assuming every repo already has benches.
- You explain why `--workspace`, `--all-targets`, or `--all-features` matter where used.

HINTS

- Think of the command pack as team muscle memory, not only as CI decoration.
- A good answer is specific enough to paste into a pipeline later.

Runnable lab

Example 3-L. `feature_matrix_lab.rs` — *Runnable lab · Feature matrix repair*

```rust
fn selected_backend(s3: bool, local: bool) -> Result<&'static str, &'static str> {
    if s3 {
        return Ok("s3");
    }

    if local {
        return Ok("local");
    }

    Err("not configured")
}

fn main() {
    println!("none = {:?}", selected_backend(false, false));
    println!("s3 = {:?}", selected_backend(true, false));
    println!("local = {:?}", selected_backend(false, true));
    println!("both = {:?}", selected_backend(true, true));
}
```

OUTPUT

```
none = Err("choose exactly one backend")
s3 = Ok("s3")
local = Ok("local")
both = Err("choose exactly one backend")
```

Review questions

- What is the operational difference between a package and a crate?
- When should you create a new crate instead of a new module?
- Why is `pub(crate)` often a better default than `pub`?
- What kinds of choices belong in Cargo features, and what kinds usually belong in runtime configuration?
- Why should benchmarking be explicit rather than assumed?

PART II

Memory, Ownership, and Data Layout

Ownership, borrowing, lifetimes, smart pointers, and the containers you reach for daily — the part that makes every later API make sense.

4 Ownership, Borrowing, and Lifetimes

5 Pointers, References, and Ownership Inside Structs

6 Pointers, References, and Ownership Inside Vectors

7 Copying Data vs Cloning Data

8 Undefined Behavior and Unsafe Rust

9 Smart Pointers and Pinning

10 Arrays, Slices, and Vectors

11 Hash Maps and Sets

12 Matrices and Multidimensional Data

13 Arena Allocation and Region-Based Memory

CHAPTER 4 · OWNERSHIP, BORROWING, AND LIFETIMES

Ownership, Borrowing, and Lifetimes

Exercises, labs, and review questions for this chapter.

Repair borrow-checker failures, annotate lifetimes only where needed, and design borrowed-input APIs

EXERCISE 1 WARM-UP COMPREHENSION

Tell the ownership story before touching syntax

OBJECTIVE

Practice describing an ownership and borrowing flow in operational terms.

STARTER PROMPT

A request parser receives `&[u8]`, validates headers, builds a routing key, and then sends an owned job to a worker queue. Describe who owns data at each step and which phases only borrow.

- Which stage owns the original request buffer?
- Which stage may borrow only temporarily?
- At what point should the worker queue receive owned data rather than references into the request buffer?

ACCEPTANCE CRITERIA

- You name an owner for the original request bytes.
- You separate local borrowed inspection from cross-boundary owned data.
- You explain why the queue boundary is normally an ownership transfer point.

HINTS

- Try to answer in terms of owners and temporary access, not in terms of syntax first.
- If a later stage can outlive the current scope, it usually should not borrow from that scope.

EXERCISE 2 CODE READING

Find the unnecessary clone

OBJECTIVE

Identify where cloning papers over an ownership design that could be simpler.

STARTER PROMPT

Review a helper that clones a request path into `path_copy`, prints it, then still uses the original path immediately after. Decide whether the clone is required and explain the cleaner alternative.

- Can a shared borrow do the same work?
- Would returning ownership from a helper be more appropriate than cloning?
- If the clone stays, what makes it economically justified rather than a reflex?

ACCEPTANCE CRITERIA

- You distinguish semantic duplication from borrow-checker work-around.
- You propose borrowing as the first repair when the data only needs read access.
- You justify any remaining clone in terms of API shape or operational cost.

HINTS

- If no independent copy is needed, start by asking whether `&str` or `&T` is enough.
- A good answer mentions when cloning is correct, not only when it is avoidable.

EXERCISE 3 IMPLEMENTATION

Accept borrowed input and return owned output

OBJECTIVE

Design an API that borrows from the caller but returns a result with independent lifetime.

STARTER PROMPT

Implement `fn build_cache_key(service: &str, route: &str) -> String` so the result format is `service:route`.

- Keep the parameters borrowed as `&str`.
- Return `String`, not `&str`.
- Do not allocate intermediate clones unless they are part of the final owned result construction.

ACCEPTANCE CRITERIA

- The function signature is exactly `fn build_cache_key(service: &str, route: &str) -> String`.
- The output matches `billing:/v1/invoices` and `search:/ready` for the provided sample inputs.
- The function returns owned data rather than borrowing from a temporary.

HINTS

- A returned `String` makes the caller independent from the input buffer lifetime.
- Either `format!` or a small `String` builder is fine here.

EXERCISE 4 DEBUGGING OR REFACTORING

Annotate lifetimes only where required

OBJECTIVE

Separate cases that need explicit lifetime relationships from cases handled by elision.

STARTER PROMPT

Decide which signatures need explicit lifetimes and write the corrected form: `fn first_word(s: &str) -> &str` and `fn pick_longer(left: &str, right: &str) -> &str`.

- Which function has only one input borrow?
- Which function returns one of several input borrows?
- Why does adding a lifetime to the first function not improve the model?

ACCEPTANCE CRITERIA

- You keep `fn first_word(s: &str) -> &str` elided.
- You annotate `pick_longer` with one explicit lifetime parameter and tie both inputs plus the output to it.
- You explain that the explicit annotation documents a relationship rather than extending storage lifetime.

HINTS

- One input borrowed reference usually makes elision sufficient.
- Multiple candidate input borrows usually force you to name the output relationship.

EXERCISE 5 DEBUGGING OR REFACTORING

Fix a borrow-checker failure without cloning

OBJECTIVE

Repair a borrow conflict by changing scope or order instead of duplicating data by default.

STARTER PROMPT

A function borrows `let first = first_word(&buffer);`, then tries `buffer.clear();`, and finally prints `first`. Refactor the flow so it compiles without cloning unless you can justify the clone economically.

- Can the use of `first` happen earlier?
- Can the borrowed value be converted into an owned value only if the later code truly needs it after mutation?
- Would a shorter helper function or inner scope make the borrow duration obvious?

ACCEPTANCE CRITERIA

- Your repair ends the borrow before the mutable operation, or deliberately converts to owned data for a justified reason.
- You do not default to cloning without explaining why independent ownership is needed.
- You explain the conflict as overlapping borrow duration, not as ownership or borrowing rule involved.

HINTS

- Ask when the immutable borrow is last used.
- If later mutation is required, the cleanest repair is often to shorten the earlier borrow's lifetime in code structure.

EXERCISE 6 DESIGN OR PRODUCTION SCENARIO

Choose ownership for a long-lived service API

OBJECTIVE

Design a Rust-native boundary for a production service that mixes parsing, normalization, storage, and background work.

STARTER PROMPT

You are designing an audit service with the flow `socket bytes -> parse route -> normalize key -> store record -> publish async notification`.

- Which functions should accept borrowed input?
- Which values must become owned before storage or async notification?
- Where would a borrowed-field struct be justified, and where would it create unnecessary lifetime coupling?
- What tradeoff are you accepting around allocation versus API simplicity?

ACCEPTANCE CRITERIA

- You keep borrowed input local to parsing and normalization where appropriate.
- You convert data to owned form before storage and async boundaries.
- You justify at least one tradeoff involving allocation cost, simplicity, or long-term maintainability.
- You avoid presenting borrowed fields in long-lived structs as the default answer.

HINTS

- Stored records and async notifications usually want independent ownership.
- The best answer names concrete owners at each subsystem edge.

Runnable lab

Example 4-L. `build_cache_key_lab.rs` — *Runnable lab · Borrowed input, owned result*

```rust
fn build_cache_key(service: &str, route: &str) -> String {
    route.to_string()
}

fn main() {
    println!("{}", build_cache_key("billing", "/v1/invoices"));
    println!("{}", build_cache_key("search", "/ready"));
}
```

OUTPUT

```
billing:/v1/invoices
search:/ready
```

Review questions

- Why is ownership usually the first question and lifetimes the second?
- What is the operational difference between `&T` and `&mut T`?
- What does a lifetime annotation describe, and what does it never do?
- When is returning `String` cleaner than returning `&str`?
- Why do queue, cache, and async boundaries often push you toward owned data?

CHAPTER 5 · POINTERS, REFERENCES, AND OWNERSHIP INSIDE STRUCTS

Pointers, References, and Ownership Inside Structs

Exercises, labs, and review questions for this chapter.

Choose field ownership, avoid lifetime-heavy structs, expose safe views, and replace self-references with stable layouts

EXERCISE 1 WARM-UP COMPREHENSION

Choose owned versus borrowed fields for two domain objects

OBJECTIVE

Practice separating a short-lived view type from a long-lived stored type instead of trying to make one struct serve both jobs.

STARTER PROMPT

Design two structs for an API gateway: `RouteMatchView` used only during request parsing, and `StoredRouteMatch` used later in metrics and retries.

- Which fields in `RouteMatchView` should borrow from the request buffer?
- Which fields in `StoredRouteMatch` should become owned before they hit metrics, retries, or persistence?
- Explain one concrete reason the two structs should not have the same field types.

ACCEPTANCE CRITERIA

- You give the view type borrowed fields only where the upstream owner is obvious and nearby.
- You give the stored type owned fields for data that must outlive the parse step.
- You explain at least one tradeoff involving allocation cost versus lifetime simplicity.

HINTS

- A parser view and a stored record usually have different jobs.
- Ask whether the second struct could sit in a `Vec`, queue, or cache without external lifetime baggage.

EXERCISE 2 CODE READING

Explain the lifetime coupling in a borrowed-field struct

OBJECTIVE

Read a struct with references and describe what the lifetime parameter means operationally rather than syntactically.

STARTER PROMPT

Given `struct HeaderView<'a> { name: &'a str, value: &'a str }`, explain what `<'a>` says, what it does not say, and where this type is a good fit.

- Who owns the bytes behind `name` and `value`?
- Why does the struct need a lifetime parameter at all?
- Why is this design usually comfortable in parsing code but awkward in caches and queues?

ACCEPTANCE CRITERIA

- You explain that the struct borrows from some external owner and therefore cannot outlive that owner.
- You state that the lifetime parameter constrains reference validity and does not extend storage lifetime.
- You identify at least one scenario where the type is a good fit and one where it becomes burdensome.

HINTS

- Treat `<'a>` as a relationship marker, not as a retention policy.
- A good answer names the owner outside the struct.

EXERCISE 3 IMPLEMENTATION

Build a struct that owns a buffer and exposes read-only views safely

OBJECTIVE

Implement an owned-buffer pattern that returns borrowed views derived from `&self` instead of storing self-references.

STARTER PROMPT

Implement `AuditLine` so it owns a raw log line and exposes a `level(&self) -> &str` method that returns the text before the first colon.

- Keep the owned field as `String`.
- Return `&str` from `level`.
- Do not store a second field borrowing from `raw`.

ACCEPTANCE CRITERIA

- The struct stores the raw line as owned `String` data.
- The `level` method returns a borrowed `&str` view derived from `&self`.
- The sample output is exactly `level = WARN` and `raw = WARN: cache miss`.

HINTS

- Own the line once, then slice it on demand.
- A method returning `&str` from `&self` is the key idea here.

EXERCISE 4 DEBUGGING OR REFACTORING

Refactor a self-referential design into stable indices

OBJECTIVE

Replace an invalid self-borrowing layout with a representation Rust can move and reason about safely.

STARTER PROMPT

You inherit the shape `struct ParsedLine<'a> { raw: String, first: &'a str }`. Refactor it so the type still owns `raw` but can recover the first token later without storing a self-reference.

- Will you store a byte range, a start/end offset pair, or a typed token ID?
- How will the accessor recover `&str` from `&self`?
- What happens if the owned buffer can mutate later?

ACCEPTANCE CRITERIA

- Your redesign removes the self-reference entirely.
- Your replacement stores stable metadata such as offsets, ranges, or IDs.
- You explain how the accessor reconstructs a borrowed view from the owned buffer.
- You note that mutation rules must preserve whatever indexing metadata you store.

HINTS

- If the accessor starts from `&self`, it can borrow from the owned buffer at call time.
- Offsets and ranges are often simpler than pointer-like designs.

EXERCISE 5 DEBUGGING OR REFACTORING

Choose Box, Rc, or Arc intentionally

OBJECTIVE

Practice mapping pointer types to actual ownership semantics instead of treating them as interchangeable heap wrappers.

STARTER PROMPT

Choose a field type for each case: a recursive AST node owned by one parent, a read-mostly UI template shared inside one thread, and a schema object shared across worker threads.

- Where is `Box<T>` the right indirection tool?
- Why is `Rc<T>` wrong at a thread boundary?
- What extra truth must hold before `Arc<T>` is enough?

ACCEPTANCE CRITERIA

- You choose `Box<T>` for the recursive single-owner case.
- You choose `Rc<T>` only for single-thread shared ownership.
- You choose `Arc<T>` only for cross-thread shared ownership and mention that `T` must still be thread-safe.

HINTS

- Reference counting answers an ownership-count question, not a mutability-design question.
- Start with 'how many owners?' and 'which threads?' before naming the type.

EXERCISE 6 DESIGN OR PRODUCTION SCENARIO

Design long-lived service structs without over-lifetime-parameterization

OBJECTIVE

Make a lifetime-aware struct design for a service pipeline without turning every downstream type into `Thing<'a>`.

STARTER PROMPT

You are designing `socket bytes -> parse headers -> build Event -> enqueue -> retry -> persist`. Decide which types are temporary borrowed views and which types should own their fields.

- Which types exist only inside parsing or validation?
- At which boundary does `Event` become fully owned?
- Where would `Arc<T>` be semantically real, and where would it merely hide unclear ownership?
- What is one allocation you would accept to simplify the rest of the system?

ACCEPTANCE CRITERIA

- You keep borrowed views local to parsing or validation.
- You convert queued, retried, or persisted data into owned forms before those boundaries.
- You justify at least one tradeoff involving allocation, indirection, or shared ownership cost.
- You avoid making all downstream service structs lifetime-parameterized by default.

HINTS

- Queues and retries are ownership boundaries.
- If a type survives past the input buffer, it should usually stop borrowing from it.

Runnable lab

Example 5-L. `audit_line_lab.rs` — *Runnable lab · Exercise 3*

```
struct AuditLine {
    raw: String,
}

impl AuditLine {
    fn new(raw: &str) -> Self {
        Self {
            raw: String::new(),
        }
    }

    fn level(&self) -> &str {
        "UNKNOWN"
    }
}

fn main() {
    let line = AuditLine::new("WARN: cache miss");
    println!("level = {}", line.level());
    println!("raw = {}", line.raw);
}
```

OUTPUT

```
level = WARN
raw = WARN: cache miss
```

Review questions

- When is a borrowed-field struct a strong design, and when is it a maintenance smell?
- Why does one borrowed field often force a lifetime parameter onto the whole struct?
- What problem does `Box<T>` solve that plain owned `T` does not?
- Why is `Arc<T>` not a synonym for 'safe to share and mutate however I want'?
- Why do offsets, ranges, or IDs often beat self-references inside owned structs?

CHAPTER 6 · POINTERS, REFERENCES, AND OWNERSHIP INSIDE VECTORS

Pointers, References, and Ownership Inside Vectors

Exercises, labs, and review questions for this chapter.

Repair reallocation hazards, switch from references to indices, and mutate vectors safely under production constraints

EXERCISE 1 WARM-UP COMPREHENSION

Narrate the invalidation event before proposing a fix

OBJECTIVE

Practice explaining why a reference into a vector stops being the right handle once the vector may grow.

STARTER PROMPT

A function borrows `let first = &workers[0];`, then later appends more workers with `push`, and finally reads `first.name`. Explain the failure in operational terms.

- What does the vector own?
- What does the borrowed reference point into?
- Why is the problem not merely 'the borrow checker being strict'?

ACCEPTANCE CRITERIA

- You explain that the reference points into the vector's current buffer.
- You state that growth may relocate the buffer and therefore invalidate the old address story.
- You propose at least one valid repair such as using an index, ending the borrow sooner, or changing representation.

HINTS

- Start with the owner: the vector owns the buffer.
- Then ask what `push` is allowed to do to that buffer.

EXERCISE 2 CODE READING

Find the hidden shape change

OBJECTIVE

Read a small loop and identify which operation changes vector shape or element positions.

STARTER PROMPT

Review a loop that stores `&sessions[i]` in a temporary, then calls either `reserve`, `insert`, `remove`, or `sort_by_key` later in the same flow. Identify which operations threaten reference validity and why.

- Which operations may relocate storage?
- Which operations may keep the same buffer but still shift positions?
- Which handle type would stay meaningful after each operation?

ACCEPTANCE CRITERIA

- You distinguish relocation hazards from position-shift hazards.
- You explain that both categories are dangerous for borrowed element references held too long.
- You choose a more durable handle such as an index or ID where appropriate.

HINTS

- Address stability and logical identity are related, but they are not the same thing.
- Sorting may preserve allocation yet still break position-based assumptions.

EXERCISE 3 IMPLEMENTATION

Return an index handle instead of assuming reference stability

OBJECTIVE

Refactor a helper so callers keep a durable index rather than an element reference or a mistaken post-push length.

STARTER PROMPT

Implement `enqueue(tasks: &mut Vec<Task>, name: &str) -> usize` so the returned handle identifies the newly pushed task correctly even if the vector later grows.

- Keep the return type as `usize`.
- Do not return a borrowed reference.
- Make the handle point at the inserted task, not at the next free slot.

ACCEPTANCE CRITERIA

- The function returns the inserted task's index.
- The sample output shows the first inserted task after additional pushes.
- The design remains valid even if the vector grows later.

HINTS

- The correct index exists either just before the push or as `len() - 1` immediately after it.
- A durable handle should survive future vector growth as long as reordering policy is understood.

EXERCISE 4 DEBUGGING OR REFACTORING

Convert a reference-heavy graph edge into IDs

OBJECTIVE

Replace stored references into a vector-backed node list with an ID or index design Rust can move and maintain safely.

STARTER PROMPT

You inherit a parser graph that stores nodes in a `Vec<Node>` and also stores `&Node` edges inside other nodes. Refactor the relationship into indices or typed IDs.

- Where do you store the nodes?
- What do edges store instead of references?
- How does lookup recover a borrowed view when needed?

ACCEPTANCE CRITERIA

- Your redesign removes long-lived references into the vector.
- Edges store indices, IDs, or another stable logical handle.
- You explain how the owner vector reborrows the node by handle at use time.

HINTS

- The owner of the collection should usually also be the place where reborrowing happens.
- Typed IDs are often easier to review than naked `usize` values in larger systems.

EXERCISE 5 DEBUGGING OR REFACTORING

Implement safe mutation while iterating

OBJECTIVE

Repair a loop that inspects a vector and also changes its shape in the same pass.

STARTER PROMPT

A retry scheduler scans jobs, appends follow-up work for failed jobs, and removes expired jobs in one loop. Rewrite the flow safely.

- Which edits can happen in place with `iter_mut`?
- Which edits should move into a second pass?
- Would `retain`, `drain`, or rebuilding be clearer than manual index juggling?

ACCEPTANCE CRITERIA

- You separate inspection from shape-changing edits, or choose a standard library pattern that does so safely.
- You explain why one pass with live element borrows plus shape changes is the real problem.
- Your final design does not depend on cloning the whole vector just to escape the issue.

HINTS

- Collect indices, IDs, or commands first.
- Then apply the structural edits once no element borrow remains active.

EXERCISE 6 DESIGN OR PRODUCTION SCENARIO

Choose Vec<T>, Vec<Box<T>>, or arena-style storage for a registry

OBJECTIVE

Select the right collection representation for a production registry with concrete stability and performance requirements.

STARTER PROMPT

You are designing a long-lived connection registry for `accept -> authenticate -> route -> retry -> drop`. Some components need fast scans, some need durable handles, and a plugin boundary may require stable addresses.

- Which parts of the system only need contiguous scanning?
- Which parts need stable logical handles rather than direct references?
- Does any subsystem truly need stable element addresses?
- What cost are you willing to pay in allocation or indirection?

ACCEPTANCE CRITERIA

- You justify one representation as the default and explain where another becomes necessary.
- You distinguish stable logical handles from stable memory addresses.
- You name at least one tradeoff involving cache locality, allocation count, or API complexity.
- You avoid choosing a heavier representation without an actual requirement for it.

HINTS

- Start with the simplest owner that meets the semantics.
- Plain vectors are often right until stable addressability or deletion semantics make them insufficient.

Runnable lab

Example 6-L. `task_handle_lab.rs` — *Runnable lab · Exercise 3*

```rust
#[derive(Debug)]
struct Task {
    name: String,
}

fn enqueue(tasks: &mut Vec<Task>, name: &str) -> usize {
    tasks.push(Task {
        name: name.to_string(),
    });
    tasks.len()
}

fn main() {
    let mut tasks = Vec::with_capacity(1);
    let first = enqueue(&mut tasks, "billing");
    enqueue(&mut tasks, "search");
    enqueue(&mut tasks, "indexer");

    println!("first = {}", tasks[first].name);
    println!("total = {}", tasks.len());
}
```

OUTPUT

```
first = billing
total = 3
```

Review questions

- What exactly does a borrowed element reference from a vector point into?
- Why are index handles often the default durable handle for `Vec<T>`?
- What is the difference between stable logical identity and stable memory address?
- When is `Vec<Box<T>>` a better fit than plain `Vec<T>`?
- Why do two-phase loops often produce the clearest vector mutation code?

CHAPTER 7 · COPYING DATA VS CLONING DATA

Copying Data vs Cloning Data

Exercises, labs, and review questions for this chapter.

Classify move/copy/clone operations, implement Clone manually, remove unnecessary clones, and choose Copy safely

EXERCISE 1 WARM-UP COMPREHENSION

Classify each duplication event correctly

OBJECTIVE

Practice separating moves, implicit copies, explicit clones, and plain borrows in one small flow.

STARTER PROMPT

Classify the operation at each marked line in a function that uses `RequestId(u64)`, `String`, `&str`, and `Arc<Schema>` values.

- Which line is a move because the type is non-Copy and ownership transfers?
- Which line is an implicit copy because the type is `Copy`?
- Which line is an explicit clone of owned data?
- Which line is only a borrow and therefore not duplication at all?

ACCEPTANCE CRITERIA

- You classify each event using Rust terms rather than vague 'copy-like' language.
- You distinguish shared ownership via `Arc::clone` from deep cloning of inner data.
- You explain one case where borrowing removes the need for duplication entirely.

HINTS

- Ask whether the old binding still owns the value afterward.
- The spelling at the call site is a clue: plain assignment, `.clone()`, `Arc::clone`, or `&value`.

EXERCISE 2 CODE READING

Find the hidden allocation budget

OBJECTIVE

Read a small service path and identify which operations are cheap scalar copies, which clone heap data, and which merely add shared owners.

STARTER PROMPT

Review a request path that duplicates a `RequestId`, clones a `String` route key, and clones an `Arc<Schema>`. Write a short cost review for the function.

- Which operation is almost certainly trivial bitwise duplication?
- Which operation likely duplicates heap-backed bytes?
- Which operation increments a reference count instead of deep-cloning the schema?
- Where would you replace cloning with borrowing if the helper only reads?

ACCEPTANCE CRITERIA

- You identify a scalar or small ID-style copy correctly.
- You identify at least one heap-backed `Clone` correctly.
- You explain `Arc::clone` as another shared owner rather than a deep copy.
- You propose one borrowing repair that would remove unnecessary allocation.

HINTS

- Read the field types first. Costs tend to follow ownership shape.
- A review answer should say what happened operationally, not just 'this is expensive.'

EXERCISE 3 IMPLEMENTATION

Implement Clone manually for a realistic type

OBJECTIVE

Write a correct manual `Clone` implementation for a type with owned and scalar fields.

STARTER PROMPT

Implement `Clone` for `JobTemplate { service: String, steps: Vec<String>, retries: usize }` so cloning preserves the data and later mutations to the clone do not affect the original.

- Clone the owned fields explicitly.
- Copy the scalar field directly.
- Do not implement `Copy` for this type.

ACCEPTANCE CRITERIA

- Your type implements `Clone` manually, not through a placeholder or stub.
- Owned fields are duplicated correctly.
- Mutating the clone after creation does not change the original value.
- You keep the type non-`Copy` because implicit duplication would be misleading here.

HINTS

- Use `.clone()` on `String` and `Vec<String>`.
- A `usize` does not need a `clone()` call in practice. Copying the value is enough.

EXERCISE 4 DEBUGGING OR REFACTORING

Remove unnecessary clones from a read-only API

OBJECTIVE

Repair an API that clones owned strings where borrowing would express the contract more accurately.

STARTER PROMPT

A helper `fn route_label(route: String) -> String` is called only to log and compare route text. Refactor the API so callers stop cloning route strings just to satisfy the signature.

- What should the parameter type become if the helper only reads?
- Should the return type stay owned or become borrowed?
- Which call sites can lose `.clone()` after the change?

ACCEPTANCE CRITERIA

- Your refactor changes the signature to borrow when read-only access is enough.
- You remove at least one unnecessary clone from the caller side.
- You explain why borrowing communicates the contract more clearly than taking ownership here.

HINTS

- If the helper only inspects text, start with `&str`.
- A better signature usually fixes more than one clone at once.

EXERCISE 5 DESIGN OR PRODUCTION SCENARIO

Decide whether a type should implement Copy

OBJECTIVE

Practice the conservative rule set for implementing `Copy` safely.

STARTER PROMPT

Evaluate whether each type should implement `Copy`: `RequestId(u64)`, `Span { start: usize, end: usize }`, `SessionKey(String)`, `SharedSchema(Arc<str>)`, and `SocketOwner(TcpStream)`.

- Which ones satisfy Rust's mechanical rules for `Copy`?
- Which ones would still be a semantic mistake even if the shape looked small?
- Where is explicit `Clone` or no duplication trait the better signal?

ACCEPTANCE CRITERIA

- You approve `Copy` only for types where implicit duplication is trivial and unsurprising.
- You reject `Copy` for heap-owning or resource-owning types.
- You explain why `Arc<T>` being cheap to clone does not make the wrapper automatically a good `Copy` candidate.

HINTS

- The `Copy` checklist is stricter than 'this seems lightweight.'
- Resource ownership and shared ownership deserve visible APIs.

EXERCISE 6 DESIGN OR PRODUCTION SCENARIO

Choose receivers and duplication boundaries together

OBJECTIVE

Design a small service API where receiver choice and duplication policy line up.

STARTER PROMPT

You are building a `ConfigBuilder`, a long-lived `ServiceConfig`, and a `publish` step that fans work out to multiple workers. Choose where methods should take `self`, `&self`, or `&mut self`, and where cloning is legitimate.

- Which method should consume `self` because it finalizes the builder?
- Which methods only need `&self` for inspection or borrowed views?
- Which mutations should use `&mut self` rather than rebuilding with a fresh clone?
- At the worker fan-out boundary, do you move, clone, or share through `Arc`?

ACCEPTANCE CRITERIA

- You use receiver choices to express ownership transitions clearly.
- You justify at least one explicit clone in economic or semantic terms.
- You avoid both extremes: cloning everything and forcing everything through shared ownership.
- You name one tradeoff involving allocation, atomic cost, or API clarity.

HINTS

- Builders often end with a consuming `build(self)` or `finish(self)`.
- Fan-out work may justify `Arc` when the underlying data is truly shared.

Runnable lab

Example 7-L. `manual_clone_lab.rs` — *Runnable lab · Manual Clone implementation*

```rust
#[derive(Debug)]
struct JobTemplate {
    service: String,
    steps: Vec<String>,
    retries: usize,
}

impl Clone for JobTemplate {
    fn clone(&self) -> Self {
        Self {
            service: String::new(),
            steps: Vec::new(),
            retries: self.retries,
        }
    }
}

fn main() {
    let original = JobTemplate {
        service: String::from("billing"),
        steps: vec![String::from("parse"), String::from("persist")],
        retries: 2,
    };

    let mut cloned = original.clone();
    cloned.steps.push(String::from("notify"));

    println!("original = {} {}", original.service, original.steps.len());
    println!("cloned = {} {}", cloned.service, cloned.steps.len());
    println!("retries = {}", cloned.retries);
}
```

OUTPUT

```
original = billing 2
cloned = billing 3
retries = 2
```

Review questions

- What is the practical difference between moving a `String`, copying a `u64`, and cloning an `Arc<T>`?
- Why is `Copy` intentionally much narrower than 'cheap enough to duplicate'?
- When is `Cow<'a, str>` a better API than always returning `String`?
- Why is `Arc::clone` not the same thing as deep-cloning the inner value?
- How do receiver choices reduce unnecessary cloning in production APIs?

CHAPTER 8 · UNDEFINED BEHAVIOR AND UNSAFE RUST

Undefined Behavior and Unsafe Rust

Exercises, labs, and review questions for this chapter.

Audit unsafe abstractions, identify UB risks in raw-pointer code, repair MaybeUninit misuse, and design safe wrappers around unsafe boundaries

EXERCISE 1 WARM-UP COMPREHENSION

Name the invariant before the pointer

OBJECTIVE

Translate one unsafe operation into explicit preconditions instead of vague confidence.

STARTER PROMPT

A helper builds `slice::from_raw_parts(ptr, len)` from a raw pointer returned by a packet parser. State the invariants that must hold before that slice is valid.

- What must be true about nullability, alignment, and bounds?
- What lifetime story makes the returned slice valid?
- What aliasing story must still hold if the slice is shared or mutable?

ACCEPTANCE CRITERIA

- You name pointer validity conditions such as non-null when required, alignment, and readable memory for `len` elements.
- You explain that the produced slice cannot outlive the real owner of the backing allocation.
- You mention aliasing requirements that would apply if references are created from the raw pointer.

HINTS

- Start with: what memory is this pointer allowed to refer to right now?
- Then ask: for how long is that still true?

EXERCISE 2 CODE READING

Find the UB risks in a raw-pointer copy helper

OBJECTIVE

Practice spotting the real missing proof obligations in a pointer-based routine.

STARTER PROMPT

Review a function that takes `dst: *mut u8`, `src: *const u8`, and `len: usize`, loops with pointer arithmetic, and is exposed as a safe public function with no checks.

- Which caller obligations are currently undocumented?
- Should the function stay safe, become `unsafe fn`, or be redesigned around slices?
- Where could null, lifetime, writability, or bounds assumptions fail?

ACCEPTANCE CRITERIA

- You identify multiple missing obligations rather than saying only 'raw pointers are dangerous.'
- You explain why a safe public API is wrong if the callee cannot actually enforce those obligations.
- You propose either an `unsafe fn` contract or a safe redesign using slices or owned buffers.

HINTS

- Ask whether the callee can validate the entire contract or only part of it.
- If the caller must uphold the truth, the API shape should say so.

EXERCISE 3 IMPLEMENTATION

Wrap unsafe internals in a safe public API

OBJECTIVE

Implement a checked wrapper that writes a fixed four-byte marker only when the buffer is large enough.

STARTER PROMPT

Implement `write_magic(buf: &mut [u8]) -> Result<(), &'static str>` so buffers shorter than four bytes return `Err("buffer too small")`, and valid buffers become `RST!` through a small unsafe raw-pointer write block.

- Keep the public signature safe.
- Check the length before any unsafe write happens.
- Write the bytes `82, 83, 84, 33` into positions `0..4`.

ACCEPTANCE CRITERIA

- Short buffers return `Err("buffer too small")` exactly.
- Long enough buffers return `Ok(())` and contain the marker `RST!`.
- The unsafe region stays small and depends on an explicit length check.

HINTS

- The unsafe block should begin only after the precondition is already true.
- This is the classic shape of a safe abstraction over unsafe internals.

EXERCISE 4 DEBUGGING OR REFACTORING

Move `assume_init` to the only valid place

OBJECTIVE

Repair a `MaybeUninit` design that treats uninitialized storage as a finished value too early.

STARTER PROMPT

You inherit code that creates `MaybeUninit<[u32; 4]>`, calls `assume_init()` immediately, and then writes only some elements afterward. Refactor the flow so initialization becomes explicit and ordered correctly.

- Where should writes occur relative to `assume_init`?
- What must be true before the final array exists as a real `[u32; 4]`?
- How would you explain the bug to a reviewer in one sentence?

ACCEPTANCE CRITERIA

- You keep the value inside `MaybeUninit` storage until all elements are written.
- You call `assume_init` only after full initialization is complete.
- You explain that reading the value early would treat uninitialized memory as a valid typed value.

HINTS

- The repair is usually about order, not about adding more unsafe.
- A valid `T` exists only when every invariant of `T` is already true.

EXERCISE 5 DEBUGGING OR REFACTORING

Decide whether the function should be safe or unsafe

OBJECTIVE

Separate APIs whose callers must uphold invariants from APIs whose callees can enforce them internally.

STARTER PROMPT

Audit `fn view<'a>(ptr: *const u8, len: usize) -> &'a [u8]`. Decide whether it should remain safe, become `unsafe fn`, or be replaced by a different signature altogether.

- Can the function verify the pointer is valid for an arbitrary `'a`?
- Is the returned lifetime actually tied to a real owner?
- Would a signature based on `&[u8]` or an owned buffer express the truth more honestly?

ACCEPTANCE CRITERIA

- You explain why the callee cannot manufacture an arbitrary valid lifetime from a raw pointer.
- You propose a more honest API shape, not only a slogan about safety.
- You distinguish caller-held obligations from callee-enforced checks clearly.

HINTS

- If the type claims more than the implementation can prove, the API is lying.
- Raw pointers plus arbitrary returned references are where lifetime fiction becomes dangerous quickly.

EXERCISE 6 DESIGN OR PRODUCTION SCENARIO

Wrap an FFI boundary without exporting C mistakes

OBJECTIVE

Design a Rust-facing wrapper around a C API while keeping the rest of the system in ordinary safe Rust.

STARTER PROMPT

A C library returns `*const c_char` error messages and `*mut Record` results. Design the Rust wrapper layer for a service that parses input, calls the library, retries failures, and eventually frees native resources.

- Where does unsafe live: in every caller, or in one adapter layer?
- How do you translate nullability, ownership transfer, and string validity into Rust types?
- What is the plan for cleanup and for preventing unwinding assumptions from staying fuzzy at the boundary?

ACCEPTANCE CRITERIA

- You isolate unsafe code in one adapter or wrapper layer.
- You translate raw pointer contracts into Rust concepts such as `Option`, `Result`, owned types, or borrowed views with clear lifetimes.
- You describe who owns native resources and who releases them.
- You mention at least one failure mode involving invalid strings, null pointers, or mismatched cleanup.

HINTS

- Good wrapper layers convert C contracts into Rust truths early.
- If the rest of the codebase still manipulates raw pointers directly, the boundary is probably too wide.

Runnable lab

Example 8-L. `write_magic_lab.rs` — *Runnable lab · Exercise 3*

```rust
fn write_magic(buf: &mut [u8]) -> Result<(), &'static str> {
    let ptr = buf.as_mut_ptr();

    unsafe {
        ptr.add(0).write(82);
        ptr.add(1).write(83);
        ptr.add(2).write(84);
        ptr.add(3).write(33);
    }

    Ok(())
}

fn main() {
    let mut short = vec![0u8; 3];
    let mut ok = vec![0u8; 4];

    println!("short = {:?}", write_magic(&mut short));
    println!("value = {:?}", write_magic(&mut ok));
    println!("buf = {}", std::str::from_utf8(&ok).unwrap());
}
```

OUTPUT

```
short = Err("buffer too small")
value = Ok(())
buf = RST!
```

Review questions

- What does an `unsafe` block let you do, and what does it not excuse?
- Why is a data race undefined behavior in Rust rather than merely an unlucky concurrency bug?
- What is the difference between a raw pointer and a reference in terms of promises made to the optimizer?
- Why is `MaybeUninit<T>` the right model for not-yet-valid storage?
- When should an API be `unsafe fn` instead of a safe function with internal unsafe code?

CHAPTER 9 · SMART POINTERS AND PINNING

Smart Pointers and Pinning

Exercises, labs, and review questions for this chapter.

Choose pointer types deliberately, break cycles with Weak, and explain why pinned futures exist

EXERCISE 1 WARM-UP COMPREHENSION

Read each pointer type as an ownership statement

OBJECTIVE

Restate what Box, Rc, Arc, RefCell, Mutex, and Weak each encode without conflating ownership count with mutation authority.

STARTER PROMPT

The chapter argues that a smart pointer encodes ownership count, mutation authority, thread sharing, and address stability. For each of Box<T>, Rc<T>, Arc<T>, RefCell<T>, Mutex<T>, and Weak<T>, write one sentence naming exactly what it provides and one naming what it does NOT provide.

- State the owner count each type implies: one, many on one thread, or many across threads.
- Say which types add mutation through a shared handle and where the aliasing rule is enforced (compile time, runtime panic, or lock acquisition).
- Explain why Weak<T> is described as a non-owning edge and what upgrade() returns.
- Identify which single type in the list speaks to address stability rather than ownership.

ACCEPTANCE CRITERIA

- Box<T> is described as one owner plus heap indirection, with no sharing, reference counting, or interior mutability.
- Rc<T> and Arc<T> are both shared ownership, differing only in non-atomic versus atomic counting and therefore single-thread versus cross-thread use.
- RefCell<T> is named as runtime-checked borrowing (panic on violation), Mutex<T> as exclusive access after lock acquisition, and neither is presented as adding owners by itself.
- Weak<T> is identified as not keeping the allocation alive, with upgrade() returning Option<Rc<T>> or Option<Arc<T>>; Pin is the only address-stability item.

HINTS

- Ownership count and mutation authority are independent questions; a composite like Rc<RefCell<T>> answers both.
- Reread the 'Choosing the right pointer type' section: owners first, then thread boundary, then mutation model.

EXERCISE 2 CODE READING

Trace the strong and weak edges in the tree sample

OBJECTIVE

Read the chapter's Node sample and account for every reference-count change and borrow that occurs at runtime.

STARTER PROMPT

Work through the smart_pointers_tree sample, where Node holds parent: RefCell<Weak<Node>> and children: RefCell<Vec<Rc<Node>>>. Explain why the graph stays acyclic and what each printed line reports.

- After root.children.borrow_mut().push(Rc::clone(&leaf)), state leaf's strong count and why it changed.
- After *leaf.parent.borrow_mut() = Rc::downgrade(&root), state root's strong count and weak count.
- Explain why leaf.parent.borrow().upgrade() must be handled as an Option rather than dereferenced directly.
- Justify the final value of Rc::strong_count(&root) printed by the program.

ACCEPTANCE CRITERIA

- The answer states that pushing Rc::clone(&leaf) raises leaf's strong count to 2, and that downgrade adds a weak (not strong) count to root.
- Rc::strong_count(&root) is correctly explained as 1, because only the leaf-to-root edge is weak and no second strong handle to root exists.
- The answer explains that upgrade() returns Option because a Weak does not keep the allocation alive, so the owner may already be gone.
- The reader identifies the children edge as the owning (strong) edge and the parent edge as the observational (weak) edge, which is why no cycle forms.

HINTS

- Each Rc::clone bumps the strong count; Rc::downgrade bumps the weak count instead.
- A strong cycle is two strong edges that point at each other; here one direction is deliberately weak.

EXERCISE 3 IMPLEMENTATION

Build a single-thread cache with Rc plus interior mutability

OBJECTIVE

Compose shared ownership with the smallest sufficient interior-mutability tool instead of reaching for RefCell by reflex.

STARTER PROMPT

Implement a single-thread counter cache shared by several handles: a struct Stats holding hits: Cell<u32> and a label: String, wrapped as Rc<Stats>. Provide fn record_hit(stats: &Rc<Stats>) that increments the count and fn snapshot(stats: &Rc<Stats>) -> u32 that reads it.

- Choose Cell<u32> rather than RefCell<u32> and justify the choice in one sentence.
- Implement record_hit using get and set (or a single update of the copied value), without handing out an interior reference.
- Clone the Rc<Stats> into a second handle and show that both handles observe the same incremented count.
- State what would change if a field needed an interior &mut reference instead of copy-out semantics.

ACCEPTANCE CRITERIA

- Stats uses Cell<u32> for the count, and record_hit mutates it through a shared &Rc<Stats> without &mut.
- snapshot returns the current count by value and does not borrow an interior reference.
- Two cloned Rc handles are shown to read the same updated count, demonstrating shared ownership of one allocation.
- The answer notes that Cell is correct here because only whole-value copy/replace is needed, and RefCell would be required only if interior references were handed out.

HINTS

- Cell::get copies the value out; Cell::set replaces it; no borrow token is involved.
- Cloning an Rc adds an owner of the same allocation, so both handles see one shared Cell.

EXERCISE 4 IMPLEMENTATION

Poll a hand-written future through Pin

OBJECTIVE

Drive an address-sensitive value to completion using Pin and the poll contract from the chapter's pinning sample.

STARTER PROMPT

Following the smart_pointers_pin_poll sample, implement a future struct Ticker { remaining: u8 } whose poll returns Poll::Pending while remaining is above zero (decrementing each call) and Poll::Ready("fired") at zero. Box::pin it and poll it in a loop until ready.

- Give poll the exact signature fn poll(self: Pin<&mut Self>, cx: &mut Context<'_>) -> Poll<Self::Output>.
- Recover a mutable reference to the fields with self.get_mut() and explain why that is permitted here.
- Build a no-op Waker and Context, pin the future with Box::pin, and poll via Future::poll(task.as_mut(), &mut cx).
- Count how many Pending results precede the single Ready for remaining = 3.

ACCEPTANCE CRITERIA

- poll has the signature taking Pin<&mut Self> and returns Poll::Ready("fired") only when remaining reaches zero.
- self.get_mut() is used to reach the fields, with a note that it is sound because Ticker is movable (Unpin).
- The driver pins with Box::pin and polls task.as_mut() in a loop until Poll::Ready is returned.
- For remaining = 3 the program reports three Pending polls before the Ready result.

HINTS

- get_mut is available because a plain struct of Copy fields is Unpin; the !Unpin restriction is what blocks projection elsewhere.
- Pin<&mut Self> is the temporary pinned view; Box::pin gives the owned pinned storage you poll repeatedly.

EXERCISE 5 DEBUGGING OR REFACTORING

Break a reference cycle and fix a misused pointer

OBJECTIVE

Diagnose a leak caused by two strong edges and a thread-safety mismatch, then repair both with the chapter's rules.

STARTER PROMPT

A single-thread graph defines Node { parent: RefCell<Option<Rc<Node>>>, children: RefCell<Vec<Rc<Node>>> } and is then moved across a thread boundary inside an Rc. The parent never drops, and the cross-thread move fails to compile. Find and fix both defects.

- Explain why a strong parent edge plus a strong children edge prevents either strong count from reaching zero.
- Change the parent edge to the correct non-owning type and adjust how the parent is set and read.
- Explain why Rc<Node> cannot cross a thread boundary and name the minimal change for the cross-thread case.
- State whether switching to Arc alone makes shared mutation safe, and what additional mechanism is required.

ACCEPTANCE CRITERIA

- The leak is correctly attributed to a strong-strong cycle, with neither side reaching a strong count of zero.
- The parent field is changed to RefCell<Weak<Node>>, set via Rc::downgrade and read via upgrade() returning an Option.
- The answer states Rc<Node> is not Send/Sync because its count is non-atomic, and that Arc<Node> is the cross-thread replacement.
- The answer makes clear that Arc fixes ownership count only, and shared mutation across threads still needs a Mutex (or message passing).

HINTS

- Owning edges stay strong; observational back-edges become weak.
- Arc<Mutex<T>> reads as shared ownership plus synchronized mutation, two separate decisions.

EXERCISE 6 DESIGN OR PRODUCTION SCENARIO

Choose pointer types for a mixed-ownership service

OBJECTIVE

Translate a service's ownership, threading, and mutation requirements into concrete composite pointer types.

STARTER PROMPT

Design the storage types for the service in the opening scenario: recursive parser state with a single owner, planning data shared on one thread, a schema shared read-only across worker threads, retry coordination mutated by several threads, and a future stored to be polled later.

- Assign a pointer type to each of the five concerns and justify owner count and thread boundary for each.
- For each shared-and-mutable concern, name the enforcement model and read the composite type literally (for example, what Arc<Mutex<T>> means).
- Explain why the stored future uses Pin<Box<dyn Future>> rather than a plain Box.
- Flag the operational concerns the locking choice introduces (lock scope, contention, poisoning, deadlock).

ACCEPTANCE CRITERIA

- Recursive parser state is Box<T> (one owner, heap indirection for finite size); shared planning data is Rc<T> or Rc<RefCell<T>> on one thread.
- The cross-thread read-only schema is Arc<T>, and the cross-thread mutable retry state is Arc<Mutex<T>>, each justified by owner count then thread boundary then mutation model.
- The stored future is Pin<Box<dyn Future>>, justified by the future's address-sensitive state machine needing to stay put while polled.
- The answer names at least two operational concerns the Mutex introduces, such as lock scope, contention, poisoning, or deadlock risk.

HINTS

- Answer the questions in order: how many owners, one thread or many, does the shared value mutate, under which model.
- Read composites as layered statements; the type literal already documents the runtime story.

Runnable lab

Complete the attach function so the parent owns the child through a strong Rc and the child only observes the parent through a Weak handle. The printed counts and the post-drop lookup confirm the graph is acyclic.

Example 9-L. `weak_parent_edge_lab.rs` — *Runnable lab · strong edges own, weak edges observe*

```
use std::cell::RefCell;
use std::rc::{Rc, Weak};
```

```rust
#[derive(Debug)]
struct Node {
    name: String,
    parent: RefCell<Weak<Node>>,
    children: RefCell<Vec<Rc<Node>>>,
}

impl Node {
    fn new(name: &str) -> Rc<Node> {
        Rc::new(Node {
            name: String::from(name),
            parent: RefCell::new(Weak::new()),
            children: RefCell::new(Vec::new()),
        })
    }
}

// TODO: attach `child` under `parent` so the parent owns the child with a
// strong edge and the child observes the parent with a weak edge.
fn attach(_parent: &Rc<Node>, _child: &Rc<Node>) {
    // TODO: push a strong Rc::clone of `child` into `parent.children`,
    // then set `child.parent` to a weak handle via Rc::downgrade.
}

fn parent_name(node: &Rc<Node>) -> String {
    node.parent
        .borrow()
        .upgrade()
        .map(|p| p.name.clone())
        .unwrap_or_else(|| String::from("none"))
}

fn main() {
    let root = Node::new("root");
    let leaf = Node::new("leaf");

    attach(&root, &leaf);

    println!("root children = {}", root.children.borrow().len());
    println!("leaf parent = {}", parent_name(&leaf));
    println!("root strong = {}", Rc::strong_count(&root));
    println!("root weak = {}", Rc::weak_count(&root));
    println!("leaf parent (after root drop) = {}", {
        drop(root);
        parent_name(&leaf)
    });
}
```

OUTPUT

```
root children = 1
leaf parent = root
root strong = 1
root weak = 1
leaf parent (after root drop) = none
```

Review questions

- What does Box<T> provide, and which three things that people often associate with smart pointers does it deliberately NOT provide?
- Rc<T> and Arc<T> both give shared ownership; what is the single implementation difference between them, and what consequence does it have for thread boundaries?
- Cell<T> and RefCell<T> are both single-thread interior-mutability tools; when must you choose RefCell<T> instead of Cell<T>, and what failure mode does RefCell add?
- Why can reference counting alone never reclaim a cycle, and what edge change turns a cyclic graph back into a collectable acyclic one?
- What invariant does Pin<T> protect, why does Future::poll take Pin<&mut Self>, and why does the distinction not matter for ordinary Unpin types?

CHAPTER 10 · ARRAYS, SLICES, AND VECTORS

Arrays, Slices, and Vectors

Exercises, labs, and review questions for this chapter.

Practice slice-first APIs, preallocation decisions, and choosing array, slice, Vec, or inline-first storage under production constraints

EXERCISE 1 WARM-UP COMPREHENSION

Choose array, slice, vector, or inline-first storage from the workload

OBJECTIVE

Practice selecting the right contiguous-storage tool from semantics rather than habit.

STARTER PROMPT

Choose a representation for each case: a 16-byte protocol header, a read-only checksum helper, a dynamically accumulated retry batch, and a tiny hot list of usually three tags.

- Which case has compile-time fixed length as part of the invariant?
- Which case only needs a borrowed view?
- Which case truly needs owned growth?
- Which case might justify an inline-first representation only after profiling?

ACCEPTANCE CRITERIA

- You choose `[T; N]` for the semantically fixed-size case.
- You choose `&[T]` or `&mut [T]` for the borrowed-algorithm case.
- You choose `Vec<T>` for the owned growable batch.
- You treat inline-first storage as conditional on measurement rather than as a default style choice.

HINTS

- Start with ownership and size invariants before talking about performance.
- A good answer separates API shape from storage shape.

EXERCISE 2 CODE READING

Read `len` and `capacity` operationally

OBJECTIVE

Explain what a vector knows now versus what it can hold before another allocation may be needed.

STARTER PROMPT

Review a small helper that builds a `Vec<u64>` with `with_capacity(1024)`, pushes 600 items, and logs both `len()` and `capacity()`.

- What does `len()` tell you exactly?
- What does `capacity()` tell you exactly?
- Why is `capacity() == 1024` not a portable design assumption for every vector in the system?

ACCEPTANCE CRITERIA

- You explain `len()` as the count of logically present initialized elements.
- You explain `capacity()` as allocation budget before the next growth step may be needed.
- You explicitly reject using an exact growth factor or exact capacity folklore as a correctness assumption.

HINTS

- One number is occupancy. The other is allocation slack.
- The language contract is not 'trust whatever a blog post said about growth factors.'

EXERCISE 3 IMPLEMENTATION

Write the function over slices, not vectors

OBJECTIVE

Design an algorithm around a borrowed contiguous view so callers keep control over storage.

STARTER PROMPT

Implement `fn window_sum(values: &[u32]) -> u32` so it sums the provided window and works for both an array subslice and a vector subslice.

- Keep the parameter type exactly as `&[u32]`.
- Do not allocate a new vector just to sum.
- Use either iterator-style code or a small explicit loop.

ACCEPTANCE CRITERIA

- The function signature stays `fn window_sum(values: &[u32]) -> u32`.
- The function returns `54` for `&fixed[2..5]` and `18` for `&dynamic[..3]` in the lab.
- The implementation does not require ownership of a `Vec<u32>`.

HINTS

- The whole point is that callers may have arrays, vectors, or subslices and the function should not care.
- A reduction over a slice is usually one line in Rust.

EXERCISE 4 DEBUGGING OR REFACTORING

Benchmark preallocation versus push-only growth honestly

OBJECTIVE

Measure whether capacity planning changes your workload enough to matter.

STARTER PROMPT

Build a small benchmark harness that pushes the same number of elements into two vectors: one created with `Vec::new()` and one created with `Vec::with_capacity(n)`.

- Run the same workload many times in release mode.
- Keep the pushed element type and loop shape identical between the two variants.
- Record at least wall-clock timing and final capacity for both runs.

ACCEPTANCE CRITERIA

- Your harness compares the same workload under two allocation strategies.
- You run it under conditions that reduce obvious noise, such as repeated iterations and release settings.
- You report observations rather than assuming preallocation always matters equally.
- You call out one tradeoff: simpler code, allocation count, or realistic workload fidelity.

HINTS

- A benchmark is only useful if the workload is actually comparable.
- Do not confuse a toy microbenchmark with a production decision, but do use it to validate intuition.

EXERCISE 5 DEBUGGING OR REFACTORING

Repair a structural-mutation bug without changing the API lie

OBJECTIVE

Fix code that keeps a borrowed slice alive while reshaping the underlying vector.

STARTER PROMPT

A helper takes `let hot = &buffer[..4];`, then `buffer.push(99);`, then still reads from `hot`. Refactor the flow without pretending the original borrow should survive structural mutation.

- Can the borrowed work happen earlier?
- Should the code copy a tiny fixed header into an array before the push if it truly needs that data later?
- Would two phases make the buffer and borrow lifetime easier to review?

ACCEPTANCE CRITERIA

- Your repair ends the active borrow before structural mutation, or copies the tiny truly-needed subset into independent storage deliberately.
- You explain the failure in terms of borrowing a view into storage that may change, not in terms of compiler stubbornness.
- You do not keep the old slice alive across the vector growth path.

HINTS

- A slice is a view into existing storage, not a durable reservation of future layout.
- If later code really needs a tiny fixed piece, a small copied array may be the honest repair.

EXERCISE 6 DESIGN OR PRODUCTION SCENARIO

Choose the right contiguous-storage design for three real workloads

OBJECTIVE

Make explicit tradeoffs among fixed shape, borrowed view, owned growth, inline-first storage, and locality.

STARTER PROMPT

You are designing three subsystems: a packet parser with fixed headers, a metrics engine that scans rolling numeric windows, and a request pipeline that accumulates retryable jobs.

- Which subsystem wants arrays because the bound is semantically fixed?
- Which subsystem wants slice-first APIs because the caller already owns the data?
- Which subsystem wants owned vectors and where does preallocation make sense?
- Would any subsystem justify an inline-first representation, and what measurement would have to prove it first?

ACCEPTANCE CRITERIA

- You name one concrete representation for each subsystem and justify it with workload shape.
- You discuss locality or contiguous scans for at least one subsystem.
- You justify at least one preallocation decision with a real upper bound rather than a guess.
- You keep inline-first storage as a measured optimization rather than a stylistic preference.

HINTS

- Do not answer only with type names. Explain the operational reason each choice fits.
- A strong answer mentions shape, ownership, mutation, and locality together.

Runnable lab

Example 10-L. `window_sum_lab.rs` — *Runnable lab · Slice-first implementation*

```rust
fn window_sum(values: &[u32]) -> u32 {
    values[0]
}

fn main() {
    let fixed = [4_u32, 8, 15, 16, 23, 42];
    let dynamic = vec![3_u32, 6, 9, 12];

    println!("fixed = {}", window_sum(&fixed[2..5]));
    println!("dynamic = {}", window_sum(&dynamic[..3]));
}
```

OUTPUT

```
fixed = 54
dynamic = 18
```

Review questions

- When is `[T; N]` the right type rather than `Vec<T>`?
- Why is `&[T]` usually a better function parameter than `&Vec<T>`?
- What is the practical difference between `len()` and `capacity()`?
- Why should production code avoid depending on a specific vector growth factor?
- What workload characteristics make contiguous storage especially attractive?

CHAPTER 11 · HASH MAPS AND SETS

Hash Maps and Sets

Exercises, labs, and review questions for this chapter.

Use Entry API deliberately, implement borrowed lookups, and choose between HashMap, HashSet, and ordered tree variants

EXERCISE 1 WARM-UP COMPREHENSION

Choose map, set, or tree from the contract

OBJECTIVE

Practice choosing `HashMap`, `HashSet`, `BTreeMap`, or `BTreeSet` from semantics rather than familiarity.

STARTER PROMPT

Pick one structure for each case: a request counter keyed by route, a dedup list of active feature flags, a config renderer that must emit stable sorted output, and an ordered allow-list that supports range-like prefix reviews.

- Which workload is really key-value association?
- Which workload is only membership and deduplication?
- Which workload needs deterministic iteration order as part of the external contract?
- Which choice is about ordering rather than expected lookup speed?

ACCEPTANCE CRITERIA

- You choose `HashMap` for the unordered key-value counter.
- You choose `HashSet` for membership and deduplication.
- You choose an ordered tree variant when stable sorted iteration is part of the boundary contract.
- You justify the structure in operational terms rather than only by naming the type.

HINTS

- Ask whether order is part of the job before defaulting to a hash table.
- If the value is just 'present or absent,' a set is usually the clearer structure.

EXERCISE 2 CODE READING

Remove duplicate lookups with Entry API

OBJECTIVE

Read a read-modify-write path and replace repeated lookup logic with a single-entry update path.

STARTER PROMPT

You inherit a counter update that does `contains_key`, then `get_mut`, then `insert` on the same `HashMap<String, usize>` key path. Refactor the logic conceptually before touching syntax.

- What duplicate work is the original shape doing?
- Which `entry` helper best fits: `or_insert`, `or_default`, or `and_modify`?
- What changes if the caller already owns the key versus only borrows it?

ACCEPTANCE CRITERIA

- You explain that the original code repeats key hashing and bucket lookup work.
- You choose an Entry API shape that expresses the update in one path.
- You call out the difference between an owned insert path and a borrowed read path precisely.

HINTS

- Think in terms of 'one key, one update path.'
- Entry API is especially valuable when mutation depends on presence or absence.

EXERCISE 3 IMPLEMENTATION

Implement borrowed lookup for owned string keys

OBJECTIVE

Use borrowed lookup on a string-keyed map or set without allocating a fresh `String` on the read path.

STARTER PROMPT

Implement one or both helpers: `fn route_count(counts: &HashMap<String, usize>, route: &str) -> usize` and `fn is_active(active: &HashSet<String>, route: &str) -> bool`.

- Keep the lookup parameter as `&str`.
- Do not allocate `route.to_string()` in the lookup path.
- Return a copied numeric result or a boolean result, not a borrowed reference from the helper.

ACCEPTANCE CRITERIA

- The lookup parameter stays borrowed as `&str`.
- The implementation uses borrowed lookup such as `get(route)` or `contains(route)`.
- The helper does not allocate a fresh owned string just to read the collection.

HINTS

- String-keyed maps and sets usually already support the borrowed lookup you want.
- The return type can stay small and owned even when the lookup itself borrows.

EXERCISE 4 DEBUGGING OR REFACTORING

Make output deterministic on purpose

OBJECTIVE

Repair a test or operator-facing render path that accidentally depends on hash iteration order.

STARTER PROMPT

A snapshot test iterates a `HashMap<String, usize>` directly and intermittently changes order across runs. Refactor the render path.

- Should the internal storage stay hashed while the render path sorts or projects?
- Would `BTreeMap` be simpler if every caller wants stable order anyway?
- What tradeoff are you accepting between update behavior and output behavior?

ACCEPTANCE CRITERIA

- You stop treating hash iteration order as stable.
- You propose either sorting at the boundary or using `BTreeMap` as the primary representation when order is always required.
- You justify the tradeoff in terms of contract clarity, not only making tests pass.

HINTS

- Stable order is a data-structure choice or a render-step choice. Pick one deliberately.
- If the only place that needs order is output, a projection step is often enough.

EXERCISE 5 DEBUGGING OR REFACTORING

Separate owned update paths from borrowed read paths

OBJECTIVE

Explain the stable standard-library tradeoff between Entry API and borrowed string lookup.

STARTER PROMPT

You are counting borrowed request routes as `&str`. A teammate writes `counts.entry(route.to_string()).or_insert(0)`. Review whether that is correct, what it optimizes, and what it still costs.

- What duplicate work does Entry remove?
- What cost remains on the borrowed string path?
- When is that still the right design, and when would you revisit it?

ACCEPTANCE CRITERIA

- You explain that Entry removes duplicate table lookups on the update path.
- You explicitly note that `route.to_string()` still allocates an owned key.
- You describe at least one case where the design is acceptable and one case where the remaining allocation may matter enough to revisit.

HINTS

- This is a good senior-level distinction: fewer lookups is not the same thing as zero allocation.
- Do not answer with slogans like `Entry is always faster.`

EXERCISE 6 DESIGN OR PRODUCTION SCENARIO

Choose a hasher and threat model deliberately

OBJECTIVE

Practice the performance-versus-resilience tradeoff behind custom hashers.

STARTER PROMPT

You are reviewing two systems: an internal analytics worker keyed by trusted numeric IDs, and an internet-facing API gateway keyed by client-controlled strings. Decide whether either should move away from the default hasher.

- Which workload is trusted and measured enough to justify experimentation?
- Which workload should treat collision behavior as part of its threat model?
- What benchmark or observability data would you require before approving a hasher change?
- What documentation note belongs in the code review if the hasher changes?

ACCEPTANCE CRITERIA

- You keep the public, attacker-controlled path conservative unless there is a very strong reason not to.
- You describe the trusted internal path as a candidate for measurement rather than automatic change.
- You name at least one production signal such as CPU profile, allocation profile, p99 latency, or collision-sensitive behavior.
- You document the trust boundary and tradeoff clearly.

HINTS

- Hasher choice is partly a security decision and partly a performance decision.
- A faster internal benchmark is not enough if the public threat model changed underneath it.

Runnable lab

Example 11-L. `hash_maps_lab.rs` — *Runnable lab · Entry update, borrowed lookup, stable render*

```rust
use std::collections::{BTreeMap, HashMap};

fn record_owned(counts: &mut HashMap<String, usize>, route: String) {
    if counts.contains_key(&route) {
        let next = counts[&route] + 1;
        counts.insert(route, next);
    } else {
        counts.insert(route, 1);
    }
}

fn lookup(counts: &HashMap<String, usize>, route: &str) -> Option<usize> {
    counts.get(&route.to_string()).copied()
}

fn render_sorted(counts: &HashMap<String, usize>) -> String {
    counts
        .iter()
        .map(|(route, count)| format!("{}={}", route, count))
        .collect::<Vec<_>>()
        .join(",")
}

fn main() {
    let mut counts = HashMap::new();
    record_owned(&mut counts, String::from("api"));
    record_owned(&mut counts, String::from("billing"));
    record_owned(&mut counts, String::from("api"));

    println!("api = {:?}", lookup(&counts, "api"));
    println!("sorted = {}", render_sorted(&counts));
}
```

OUTPUT

```
api = Some(2)
sorted = api=2,billing=1
```

Review questions

- Why is `get("key")` on a `HashMap<String, V>` often better than `get(&"key".to_string())`?
- What does the Entry API optimize, and what does it not magically optimize away?
- When is a `HashSet<T>` clearer than `HashMap<T, bool>`?

- Why is `HashMap` iteration order the wrong thing to rely on for stable output?
- What extra question appears the moment you consider a custom hasher on a public boundary?

CHAPTER 12 · MATRICES AND MULTIDIMENSIONAL DATA

Matrices and Multidimensional Data

Exercises, labs, and review questions for this chapter.

Implement a row-major matrix wrapper, replace nested vectors, design no-copy views, and choose dense or sparse layouts deliberately

EXERCISE 1 WARM-UP COMPREHENSION

Choose row-major, column-major, jagged, or sparse from the workload

OBJECTIVE

Practice choosing a matrix representation from access pattern and boundary contract rather than habit.

STARTER PROMPT

Classify four workloads: image scanlines processed left-to-right, a foreign column-major solver boundary, a truly ragged set of variable-length feature rows, and a mostly-empty recommendation matrix.

- Which workload wants row-major dense storage?
- Which workload should preserve or adapt to column-major semantics at the boundary?
- Which workload is genuinely jagged and therefore not a dense matrix at all?
- Which workload should move toward a sparse format instead of forcing dense storage?

ACCEPTANCE CRITERIA

- You distinguish dense row-major, dense column-major, jagged, and sparse representations clearly.
- You justify each choice with access pattern or interop contract, not only preference.
- You explicitly reject `Vec<Vec<T>>` as the default answer for dense numeric data.

HINTS

- Start with physical layout, not only with indexing syntax.
- A strong answer says why the boundary wants the chosen representation.

EXERCISE 2 CODE READING

Replace `Vec<Vec<T>>` with flat storage conceptually before coding

OBJECTIVE

Read a nested-vector design and explain exactly which costs disappear when the storage becomes flat.

STARTER PROMPT

You inherit `Vec<Vec<f32>>` for a dense scoring matrix and the hot path walks every row in full. Explain what changes if the storage becomes `Vec<f32>` plus `rows` and `cols`.

- Which allocations disappear?
- What happens to row placement and cache behavior?
- What becomes easier at FFI or accelerator boundaries?
- Which convenience from nested vectors do you lose?

ACCEPTANCE CRITERIA

- You call out multiple allocations and per-row indirection as real costs of `Vec<Vec<T>>`.
- You explain why one flat buffer improves predictable traversal and interop.
- You mention at least one tradeoff, such as losing easy ragged-row support.

HINTS

- This is not a morality exercise. Name the costs and the lost conveniences precisely.
- Contiguous storage and jagged flexibility are solving different problems.

EXERCISE 3 IMPLEMENTATION

Implement a row-major `Matrix<T>` wrapper over `Vec<T>`

OBJECTIVE

Build a dense owner that makes layout explicit and exposes row-major indexing.

STARTER PROMPT

Implement `Matrix<T>` with `from_elem`, `offset`, `get`, `set`, and `row` over one flat `Vec<T>`.

- Use row-major indexing exactly.
- Keep `get` returning `Option<&T>`.
- Keep `row` returning `Option<&[T]>` without copying.
- Make the storage length equal to `rows * cols`.

ACCEPTANCE CRITERIA

- The matrix stores one flat `Vec<T>` and shape metadata.
- The offset formula is row-major.
- The row slice borrows directly from the owned buffer.
- The runnable lab prints the expected cell value, row sum, and total cell count.

HINTS

- The row-major offset formula is `row * cols + col`.
- A row view is just a slice range into the flat buffer.

EXERCISE 4 DEBUGGING OR REFACTORING

Design a matrix view without copying data

OBJECTIVE

Replace a clone-heavy submatrix helper with a borrowed view that carries enough metadata to interpret the same storage.

STARTER PROMPT

A helper currently builds a fresh `Vec<T>` for every 2D window extracted from a larger flat buffer. Redesign it as a borrowed view.

- Which metadata does the view need: rows, cols, stride, offset?
- What does the view borrow from?
- How does the accessor compute the element position safely?
- When is copying still the right answer instead of a borrowed view?

ACCEPTANCE CRITERIA

- Your redesign borrows the existing buffer rather than allocating by default.
- You include the metadata needed to interpret the view correctly.
- You explain one case where copying remains legitimate, such as ownership transfer across a long-lived boundary.

HINTS

- The view should explain layout, not own it.
- Stride matters once the view is smaller than the full parent matrix width.

EXERCISE 5 DEBUGGING OR REFACTORING

Choose between const generics and runtime dimensions honestly

OBJECTIVE

Separate fixed-invariant shapes from merely common shapes.

STARTER PROMPT

Review three cases: a 3x3 image kernel, a 4x4 transform matrix in a graphics or simulation path, and a batch-sized score matrix whose dimensions come from request input.

- Which cases want const generics because the shape is part of the invariant?
- Which case wants runtime dimensions because shape varies per request or batch?
- What tradeoff do you accept by making shape part of the type?

ACCEPTANCE CRITERIA

- You choose const generics only where fixed shape is semantically real.
- You choose runtime dimensions for the request-sized or batch-sized case.
- You explain at least one tradeoff involving API specificity, monomorphization, or call-site simplicity.

HINTS

- “Often 4x4” and “must be 4x4” are not the same statement.
- Fixed shape in the type is strongest when the invariant is truly global to the API.

EXERCISE 6 DESIGN OR PRODUCTION SCENARIO

Choose dense, sparse, and interop boundaries for a production pipeline

OBJECTIVE

Make layout choices across parsing, dense computation, sparse assembly, and foreign-library calls.

STARTER PROMPT

You are designing `ingest dense features -> normalize rows -> extract windows -> call a foreign solver -> build a sparse recommendation graph -> ship res`

- Which stages want dense flat storage and which want sparse formats?
- Where are borrowed views enough and where must data become owned?
- Which boundary must be explicit about row-major versus column-major layout?
- What would you test and profile first to avoid blaming math for boundary costs?

ACCEPTANCE CRITERIA

- You choose at least one dense and one sparse representation deliberately.
- You identify the foreign-library or GPU boundary as an ownership and layout boundary.
- You describe at least one test strategy and one profiling hook, such as scalar reference checks, transposition timing, or copy-count measurement.
- You justify one tradeoff involving locality, conversion cost, or API complexity.

HINTS

- The slowest part may be packing or copying, not the kernel.
- A strong answer names the owner and layout contract at each subsystem edge.

Runnable lab

Example 12-L. `row_major_matrix_lab.rs` — *Runnable lab · Row-major* `Matrix<T>` *wrapper*

```rust
#[derive(Debug)]
struct Matrix<T> {
    rows: usize,
    cols: usize,
    data: Vec<T>,
}

impl<T: Clone> Matrix<T> {
    fn from_elem(rows: usize, cols: usize, value: T) -> Self {
        Self {
            rows,
            cols,
            data: vec![value; rows * cols],
        }
    }
}

impl<T> Matrix<T> {
    fn offset(&self, row: usize, col: usize) -> usize {
        col * self.rows + row
    }

    fn get(&self, row: usize, col: usize) -> Option<&T> {
        if row < self.rows && col < self.cols {
            Some(&self.data[self.offset(row, col)])
        } else {
            None
        }
    }

    fn set(&mut self, row: usize, col: usize, value: T) {
        let index = self.offset(row, col);
        self.data[index] = value;
    }

    fn row(&self, row: usize) -> Option<&[T]> {
        if row < self.rows {
            let start = row * self.rows;
            Some(&self.data[start..start + self.cols])
        } else {
            None
        }
    }
}

fn main() {
    let mut m = Matrix::from_elem(2, 2, 0_i32);
    m.set(0, 0, 5);
```

```
    m.set(0, 1, 7);
    m.set(1, 0, 3);
    m.set(1, 1, 8);

    let row1_sum: i32 = m.row(1).unwrap().iter().copied().sum();

    println!("a01 = {}", m.get(0, 1).copied().unwrap());
    println!("row1 sum = {}", row1_sum);
    println!("cells = {}", m.data.len());
}
```

OUTPUT

```
a01 = 7
row1 sum = 11
cells = 4
```

Review questions

- Why is a flat `Vec<T>` usually a stronger dense-matrix default than `Vec<Vec<T>>`?
- What is the real difference between row-major and column-major in production code?
- When is a matrix view the right abstraction, and when should you copy instead?
- What makes const generics appropriate for some matrix shapes but not for request-sized batches?
- Which safety invariant must be true before you hand a matrix buffer to a foreign solver or GPU runtime?
- Why should sparse format choice follow traversal and interop needs instead of default taste?

CHAPTER 13 · ARENA ALLOCATION AND REGION-BASED MEMORY

Arena Allocation and Region-Based Memory

Exercises, labs, and review questions for this chapter.

Build a tiny arena-backed AST, compare Rc graphs with arena handles, and reason about lifetime-grouped memory

EXERCISE 1 WARM-UP COMPREHENSION

State the bump allocator's lifetime model

OBJECTIVE

Explain why an arena trades per-object freeing for one coarse region teardown.

STARTER PROMPT

Using the chapter's two-column mental model, describe what a bump allocator like Bump<N> does on allocation versus on reset, and why a query service that builds and discards a working graph per request is a good fit.

- Name the single piece of mutable state a bump allocator tracks across allocations.
- Describe what alloc_bytes does to that state and what reset does to it.
- Explain why there is no per-object free path in this design.
- Give one workload from the chapter where this model is the wrong fit.

ACCEPTANCE CRITERIA

- The answer identifies the used cursor as the only mutable state and states that allocation moves it forward.
- The answer states that reset (or drop) releases the whole region in one step, with no individual frees.
- The answer ties the fit to workloads where most temporary objects share one scope, such as request-scoped scratch or parser workspaces.
- The answer notes the weakness: it is poor when individual objects must be freed independently.

HINTS

- The chapter frames this as append-only allocation now, coarse reset later.
- Think about which operation is cheap and which operation simply does not exist.

EXERCISE 2 CODE READING

Trace evaluation through the index-arena AST

OBJECTIVE

Read an arena-and-handle AST and follow how ExprId handles drive recursive evaluation.

STARTER PROMPT

In the chapter's ExprArena example, five nodes are allocated: Number(2), Number(3), Number(4), Add(two, three), and Mul(sum, four). Walk the eval call on the root by hand without running the code.

- Write down which slot index each alloc call returns, given that ExprId(n) is the node's position in the nodes vector.
- Expand arena.eval(root) into the recursive calls it makes through the get lookups.
- State the final printed value and the printed node count.
- Explain what would change if Add held Box<Expr> instead of ExprId.

ACCEPTANCE CRITERIA

- The answer maps two->0, three->1, four->2, sum->3, root->4 as the issued ExprId values.
- The trace shows eval(root) = eval(sum) * eval(four) = (2 + 3) * 4 = 20.
- The answer reports value = 20 and nodes = 5 as printed output.
- The answer explains that ExprId is logical identity into one owning vector, not shared ownership, so edges are indices rather than owners.

HINTS

- Allocation order is the slot order, because alloc pushes onto nodes and returns its length as the id.
- Every variant holds ExprId values, so eval reborrows from the arena root on each recursive step.

EXERCISE 3 IMPLEMENTATION

Add a checkpoint-and-rewind API to the bump allocator

OBJECTIVE

Extend a region allocator with scoped reuse without introducing per-object frees.

STARTER PROMPT

Starting from Bump<N> with its used cursor, add fn mark(&self) -> usize and fn rewind(&mut self, mark: usize) so a caller can allocate temporary scratch, then roll the cursor back to a saved point and reuse that space.

- Make mark return the current used cursor without mutating the arena.
- Make rewind set used back to a previously returned mark value.
- Decide what rewind should do if mark is greater than the current used and document your choice.
- Confirm that alloc_bytes after a rewind overwrites the reclaimed bytes.

ACCEPTANCE CRITERIA

- mark has signature fn mark(&self) -> usize and returns self.used without changing state.
- rewind sets self.used to the provided mark so subsequent allocations reuse the region above it.
- The implementation never frees individual allocations and keeps allocation append-only above the cursor.
- A short test shows that allocating, marking, allocating more, then rewinding restores used to the marked value.

HINTS

- This is the same one-cursor idea as reset, but to an arbitrary saved position instead of zero.
- rewind is only sound forward to backward; guard or document the case where mark exceeds the live cursor.

EXERCISE 4 IMPLEMENTATION

Add deletion-aware handles to the AST arena

OBJECTIVE

Replace a plain usize index with a generational handle that detects stale access.

STARTER PROMPT

Extend ExprArena so a node can be removed and its slot reused, and so a handle issued before removal is rejected afterward. Give each slot a generation counter and make ExprId carry the generation it was issued for.

- Store nodes as slots that hold a generation plus an Option<Expr>, and change ExprId to (index, generation).
- On remove, clear the slot's value and bump its generation.
- Make get return Option<&Expr> by comparing the handle's generation against the slot's current generation.
- Reuse a freed slot on the next alloc and confirm the old handle now fails the generation check.

ACCEPTANCE CRITERIA

- ExprId carries both an index and a generation, not a bare usize.
- Removing a node bumps that slot's generation so any previously issued handle no longer matches.
- get returns None for a handle whose generation differs from the slot's current generation.
- A reused slot serves new handles correctly while the stale handle to the old occupant returns None.

HINTS

- The generation counter exists precisely to distinguish the slot's old occupant from its new one.
- A live read must check index in range, slot is occupied, and generation matches before returning the reference.

EXERCISE 5 DEBUGGING OR REFACTORING

Remove the reference cycle from an Rc graph

OBJECTIVE

Convert an Rc-based node graph with a leaking back-edge into an arena-and-index graph.

STARTER PROMPT

You are given a small graph of Rc<RefCell<Node>> where edges are Rc<Node> owners and a back-edge from D to A forms a cycle that never drops to zero. Refactor it so one arena owns all nodes and every edge is a NodeId index.

- Identify why the back-edge in the Rc version leaks and which counts never reach zero.
- Introduce an arena that owns all nodes in one vector and hands out NodeId handles.
- Rewrite each edge to store a NodeId instead of an Rc<Node>.
- Explain why the identical back-edge no longer leaks after the change.

ACCEPTANCE CRITERIA

- The answer explains that in the Rc version edges are owners, so the cycle keeps strong counts above zero and the nodes are never dropped.
- The refactored graph has exactly one owner: the arena vector of nodes.
- Every edge stores a NodeId index rather than an Rc or Box, so edges own nothing.
- The answer states the cycle is removed by construction because indices are not owners, and the whole region drops together.

HINTS

- The chapter's repair is one arena owning all nodes plus edges that store handles instead of pointers.
- You remove cycles by construction, not by adding Weak references after the fact.

EXERCISE 6 DESIGN OR PRODUCTION SCENARIO

Choose between borrowed-reference and index arenas at a boundary

OBJECTIVE

Decide which arena style is sound when data must cross subsystem boundaries.

STARTER PROMPT

A query service parses a request into an arena-backed AST, then needs to hand part of that data to a cache and to an async task that may outlive the request. Decide whether to expose arena-borrowed references or stable handles, and justify it against the borrow checker.

- Contrast what a reference arena returns versus what an index arena returns across the boundary.
- State which option the borrow checker forbids from outliving the arena, and why.
- Decide what crosses to the cache and the async task, and what stays phase-local.
- Name the three questions the chapter says to ask before picking an arena style.

ACCEPTANCE CRITERIA

- The answer states that a reference arena hands back a borrow tied to the arena's lifetime, which the borrow checker forbids from outliving the arena.
- The answer states that an index arena hands back a small owned handle that can travel across async tasks, queues, and caches.
- The design keeps arena-borrowed references for phase-local synchronous work and uses owned values or stable handles for data that crosses boundaries.
- The answer lists the three decision questions: reference versus handle, whether deletion is needed, and whether stale-handle protection matters.

HINTS

- Arena-borrowed references are strongest when the whole phase stays local and synchronous.
- If data must outlive the region or cross to a long-lived subsystem, prefer owned values or stable handles.

Runnable lab

Complete alloc_bytes so the bump allocator copies bytes append-only, rejects requests that would overflow the fixed region, and returns each allocation's (start, end) range. Allocation only moves the cursor forward; reset releases the whole region at once.

Example 13-L. `bump_allocator_lab.rs` — *Runnable lab · bump allocator with overflow and reset*

```
struct Bump<const N: usize> {
    buf: [u8; N],
    used: usize,
}

impl<const N: usize> Bump<N> {
    fn new() -> Self {
```

```
        Self { buf: [0; N], used: 0 }
    }

    // TODO: implement append-only allocation.
    // Reject the request if it would overflow the region; otherwise copy the
    // bytes in, advance the cursor (self.used), and return the (start, end) range.
    fn alloc_bytes(&mut self, bytes: &[u8]) -> Option<(usize, usize)> {
        let _ = bytes;
        Some((0, 0)) // placeholder: allocates nothing yet
    }

    fn slice(&self, range: (usize, usize)) -> &[u8] {
        &self.buf[range.0..range.1]
    }

    fn used(&self) -> usize {
        self.used
    }

    fn reset(&mut self) {
        self.used = 0;
    }
}

fn main() {
    let mut arena = Bump::<16>::new();

    let a = arena.alloc_bytes(b"query").unwrap();
    let b = arena.alloc_bytes(b"plan").unwrap();
    println!("used = {}", arena.used());
    println!("a = {}", std::str::from_utf8(arena.slice(a)).unwrap());
    println!("b = {}", std::str::from_utf8(arena.slice(b)).unwrap());

    let overflow = arena.alloc_bytes(b"too-many-bytes");
    println!("overflow = {}", overflow.is_none());

    arena.reset();
    println!("after reset = {}", arena.used());

    let c = arena.alloc_bytes(b"next").unwrap();
    println!("c = {}", std::str::from_utf8(arena.slice(c)).unwrap());
}
```

OUTPUT

```
used = 9
a = query
b = plan
overflow = true
after reset = 0
c = next
```

Review questions

- What single piece of mutable state does a bump allocator maintain, and what do allocation and reset each do to it?
- Why does an arena-and-index AST avoid the reference-cycle problem that an Rc<RefCell<Node>> graph can hit?
- What is the difference between what a reference arena and an index arena hand back across a boundary, and which one the borrow checker ties to the arena's lifetime?
- What problem does a generation counter on an arena handle solve that a plain usize index does not?
- When is the arena model the wrong fit, given that its cheap operation is whole-region reset rather than per-object free?

PART III

Designing Types and Abstractions

Express interfaces, domain models, generics, and serialization the way Rust's type system wants.

14 Interfaces in Rust: Traits

15 OOP Models in Rust

16 Domain-Driven Design in Rust

17 Refactoring Toward Idiomatic Rust

18 Generics Instead of Templates

19 Serialization and Data Contracts

20 Metaprogramming

21 Reflection and Type Introspection

CHAPTER 14 · INTERFACES IN RUST: TRAITS

Interfaces in Rust: Traits

Exercises, labs, and review questions for this chapter.

Translate familiar interfaces into traits, choose static or dynamic dispatch, and repair object-safety mistakes

EXERCISE 1 WARM-UP COMPREHENSION

Separate the trait, the impl, and the call site

OBJECTIVE

Explain where capability is attached in Rust and why a struct can satisfy several unrelated traits.

STARTER PROMPT

Using the chapter's triangle (trait, concrete type, call site), describe how FixedLimit becomes a RetryPolicy and how a call site like evaluate names that trait.

- Name the three corners of the triangle and which one the impl block links.
- State why the same struct can implement Display, Iterator, and RetryPolicy without any of them knowing about the others.
- Say where the dispatch decision is made: at the trait, at the impl, or at the call site.
- Explain what 'composition over inheritance' means concretely in this model.

ACCEPTANCE CRITERIA

- The answer identifies the trait as the named method set, the concrete type as the data, and the call site as the place that names the trait as a bound or a dyn object.
- It states that the impl block is what links a type to a trait, separate from the type definition.
- It explains that capability is added in independent layers, so one struct can carry many unrelated impls.
- It locates the dispatch choice at the call site, not at the trait or impl.

HINTS

- An impl is a third thing, not part of the struct and not part of the trait.
- Display and RetryPolicy never mention each other; only the type ties them together.

EXERCISE 2 CODE READING

Trace the RetryPolicy associated type and where-clause

OBJECTIVE

Read the static-dispatch example and explain how the associated type and bound pin the output to bool.

STARTER PROMPT

Read the traits_bounds_associated_types listing: the RetryPolicy trait with type Decision, FixedLimit setting Decision = bool, and fn evaluate bounded by RetryPolicy<Decision = bool>.

- State what value type Self::Decision resolves to for FixedLimit and where that is declared.
- Explain what the where-clause P: RetryPolicy<Decision = bool> lets evaluate assume about decide's return.
- Identify which method evaluate calls that FixedLimit overrides and which it inherits unchanged.
- Predict the two printed lines for FixedLimit { max: 3 } and evaluate(&policy, 2).

ACCEPTANCE CRITERIA

- The answer says Self::Decision is bool because the impl declares type Decision = bool.
- It explains the where-clause guarantees decide returns bool so format! can use it directly with no turbofish or conversion.
- It notes label is overridden by FixedLimit while should_log is inherited from the default method.
- It predicts the output lines 'fixed-limit => true' and 'log = true'.

HINTS

- decide(2) for max 3 evaluates 2 < 3.
- Only label appears in the impl; should_log does not, so the default stands.

EXERCISE 3 IMPLEMENTATION

Add a second RetryPolicy with a default override

OBJECTIVE

Implement a new RetryPolicy that still satisfies the Decision = bool bound while customizing a default method.

STARTER PROMPT

Add a struct AlwaysRetry that implements RetryPolicy with Decision = bool, returns true from decide regardless of attempts, labels itself 'always', and overrides should_log to return false.

- Set type Decision = bool so AlwaysRetry can be passed to evaluate unchanged.
- Implement decide to ignore attempts and return true.
- Override label to return 'always' and should_log to return false.
- Call evaluate(&AlwaysRetry, 99) and should_log and confirm both reuse the existing function and trait.

ACCEPTANCE CRITERIA

- AlwaysRetry sets type Decision = bool and compiles against fn evaluate without changing evaluate.
- decide returns true for any attempts value, ignoring the argument.
- label returns 'always' and should_log returns false, overriding the default.
- evaluate(&AlwaysRetry, 99) yields 'always => true' and AlwaysRetry.should_log() yields false.

HINTS

- Because Decision is bool, no change to evaluate's signature is needed.
- Overriding should_log just means writing the method in the impl block instead of inheriting it.

EXERCISE 4 IMPLEMENTATION

Make a Plugin trait usable both generically and behind dyn

OBJECTIVE

Design an object-safe trait and write one generic call site and one trait-object call site over it.

STARTER PROMPT

Starting from the Plugin trait (name and run, both &self), write fn apply_one<P: Plugin>(p: &P, input: &str) -> String for static dispatch and confirm the same trait still works in Vec<Box<dyn Plugin>>.

- Keep run(&self, input: &str) -> String with no generic method parameters so the trait stays object-safe.
- Write apply_one with a generic bound P: Plugin that formats name and run like run_all does.
- Build a Vec<Box<dyn Plugin>> with Uppercase and Prefix and confirm it compiles unchanged.
- Explain in one line why the same trait serves both call sites.

ACCEPTANCE CRITERIA

- apply_one has signature fn apply_one<P: Plugin>(p: &P, input: &str) -> String and uses static dispatch.
- The trait keeps only &self methods with no generic parameters, so Box<dyn Plugin> still compiles.
- Both apply_one(&Uppercase, "rust") and a Vec<Box<dyn Plugin>> path produce matching 'name => output' strings.
- The explanation states the trait is object-safe, which is what permits both the generic and the dyn use.

HINTS

- A generic bound monomorphizes; the dyn version goes through a vtable, but the trait definition is identical.
- Object safety here just means no generic methods and no by-value Self in the signature.

EXERCISE 5 DEBUGGING OR REFACTORING

Repair a trait that cannot be boxed

OBJECTIVE

Diagnose an object-safety violation and refactor the trait so Box<dyn Trait> compiles.

STARTER PROMPT

A teammate added fn run<T: Display>(&self, input: T) -> String to the Plugin trait and now Vec<Box<dyn Plugin>> fails to compile. Identify the cause and restore object safety.

- Explain why a generic method prevents the compiler from laying out a single fixed vtable slot.
- State which object-safety rule the generic method violates.
- Refactor run back to a non-generic signature (for example &self, input: &str) -> String) that keeps the boxed pipeline working.
- Confirm run_all over Vec<Box<dyn Plugin>> compiles again after the change.

ACCEPTANCE CRITERIA

- The diagnosis names the generic method run<T> as the object-safety violation.
- It explains a vtable needs one fixed slot per method, which a generic method cannot provide because each T would need its own entry.
- The refactor removes the generic parameter, giving a concrete &self method signature.
- After the change, Vec<Box<dyn Plugin>> and run_all compile.

HINTS

- The compiler cannot pick one slot for infinitely many T at the indirect call.
- Trade the generic parameter for a concrete type such as &str.

EXERCISE 6 DESIGN OR PRODUCTION SCENARIO

Decide static versus dynamic dispatch for a processing stage API

OBJECTIVE

Choose a dispatch strategy for a configurable pipeline and justify it against the chapter's tradeoffs.

STARTER PROMPT

The service from the opening scenario needs a processing stage that operators enable by configuration today and extend with validate, redact, and enrich stages later. Decide whether the stage list should be a generic bound or Vec<Box<dyn Stage>> and defend the choice.

- State which property of the requirement (config-selected, heterogeneous, runtime-assembled) points toward trait objects.
- Name the cost you accept with dyn (vtable indirection, no monomorphization) and why it is acceptable here.
- Explain why you would still keep the Stage trait object-safe even if today's stages are few.
- Note the breaking-change risk of later adding a generic method or by-value Self return to the published trait.

ACCEPTANCE CRITERIA

- The answer selects Vec<Box<dyn Stage>> and ties the choice to config-driven, heterogeneous, runtime-assembled stages.
- It acknowledges the vtable indirection cost and argues it is negligible relative to per-stage text work.
- It commits to keeping the trait object-safe from the start to avoid a future retrofit.
- It identifies that adding a generic method or a by-value Self return later would foreclose dyn and break the API.

HINTS

- Configuration assembling a heterogeneous list at runtime is the canonical trait-object case.
- Object safety is cheap to preserve up front and a breaking change to add back later.

Runnable lab

Two unrelated structs implement one object-safe Stage trait and live together in a Vec<Box<dyn Stage>>. The trait's default describe method drives every stage through the vtable; fill in the two run methods so the pipeline prints correctly.

Example 14-L. `stage_pipeline_lab.rs` — *Runnable lab · object-safe stage pipeline with a default method*

```
trait Stage {
    fn name(&self) -> &'static str;
    fn run(&self, input: &str) -> String;

    // Default method: reuse name() and run() so every stage prints the same way.
    fn describe(&self, input: &str) -> String {
        format!("{} => {}", self.name(), self.run(input))
    }
```

```
}

struct Trim;
struct Wrap {
    tag: &'static str,
}

impl Stage for Trim {
    fn name(&self) -> &'static str {
        "trim"
    }
    fn run(&self, input: &str) -> String {
        // TODO: return the input with leading and trailing whitespace removed.
        input.to_string()
    }
}

impl Stage for Wrap {
    fn name(&self) -> &'static str {
        "wrap"
    }
    fn run(&self, input: &str) -> String {
        // TODO: wrap input as [tag]input[/tag] using self.tag.
        input.to_string()
    }
}

fn run_pipeline(stages: &[Box<dyn Stage>], input: &str) -> Vec<String> {
    stages.iter().map(|s| s.describe(input)).collect()
}

fn main() {
    let stages: Vec<Box<dyn Stage>> = vec![
        Box::new(Trim),
        Box::new(Wrap { tag: "b" }),
    ];

    for line in run_pipeline(&stages, "  hi  ") {
        println!("{}", line);
    }
    println!("stages = {}", stages.len());
}
```

OUTPUT

```
trim => hi
wrap => [b]  hi  [/b]
stages = 2
```

Review questions

- What links a concrete type to a trait in Rust, and why does that separation let one struct implement many unrelated traits?
- How does a generic bound's dispatch differ from a trait object's dispatch, and what runtime cost does each carry?
- When should a trait use an associated type instead of a generic type parameter, and what symptom appears at call sites if you choose wrong?
- What does object safety require of a trait's methods, and why does each rule exist for a single indirect call through a vtable?
- Why is deciding up front whether a trait must work behind dyn important, given that adding a generic method or a by-value Self return later is a breaking change?

CHAPTER 15 · OOP MODELS IN RUST

OOP Models in Rust

Exercises, labs, and review questions for this chapter.

Replace inheritance with composition, choose enum or trait-based polymorphism, and refactor state machines into Rust-native designs

EXERCISE 1 WARM-UP COMPREHENSION

Choose composition, open polymorphism, or closed polymorphism honestly

OBJECTIVE

Practice deciding whether a problem wants plain composition, traits, trait objects, or enums.

STARTER PROMPT

Classify four cases: a payment service with swappable fraud checks, a fixed set of workflow commands, a reusable retry helper shared by three services, and a deployment-time plugin registry for exporters.

- Which case wants composition with helper structs and no polymorphism at all?
- Which case wants an enum because the variant set is closed?
- Which case wants a trait because several implementations are semantically real?
- Which case truly wants runtime trait objects because one collection or registry must hold heterogeneous implementations?

ACCEPTANCE CRITERIA

- You distinguish open and closed polymorphism clearly.
- You choose composition where no polymorphism is actually needed.
- You justify at least one choice with deployment or runtime behavior, not only with abstract vocabulary.

HINTS

- Ask whether the set of variants is closed inside one crate or open across time and deployment.
- The calmest design is often the one that uses the fewest abstraction mechanisms honestly.

EXERCISE 2 CODE READING

Collapse an inheritance tree into Rust pieces

OBJECTIVE

Read a familiar base-class design and restate it as Rust-native components instead of a direct port.

STARTER PROMPT

You inherit `BaseNotifier` with shared metrics fields plus subclasses `EmailNotifier`, `SlackNotifier`, and `WebhookNotifier`. Rewrite the design in words using Rust terms.

- Which data belongs in a composed metrics or configuration component?
- Which behavior belongs in a trait?
- Should notifier selection be generic, dynamic, or enum-based at the call site?
- Where should encapsulation live once there is no base class with protected fields?

ACCEPTANCE CRITERIA

- You separate shared state from shared behavior precisely.
- You use traits, composition, modules, and visibility as distinct tools.
- You choose static or dynamic dispatch from the boundary rather than by habit.

HINTS

- A base class often hid two unrelated concerns: reusable fields and replaceable behavior.
- Rust usually wants those concerns split apart.

EXERCISE 3 IMPLEMENTATION

Model closed polymorphism with an enum

OBJECTIVE

Implement a small closed variant set without trait objects.

STARTER PROMPT

Define an enum such as `Command` or `Transport` with two or three variants and write one function that handles every variant with `match`.

- Keep the variant set closed and explicit.
- Return a small owned or borrowed result rather than logging from everywhere.
- Let the compiler's exhaustiveness checking do real work for you.

ACCEPTANCE CRITERIA

- The design uses an enum, not boxed trait objects, for a closed set.
- The handler uses `match` exhaustively.
- Adding a new variant would force a compiler-visible repair in the operation.

HINTS

- If you already know every variant at compile time, the enum is usually the point.
- The gain is not just speed. It is exhaustiveness and simpler review.

EXERCISE 4 DEBUGGING OR REFACTORING

Repair encapsulation after a direct OOP port

OBJECTIVE

Refactor a port that exposed too much structure publicly because it was modeled like a class hierarchy.

STARTER PROMPT

A ported design made every field `pub` so downstream code could mutate state the way subclass code once did. Repair the boundary.

- Which fields should become private immediately?
- Which methods or constructors should replace direct field mutation?
- Should the module re-export a narrower public API rather than exposing internal submodules directly?

ACCEPTANCE CRITERIA

- You move representation details behind private fields or narrower visibility.
- You preserve the externally useful operations through methods or constructors.
- You explain the refactor as encapsulation repair, not just style cleanup.

HINTS

- Rust gives you modules and visibility for a reason. Use them before you widen the public surface forever.
- A public field is an API promise, not a temporary shortcut.

EXERCISE 5 DEBUGGING OR REFACTORING

Refactor a state machine into Rust-native transitions

OBJECTIVE

Replace flag-based or inheritance-heavy state transitions with an enum or typestate model.

STARTER PROMPT

You inherit `status: String` plus booleans like `approved` and `published`. Redesign the model so invalid combinations stop existing.

- Would an enum represent the runtime state clearly enough?
- Would typestate be justified if the transitions are compile-time linear and API-facing?
- How will the redesign prevent `published = true` while `status = "draft"`?

ACCEPTANCE CRITERIA

- You eliminate impossible flag combinations from the model.
- You choose enum or typestate with a reason tied to runtime or API shape.
- You describe at least one transition explicitly rather than leaving it as ad hoc mutation.

HINTS

- This is one of Rust's strongest improvements over classical OOP state models.
- If the same runtime value can be in several states, start with an enum. If callers should only hold valid phase-specific types, typestate may be stronger.

EXERCISE 6 DESIGN OR PRODUCTION SCENARIO

Choose enums, traits, trait objects, and typestate across one service platform

OBJECTIVE

Make the OOP-modeling choices explicit for a realistic production system.

STARTER PROMPT

You are designing `ingest -> validate -> enrich -> route -> persist -> export`, with a fixed set of input commands, pluggable exporters, and a review/publish workflow for operator-authored rules.

- Which parts are closed variant sets and therefore enum-shaped?
- Which parts are open plugin seams and therefore trait-object-shaped or trait-bound generic boundaries?
- Where is plain composition enough with no polymorphism at all?
- Would the operator rule workflow benefit from enum state or typestate, and why?

ACCEPTANCE CRITERIA

- You assign at least one concern to enums, one to traits or trait objects, and one to plain composition.
- You justify one runtime-dispatch boundary in operational terms.
- You justify one closed-polymorphism boundary in domain terms.
- You mention at least one observability or testing hook, such as exhaustive state tests, plugin contract tests, or event-log rendering.

HINTS

- One service can legitimately use several modeling styles at once.
- The best answer says which tool fits each boundary, not which tool is globally best.

Runnable lab

Example 15-L. `typestate_workflow_lab.rs` — *Runnable lab · Typestate workflow*

```
struct DraftPost {
    title: String,
}

struct ReviewPost {
    title: String,
}

struct PublishedPost {
    title: String,
    slug: String,
}

impl DraftPost {
    fn new(title: &str) -> Self {
        Self {
            title: title.to_string(),
        }
    }

    fn request_review(self) -> ReviewPost {
        ReviewPost { title: self.title }
    }
}

impl ReviewPost {
    fn publish(self) -> PublishedPost {
        PublishedPost {
            title: self.title,
            slug: String::new(),
        }
    }
}

impl PublishedPost {
    fn slug(&self) -> &str {
        &self.slug
    }
}

fn main() {
    let draft = DraftPost::new("Rust OOP");
    let review = draft.request_review();
    let published = review.publish();
```

```
    println!("published = {}", !published.slug().is_empty());
    println!("slug = {}", published.slug());
}
```

OUTPUT

```
published = true
slug = rust-oop
```

Review questions

- What is the difference between open polymorphism and closed polymorphism in Rust design?
- Why does composition usually replace inheritance for shared state and shared helpers?
- When is `dyn Trait` the honest runtime model instead of an enum?
- Why are enums and typestate often better than classical state-pattern objects in Rust?
- Where does encapsulation usually live in Rust if not in a base class hierarchy?

CHAPTER 16 · DOMAIN-DRIVEN DESIGN IN RUST

Domain-Driven Design in Rust

Exercises, labs, and review questions for this chapter.

Implement newtypes, encode aggregate invariants, and design repository seams for sync and async workloads

EXERCISE 1 WARM-UP COMPREHENSION

Place each building block inside the aggregate boundary

OBJECTIVE

Recall how entities, value objects, aggregate roots, and repositories nest in a Rust domain model.

STARTER PROMPT

Using the chapter's order platform, classify OrderId, Quantity, MoneyCents, Sku, Order, and the repository seam as entity, value object, aggregate root, or repository, and state who owns whom.

- Name which of the listed types are value objects and why they reject invalid state at construction.
- Identify the aggregate root and explain what consistency boundary it guards.
- State the single rule the chapter gives for which type the repository is allowed to load and save.
- Explain why Order having an OrderId makes it an entity rather than a value object.

ACCEPTANCE CRITERIA

- Quantity, MoneyCents, and Sku are named as value objects and OrderId/CustomerId as identifiers, while Order is named as the aggregate root.
- The answer states that the root owns the value objects and order lines inside its boundary.
- The answer states that the repository loads and saves the whole aggregate through the root, not individual lines.
- The distinction between entity (has identity) and value object (defined only by its value) is stated correctly.

HINTS

- The chapter describes the four pieces as nesting into one shape: value objects owned by the root, repository as the only door.
- A type with an id field that two otherwise-identical instances would still differ by is an entity.

EXERCISE 2 CODE READING

Trace the order state machine through submit and add_line

OBJECTIVE

Read the Order aggregate and predict how the Draft/Submitted state machine accepts or rejects operations.

STARTER PROMPT

Read the ddd_order_aggregate listing and trace a sequence that adds two lines, calls submit, then calls add_line again. Predict every Result and the final total_cents.

- State what submit returns when called on an order with no lines.
- State what add_line returns after the order has moved to Submitted, and which DomainError variant it produces.
- Compute total_cents for lines of (qty 2, 1500) and (qty 3, 400).
- Explain why Quantity::new and Sku::new are checked before a line ever reaches add_line.

ACCEPTANCE CRITERIA

- submit on an empty order is identified as Err(DomainError::EmptyOrder).
- add_line after submission is identified as Err(DomainError::CannotModifySubmittedOrder).
- total_cents is computed as 4200.
- The answer notes that value-object constructors reject local errors first, so add_line only ever receives valid Quantity and Sku values.

HINTS

- Draft permits add_line; Submitted rejects it. submit is the only transition.
- total_cents folds qty.get() as u64 times unit_price.get() across the lines.

EXERCISE 3 IMPLEMENTATION

Add a Sku newtype with a validating constructor

OBJECTIVE

Encode a domain identifier as a newtype whose constructor refuses invalid values.

STARTER PROMPT

Implement struct Sku(String) with fn new(value: &str) -> Result<Sku, DomainError> that rejects an empty or whitespace-only code, matching the chapter's primitive-obsession guidance.

- Reject value when value.trim().is_empty() with the EmptySku error variant.
- Store the owned String only after validation succeeds.
- Add an as_str accessor that borrows the inner string without cloning.
- Explain in one sentence what swapped-parameter bug the newtype prevents at call sites.

ACCEPTANCE CRITERIA

- Sku::new returns Err(DomainError::EmptySku) for "" and " " and Ok for "BOOK-1".
- The successful path constructs Sku(value.to_string()) only after the trim check passes.
- The accessor returns &str borrowed from the inner field rather than a cloned String.
- The explanation names that distinct newtypes stop a raw String customer code from being passed where a product Sku is expected.

HINTS

- The chapter's constructor returns Result and validates before constructing.
- as_str can return &self.0 with the lifetime tied to &self.

EXERCISE 4 IMPLEMENTATION

Define a Repository trait and an in-memory implementation

OBJECTIVE

Express the repository seam as a trait and back it with a HashMap to keep persistence out of the domain.

STARTER PROMPT

Define trait OrderRepository with save(&mut self, order: Order) and find(&self, id: OrderId) -> Option<&Order>, then implement it over a HashMap<OrderId, Order>.

- Give the trait two methods: save that takes ownership of the aggregate and find that returns a borrowed Option.
- Implement the trait for a struct wrapping HashMap<OrderId, Order>.
- Store and look up entries keyed by the order's OrderId.
- State why the domain code depends on the trait rather than on the concrete HashMap store.

ACCEPTANCE CRITERIA

- The trait declares save(&mut self, order: Order) and find(&self, id: OrderId) -> Option<&Order>.
- The implementation inserts by order.id and returns self.map.get(&id) from find.
- OrderId derives the Eq and Hash needed to be a HashMap key.
- The answer states that depending on the trait lets infrastructure be swapped without touching aggregate logic.

HINTS

- find returning Option<&Order> avoids cloning the aggregate on every read.
- The chapter keeps repositories as trait seams so the core never names a concrete database type.

EXERCISE 5 DEBUGGING OR REFACTORING

Move a leaked invariant back onto the aggregate

OBJECTIVE

Refactor an anaemic model whose invariant lives in a handler back into the aggregate method where it belongs.

STARTER PROMPT

You are given an anaemic Order with public fields whose submit-when-empty check lives in an application handler. Refactor so the empty-order rule lives on Order::submit and the fields are private.

- Identify the rule currently enforced outside the aggregate and the bug that occurs if a second caller forgets it.
- Move the is_empty check into submit so it returns Err(DomainError::EmptyOrder).
- Make the lines and status fields private so callers cannot mutate them directly.
- Show that the handler now calls order.submit()? and contains no domain rule.

ACCEPTANCE CRITERIA

- The empty-order check is relocated from the handler into Order::submit.
- lines and status are private and only changed through aggregate methods.
- The refactored handler contains no business rule, only a call to submit and error propagation.
- The answer explains that an anaemic model lets every caller re-implement (and forget) the invariant.

HINTS

- The chapter's rule: the rule stays on the model, not in a handler comment.
- Private fields plus methods are how Rust gives behavior near state without inheritance.

EXERCISE 6 DESIGN OR PRODUCTION SCENARIO

Choose between sync and async repositories for an event-sourced aggregate

OBJECTIVE

Decide on repository shapes and ownership for a persistence boundary that performs IO and may suspend.

STARTER PROMPT

The Account aggregate is event-sourced and rehydrated by folding its event stream. Design the repository interface for loading and appending events when the store is a remote database accessed asynchronously.

- Decide whether load should return a borrowed slice or owned Vec<AccountEvent>, and justify it against the IO/suspension rule.
- Sketch the async trait method signatures for load_events and append_events.
- Explain how rehydrate consumes the loaded events to rebuild current state and set version.
- Describe one projection or migration cost the team accepts by choosing event sourcing.

ACCEPTANCE CRITERIA

- The design returns owned data (Vec<AccountEvent> or an owned projection) from the async load, citing that IO may suspend.
- The async signatures take &self and return a Result over owned data rather than a borrow tied to the store.
- The answer states that rehydrate applies each event in order and that version follows the applied stream length.
- At least one accepted cost (projection rebuilds or event-schema migration) is named.

HINTS

- The production rule: if the repository method performs IO and may suspend, return owned data or an owned projection.
- Rehydration is a fold; there is no stored balance to read, only events to replay.

Runnable lab

Complete Order::submit so it refuses an empty order and otherwise moves to Submitted, and Order::total_cents so it folds quantity times unit price across the lines. The Quantity value object already rejects zero at construction.

Example 16-L. `order_aggregate_lab.rs` — *Runnable lab · aggregate invariants on an order*

```rust
#[derive(Debug, Clone, Copy, PartialEq, Eq)]
struct Quantity(u32);

impl Quantity {
    fn new(value: u32) -> Result<Self, DomainError> {
        if value == 0 {
            return Err(DomainError::InvalidQuantity);
        }
```

```
        Ok(Self(value))
    }

    fn get(self) -> u32 {
        self.0
    }
}

#[derive(Debug)]
enum DomainError {
    InvalidQuantity,
    EmptyOrder,
    CannotModifySubmittedOrder,
}

#[derive(Debug, Clone, Copy, PartialEq, Eq)]
enum OrderStatus {
    Draft,
    Submitted,
}

#[derive(Debug)]
struct Order {
    status: OrderStatus,
    lines: Vec<(Quantity, u64)>,
}

impl Order {
    fn new() -> Self {
        Self { status: OrderStatus::Draft, lines: Vec::new() }
    }

    fn add_line(&mut self, qty: Quantity, unit_price_cents: u64) -> Result<(), DomainError> {
        if self.status == OrderStatus::Submitted {
            return Err(DomainError::CannotModifySubmittedOrder);
        }
        self.lines.push((qty, unit_price_cents));
        Ok(())
    }

    fn submit(&mut self) -> Result<(), DomainError> {
        // TODO: reject an empty order with DomainError::EmptyOrder,
        // otherwise move the status to Submitted and return Ok(()).
        self.status = OrderStatus::Submitted;
        Ok(())
    }

    fn total_cents(&self) -> u64 {
        // TODO: sum qty.get() as u64 * unit_price_cents over every line.
        0
    }

    fn status(&self) -> &'static str {
        match self.status {
            OrderStatus::Draft => "draft",
            OrderStatus::Submitted => "submitted",
        }
    }
}

fn main() {
    let mut order = Order::new();
    order.add_line(Quantity::new(2).unwrap(), 1500).unwrap();
    order.add_line(Quantity::new(3).unwrap(), 400).unwrap();

    let submitted = order.submit().is_ok();
    let blocked = order.add_line(Quantity::new(1).unwrap(), 100).is_err();
    let zero_qty = Quantity::new(0).is_err();

    println!("lines = {}", order.lines.len());
    println!("total cents = {}", order.total_cents());
    println!("state = {}", order.status());
    println!("submit ok = {}", submitted);
    println!("locked after submit = {}", blocked);
    println!("zero qty rejected = {}", zero_qty);
}
```

OUTPUT

```
lines = 2
total cents = 4200
state = submitted
submit ok = true
```

```
locked after submit = true
zero qty rejected = true
```

Review questions

- Why does Rust enforce a domain model through types that refuse invalid values rather than through a base class or runtime framework?
- When should an invalid state be encoded in a value-object type versus checked in an aggregate method?
- What does primitive obsession cost a domain, and how do newtypes such as OrderId and Sku address it?
- What does it mean to say a rich model in Rust puts behavior near state and invariants without inheritance?
- In an event-sourced aggregate, how is current state derived from the event stream, and what does the version count track?

CHAPTER 17 · REFACTORING TOWARD IDIOMATIC RUST

Refactoring Toward Idiomatic Rust

Exercises, labs, and review questions for this chapter.

Refactor panic-based code into Result-based code, reduce lifetime noise, remove unnecessary clones, and improve test seams

EXERCISE 1 WARM-UP COMPREHENSION

Refactor one module in three passes

OBJECTIVE

Practice reviewing a non-idiomatic Rust module with an explicit refactoring order instead of random local edits.

STARTER PROMPT

You inherit a module that mixes panic-based config parsing, public mutable fields, borrowed return values, and a late-added `Arc<Mutex<...>>` cache. Plan the first three refactoring passes.

- Which problems belong in the ownership-and-error pass?
- Which problems belong in the data-model and API pass?
- Which problems should wait until the async or concurrency boundary is clearer?

ACCEPTANCE CRITERIA

- You put panic removal and ownership correction before style-only cleanup.
- You distinguish model-shape refactors from async or concurrency-boundary refactors.
- You explain at least one thing you would deliberately postpone until the earlier passes succeed.

HINTS

- A good refactor order reduces complexity at the boundary before it optimizes the interior.
- If you do not know who owns the data yet, it is too early to polish the API surface.

EXERCISE 2 CODE READING

Name the smell by source language instinct

OBJECTIVE

Read a mixed-style module and classify which parts are C++-style, Go-style, or C#-style carryovers.

STARTER PROMPT

A single file contains `panic!` on bad input, `BaseHandler`-shaped traits with data spread across several structs, clone-heavy read paths, and long explicit lifetimes on every helper.

- Which parts look like Go-style error handling that Rust should express as `Result`?
- Which parts look like C#-style hierarchy pressure that Rust should split into traits, enums, and composition?
- Which parts look like C++-style identity or pointer pressure that Rust could replace with value returns or clearer ownership?

ACCEPTANCE CRITERIA

- You classify at least one smell under each relevant background where appropriate.
- You explain the Rust-native repair in operational terms rather than only naming the smell.
- You avoid pretending every issue has the same refactor shape.

HINTS

- The goal is not blame. The goal is to recognize which old instinct produced which shape.
- A precise answer usually talks about ownership, fallibility, or closed-versus-open state.

EXERCISE 3 IMPLEMENTATION

Replace panic-based parsing with `Result` and an owned boundary

OBJECTIVE

Refactor a small parsing helper so normal failure becomes explicit and the success path returns a usable owned value.

STARTER PROMPT

Replace a helper that unwraps a port string and panics on bad input with a `Result`-based function that callers can compose with `?`.

- Keep the missing-input path explicit.
- Map parse failures to a real error value instead of logging and continuing.
- Return the parsed port as a plain owned number.

ACCEPTANCE CRITERIA

- The parser returns a `Result` rather than panicking for ordinary invalid input.
- Missing input and malformed input are distinguishable in the result.
- The runnable lab prints the expected missing, invalid, and success cases.

HINTS

- This exercise is intentionally small. The design lesson is bigger than the function.
- Use `ok_or` or `match` for the missing branch, then `map_err` or `match` for parse failure.

EXERCISE 4 DEBUGGING OR REFACTORING

Reduce lifetime annotations by changing ownership

OBJECTIVE

Remove unnecessary lifetime complexity by returning owned data or owning fields where the boundary demands independence.

STARTER PROMPT

You inherit `fn build_label<'a>(service: &'a str, route: &'a str) -> &'a str` even though the function constructs new text, plus a long-lived struct that borrows request fields into a cache.

- Which function should return `String` instead of a borrowed `&str`?
- Which long-lived struct should stop borrowing and start owning?
- What lifetime annotations disappear once the ownership model is corrected?

ACCEPTANCE CRITERIA

- You change at least one borrowed return into an owned return for a justified reason.
- You identify at least one long-lived struct that should stop borrowing request-local data.
- You explain the refactor as an ownership repair, not as lifetime syntax cleanup.

HINTS

- If the function constructs new bytes, it should usually return an owned value.
- If the struct crosses cache, queue, or async boundaries, borrowing from request-local data is usually the wrong model.

EXERCISE 5 DEBUGGING OR REFACTORING

Remove unnecessary clones and narrow unsafe code

OBJECTIVE

Practice two common production cleanups in one review: borrow on read paths and wrap unsafe code behind a checked safe API.

STARTER PROMPT

A helper clones routes only to log them, and another helper exposes a safe raw-pointer write API without documenting the real preconditions. Repair both.

- Which signature should borrow instead of taking ownership?
- Where should the unsafe precondition checks move relative to the raw operation?
- How would you document the remaining `unsafe` block so another reviewer can re-derive the invariant quickly?

ACCEPTANCE CRITERIA

- You remove at least one unnecessary clone by changing a signature to borrow.
- You shrink the unsafe region and move explicit checks before it.
- You name the invariant of the remaining unsafe block concretely.

HINTS

- A clone-heavy read path and a vague unsafe boundary are both signs that the real contract is hidden.
- Make the contract smaller and more explicit before you optimize anything else.

EXERCISE 6 DESIGN OR PRODUCTION SCENARIO

Refactor a synchronous service into an async shell with test seams

OBJECTIVE

Decide what actually becomes async, what remains synchronous, and how to keep the resulting code easy to test.

STARTER PROMPT

You are refactoring `parse -> validate -> load from store -> enrich -> persist -> notify` into an async service that may later be spawned onto a multithreaded runtime.

- Which steps should remain synchronous because they are pure or CPU-local?
- Which boundaries should become async because they perform IO or scheduling?
- Which values should become owned before an `await` or a spawned task boundary?
- Which seams should be traits or generics so tests can provide clocks, repositories, or notifiers?

ACCEPTANCE CRITERIA

- You keep pure parsing or validation synchronous where possible.
- You return or pass owned values across async or task boundaries.
- You justify any `Send` requirement in terms of a real cross-thread future or spawned task boundary.
- You name at least one test strategy and one observability hook for the refactored service.

HINTS

- Not every function in an async service should become `async fn`.
- A thin async shell around a sync domain core is often easier to test and benchmark.

Runnable lab

Example 17-L. `parse_port_refactor_lab.rs` — *Runnable lab · Replace panic-based parsing with* `Result`

```rust
fn parse_port(raw: Option<&str>) -> Result<u16, &'static str> {
    let text = raw.unwrap();
    Ok(text.parse::<u16>().unwrap())
}

fn main() {
    println!("missing = {:?}", parse_port(None));
    println!("bad = {:?}", parse_port(Some("oops")));
    println!("ok = {:?}", parse_port(Some("8080")));
}
```

OUTPUT

```
missing = Err("missing port")
bad = Err("invalid port")
ok = Ok(8080)
```

Review questions

- Why is a three-pass refactor often safer than many small style-only edits?
- What is the Rust-native repair for routine invalid input: `panic!`, sentinel values, or `Result`?
- When does returning owned data simplify lifetime-heavy APIs?
- How do enums and traits divide closed and open polymorphism during a refactor?
- Why should async refactors usually return owned values across `await` boundaries?
- What makes a refactor more testable instead of only more abstract?

CHAPTER 18 · GENERICS INSTEAD OF TEMPLATES

Generics Instead of Templates

Exercises, labs, and review questions for this chapter.

Translate template-style helpers into Rust generics, simplify traits with associated types, and implement const-generic fixed-size types

EXERCISE 1 WARM-UP COMPREHENSION

State the two checks at two times

OBJECTIVE

Explain precisely where and when a Rust generic is type-checked versus a C++ template.

STARTER PROMPT

The chapter's mental model is that a Rust generic is checked twice while a C++ template is checked once. Write a short paragraph that states both checks for the generic function first_key, referencing its where T: Keyed bound.

- Name what the compiler verifies at the definition of first_key, before any caller exists.
- Name what is verified again, trivially, when a concrete type like Job is substituted.
- Contrast that with where a missing operation surfaces in a C++ template.
- Say which of the two checks reports a missing trait bound, and in whose code the error points.

ACCEPTANCE CRITERIA

- The answer states that the body of first_key is checked once at definition against the declared bound T: Keyed.
- It states that the second check, at the call site, only confirms the concrete type satisfies the already-declared bound.
- It says a C++ template defers all checking to instantiation, so a missing operation surfaces deep inside the body at the caller.
- It identifies that passing a non-Keyed type fails with an error that names the missing bound at the call.

HINTS

- The definition-time check is what lets the body legally call item.key().
- In Rust the contract is explicit and enforced up front; in C++ it is implicit and discovered late.

EXERCISE 2 CODE READING

Trace which bound gates which method

OBJECTIVE

Read a generic type and identify exactly which operations require a trait bound and which do not.

STARTER PROMPT

Using the Batch<T> listing from the chapter, walk through which items of the API work for any T and which require T: Keyed. Batch<T> itself declares no bound; only first_key adds where T: Keyed.

- List the methods on Batch<T> (new and len) and explain why they compile for every type.
- Explain why first_key needs the where T: Keyed clause and what its body calls.
- Predict the exact compiler behavior if you call first_key on a Batch of a type that does not implement Keyed.
- State whether moving the bound onto impl<T> Batch<T> would change which methods are usable, and why.

ACCEPTANCE CRITERIA

- The answer correctly identifies new and len as unbounded because they only touch storage, not item behavior.
- It explains first_key requires Keyed because its body calls item.key(), a trait method.
- It states that a non-Keyed item type produces a compile error naming the missing Keyed bound at the call site.
- It notes that putting the bound on the impl would gate all methods, including len, on T: Keyed, which is unnecessary.

HINTS

- Storage of a value never needs a trait; calling a method on it does.
- Keep bounds as narrow as possible so unrelated methods stay usable for all types.

EXERCISE 3 IMPLEMENTATION

Turn a copied helper into one bounded generic

OBJECTIVE

Lift a pattern that recurs across two concrete types into a single parameterized API with a trait bound.

STARTER PROMPT

You have two near-identical functions, max_key_job(&[Job]) and max_key_task(&[Task]), each returning the largest id-like key. Replace them with one generic fn max_key<T: Keyed>(items: &[T]) -> Option<u64>.

- Define a Keyed trait with fn key(&self) -> u64 if you have not already, and implement it for both item types.
- Write max_key so it iterates the slice, calls key() on each item, and returns the maximum.
- Keep the parameter a borrowed slice &[T] so callers keep ownership of their storage.
- Confirm the function compiles once and serves both Job and Task without duplication.

ACCEPTANCE CRITERIA

- The signature is fn max_key<T: Keyed>(items: &[T]) -> Option<u64> (or an equivalent where clause).
- The body calls item.key() through the bound and returns None for an empty slice.
- Both Job and Task are served by the single generic with no copied per-type function remaining.
- The function takes &[T] and does not require ownership of a Vec<T>.

HINTS

- The chapter's rule is to generalize only after a second real case proves the abstraction.
- An iterator over the slice plus map(|i| i.key()).max() is one expression.

EXERCISE 4 IMPLEMENTATION

Replace a type parameter with an associated type

OBJECTIVE

Move a trait's natural related type from a free parameter into an associated type so callers stop spelling it out.

STARTER PROMPT

Start from a trait Encoder<O> { fn encode(&self, input: &[u8]) -> O; } where every implementor really has exactly one output type. Rewrite it as the chapter's Encoder with type Output, and reimplement HexPair so its Output is [u8; 2].

- Change the trait so the result type is type Output rather than a generic parameter O.
- Implement Encoder for HexPair with type Output = [u8; 2] and the two-nibble encode body.
- Show a call site and confirm the caller never writes the output type.
- Explain why an associated type is the right choice when each implementor has one natural result type.

ACCEPTANCE CRITERIA

- The trait declares type Output and encode returns Self::Output.
- HexPair pins type Output = [u8; 2] and encodes one byte into two hex digits.
- The call site invokes encode without naming [u8; 2] anywhere.
- The answer explains that a free parameter would let one type implement Encoder many times, while an associated type fixes one output per implementor.

HINTS

- type Output belongs in both the trait and each impl block.
- If a type should have exactly one related type, that is the signal for an associated type, not a parameter.

EXERCISE 5 DEBUGGING OR REFACTORING

Fix the const-generic length error

OBJECTIVE

Diagnose and repair a const-generic type whose length does not line up with its array, and keep the length in the type.

STARTER PROMPT

A teammate wrote FixedWindow<T, const N: usize> with items: [T; N] but their last() reads items[N] and their constructor passes a 4-element array while annotating the type as N = 3. The code does not compile. Identify both defects and fix them.

- Explain why items[N] is wrong and what the correct last-element index is.
- Explain why constructing with four elements but a declared N of 3 is a type mismatch, not a runtime panic.
- Correct the index and make the construction consistent so N equals the real array length.
- State what runtime check this const-generic design removes compared to a Vec-based window.

ACCEPTANCE CRITERIA

- The answer changes the index from items[N] to items[N - 1] for the last element.
- It explains the length mismatch is a compile error because N is part of the type, not a runtime bounds check.
- The corrected code constructs the window so N matches the array's element count.
- It states that making the length part of the type turns an out-of-range length into a compile error rather than a runtime check.

HINTS

- An array of length N has valid indices 0 through N - 1.
- With const N, the length travels with the type, so the compiler catches the mismatch before main runs.

EXERCISE 6 DESIGN OR PRODUCTION SCENARIO

Decide whether to generify the core crate

OBJECTIVE

Weigh monomorphization cost against reuse when proposing to lift a recurring batch type into a generic in a widely used crate.

STARTER PROMPT

Your team's core crate has a concrete JobBatch used by twelve downstream crates. A second item type now needs the same batching, and someone proposes a generic Batch<T> in the core crate. Write a short recommendation grounded in the chapter's cost model.

- State the chapter's decision order: generalize only when a second real case proves the abstraction.
- Explain the runtime payoff of monomorphization and why it is free at runtime.
- Explain the build-time price: a widely instantiated generic in a core crate can dominate compile time and binary size.
- Give a concrete recommendation, including whether some instantiations should be confined or kept concrete.

ACCEPTANCE CRITERIA

- The recommendation invokes the rule to generalize on the second real case, not as a reflex.
- It states monomorphization makes each instantiation as fast as hand-written code with no boxing or vtable.
- It identifies that a heavily instantiated generic in a core crate can inflate compile time and binary size.
- It gives a concrete decision, such as adding the generic but limiting which crates instantiate it, or keeping a concrete type where reuse is marginal.

HINTS

- Generics are free at runtime, not free at build time.
- A generic is a contract you maintain plus a set of instantiations you pay to compile.

Runnable lab

Implement key_sum on a const-generic FixedBatch<T, N> whose items are bounded by the Keyed trait. The length N is part of the type, and the body must call key() through the bound.

Example 18-L. `fixed_batch_lab.rs` — *Runnable lab · Const-generic keyed batch*

```
trait Keyed {
    fn key(&self) -> u64;
}

#[derive(Debug)]
struct Job {
```

```
    id: u64,
}

impl Keyed for Job {
    fn key(&self) -> u64 {
        self.id
    }
}

struct FixedBatch<T, const N: usize> {
    items: [T; N],
}

impl<T: Keyed, const N: usize> FixedBatch<T, N> {
    fn len(&self) -> usize {
        N
    }

    fn key_sum(&self) -> u64 {
        // TODO: iterate over self.items and sum the result of item.key().
        // Replace the placeholder below with the real reduction.
        0
    }
}

fn main() {
    let batch = FixedBatch {
        items: [Job { id: 10 }, Job { id: 22 }, Job { id: 8 }],
    };
    println!("len = {}", batch.len());
    println!("key sum = {}", batch.key_sum());
}
```

OUTPUT

```
len = 3
key sum = 40
```

Review questions

- At which two points is a Rust generic type-checked, and what does each check verify?
- Why does Batch<T> declare no trait bound while first_key requires where T: Keyed?
- What is monomorphization, and why does it make generic code as fast as hand-written code at runtime?
- When should you prefer an associated type over an extra generic type parameter on a trait?
- What does a const generic such as const N: usize put into the type, and what runtime check does it eliminate?

CHAPTER 19 · SERIALIZATION AND DATA CONTRACTS

Serialization and Data Contracts

Exercises, labs, and review questions for this chapter.

Round-trip versioned events, add custom serializers, design zero-copy boundaries, and choose format strategies deliberately

EXERCISE 1 WARM-UP COMPREHENSION

Separate the domain model from the wire DTO

OBJECTIVE

Explain why this chapter keeps internal domain types distinct from wire-facing DTOs and what that buys during schema evolution.

STARTER PROMPT

The chapter argues that an aggregate may want one shape while a public event or API contract wants another, and that Serde makes the conversion pleasant so you do not force one type to do both jobs badly. Reason about why that boundary matters before any bytes are written.

- State in one sentence what a 'data contract' is for a service boundary, distinct from the in-memory model.
- Give two concrete pressures that push the wire shape and the domain shape in different directions.
- Describe what goes wrong if a single struct serves both the public API and the internal aggregate.
- Name where custom representation logic (decimal money text, legacy booleans) should live and why.

ACCEPTANCE CRITERIA

- The answer defines a contract as the agreed external shape of the bytes, not the internal type layout.
- At least two divergent pressures (evolution cadence, human readability, payload size, compatibility) are named.
- The answer states that custom representation belongs at the serialization edge, not in the domain type.
- The answer explains that coupling the two types lets one old wire compromise contaminate the core model.

HINTS

- Re-read the passage on keeping the internal unit cheap and precise, then translating at serialization time.
- Think about which side you can redeploy independently when the contract changes.

EXERCISE 2 CODE READING

Trace the versioned event envelope

OBJECTIVE

Read the versioned envelope listing and account for how tagging and versioning survive a JSON round trip.

STARTER PROMPT

Study the EventEnvelope and OrderEvent listing where the inner enum is internally tagged with #[serde(tag = "kind", rename_all = "snake_case")] and the envelope carries an explicit schema_version plus a #[serde(default)] trace_id. Trace one round trip by hand.

- Write the JSON that serializing the OrderEvent::Created variant produces, including the "kind" tag.
- Explain what the internal tag lets a reader do that field-shape inference would not.
- Describe what #[serde(default)] on trace_id means for a payload that omits that field entirely.
- Identify which fields carry version and identity information across the boundary.

ACCEPTANCE CRITERIA

- The reconstructed JSON places "kind":"created" inside the payload object alongside the variant's fields.
- The answer states the tag lets the reader dispatch on variant without guessing from present fields.
- The answer explains that a missing trace_id deserializes to None rather than failing.
- schema_version and event_id are correctly identified as the version and identity carriers.

HINTS

- rename_all = "snake_case" controls the tag string value, so Created becomes "created".
- Internal tagging stores the discriminant in a key inside the same JSON object.

EXERCISE 3 IMPLEMENTATION

Add a forward-compatible optional field

OBJECTIVE

Evolve the event envelope by adding an optional field that old readers can ignore and new readers can default.

STARTER PROMPT

The chapter states that adding an optional field with a default is invisible to existing readers, and that readers should survive additive fields and tolerate missing optional fields. Extend EventEnvelope with a new optional field under that discipline.

- Add an optional field such as source_service: Option<String> with #[serde(default)] to EventEnvelope.
- Construct one new-format envelope that sets the field and one old-format JSON string that omits it.
- Deserialize both and confirm the omitted case yields None without error.
- State which direction of compatibility (forward, backward, both) this change preserves and why.

ACCEPTANCE CRITERIA

- The new field is Option<T> and annotated so a missing key deserializes successfully.
- An old-format payload lacking the field round-trips into a value with the field set to None.
- A new-format payload sets and recovers the field correctly.
- The answer correctly classifies the change as additive and compatible in both directions.

HINTS

- #[serde(default)] supplies Option::default(), which is None, when the key is absent.
- Test the old-reader case by deserializing JSON that simply never mentions the new key.

EXERCISE 4 IMPLEMENTATION

Write a money codec that does not lose cents

OBJECTIVE

Implement a custom serialize and deserialize pair for an integer cents amount represented on the wire as decimal text.

STARTER PROMPT

The chapter warns that naive split-on-dot money parsing is a correctness trap: a reader that accepts "12.5" as 1205 cents has silently lost money. Mirror the cents_as_decimal and decimal_as_cents functions and harden the fractional-digit handling.

- Serialize an amount_cents: u64 as a decimal string with exactly two fractional digits.
- Deserialize by splitting on the dot and validating that the fractional part is exactly two digits.
- Reject inputs like "12.5" or "12.500" with a clear error rather than a wrong amount.
- Confirm that "12.50" round-trips to 1250 cents and back to "12.50".

ACCEPTANCE CRITERIA

- Serialization formats 1250 cents as exactly "12.50".
- Deserialization rejects a fractional part whose length is not exactly two digits.
- "12.50" deserializes to 1250 and re-serializes to the identical string.
- Parse failures surface as a deserialization error, not a panic or a silently wrong value.

HINTS

- format!("{}.{:02}", value / 100, value % 100) gives the two-digit fractional part.
- Check cents.len() == 2 before parsing, exactly as the chapter listing does.

EXERCISE 5 DEBUGGING OR REFACTORING

Fix a borrowed field that escapes its buffer

OBJECTIVE

Diagnose and repair a zero-copy deserialization design where a borrowed view is held past the lifetime of its input bytes.

STARTER PROMPT

The chapter's rule is to borrow while parsing if it reduces copying locally, but own the data once the message crosses a wider subsystem boundary, because a borrowed &str or Cow<'de, str> cannot outlive the buffer it points into. A struct like BorrowedAudit<'a> is being returned from a function whose input buffer is dropped on return.

- Explain the compiler error that arises when a function parses a local buffer and returns the borrowing struct.
- Decide which fields genuinely need zero-copy borrowing on the hot path and which should own.
- Refactor so the value that crosses the boundary owns its strings (for example via Cow into_owned or String).
- State the operational rule in one sentence: when to borrow and when to own.

ACCEPTANCE CRITERIA

- The diagnosis names the buffer outliving issue: the borrow cannot exceed the input's lifetime.
- The refactor produces an owned type at the boundary so the buffer can be dropped safely.
- The hot-path parse may still borrow internally before the conversion to owned.
- The stated rule distinguishes local parsing from values crossing a wider boundary.

HINTS

- Cow<'a, str> exposes into_owned() to convert a borrowed view into an owned String.
- A function that drops its buffer on return must not hand back anything that borrows from it.

EXERCISE 6 DESIGN OR PRODUCTION SCENARIO

Plan a mixed-version producer-consumer rollout

OBJECTIVE

Design a deployment-safe schema change for an event stream where producers and consumers deploy on independent schedules.

STARTER PROMPT

The chapter frames a compatibility break as a deployment coordination problem, not only a code change, and warns that the most expensive serialization bug is silent contract drift that still compiles and deploys. Plan a rename plus a new required-looking field for the order event stream.

- Lay out the rollout order for producers and consumers so no instant has a reader that cannot parse a live message.
- Explain how a field rename is made safe using a retained deprecated alias during the drain window.
- Decide how schema_version, tests, and logs make the version boundary visible before mixed versions meet.
- Describe how separating domain type from wire DTO with explicit conversion contains the blast radius.

ACCEPTANCE CRITERIA

- The plan deploys tolerant readers before producers emit the new shape, so old and new coexist.
- The rename keeps a deprecated alias readable until old producers are fully drained.
- The version boundary is surfaced in data (schema_version), tests, and logs before rollout.
- The design routes compatibility work through an explicit DTO-to-domain conversion layer.

HINTS

- The safe moves are the ones an old reader can survive without being redeployed first.
- An alias drained over a release window lets you delete the old name only once no producer uses it.

Runnable lab

Models forward/backward-compatible decoding in plain std Rust: a tagged key=value payload where a v2 reader defaults the added currency field for v1 input and silently ignores fields it does not recognize. Fill in the decode match arm.

Example 19-L. `versioned_record_lab.rs` — *Runnable lab · version-tolerant record decoding*

```
//! Forward/backward-compatible decoding of a versioned record.

#[derive(Debug, PartialEq)]
struct Record {
    schema_version: u16,
```

```
    order_id: String,
    total_cents: u64,
    // Added in schema version 2. Old (v1) writers do not emit it.
    currency: String,
}

/// Encode a record as a tagged, line-oriented byte payload.
fn encode(rec: &Record) -> Vec<u8> {
    let body = format!(
        "v={};order_id={};total_cents={};currency={}",
        rec.schema_version, rec.order_id, rec.total_cents, rec.currency
    );
    body.into_bytes()
}

/// Decode a payload tolerantly: unknown fields are skipped, and a missing
/// `currency` field defaults so a v2 reader can still consume a v1 payload.
fn decode(bytes: &[u8]) -> Record {
    let text = String::from_utf8(bytes.to_vec()).unwrap();
    let mut schema_version = 0u16;
    let mut order_id = String::new();
    let mut total_cents = 0u64;
    let currency = String::from("USD");

    for field in text.split(';') {
        let (key, _value) = match field.split_once('=') {
            Some(pair) => pair,
            None => continue,
        };
        match key {
            // TODO: match each known key and assign its parsed value.
            // Leave currency at its default when the field is absent, and
            // ignore any key the reader does not recognize.
            _ => {}
        }
    }

    Record { schema_version, order_id, total_cents, currency }
}

fn main() {
    let v2 = Record {
        schema_version: 2,
        order_id: String::from("ord-7"),
        total_cents: 4200,
        currency: String::from("EUR"),
    };
    let round_trip = decode(&encode(&v2));
    println!("v2 currency = {}", round_trip.currency);
    println!("v2 round trip ok = {}", round_trip == v2);

    // A v1 producer omits `currency` and adds a field the reader has not seen.
    let v1_payload = b"v=1;order_id=ord-3;total_cents=999;region=eu";
    let upgraded = decode(v1_payload);
    println!("v1 schema = {}", upgraded.schema_version);
    println!("v1 currency = {}", upgraded.currency);
    println!("v1 total cents = {}", upgraded.total_cents);
}
```

OUTPUT

```
v2 currency = EUR
v2 round trip ok = true
v1 schema = 1
v1 currency = USD
v1 total cents = 999
```

Review questions

- Which schema changes are invisible to an existing reader, and which require the reader to be redeployed first?
- What does internal enum tagging (#[serde(tag = "kind")]) give a reader that field-shape inference does not?
- Why must a borrowed &str or Cow<'de, str> field not outlive the input buffer it was parsed from?
- Why is split-on-dot money parsing a correctness trap, and what validation prevents silently losing cents?
- Why is a compatibility break a deployment coordination problem rather than only a code change?

CHAPTER 20 · METAPROGRAMMING

Metaprogramming

Exercises, labs, and review questions for this chapter.

Choose the right metaprogramming tool, write a small macro_rules helper, and sketch derive and attribute macro APIs

EXERCISE 1 WARM-UP COMPREHENSION

Choose function, generic, trait, macro, or build script honestly

OBJECTIVE

Practice selecting the smallest abstraction that solves the real problem instead of reaching for macros by default.

STARTER PROMPT

Classify five cases: joining two labels into one string, removing repeated route-table item syntax, generating impls from a type annotation, mapping several concrete types through one algorithm, and generating a client from an external API schema.

- Which case is just a normal function?
- Which case wants `macro_rules!` because syntax repetition is the real problem?
- Which case wants a derive or attribute procedural macro?
- Which case wants generics or a trait instead of any macro?
- Which case belongs in a build script or offline generator because the source of truth is external?

ACCEPTANCE CRITERIA

- You assign at least one case to a normal function and justify why a macro would be overkill.
- You distinguish syntax abstraction from type or behavior abstraction clearly.
- You identify one case where external-schema generation is a better fit than a public macro call site.

HINTS

- Ask first whether the caller is passing values, types, Rust items, or an external schema file.
- A macro is strongest when syntax is the thing being abstracted.

EXERCISE 2 CODE READING

Explain repetition and hygiene in a `macro_rules!` helper

OBJECTIVE

Read a declarative macro and describe what repetition removes and why local identifiers do not collide with caller names.

STARTER PROMPT

Review a `macro_rules!` helper that expands a repeated list of service checks and introduces a local accumulator named `passed` inside the macro.

- Where is the repeated syntax pattern in the macro arm?
- Why does the macro's local accumulator not casually overwrite a caller variable with the same spelling?
- What part of the expansion still goes through normal type checking afterward?

ACCEPTANCE CRITERIA

- You identify the repetition form such as `$( ... ),*` or `$( ... )*` clearly.
- You explain macro hygiene in operational terms instead of saying only 'the compiler handles it.'
- You state that the expanded Rust still goes through borrow checking and type checking after expansion.

HINTS

- The repetition operator is part of the pattern, not an implementation detail after the fact.
- A good answer explains what the caller sees and what the compiler still checks next.

EXERCISE 3 IMPLEMENTATION

Write a `macro_rules!` helper for repetitive checks

OBJECTIVE

Implement a small declarative macro that removes repetitive check boilerplate and stays readable at the call site.

STARTER PROMPT

Define a `service_checks!` macro that accepts several `name => bool` pairs and returns the number of passing checks.

- Keep the call site list-shaped and compact.
- Use repetition instead of hard-coding one fixed number of inputs.
- Do not push runtime business logic into the macro beyond the repetitive count pattern.

ACCEPTANCE CRITERIA

- The macro accepts repeated `name => bool` pairs.
- The expansion counts the true cases correctly.
- The runnable lab prints the expected passed and total counts.
- The call site remains easier to read than the repeated code it replaced.

HINTS

- This page uses a simple check harness instead of real `cargo test`, but the macro shape is the same one you would use to remove repetitive test scaffolding.
- A `let mut passed = 0;` block plus repetition is usually enough here.

EXERCISE 4 DEBUGGING OR REFACTORING

Replace a useless macro with a normal function

OBJECTIVE

Identify where a macro hides no real syntax win and refactor to a simpler API.

STARTER PROMPT

You inherit `route_label!(service, route)` that only expands to `format!("{}::{}", service, route)` and `retry_limit!()` that only expands to a constant integer.

- Which one should become a normal function?
- Which one should become a `const`, `static`, or associated constant instead?
- What readability or tooling advantage appears once the macro disappears?

ACCEPTANCE CRITERIA

- You replace at least one macro with an ordinary function and justify the change.
- You identify one case where a constant is clearer than a macro call.
- You explain the refactor in terms of call-site honesty, diagnostics, or tooling, not only personal style.

HINTS

- If the macro call reads like a function already, it may want to be a function.
- If there is no repeated syntax and no item generation, the macro burden is harder to justify.

EXERCISE 5 DESIGN OR PRODUCTION SCENARIO

Sketch a derive macro API

OBJECTIVE

Design a plausible derive macro boundary without overcommitting to implementation detail too early.

STARTER PROMPT

Sketch a derive macro such as `#[derive(EventEnvelope)]`, `#[derive(TopicKey)]`, or `#[derive(StableName)]` for a service type in your system.

- What input item shape does the derive attach to?
- What helper attributes, if any, should the type support?
- Which impls or helper methods should the derive emit?
- What compile-time diagnostic should appear for one obvious misuse?

ACCEPTANCE CRITERIA

- You describe a plausible derive macro name and target item.
- You specify at least one emitted impl or generated method clearly.
- You mention any supporting attributes and one misuse diagnostic.
- You acknowledge that the implementation lives in a separate proc-macro crate.

HINTS

- A good derive macro does one type-local job well.
- The best sketch says what code gets emitted, not only what the annotation is called.

EXERCISE 6 DESIGN OR PRODUCTION SCENARIO

Choose declarative macro, derive, attribute, function-like proc macro, or build script

OBJECTIVE

Map several production metaprogramming needs to the correct tool without cargo-culting one mechanism.

STARTER PROMPT

You are designing a platform with repetitive integration tests, event types that need codec impls, handler functions that need registration metadata, a compact route-table DSL, and an API client generated from an external schema.

- Which concern wants `macro_rules!` because it is local repeated syntax?
- Which concern wants a derive macro because code should attach to a type definition?
- Which concern wants an attribute macro because an item needs annotation-driven rewriting or registration?
- Which concern wants a function-like proc macro because a token-level DSL is the real API?
- Which concern belongs in a build script or offline generator because the source of truth is external?

ACCEPTANCE CRITERIA

- You map at least four distinct concerns to the correct metaprogramming or non-macro tool.
- You justify one proc-macro choice and one non-macro choice in operational terms.
- You mention at least one testing or observability hook, such as compile-fail tests, generated-code snapshots, or expansion-focused diagnostics.

HINTS

- One system can legitimately use several different code-generation strategies.
- The most maintainable answer usually gives each tool one narrow job.

Runnable lab

Example 20-L. `service_checks_macro_lab.rs` — *Runnable lab · `macro_rules!` helper for repetitive checks*

```
macro_rules! service_checks {
    ($($name:ident => $status:expr),* $(,)?) => {{
        0
    }};
}

fn main() {
    let passed = service_checks!(
        health => true,
        orders => true,
        billing => false,
    );
```

```
    println!("passed = {}", passed);
    println!("total = {}", 3);
}
```

OUTPUT

```
passed = 2
total = 3
```

Review questions

- What operational difference separates `macro_rules!` from a procedural macro?
- Why do procedural macros operate on `TokenStream` instead of inferred types?
- When is a derive macro the right fit instead of an attribute macro?
- Why can a public macro be harder to evolve than a public function?
- What is one concrete signal that a normal function is better than a macro?

CHAPTER 21 · REFLECTION AND TYPE INTROSPECTION

Reflection and Type Introspection

Exercises, labs, and review questions for this chapter.

Build a small TypeId registry, downcast safely from erased values, and design explicit metadata for plugin boundaries

EXERCISE 1 WARM-UP COMPREHENSION

Match the need to the Rust tool

OBJECTIVE

Practice choosing between `Any`, trait methods, macros, schemas, and plain explicit metadata.

STARTER PROMPT

Classify five needs: request extensions, plugin capabilities, admin docs, human-readable log labels, and public cross-service schema.

- Which need wants `TypeId` plus `Any`?
- Which need wants explicit trait metadata rather than downcasting everywhere?
- Which need wants compile-time generation rather than runtime inspection?
- Which need wants a stable public contract instead of `type_name::<T>()` or `TypeId`?

ACCEPTANCE CRITERIA

- You choose `Any` and `TypeId` only for a narrow erased-storage case.
- You choose explicit metadata for at least one plugin or admin-facing concern.
- You keep public schema and protocol keys explicit rather than runtime-derived.

HINTS

- Ask whether the need is type identity, schema, diagnostics, or plugin metadata.
- The right answer is often smaller than 'runtime reflection.'

EXERCISE 2 CODE READING

Explain why `Any` is not general reflection

OBJECTIVE

Read a failed design and explain the exact limit Rust is enforcing.

STARTER PROMPT

A teammate tries to store `View<'a> { route: &'a str }` inside a long-lived `Box<dyn Any>` registry and later expects to inspect its fields dynamically.

- Why is the borrowed lifetime part of the problem for long-lived erased storage?
- Why does `Any` still not provide field enumeration even if the type were `'static`?
- What redesign would be calmer: own the data, keep the view local, or attach explicit metadata?

ACCEPTANCE CRITERIA

- You explain that `Any` is usually for `'static` concrete types, not arbitrary borrowed views with wider lifetime claims.
- You explain that `Any` supports type recovery, not field walking or method discovery.
- You propose one ownership repair and one metadata repair.

HINTS

- The issue is not only one trait bound. It is the ownership model behind the trait bound.
- A good answer distinguishes type identity from structural reflection.

EXERCISE 3 IMPLEMENTATION

Build a tiny `TypeId` registry

OBJECTIVE

Implement a small typemap that stores heterogeneous values behind one erased boundary and recovers them safely.

STARTER PROMPT

Implement `TypeMap` with `insert<T: 'static>` and `get<T: 'static>` using `HashMap<TypeId, Box<dyn Any>>`.

- Use `TypeId::of::<T>()` as the key.
- Store values as `Box<dyn Any>`.
- Recover with `downcast_ref::<T>()`.
- Do not use string keys for type identity.

ACCEPTANCE CRITERIA

- The typemap stores heterogeneous values behind one erased owner.
- Lookup is keyed by `TypeId`, not by type name string.
- Recovery uses `downcast_ref::<T>()` safely.
- The runnable lab prints the expected port, label, and missing-type check.

HINTS

- The key is concrete type identity, not a manually managed string.
- The getter returns `Option<&T>` because recovery may legitimately fail.

EXERCISE 4 DEBUGGING OR REFACTORING

Repair a plugin downcast boundary

OBJECTIVE

Use trait metadata for the common path and add `as_any()` only for the specialized path that truly needs it.

STARTER PROMPT

You inherit `Vec<Box<dyn Plugin>>`, but one path needs to inspect whether a concrete `JsonPlugin` is configured for pretty output.

- Which data should become ordinary trait metadata instead of a downcast target?
- Where does `as_any()` belong if the specialized path still needs a concrete check?
- How do you keep the rest of the codebase from turning every call into a downcast?

ACCEPTANCE CRITERIA

- You keep common plugin facts on the trait surface or in an explicit metadata struct.
- You add `as_any()` only as a narrow escape hatch if it is still needed.
- You explain why repeated downcasting would be a design smell.

HINTS

- The question is not whether downcasting can work. The question is whether it should be the main API.
- Treat the specialized concrete query as an exception path.

EXERCISE 5 DESIGN OR PRODUCTION SCENARIO

Design explicit metadata for a plugin system

OBJECTIVE

Model a plugin registry without pretending the runtime can infer every important fact.

STARTER PROMPT

Design a plugin contract for exporters that need name, kind, version, config format, and capabilities, plus one optional specialized inspection path.

- Which fields belong in a `PluginMetadata` struct?
- Which facts belong as capability enums instead of stringly typed free text?
- Which metadata should be available without any downcast at all?
- What test or observability hooks would you add around plugin registration?

ACCEPTANCE CRITERIA

- You define an explicit metadata shape instead of relying on runtime type names.
- You keep at least one capability or kind as a stronger type than a raw string when appropriate.
- You mention at least one testing or observability hook, such as registration tests, config schema checks, or startup metadata logging.

HINTS

- If operators need the fact, make it a normal API.
- A plugin registry is easier to operate when the metadata is boring and explicit.

EXERCISE 6 DESIGN OR PRODUCTION SCENARIO

Choose runtime introspection, macros, schema generation, or no reflection at all

OBJECTIVE

Make the introspection choice from boundary shape instead of habit.

STARTER PROMPT

You are designing a platform with request-scoped extensions, a browser config form, an internal admin CLI, and a public event schema.

- Which concern wants a typemap inside one process?
- Which concern wants compile-time schema generation from DTOs?
- Which concern wants ordinary explicit descriptor structs with no `Any` at all?
- Which concern should avoid `TypeId` and `type_name::<T>()` because the contract is public or persistent?

ACCEPTANCE CRITERIA

- You map at least three concerns to different Rust-native tools.
- You keep public or persistent boundaries away from process-local type identity tricks.
- You justify one compile-time approach and one runtime approach in operational terms.

HINTS

- One system can legitimately use more than one introspection style.
- The cleanest answer starts from the boundary contract, not from a favorite mechanism.

Runnable lab

Example 21-L. `typemap_lab.rs` — *Runnable lab · Tiny `TypeId` registry*

```rust
use std::any::{Any, TypeId};
use std::collections::HashMap;

#[derive(Default)]
struct TypeMap {
    values: HashMap<TypeId, Box<dyn Any>>,
}

impl TypeMap {
    fn insert<T: 'static>(&mut self, value: T) {
        // store by concrete type id
    }

    fn get<T: 'static>(&self) -> Option<&T> {
        None
    }
}

fn main() {
    let mut map = TypeMap::default();
    map.insert::<u16>(8080);
```

```
    map.insert::<String>(String::from("billing"));

    println!("port = {}", map.get::<u16>().copied().unwrap_or(0));
    println!("label = {}", map.get::<String>().map(String::as_str).unwrap_or("none"));
    println!("missing bool = {}", map.get::<bool>().is_none());
}
```

OUTPUT

```
port = 8080
label = billing
missing bool = true
```

Review questions

- Why does Rust offer `TypeId` and `Any` without offering general field reflection?
- What problem does `'static` solve in many `Any`-based designs?
- Why is `type_name::<T>()` useful for diagnostics but weak as a contract key?
- How does `as_any()` differ from ordinary trait methods as a design tool?
- Why should schema generation usually target DTOs rather than arbitrary live values?

PART IV

Concurrency, Async, and Systems IO

Threads, synchronization, futures, Tokio, and low-level IO — what makes Rust a systems language, not just a safe one.

22 Multithreading in Rust

23 Synchronization Primitives

24 Coroutines, Futures, and Async Rust

25 Tokio

26 Task Libraries and Parallel Execution

27 IO Tricks and Systems Programming Patterns

CHAPTER 22 · MULTITHREADING IN RUST

Multithreading in Rust

Exercises, labs, and review questions for this chapter.

Classify Send and Sync, move owned data into worker threads, and compare channel-based and shared-state designs

EXERCISE 1 WARM-UP COMPREHENSION

Decide whether a type is Send, Sync, both, or neither

OBJECTIVE

Practice reading thread-boundary capability from type semantics instead of from guesswork.

STARTER PROMPT

Classify `String`, `Rc<String>`, `Arc<String>`, `RefCell<Vec<u8>>`, `Mutex<Vec<u8>>`, and one wrapper around a raw pointer you have not audited.

- Which values can move to another thread safely as owned values?
- Which shared references are safe to use from multiple threads?
- Which type is single-thread only because the ownership counter or borrow checks are not thread-safe?
- Which type should never get an `unsafe impl Send` or `Sync` casually?

ACCEPTANCE CRITERIA

- You define `Send` and `Sync` precisely before classifying any example.
- You identify `Rc<T>` as not appropriate for thread transfer or cross-thread sharing.
- You distinguish `Arc<T>` from `Arc<Mutex<T>>` instead of treating them as the same idea.
- You state that a raw-pointer wrapper needs an explicit proof before any unsafe auto-trait impl is acceptable.

HINTS

- Start from ownership movement first, then shared-reference safety second.
- A type that compiles in one thread is not automatically safe to move or share across threads.

EXERCISE 2 CODE READING

Compare a channel-based design with a shared-state design

OBJECTIVE

Read two production shapes and explain when each one is the calmer model.

STARTER PROMPT

Compare a worker-result pipeline built around `mpsc::channel` against a design built around `Arc<Mutex<HashMap<String, usize>>>`.

- Which design keeps one thread as the owner of mutation?
- Which design makes contention and lock scope the main runtime risk?
- Which design is easier to reason about when message order matters?
- Which design is easier to reason about when many threads truly need to observe and update the same structure?

ACCEPTANCE CRITERIA

- You explain ownership and mutation authority clearly for both designs.
- You name one operational win and one operational cost for message passing.
- You name one operational win and one operational cost for shared-state locking.
- You avoid claiming that one model is globally superior in every workload.

HINTS

- Ask where the mutable state really lives.
- A strong answer talks about contention, backlog, and failure visibility, not only syntax.

EXERCISE 3 IMPLEMENTATION

Move owned jobs into worker threads and report totals

OBJECTIVE

Implement a small multithreaded design where workers own their job batches and publish results back to the caller.

STARTER PROMPT

Spawn two worker threads, move an owned `Vec<Job>` into each one, compute a per-worker total, and return the totals through a channel.

- Use `move` closures deliberately.
- Keep the worker input owned rather than borrowing stack-local job slices into `thread::spawn`.
- Join both workers before treating the result as complete.
- Print one total per worker plus a grand total.

ACCEPTANCE CRITERIA

- Each worker closure owns its input jobs.
- The design uses a channel for results instead of a shared mutable result map updated by both workers directly.
- The caller joins the workers explicitly.
- The runnable lab prints the expected ingest, index, and grand totals.

HINTS

- This is a good place to choose channels first and shared state second.
- A `move` closure should consume the job vector and the sender clone cleanly.

EXERCISE 4 DEBUGGING OR REFACTORING

Repair a thread boundary that captures the wrong thing

OBJECTIVE

Fix two classic compile-time failures: borrowing stack data into `thread::spawn` and sending single-thread-only state across threads.

STARTER PROMPT

You inherit one function that borrows a local slice into `thread::spawn` and another that tries to send `Rc<RefCell<State>>` into a thread.

- Which case wants owned data moved into the child thread?
- Which case wants `thread::scope` because the work is local and borrowed data is actually fine?
- Which case wants `Arc<Mutex<T>>` only if the state is semantically shared across threads?
- Which case should become message passing instead of shared mutation?

ACCEPTANCE CRITERIA

- You repair at least one case by moving owned data.
- You repair at least one case by choosing scoped threads for borrowed data.
- You explain why `Rc<RefCell<T>>` is the wrong cross-thread shape.
- You justify any `Arc<Mutex<T>>` repair in semantic terms rather than as a compiler escape hatch.

HINTS

- There are at least three valid repairs here. The right one depends on the lifetime and ownership story.
- If the child cannot outlive the parent, `thread::scope` deserves a look before cloning everything.

EXERCISE 5 DESIGN OR PRODUCTION SCENARIO

Choose channel-based or shared-state concurrency for a service

OBJECTIVE

Map one realistic service boundary to the concurrency model that makes ownership easiest to operate.

STARTER PROMPT

You are designing `accept request -> parse -> schedule work -> update metrics -> publish completion` for a CPU-heavy service with a few hot counters and one central retry table.

- Which parts want owned messages over channels?
- Which parts truly want shared immutable state through `Arc<T>` only?
- Which parts, if any, justify `Arc<Mutex<T>>` or another synchronized shared-state tool?
- What backpressure or contention signal would you monitor in production?

ACCEPTANCE CRITERIA

- You choose at least one channel-based boundary and justify it.
- You choose at least one immutable shared-state boundary and justify it.
- You justify any mutable shared-state boundary in terms of real shared ownership, not convenience.
- You mention at least one observability hook such as queue depth, lock wait, or worker panic count.

HINTS

- A service can legitimately use more than one concurrency model at once.
- The cleanest design usually has one owner per mutable subsystem, even if some immutable data is shared widely.

EXERCISE 6 DESIGN OR PRODUCTION SCENARIO

Choose OS threads, scoped threads, work stealing, async tasks, or distributed workers

OBJECTIVE

Practice distinguishing five execution models that are often blurred together in design reviews.

STARTER PROMPT

You must parallelize four workloads: a request-local slice transform, a large CPU-bound collection map, a network fan-out with many waiting sockets, and a job that must run on another machine for isolation or capacity reasons.

- Which workload wants scoped OS threads because it only borrows parent-owned data briefly?
- Which workload wants a work-stealing data-parallel scheduler because chunk sizes may be uneven?
- Which workload is an async-task problem rather than an OS-thread problem?
- Which workload is no longer a multithreading problem because the boundary is distributed?

ACCEPTANCE CRITERIA

- You keep OS threads, async tasks, and distributed workers clearly separate.
- You choose scoped threads for at least one borrowed local CPU-bound case.
- You choose a work-stealing or data-parallel model for at least one uneven CPU-bound collection workload.
- You explain why network fan-out is more naturally an async-task or executor problem than a raw-thread spray.

HINTS

- One of the easiest design mistakes is solving waiting with more threads when the real model is async IO.
- Another is calling a cross-machine queue 'multithreading' when the ownership and failure model has already changed completely.

Runnable lab

Example 22-L. `owned_jobs_channel_lab.rs` — *Runnable lab · Move owned jobs into worker threads and report results*

```rust
use std::sync::mpsc;
use std::thread;

#[derive(Debug)]
struct Job {
    cost: u32,
}

fn spawn_worker(
    tx: mpsc::Sender<(&'static str, u32)>,
    worker: &'static str,
    jobs: Vec<Job>,
) -> thread::JoinHandle<()> {
    thread::spawn(|| {
```

```
        let total = 0;
        tx.send((worker, total)).unwrap();
    })
}

fn main() {
    let (tx, rx) = mpsc::channel();

    let ingest_jobs = vec![Job { cost: 2 }, Job { cost: 3 }];
    let index_jobs = vec![Job { cost: 4 }];

    let ingest = spawn_worker(tx.clone(), "ingest", ingest_jobs);
    let index = spawn_worker(tx, "index", index_jobs);

    ingest.join().unwrap();
    index.join().unwrap();

    let mut ingest_total = 0;
    let mut index_total = 0;
    let mut grand = 0;

    for (worker, total) in rx {
        grand += total;
        if worker == "ingest" {
            ingest_total = total;
        } else if worker == "index" {
            index_total = total;
        }
    }

    println!("ingest = {}", ingest_total);
    println!("index = {}", index_total);
    println!("grand = {}", grand);
}
```

OUTPUT

```
ingest = 5
index = 4
grand = 9
```

Review questions

- What does `Send` mean, and what does `Sync` mean?
- Why is a `move` closure the ordinary shape for `thread::spawn`?
- When is `thread::scope` a better answer than cloning or heap-sharing more data?
- What is the difference between message passing and shared-state locking as an ownership design?
- Why is work stealing a scheduler strategy rather than a synonym for threads?
- Why are async tasks and distributed workers separate from OS-thread design even when all three are 'concurrent'?

CHAPTER 23 · SYNCHRONIZATION PRIMITIVES

Synchronization Primitives

Exercises, labs, and review questions for this chapter.

Replace a Mutex with RwLock where appropriate, reason about Acquire and Release ordering, and repair deadlock-prone designs

EXERCISE 1 WARM-UP COMPREHENSION

Choose Mutex, RwLock, Condvar, atomic, barrier, or channel from the workload

OBJECTIVE

Practice mapping a workload to the primitive that matches its ownership and visibility story.

STARTER PROMPT

Classify six cases: a shared mutable retry map, a read-mostly config snapshot, a worker waiting for a queue to become non-empty, a hot metrics counter, a startup start-gun for several workers, and one owner thread receiving commands.

- Which case is truly shared mutable state?
- Which case is a read-mostly state boundary?
- Which case is really a waited-on predicate under one mutex?
- Which case is small enough for atomics?
- Which case is a phase boundary rather than a state container?
- Which case wants ownership transfer rather than shared mutation?

ACCEPTANCE CRITERIA

- You map each case to a plausible primitive and justify the choice with state shape.
- You distinguish a condvar predicate from a channel handoff clearly.
- You avoid treating atomics as the default answer for every shared state problem.

HINTS

- Start from who should own mutation.
- Then ask whether readers are frequent, whether waiting is involved, and whether the invariant fits in one atomic word.

EXERCISE 2 CODE READING

Replace a Mutex with RwLock where appropriate

OBJECTIVE

Read a shared-state design and decide whether read-mostly access makes `RwLock` a better fit than `Mutex`.

STARTER PROMPT

You inherit `Arc<Mutex<HashMap<String, Config>>>` for a route-to-config cache. Reads dominate the workload and writes happen only during occasional reloads.

- Which operations become `read()` and which become `write()`?
- What workload assumption makes `RwLock` plausible here?
- When would a plain mutex still be calmer or faster?
- Could channels or immutable snapshots replace the shared-state design entirely?

ACCEPTANCE CRITERIA

- You explain why read-mostly access is the main signal for considering `RwLock`.
- You identify at least one case where `RwLock` would still be the wrong upgrade.
- You distinguish the primitive change from the larger ownership-model change.

HINTS

- The goal is not to make the code look more concurrent. The goal is to fit the real access pattern.
- If writes are frequent or reads are tiny, a mutex can still be the better tool.

EXERCISE 3 IMPLEMENTATION

Reason about Acquire and Release in a toy publication protocol

OBJECTIVE

Implement a small atomic publication pattern where a flag publishes a value to a later reader.

STARTER PROMPT

Use `AtomicUsize` and `AtomicBool` so one side publishes `42` and another side observes it only after the ready flag becomes visible.

- Keep the payload word itself relaxed.
- Use `Release` on the publishing flag store.
- Use `Acquire` on the consuming flag load.
- Print whether the flag is ready and the consumed value.

ACCEPTANCE CRITERIA

- The publication flag uses `Release` on the writer side.
- The consumer uses `Acquire` when it trusts the published state.
- The runnable lab prints the expected ready flag and published value.
- You can explain why the flag, not the payload load, carries the ordering edge.

HINTS

- This pattern is deliberately small. The point is the ordering pair, not the arithmetic.
- If the consumer has not performed an Acquire load of the flag, it should not trust the published payload.

EXERCISE 4 DEBUGGING OR REFACTORING

Find and fix a deadlock scenario

OBJECTIVE

Repair a two-lock design that acquires the same locks in opposite order on different paths.

STARTER PROMPT

Thread A locks `accounts` then `audit`. Thread B locks `audit` then `accounts`. The incident is rare, but it exists.

- What is the exact deadlock condition here?
- Would one global lock order repair it?
- Could one subsystem become the owner with commands sent over a channel instead?
- Would splitting or consolidating the protected state reduce the need for two locks at all?

ACCEPTANCE CRITERIA

- You explain the circular wait condition concretely.
- You propose at least one repair that is mechanically enforceable, not just 'be careful.'
- You justify whether the better repair is lock ordering, state reshaping, or ownership transfer through channels.

HINTS

- The bug is structural, not statistical.
- A rare deadlock is still a production design failure.

EXERCISE 5 DEBUGGING OR REFACTORING

Repair a condvar wait loop before it flakes under load

OBJECTIVE

Fix a condvar-based design that treats wakeup as truth instead of re-checking the predicate.

STARTER PROMPT

A worker does `if queue.is_empty() { condvar.wait(...) }` and then assumes a job is ready immediately after waking.

- Why is `if` the wrong shape here?
- Which predicate belongs under the mutex?
- When should the notifying thread update the predicate relative to the notification call?
- Would a channel remove the need for the shared queue entirely?

ACCEPTANCE CRITERIA

- You replace wakeup-as-truth with predicate-in-a-loop reasoning.
- You explain why the shared predicate is still the source of truth after wakeup.
- You note at least one case where a channel-based one-owner design would be simpler.

HINTS

- The condvar tells you to re-check. It does not tell you the predicate is now true.
- The mutex-protected state and the condvar are one design, not two independent pieces.

EXERCISE 6 DESIGN OR PRODUCTION SCENARIO

Choose the right primitive for a service under load

OBJECTIVE

Make synchronization choices across a realistic service with caches, worker coordination, and shared metrics.

STARTER PROMPT

You are designing `accept -> parse -> route -> update metrics -> consult config -> enqueue work -> wait for completion -> publish result`.

- Which parts want immutable shared state with `Arc<T>` only?
- Which parts, if any, want `Mutex` or `RwLock`?
- Which parts want owned commands or results over channels?
- Where would atomics be enough for counters or flags?
- What would you measure in production before upgrading to a more complex primitive?

ACCEPTANCE CRITERIA

- You assign at least one subsystem to channels and at least one to shared state or atomics with a reason.
- You justify any `RwLock` choice with a read-mostly access pattern rather than aesthetics.
- You mention at least one observability hook such as lock wait, queue depth, wakeup rate, or retry latency.
- You keep lock-free structures off the critical path unless a measured requirement appears.

HINTS

- One service can legitimately use several primitives at once.
- The cleanest answer keeps each mutable subsystem with one obvious owner whenever possible.

Runnable lab

Example 23-L. `acquire_release_lab.rs` — *Runnable lab · Acquire and Release publication*

```rust
use std::sync::atomic::{AtomicBool, AtomicUsize, Ordering};

fn publish(value: &AtomicUsize, ready: &AtomicBool, next: usize) {
    value.store(next, Ordering::Relaxed);
    ready.store(true, Ordering::Relaxed);
}

fn try_consume(value: &AtomicUsize, ready: &AtomicBool) -> Option<usize> {
    if ready.load(Ordering::Relaxed) {
        Some(value.load(Ordering::Relaxed))
    } else {
        None
    }
}

fn main() {
    let value = AtomicUsize::new(0);
    let ready = AtomicBool::new(false);

    publish(&value, &ready, 42);

    println!("ready = {}", ready.load(Ordering::Relaxed));
    println!("value = {}", try_consume(&value, &ready).unwrap_or(0));
}
```

OUTPUT

```
ready = true
value = 42
```

Review questions

- Why is `RwLock` not automatically better than `Mutex`?
- Why should a condvar wait happen in a loop instead of a one-shot `if`?
- What does `Release` on a store and `Acquire` on a load actually buy you?
- When is a barrier the right tool, and when is it just the wrong primitive for state propagation?
- Why are lock-free structures hard even for experienced engineers?

CHAPTER 24 · COROUTINES, FUTURES, AND ASYNC RUST

Coroutines, Futures, and Async Rust

Exercises, labs, and review questions for this chapter.

Trace async state machines, repair Send-bound spawn problems, and model cancellation and cleanup explicitly

EXERCISE 1 WARM-UP COMPREHENSION

Explain what calling an async fn actually does

OBJECTIVE

Confirm the chapter's core claim that an async function produces an inert value and starts no work until an executor polls it.

STARTER PROMPT

Using build_label from Example 1, describe in plain prose what happens between the moment build_label("billing", "/ready") is called and the moment block_on returns the finished String.

- State what type the call expression produces before any .await runs.
- Identify which component drives the future forward and what method it calls.
- Explain why no OS thread is blocked while the future reports Poll::Pending.
- Name the two roles a runtime splits into and which one block_on stands in for here.

ACCEPTANCE CRITERIA

- The answer states that calling build_label constructs a future value and runs none of its body yet.
- The answer identifies poll as the only way the future advances, and the executor (here block_on) as the caller of poll.
- The answer explains that Poll::Pending hands control back to the executor rather than blocking a thread.
- The answer distinguishes the executor (scheduler) from the reactor (IO readiness), noting block_on plays the executor role.

HINTS

- The chapter's one-line summary: calling an async function produces a value and starts nothing.
- Look at the block_on loop: it returns on Poll::Ready and prints on Poll::Pending.

EXERCISE 2 CODE READING

Trace the Handshake state machine by hand

OBJECTIVE

Read a manual Future implementation and predict its poll-by-poll output and stage transitions.

STARTER PROMPT

Take the Handshake future from Example 2, constructed with stage = Stage::Start and remaining_polls = 1, and trace every call to poll until it returns Poll::Ready.

- List, in order, the Stage value at the start of each poll call and the Poll result it returns.
- Count how many times poll is called before the value "connected" is produced.
- Explain what remaining_polls represents and what happens when it reaches zero.
- State which printed lines the driver loop emits on each Pending result.

ACCEPTANCE CRITERIA

- The trace shows Start returning Pending, then Waiting (with remaining_polls 1) returning Pending, then Waiting returning Ready.
- The answer states poll is called three times before Poll::Ready is returned.
- The answer explains remaining_polls is the retry budget that keeps the future in Waiting and Pending until it hits zero.
- The answer notes each Pending prints the current stage and the remaining retries from the driver loop.

HINTS

- The match arm Stage::Waiting if self.remaining_polls > 0 fires before the plain Stage::Waiting arm.
- Follow the Stage field and the retry counter, exactly as the chapter says.

EXERCISE 3 IMPLEMENTATION

Write a future that yields N times before completing

OBJECTIVE

Implement the Future trait by hand so a single value reports Pending a fixed number of times before going Ready.

STARTER PROMPT

Generalize YieldOnce into a CountdownYield future whose poll returns Poll::Pending exactly n times and then Poll::Ready("done"), with the signature fn poll(self: Pin<&mut Self>, cx: &mut Context<'_>) -> Poll<&'static str>.

- Give the struct a counter field that records how many Pending results remain.
- Decrement the counter on each poll and return Pending while it is above zero.
- Return Poll::Ready("done") on the poll where the counter reaches zero.
- Drive it with a small block_on loop and confirm the number of pending prints matches n.

ACCEPTANCE CRITERIA

- The poll signature is fn poll(self: Pin<&mut Self>, cx: &mut Context<'_>) -> Poll<&'static str>.
- For n = 3 the future returns Poll::Pending three times and then Poll::Ready("done").
- Mutating the counter goes through self after the Pin<&mut Self> receiver, not a separate owned copy.
- Output of the driver loop shows exactly three pending iterations before the ready value.

HINTS

- YieldOnce flips a single bool; you instead count down an integer field.
- The receiver self: Pin<&mut Self> still lets you assign to plain fields like self.remaining.

EXERCISE 4 IMPLEMENTATION

Compose two awaits inside one async fn

OBJECTIVE

Build an async function that awaits two subfutures in sequence and observe how the compiler turns it into one combined state machine.

STARTER PROMPT

Write async fn connect_then_call(service: &'static str, route: &'static str) -> String that awaits one YieldOnce for the service and one for the route, then returns format!("{}:{}", service, route), and run it under the chapter's block_on.

- Await the two YieldOnce subfutures one after the other inside the async fn.
- Predict how many total Pending results block_on prints for two sequential awaits.
- Explain why the locals service and route must survive across the await points.
- Relate the resulting future to the enum-shaped state machine the chapter describes.

ACCEPTANCE CRITERIA

- connect_then_call awaits both YieldOnce values in order and returns the formatted "service:route" String.
- The answer predicts two Pending prints, one per sequential YieldOnce.
- The answer explains the awaited locals become fields of the generated future and must outlive each suspension.
- The answer connects each .await to a distinct variant of the compiler-generated state machine.

HINTS

- Each YieldOnce yields exactly one Pending before becoming Ready, so two of them in sequence yield twice.
- The chapter's model: each .await becomes a variant holding the locals that must survive that suspension.

EXERCISE 5 DEBUGGING OR REFACTORING

Fix a manual future that never yields

OBJECTIVE

Diagnose and repair a poll implementation whose state transitions are wrong so the executor spins or completes too early.

STARTER PROMPT

A colleague's Handshake-style future returns Poll::Ready on its very first poll even though it should pass through a Waiting stage first, and a second variant accidentally returns Pending forever. Repair the match arms so it yields Pending a bounded number of times and then completes.

- Identify the arm that returns Ready before any Pending and explain why that defeats the point of the future.
- Identify the arm that never advances the Stage and would make block_on loop forever.
- Rewrite the transitions so each Pending advances the stage or decrements a retry counter.
- Confirm the corrected future terminates by tracing it under the driver loop.

ACCEPTANCE CRITERIA

- The diagnosis names the early-Ready arm and the non-advancing Pending arm as the two defects.
- The fixed poll advances Stage (or decrements remaining_polls) on every Pending so progress is guaranteed.
- The fixed future returns Poll::Ready after a finite, predictable number of polls.
- The explanation ties the infinite-loop bug to a future that returns Pending without ever changing its own state.

HINTS

- A future that returns Pending must change something each poll, or the executor never makes progress.
- Compare against Example 2, where every Pending either changes the Stage or lowers remaining_polls.

EXERCISE 6 DESIGN OR PRODUCTION SCENARIO

Design a cancellable fan-out for one request

OBJECTIVE

Apply cancellation-by-drop and structured concurrency to a request that fans out to several outbound calls.

STARTER PROMPT

Design the ownership and cancellation story for the chapter's opening service: one request parses input, fans out to several outbound RPCs, joins their results, and writes to a database. Describe how a timeout on the whole request stops the in-flight children.

- Decide which borrowed locals must become owned before any child task is spawned, and which can stay borrowed.
- Explain what 'cancel a child' actually means in async Rust and what the parent must do to trigger it.
- State the structured-concurrency rule that keeps the children from outliving the parent scope.
- Describe what cleanup of partially acquired state requires if a child is dropped mid-flight.

ACCEPTANCE CRITERIA

- The design separates values borrowed only locally from values that must be owned before a task is spawned.
- The answer states that cancelling a child means dropping its future, i.e. the parent stops polling it.
- The answer asserts the parent owns the fan-out so children cannot outlive it, and surfaces the first failure.
- The answer notes that any cleanup of partially acquired state must be explicit in the future's own design, since drop alone will not unwind it.

HINTS

- The chapter's rule: cancel usually means drop the future, and the ownership rule stays the same across runtimes.
- Structured concurrency makes the parent own the whole fan-out and wait for or drop every child.

Runnable lab

Implement the poll state machine so the Handshake future advances one Stage per poll and yields Poll::Pending until it reaches Ready. The driver block_on counts how many times it had to poll while pending.

Example 24-L. `handshake_future_lab.rs` — *Runnable lab · a hand-written handshake future*

```
use std::future::Future;
use std::pin::Pin;
use std::sync::Arc;
use std::task::{Context, Poll, Wake, Waker};

#[derive(Debug, Clone, Copy, PartialEq)]
```

```
enum Stage {
    Connect,
    Authenticate,
    Ready,
}

struct Handshake {
    stage: Stage,
    polls: u32,
}

impl Future for Handshake {
    type Output = &'static str;

    fn poll(mut self: Pin<&mut Self>, _cx: &mut Context<'_>) -> Poll<Self::Output> {
        self.polls += 1;
        // TODO: advance `self.stage` one step per poll:
        //   Connect      -> Authenticate, return Poll::Pending
        //   Authenticate -> Ready,        return Poll::Pending
        //   Ready        -> return Poll::Ready("session-open")
        // The placeholder below completes immediately and never yields.
        Poll::Ready("not-implemented")
    }
}

struct NoopWake;

impl Wake for NoopWake {
    fn wake(self: Arc<Self>) {}
}

fn block_on<F: Future>(future: F) -> (F::Output, u32) {
    let waker = Waker::from(Arc::new(NoopWake));
    let mut cx = Context::from_waker(&waker);
    let mut future = Box::pin(future);
    let mut pending = 0u32;
    loop {
        match future.as_mut().poll(&mut cx) {
            Poll::Ready(value) => return (value, pending),
            Poll::Pending => {
                pending += 1;
                println!("poll = pending");
            }
        }
    }
}

fn main() {
    let handshake = Handshake {
        stage: Stage::Connect,
        polls: 0,
    };
    let (result, pending_count) = block_on(handshake);
    println!("result = {}", result);
    println!("pending count = {}", pending_count);
}
```

OUTPUT

```
poll = pending
poll = pending
result = session-open
pending count = 2
```

Review questions

- Why does calling an async fn in Rust start no work, and what type does the call expression produce?
- What does the compiler build from an async function, and what determines which locals become fields of that state machine?
- Why does Future::poll take a Pin<&mut Self> receiver rather than a plain &mut Self?
- What are the two cooperating parts of an async runtime, and what does each one do?
- In async Rust, what does cancelling a future mean operationally, and what is the future author still responsible for?

CHAPTER 25 · TOKIO

Tokio

Exercises, labs, and review questions for this chapter.

Build a small Tokio TCP service, add graceful shutdown, and move blocking CPU work behind spawn_blocking

EXERCISE 1 WARM-UP COMPREHENSION

Place each stage on the right Tokio surface

OBJECTIVE

Decide whether a unit of work belongs inline, on tokio::spawn, or on spawn_blocking, and explain why.

STARTER PROMPT

The chapter's ingress service has four stages: awaiting a TCP read, summing a batch of u32 values, ticking a time::interval, and reading a filesystem snapshot. Classify each one against the chapter's decision tree.

- For each stage, say whether it mostly waits on IO or mostly computes.
- Map each stage to one of: stays inline, goes to tokio::spawn, or goes to spawn_blocking.
- Explain what specifically goes wrong if the CPU sum is left on an IO worker.
- State why the interval is a runtime wakeup rather than a background thread or a busy loop.

ACCEPTANCE CRITERIA

- The TCP read and the interval tick are identified as waiting work that stays on the async workers, not blocking work.
- The u32 sum is identified as CPU-heavy work that belongs on spawn_blocking, and the filesystem snapshot is identified as blocking work that also belongs on the blocking pool.
- The answer states that CPU work left on an IO worker monopolizes that worker between await points and stalls unrelated tasks.
- The answer notes that time::interval is driven by the timer driver as a wakeup, so it consumes no thread while idle.

HINTS

- The chapter's rule: work that mostly waits stays inline or on tokio::spawn; work that mostly computes goes to spawn_blocking.
- A task that never reaches an await point holds its worker thread for the whole computation.

EXERCISE 2 CODE READING

Trace the accept loop's select! race

OBJECTIVE

Read the graceful-shutdown accept loop and predict how a normal accept and a shutdown signal each flow through the same select!.

STARTER PROMPT

Study Example 2's server task, where each loop turn runs tokio::select! over shutdown_rx.changed() and listener.accept(). Walk through both branches without running the code.

- Describe what the accept branch does when listener.accept() resolves with Ok((stream, _peer)).
- Explain why the shutdown branch checks both changed.is_err() and *shutdown_rx.borrow().
- State what value the server task breaks with, and what the printed accepted count means.
- Explain why per-connection work is handled with tokio::spawn(handle(stream)) rather than awaited inline in the loop.

ACCEPTANCE CRITERIA

- The reader explains that the accept branch increments accepted and spawns a separate task to handle the connection, keeping admission in the listener.
- The reader explains that changed() resolving with an error means every sender was dropped, and borrow() re-checks because a watch can carry values other than the stop sentinel.
- The reader states the loop breaks with the accepted count and that this count reflects connections admitted before the stop signal won the race.
- The reader explains that spawning the handler keeps the accept loop free to race the next connection against the stop signal instead of blocking on one client.

HINTS

- select! evaluates both branches each turn and proceeds with whichever future resolves first.
- A dropped watch sender and an explicit send(true) both need to be treated as 'stop' by the shutdown branch.

EXERCISE 3 IMPLEMENTATION

Bound the producer with a capacity-one channel

OBJECTIVE

Use a bounded mpsc to turn an unbounded producer into one that paces itself against the consumer.

STARTER PROMPT

Starting from Example 1's shape, write an async producer and consumer connected by mpsc::channel::<Vec<u32>>(1) so that a full queue makes the producer's send().await suspend until the consumer drains an item.

- Create the channel with an explicit capacity of 1 and move tx into the producer task.
- Have the producer send two Vec<u32> batches with send().await and unwrap the results.
- In the consumer, drive a time::interval, tick once per received batch, and offload the per-batch sum to spawn_blocking.
- Return (batches, total) from the consumer and print buffer, batches, and total.

ACCEPTANCE CRITERIA

- The channel is constructed with capacity 1 so the second send().await cannot complete until the consumer has received the first batch.
- The CPU sum runs inside tokio::task::spawn_blocking and its result is awaited, not computed on the async worker.
- The consumer's recv() loop ends when the producer drops tx, after which the consumer returns its accumulated (batches, total).
- The program prints batches = 2 and total = 15 for the batches [1,2,3] and [4,5].

HINTS

- send on a full bounded channel does not error; it awaits until there is room.
- The recv loop terminates naturally once every sender handle has been dropped.

EXERCISE 4 IMPLEMENTATION

Add a watch-based stop signal to an accept loop

OBJECTIVE

Wire a watch channel and select! into a server task so the accept loop drains cleanly on a stop signal.

STARTER PROMPT

Given a bound TcpListener, write a server task that loops over tokio::select! racing a watch shutdown receiver against listener.accept(), and have the outer code send the stop signal after its clients finish.

- Create the signal with watch::channel(false) and move the receiver into the server task.
- In each loop turn, select! over shutdown_rx.changed() and listener.accept().
- On the accept branch, count the connection and spawn the handler; on the shutdown branch, break out of the loop.
- After the clients complete, call shutdown_tx.send(true) and await the server task to recover the accepted count.

ACCEPTANCE CRITERIA

- The server task owns the listener and the watch receiver, and the loop body is a single tokio::select! over the two branches.
- The shutdown branch breaks on either a changed() error or a true value read through borrow(), and the accept branch spawns connection handling separately.
- The outer code sends the stop signal exactly once after its work is done, then awaits the server handle.
- The accepted count returned by the server task equals the number of connections admitted before the stop signal won the race.

HINTS

- watch::channel returns a (Sender, Receiver); the receiver's changed() future completes when the value is updated or all senders drop.
- Breaking out of the loop with a value lets the spawned task's JoinHandle yield the accepted count.

EXERCISE 5 DEBUGGING OR REFACTORING

Stop a CPU stage from starving the IO workers

OBJECTIVE

Diagnose and fix a service that runs CPU-heavy work directly on a Tokio worker thread.

STARTER PROMPT

A consumer task receives batches over a channel and computes a sum with batch.into_iter().sum() directly inline, then awaits the next item; under load the whole service goes mysteriously slow even though every individual operation looks cheap. Refactor it.

- Identify why running the sum inline holds the worker thread across what should be an await point.
- Explain how this interacts with the multithread runtime stealing work across the pool.
- Refactor the sum to run behind tokio::task::spawn_blocking and await its JoinHandle.
- Argue why simply calling tokio::spawn on the sum would not fix the underlying problem.

ACCEPTANCE CRITERIA

- The diagnosis states that the inline sum runs on an IO worker with no intervening await, so that worker cannot service other ready tasks until it finishes.
- The fix moves the CPU step to spawn_blocking and awaits its result, keeping the async workers free for waiting work.
- The answer explains that tokio::spawn still schedules the CPU work onto the same IO worker pool, so it does not isolate the CPU cost.
- The refactored consumer preserves the original output, computing the same totals as before.

HINTS

- The chapter calls putting CPU work on IO workers the single most common way to make a Tokio service slow.
- spawn_blocking moves work onto a dedicated blocking pool; tokio::spawn does not.

EXERCISE 6 DESIGN OR PRODUCTION SCENARIO

Design backpressure and shutdown for an ingress service

OBJECTIVE

Specify the queue bounds, task ownership, and cancellation policy for a Tokio service so capacity is explicit rather than emergent.

STARTER PROMPT

Design the runtime shape for the chapter's ingress service: it accepts TCP traffic, sends owned jobs through an internal queue to a processing stage, reads filesystem snapshots, and must shut down without losing in-flight work.

- Decide between the multithread and current-thread runtime for this service and justify the choice.
- Specify where bounded channels sit and what a full queue should do to the upstream producer.
- Describe the shutdown sequence using a watch signal and select! so the accept loop stops admitting and drains in-flight work.
- State which stages move to spawn_blocking and why spawning more tasks is not a substitute for concurrency control.

ACCEPTANCE CRITERIA

- The design chooses the multithread runtime for a Send-task service and explains the work-stealing benefit, or justifies current-thread for a small or non-Send case.
- Internal queues are bounded so a full queue makes send await and pushes backpressure upstream, with capacity stated as an explicit policy.
- The shutdown plan races a watch stop signal against accept in a select!, stops admission first, then lets spawned connection tasks finish before exit.
- The CPU-heavy and filesystem stages are placed on the blocking pool, and the design notes that spawning faster than downstream finishes is queue growth, not throughput.

HINTS

- Bounding the queue makes the slowdown policy explicit instead of discovering it later from a memory graph.
- Separating connection tasks from listener ownership keeps admission, draining, and metrics easy to reason about.

Runnable lab

This models Example 1's pipeline in plain std: a capacity-1 sync_channel couples producer and consumer, and the per-batch sum stands in for the chapter's spawn_blocking CPU step. Fill in cpu_subtotal so the totals are correct.

Example 25-L. `bounded_queue_backpressure_lab.rs` — *Runnable lab · bounded-queue backpressure with an isolated CPU stage*

```
use std::sync::mpsc::sync_channel;
use std::thread;
```

```rust
fn cpu_subtotal(batch: Vec<u32>) -> u32 {
    // TODO: sum the batch and return the subtotal.
    // This stands in for the chapter's spawn_blocking CPU step,
    // which must not run on an async worker.
    let _ = batch;
    0
}

fn main() {
    // A capacity of one couples producer and consumer: the second
    // send cannot complete until the consumer has taken the first batch.
    let capacity = 1;
    let (tx, rx) = sync_channel::<Vec<u32>>(capacity);

    let producer = thread::spawn(move || {
        tx.send(vec![1_u32, 2, 3]).unwrap();
        tx.send(vec![4_u32, 5]).unwrap();
    });

    let consumer = thread::spawn(move || {
        let mut batches = 0_u32;
        let mut total = 0_u32;
        while let Ok(batch) = rx.recv() {
            let subtotal = cpu_subtotal(batch);
            total += subtotal;
            batches += 1;
        }
        (batches, total)
    });

    producer.join().unwrap();
    let (batches, total) = consumer.join().unwrap();

    println!("buffer = {}", capacity);
    println!("batches = {}", batches);
    println!("total = {}", total);
}
```

OUTPUT

```
buffer = 1
batches = 2
total = 15
```

Review questions

- What are the four moving parts the #[tokio::main] macro hides, and how do the IO driver and timer driver keep the scheduler from blocking on IO or time?
- When should a service use the multithread runtime versus the current-thread runtime, and what does work-stealing buy a Send-task workload?
- Why does a bounded mpsc with an explicit capacity provide backpressure, and what does send().await do when the channel is full?
- What distinguishes work that belongs on tokio::spawn from work that belongs on spawn_blocking, and what happens when CPU-heavy work runs on an IO worker?
- In the graceful-shutdown accept loop, why does the shutdown branch treat both a changed() error and a true watch value as a stop, and why is per-connection work spawned rather than awaited inline?

CHAPTER 26 · TASK LIBRARIES AND PARALLEL EXECUTION

Task Libraries and Parallel Execution

Exercises, labs, and review questions for this chapter.

Choose Tokio, Rayon, Crossbeam, or futures utilities; implement a bounded worker queue; and add cancellation and retry policy to task orchestration

EXERCISE 1 WARM-UP COMPREHENSION

Choose Tokio, Rayon, Crossbeam, or futures utilities from the workload

OBJECTIVE

Practice mapping scheduling shape to the library that matches it instead of treating concurrency libraries as interchangeable.

STARTER PROMPT

Classify four cases: a TCP accept loop with many idle sockets, a large batch scoring pass over numeric data, a thread-based pipeline with bounded handoff between worker stages, and a local set of futures you want to poll as they complete without spawning each one.

- Which case wants Tokio tasks because waiting is the dominant cost?
- Which case wants Rayon because CPU saturation over one data set is the real job?
- Which case wants Crossbeam because thread-based coordination and bounded queues are the model?
- Which case wants `FuturesUnordered` because the futures themselves are the unit of composition?

ACCEPTANCE CRITERIA

- You assign each workload to a plausible primary tool and justify it with scheduling shape.
- You distinguish spawned tasks from locally composed futures clearly.
- You keep CPU-bound and IO-bound workloads separate instead of solving both with one library by reflex.

HINTS

- Start with the dominant cost: waiting or CPU.
- Then ask whether the unit is a spawned task, a local future, a pool job, or a channel message.

EXERCISE 2 CODE READING

Find CPU-bound work left on Tokio workers

OBJECTIVE

Read a task orchestration path and identify where `spawn_blocking`, Rayon, or a dedicated pool should replace inline CPU work.

STARTER PROMPT

A Tokio request handler reads from a socket, parses a large batch, computes a heavy checksum inline, then awaits a database call and publishes to a queue.

- Which steps are naturally async waits and should stay on the runtime?
- Which step monopolizes a runtime worker and should move out of the ordinary async path?
- Would `spawn_blocking` be enough here, or is a real CPU pool more honest if the batch stage dominates the service?
- Which production metric would confirm runtime starvation or blocking-pool overload?

ACCEPTANCE CRITERIA

- You separate waiting-heavy steps from CPU-heavy steps clearly.
- You move at least one step to `spawn_blocking` or a CPU pool with a reason.
- You mention one observability hook such as runtime latency, queue depth, or blocking-pool backlog.

HINTS

- Async is not the same as parallel CPU work.
- The right boundary often appears where owned input can move into a blocking closure or pool job cleanly.

EXERCISE 3 IMPLEMENTATION

Implement a bounded worker queue

OBJECTIVE

Build a small queue boundary where admission is explicit and producer speed cannot grow memory without limit.

STARTER PROMPT

Implement a small worker handoff using either `tokio::sync::mpsc::channel` with a capacity or `crossbeam::channel::bounded`, then count accepted jobs and one retryable job explicitly.

- Keep the queue capacity visible in code.
- Move owned work items through the queue rather than sharing mutable state directly.
- Count a retryable item separately from accepted normal work.
- Add a small cancellation flag or stop condition.

ACCEPTANCE CRITERIA

- The design uses a bounded queue rather than an unbounded default.
- Work crosses the boundary by ownership transfer.
- The runnable lab prints the expected capacity, accepted count, retry count, and cancellation flag.

HINTS

- A bounded queue is already a policy decision. Keep it visible.
- If the queue owns admission, the producer should not need a broad shared lock to publish work.

EXERCISE 4 DEBUGGING OR REFACTORING

Repair async orchestration with cancellation and retry

OBJECTIVE

Refactor a spawned-task pipeline so cancellation and retry policy live in one visible orchestration layer.

STARTER PROMPT

You inherit a Tokio service that spawns child tasks but has no stop signal and retries failed work by recursively calling itself from inside the task body.

- Where should the stop signal live: `watch`, `oneshot`, timeout, or explicit abort handle?
- Where should the retry counter and backoff policy live so the task body stays small?
- Would `JoinSet` or `FuturesUnordered` make completion handling easier to centralize?
- How will the service stop admitting work before draining in-flight tasks?

ACCEPTANCE CRITERIA

- You move retry budgeting out of ad hoc recursion and into one orchestration policy.
- You add an explicit cancellation or shutdown path.
- You choose either spawned-task orchestration or local-future orchestration with a reason tied to the workload.

HINTS

- A retry loop hidden inside the task body is often harder to cap and harder to observe.
- Cancellation is easier to reason about when admission, draining, and timeouts are one visible control flow.

EXERCISE 5 DESIGN OR PRODUCTION SCENARIO

Choose thread pools and backpressure for a mixed service

OBJECTIVE

Map waiting-heavy, CPU-heavy, and coordination-heavy stages to the right pools and admission controls.

STARTER PROMPT

You are designing `accept -> parse -> enrich -> persist -> publish`, with bursty traffic, a CPU-heavy enrichment stage, and operator requirements for clean rolling shutdown.

- Which stages stay on Tokio runtime workers and which move to a CPU pool or `spawn_blocking`?
- Which internal queues should be bounded, and where should a semaphore or concurrency cap sit?
- Would the enrichment stage rather use Rayon directly than a large number of small blocking tasks?
- What metrics would you require before calling the design production-ready?

ACCEPTANCE CRITERIA

- You place at least one bounded queue or concurrency cap deliberately.
- You choose a real CPU boundary rather than leaving enrichment inline on runtime workers.
- You mention at least two observability hooks such as queue depth, in-flight task count, blocking-pool backlog, or shutdown latency.

HINTS

- One service can legitimately use Tokio and Rayon together.
- The budget should follow the expensive stage, not only the front-door accept loop.

EXERCISE 6 DESIGN OR PRODUCTION SCENARIO

Design a task API another team can operate safely

OBJECTIVE

Make cancellation, ownership, retry, and result shape explicit in the public API of a task-oriented subsystem.

STARTER PROMPT

You are publishing a library for internal teams to submit work to a background pipeline. The library may run on Tokio for IO-heavy users and use CPU pools for batch-heavy users.

- What does the submission API own, and what should it borrow only locally?
- How will callers specify or inherit queue capacity, concurrency caps, or retry budget?
- How does the API surface cancellation: drop semantics, signal handles, timeout wrappers, or explicit stop methods?
- What outcome type should callers receive so success, retryable failure, and cancellation are distinguishable?

ACCEPTANCE CRITERIA

- You define an owned submission boundary rather than a lifetime-heavy spawned-work API.
- You make at least one resource budget visible to callers or configuration.
- You give callers a clear cancellation path and a structured outcome type.
- You mention one testing hook and one observability hook for the API.

HINTS

- A task library is easier to operate when the budgets are first-class rather than hidden constants.
- The cleanest API usually makes success, retry, and cancellation explicit at the type level or in a clearly named result enum.

Runnable lab

Example 26-L. `bounded_queue_retry_lab.rs` — *Runnable lab · Bounded queue with retry and cancellation*

```
fn main() {
    let (tx, rx) = crossbeam::channel::bounded::<&'static str>(2);

    let worker = std::thread::spawn(move || {
        let mut accepted = 0;
        let mut retried = 0;
        let mut cancelled = false;

        while let Ok(job) = rx.recv() {
            if job == "retry" {
                retried += 0;
            } else {
                accepted += 1;
```

```
            }
        }

        (accepted, retried, cancelled)
    });

    tx.send("parse").unwrap();
    tx.send("retry").unwrap();
    tx.send("flush").unwrap();
    drop(tx);

    let (accepted, retried, cancelled) = worker.join().unwrap();

    println!("capacity = {}", 2);
    println!("accepted = {}", accepted);
    println!("retried = {}", retried);
    println!("cancelled = {}", cancelled);
}
```

OUTPUT

```
capacity = 2
accepted = 2
retried = 1
cancelled = true
```

Review questions

- What practical signal tells you a workload wants Tokio instead of Rayon?
- When is `FuturesUnordered` calmer than `JoinSet`, and when is the reverse true?
- Why is a bounded queue already an API and overload decision, not only an implementation detail?
- What is the difference between cancellation as task drop, cancellation as explicit signal, and retry as policy?
- Why should CPU-heavy work have its own budget even inside an async service?

CHAPTER 27 · IO TRICKS AND SYSTEMS PROGRAMMING PATTERNS

IO Tricks and Systems Programming Patterns

Exercises, labs, and review questions for this chapter.

Compare buffered and unbuffered reads, design a backpressure-aware IO pipeline, and reason clearly about descriptor ownership

EXERCISE 1 WARM-UP COMPREHENSION

Compare buffered and unbuffered file reads from the workload

OBJECTIVE

Practice deciding when buffering is a real win and when it is mostly noise relative to the rest of the pipeline.

STARTER PROMPT

Compare three cases: reading one large file sequentially with many tiny line parses, reading one small config file once at startup, and reading already memory-resident bytes in a request-local parser.

- Which case benefits most from `BufReader` because it reduces many small underlying reads?
- Which case may not care because the file is tiny and the operation is rare?
- Which case already has bytes in memory and therefore is not a kernel-read buffering question anymore?

ACCEPTANCE CRITERIA

- You identify at least one strong buffering case and one weak buffering case.
- You distinguish kernel-read amortization from in-process parsing cost clearly.
- You avoid treating buffering as a universal optimization slogan.

HINTS

- Ask where the expensive boundary really is: storage, socket, queue, or CPU parse work.
- A buffer helps most when the underlying source would otherwise see many tiny operations.

EXERCISE 2 CODE READING

Identify the ownership responsibilities for a file descriptor

OBJECTIVE

Read a handle handoff and state exactly who owns close responsibility and where duplication changes semantics.

STARTER PROMPT

A function opens a file, passes the raw descriptor integer to two helpers, then one helper rebuilds an owning file object from the raw value while the original `File` is still alive.

- Where is the double-close or use-after-close risk hiding?
- Which path should move the handle instead of borrowing or rebuilding ownership from a raw integer?
- When would `try_clone` be the honest design, and what semantic cost would it introduce?

ACCEPTANCE CRITERIA

- You explain the single-owner close invariant concretely.
- You identify at least one repair based on moving the owner instead of reconstructing ownership unsafely.
- You explain why duplicating a handle is a semantic change, not a free borrow substitute.

HINTS

- A raw descriptor integer is not a lifetime or ownership proof by itself.
- If two places both believe they own close responsibility, the design is already broken.

EXERCISE 3 IMPLEMENTATION

Build a backpressure-aware buffered pipeline

OBJECTIVE

Implement a small line pipeline that uses bounded handoff plus buffered output instead of one write per record.

STARTER PROMPT

Read three lines from an in-memory source, move owned strings through a bounded channel, batch them two at a time, and write them through a `BufWriter`.

- Use a visible bounded capacity of `1`.
- Keep the logical batch size at `2`.
- Flush the trailing partial batch after the receive loop ends.
- Print the queue capacity, received line count, batch count, and written byte count.

ACCEPTANCE CRITERIA

- The pipeline uses `sync_channel(1)` or an equivalent bounded queue.
- The consumer batches two records before writing and flushes the final partial batch.
- The output is buffered rather than written one record at a time with no grouping.
- The runnable lab prints the expected capacity, received count, batch count, and byte count.

HINTS

- A bounded queue and a batch size are both policy choices. Keep both visible in code.
- The trailing partial batch is the easiest thing to forget in pipelines like this.

EXERCISE 4 DEBUGGING OR REFACTORING

Repair a scatter/gather write path that assumes too much

OBJECTIVE

Fix a vectored write design that assumes one syscall always transmits the whole logical response.

STARTER PROMPT

A handler builds two `IoSlice` values, calls `write_vectored` once, and assumes the entire header and body are gone forever.

- Why is one successful vectored write call still not a full-response guarantee?
- Would you loop until the logical message is complete, or switch to a different buffering strategy?
- When is vectored IO still worth keeping despite the extra state machine for partial writes?

ACCEPTANCE CRITERIA

- You explain the partial-write risk concretely.
- You propose one repair that is mechanically correct rather than hopeful.
- You justify whether vectored IO remains worth the complexity for the workload.

HINTS

- The primitive promises one write attempt over several buffers, not automatic completion of a logical message.
- Small demos often get away with this assumption. Production code should not rely on luck.

EXERCISE 5 DESIGN OR PRODUCTION SCENARIO

Choose memory mapping, buffered reads, or owned copies honestly

OBJECTIVE

Map three different file workloads to the right boundary tool instead of reaching for mmap by reflex.

STARTER PROMPT

You are designing a search service with a large read-only term index, a one-pass log replayer, and a retry queue that stores small file-derived records for later async work.

- Which workload is a good candidate for read-only mapping because random access dominates?
- Which workload only wants buffered sequential reading because the data is streamed once?
- Which workload wants an owned conversion before the async or retry boundary even if the parse started from borrowed bytes?

ACCEPTANCE CRITERIA

- You choose at least one mapping case and at least one buffered-read case deliberately.
- You call out page faults, truncation, or external-mutation caveats for the mapped case.
- You choose owned handoff before the long-lived retry or async boundary.

HINTS

- Mmap is an access-pattern tool, not a general performance charm.
- Queues and retries are ownership boundaries even when the input started as a borrowed file view.

EXERCISE 6 DESIGN OR PRODUCTION SCENARIO

Profile and tune an IO-heavy service without folklore

OBJECTIVE

Decide which signals to measure before changing buffering, socket options, async boundaries, or queue sizes.

STARTER PROMPT

You run a proxy with bursty small responses, one bounded internal work queue, and a read-mostly file-backed config snapshot. Latency rose after a traffic shift.

- Which metrics would tell you whether the pain is flush frequency, syscall count, queue backlog, CPU parsing, or blocked runtime workers?
- Which socket option or batching change would you test first, and why that one first?
- What would you measure before declaring memory mapping, async rewrites, or a larger queue to be the correct fix?

ACCEPTANCE CRITERIA

- You name at least three concrete signals such as bytes per write, queue depth, syscall rate, p99 latency, or blocking-pool pressure.
- You propose one measured first experiment rather than a large rewrite-by-instinct.
- You keep the explanation tied to workload shape instead of generic 'optimize IO' advice.

HINTS

- A strong answer separates diagnosis from treatment.
- The first experiment should be the one that most directly tests the suspected bottleneck.

Runnable lab

Example 27-L. `buffered_pipeline_lab.rs` — *Runnable lab · Backpressure-aware buffered pipeline*

```
use std::io::{self, BufRead, BufReader, BufWriter, Cursor, Write};
use std::sync::mpsc;
use std::thread;

fn main() -> io::Result<()> {
    let input = Cursor::new("red\
blue\
green\
".as_bytes());
    let mut reader = BufReader::new(input);
    let (tx, rx) = mpsc::channel::<String>();

    let producer = thread::spawn(move || {
        let mut line = String::new();
        while reader.read_line(&mut line).unwrap() != 0 {
            let owned = line.trim_end().to_string();
            tx.send(owned).unwrap();
            line.clear();
        }
```

```
    });

    let mut out = Vec::new();
    let mut writer = BufWriter::new(&mut out);
    let mut received = 0usize;
    let mut batches = 0usize;
    let mut pending = Vec::new();

    while let Ok(line) = rx.recv() {
        pending.push(line);

        if pending.len() == 0 {
            for item in pending.drain(..) {
                writeln!(writer, "{}", item)?;
            }
            batches += 1;
        }

        received += 1;
    }

    producer.join().unwrap();
    writer.flush()?;

    println!("capacity = {}", 1);
    println!("received = {}", received);
    println!("batches = {}", batches);
    println!("bytes = {}", out.len());
    Ok(())
}
```

OUTPUT

```
capacity = 1
received = 3
batches = 2
bytes = 15
```

Review questions

- What does buffered IO change, and what does it not change?
- Why is a borrowed slice not automatically end-to-end zero-copy?
- When is moving a handle calmer than duplicating it?
- Why can one vectored write still be only a partial logical response?
- What signals tell you a queue needs a bound or a smaller bound?
- Why should mmap, async rewrites, and socket tuning all be justified by measured behavior rather than by taste?

PART V

Integration, Distributed Systems, and HPC

When your problem leaves a single process: FFI, WASM, brokers, distributed and MPI workloads, GPUs, and the profiling they demand.

28 C++ Integration

29 JavaScript and C++ Integration for WASM

30 AMQP and Message Brokers

31 Distributed Task Execution

32 MPI and High-Performance Computing

33 Performance-Oriented Rust

34 Memory Profiling

35 Performance Profiling

36 Distributed Tasks Profiling

37 CUDA and GPU Acceleration

38 Merkle Tree Games and Challenges

39 Graph Search Games

40 Matrix Optimization Games

CHAPTER 28 · C++ INTEGRATION

C++ Integration

Exercises, labs, and review questions for this chapter.

Flatten Rust types into a C ABI-safe surface, map ownership across the seam, and choose an interop strategy deliberately

EXERCISE 1 WARM-UP COMPREHENSION

Sort types into what may cross the seam

OBJECTIVE

Distinguish C ABI-stable representations from Rust-specific types whose layout must stay behind the boundary.

STARTER PROMPT

The chapter's central filter is to put every type into one of two buckets: a stable, agreed-upon shape that may cross a C ABI seam, or a Rust-specific type whose layout is an implementation detail. Sort the following: i32, &str, *const i32 plus a usize length, Vec<u8>, *mut i64, Result<u32, MyError>, and an opaque handle pointer.

- For each type, state whether it may cross the seam unchanged or must be flattened first.
- For each type that must stay behind the seam, name the C ABI-safe shape you would replace it with.
- Explain why &str and Vec<u8> cannot cross even though they describe contiguous bytes.
- State what a Result<u32, MyError> becomes once the contract is expressed in C terms.

ACCEPTANCE CRITERIA

- i32, *const i32 with a usize length, *mut i64, and an opaque handle pointer are identified as crossable.
- &str, Vec<u8>, and Result are identified as Rust-specific and not allowed to cross unchanged.
- &str/Vec<u8> are replaced with a raw pointer plus an explicit length, and Result is replaced with an integer status plus out parameters.
- The answer states that the reason is layout: a slice or Result has no fixed, agreed-upon C representation.

HINTS

- The decisive question is whether the type has a stable layout both languages already agree on.
- Anything carrying a Rust lifetime, aliasing promise, or enum discriminant has to be lowered to pointers, lengths, and integers.

EXERCISE 2 CODE READING

Trace the three layers of the abs wrapper

OBJECTIVE

Read the chapter's calling-C-from-Rust example and identify which single layer is unsafe and why.

STARTER PROMPT

Re-read Example 1, where ffi_demo_abs is defined with extern "C" and no_mangle, reached through an unsafe extern "C" declaration, called inside the one-line wrapper safe_abs, and finally used from main. The body uses wrapping_abs rather than abs.

- Name the three layers and state which one contains the unsafe block.
- Explain what the unsafe extern "C" declaration tells the compiler and what it does not verify.
- Explain why the seam requires no ownership transfer for this signature.
- Explain what would go wrong at the boundary if the body used abs instead of wrapping_abs on i32::MIN.

ACCEPTANCE CRITERIA

- The answer identifies the foreign declaration, the unsafe wrapper safe_abs, and the safe main, with only safe_abs holding unsafe.
- It states that the declaration asserts the symbol and signature exist elsewhere but the compiler cannot check the foreign side honors them.
- It explains the seam is plain i32 -> i32, so nothing is allocated or freed across it.
- It explains that abs on i32::MIN panics, and a panic unwinding across a C ABI boundary is undefined behavior, which wrapping_abs avoids.

HINTS

- Only the actual foreign call needs unsafe; the surrounding wrapper and main stay safe.
- The panic policy at the boundary is the real reason for choosing wrapping_abs over abs.

EXERCISE 3 IMPLEMENTATION

Export a length-prefixed buffer reader

OBJECTIVE

Build a C ABI-safe exported function that takes a raw pointer plus length, validates null, and reports failure through an integer status.

STARTER PROMPT

Following the shape of sum_i32s from Example 2, write a function fn max_i32(ptr: *const i32, len: usize, out_max: *mut i32) -> i32 that writes the maximum element through the out parameter and returns 0 on success.

- Return a distinct nonzero status if out_max is null, and another if ptr is null.
- Decide and document what status you return when len is 0 so the contract is unambiguous.
- Build the slice with std::slice::from_raw_parts only after both pointers are known non-null.
- Write the result through *out_max only on the success path and return 0.

ACCEPTANCE CRITERIA

- The signature is exactly fn max_i32(ptr: *const i32, len: usize, out_max: *mut i32) -> i32 marked extern "C".
- Null out_max and null ptr each return a distinct nonzero status before any dereference.
- The empty-input case returns a defined nonzero status rather than reading from an empty slice or writing a garbage maximum.
- The slice is constructed inside an unsafe block with a SAFETY comment, and *out_max is written only on success.

HINTS

- Order matters: check the out pointer first, then the input pointer, exactly as the chapter's guard sequence does.
- from_raw_parts still requires a non-null, aligned pointer even when len is 0, so an empty input is a contract decision, not a free pass.

EXERCISE 4 IMPLEMENTATION

Map ownership across the seam with an opaque handle

OBJECTIVE

Express Rust-owned memory across a C ABI boundary using an opaque handle and an explicit destroy function.

STARTER PROMPT

Design a tiny C ABI surface for a Rust-owned counter: a create function returning an opaque handle, an increment function taking that handle, a read function, and a destroy function. The Rust struct behind the handle must never appear in the C signatures.

- Define the create function as returning *mut CounterHandle built from Box::into_raw on a boxed Rust struct.
- Have increment and read take the raw handle, null-check it, and reconstruct a reference without taking ownership.
- Define destroy so it takes the handle back with Box::from_raw exactly once and drops it.
- State in a comment who allocates, who may mutate, and who frees, so ownership is readable from the signatures.

ACCEPTANCE CRITERIA

- The four extern "C" functions expose only the opaque pointer, integers, and out parameters, never the Rust struct by value.
- create uses Box::into_raw and destroy uses Box::from_raw, with destroy consuming the handle exactly once.
- increment and read null-check the handle and borrow it without freeing it.
- A comment records that Rust owns the allocation, the caller holds the handle, and only destroy frees it.

HINTS

- Box::into_raw hands ownership out as a pointer; Box::from_raw is the matching reclaim, and calling it twice is a double free.
- An opaque handle is just a pointer to a type the foreign side never sees the fields of.

EXERCISE 5 DEBUGGING OR REFACTORING

Stop an unwind from crossing the boundary

OBJECTIVE

Find and fix an exported function that can let a Rust panic unwind across a C ABI boundary.

STARTER PROMPT

An exported extern "C" function parses a caller buffer with code that can panic (an indexing operation and an unwrap on a parse), then returns the parsed value. Reviewers flagged that a panic here is undefined behavior because it would unwind through the foreign caller's frames.

- Identify every operation in the body that can panic and explain why each is a boundary hazard.
- Refactor the panicking operations into checked forms that return a status instead of panicking.
- Decide whether to also wrap the body in std::panic::catch_unwind, and justify the choice.
- Confirm that on every path the function returns an integer status rather than unwinding.

ACCEPTANCE CRITERIA

- Each panic source (indexing, unwrap, slicing, arithmetic) is named and replaced with a checked operation or guard.
- Failure is reported through a distinct nonzero status code, never by panicking.
- The answer states the rule plainly: no cross-language unwinding, translate failure to a status before control leaves the function.
- If catch_unwind is used, the answer explains it is a backstop and the primary fix is removing the panics.

HINTS

- The most dangerous thing crossing the seam is an in-flight unwind, not a bad value.
- get, checked_add, and parse returning Result are the checked replacements for indexing, raw arithmetic, and unwrap.

EXERCISE 6 DESIGN OR PRODUCTION SCENARIO

Choose a binding tool for a legacy C++ subsystem

OBJECTIVE

Apply the chapter's decision tree to select between a hand-written C shim, bindgen, cxx, and autocxx for a real integration.

STARTER PROMPT

A product is replacing one hot native subsystem with Rust while keeping the existing host process, plugin loader, and native test harness. You must reach a large legacy C++ surface, you control the Rust side but not the historical C++ headers, and the requirement is a narrow ABI contract with explicit layout, pointer validity, ownership, status-code errors, and no cross-language unwinding.

- State the two questions the chapter's decision tree turns on and answer them for this scenario.
- Recommend one of a hand-written C shim, bindgen, cxx, or autocxx, and justify it against the size of the C++ surface and who controls each side.
- Explain what part of the contract the tool does not write for you and remains your responsibility.
- Describe how you would test and sanitize the resulting FFI layer before shipping.

ACCEPTANCE CRITERIA

- The answer frames the choice around whether you control both sides and how large the C++ surface is.
- A specific tool is recommended with reasoning tied to a large C++ surface you do not fully control.
- The answer states that the tool reduces boilerplate but does not change the ownership, panic, and ABI-stability contract you still owe.
- A concrete testing and sanitizing plan is given, such as exercising the seam under the native harness with a sanitizer build.

HINTS

- The tool changes how much boilerplate is written for you, not the binary contract you are responsible for.
- A deliberate C ABI seam around C++ buys a smaller contract surface and fewer standard-library assumptions.

Runnable lab

Model the exported sum_i32s seam from Example 2 in plain std Rust: the caller owns the input buffer and the output slot, you only borrow both, and every failure is reported as a distinct integer status rather than a Result.

Example 28-L. `cpp_integration_status_seam_lab.rs` — *Runnable lab · status-code seam over a caller buffer*

```
// Status codes mirror the C ABI seam from the chapter: 0 = success,
// nonzero = a distinct failure the caller can branch on without a Result.
const STATUS_OK: i32 = 0;
```

```
const STATUS_NULL_OUT: i32 = 1;
const STATUS_NULL_INPUT: i32 = 2;

// Models the exported `sum_i32s` seam: the caller owns both the input buffer
// and the output storage; this function only borrows them for the call.
// Option<&...> stands in for a possibly-null pointer; &mut i64 is the out slot.
fn sum_i32s(input: Option<&[i32]>, out_total: Option<&mut i64>) -> i32 {
    // TODO: implement the guard sequence and the sum.
    // 1. If out_total is None, return STATUS_NULL_OUT.
    // 2. If input is None, return STATUS_NULL_INPUT.
    // 3. Otherwise widen each i32 to i64, sum them, write the total
    //    through the out parameter, and return STATUS_OK.
    let _ = (&input, &out_total);
    STATUS_OK
}

fn main() {
    let values = [3_i32, 4, 5];

    let mut total = -1_i64;
    let status = sum_i32s(Some(&values), Some(&mut total));
    println!("status = {}", status);
    println!("total = {}", total);

    let mut ignored = 0_i64;
    println!("null input status = {}", sum_i32s(None, Some(&mut ignored)));
    println!("null out status = {}", sum_i32s(Some(&values), None));
}
```

OUTPUT

```
status = 0
total = 12
null input status = 2
null out status = 1
```

Review questions

- What are the four FFI fundamentals the chapter says almost every Rust interop layer is built from?
- Why must a Rust Result<T, E> be re-expressed as an integer status plus out parameters before crossing a C ABI boundary?
- What does no_mangle and the extern "C" calling convention each contribute when exporting a Rust function to C or C++?
- Why is an in-flight unwind, a Rust panic or a C++ exception, more dangerous to let cross the seam than a bad value?
- Which two questions drive the choice between a hand-written C shim, bindgen, cxx, and autocxx?

CHAPTER 29 · JAVASCRIPT AND C++ INTEGRATION FOR WASM

JavaScript and C++ Integration for WASM

Exercises, labs, and review questions for this chapter.

Export Rust to JavaScript with wasm-bindgen, choose serialization and memory boundaries, and reason about JS/WASM copying costs

EXERCISE 1 WARM-UP COMPREHENSION

State the two-heaps model in your own words

OBJECTIVE

Explain why every non-numeric value crossing the WebAssembly boundary is copied rather than shared.

STARTER PROMPT

The chapter says the shape worth memorizing is two separate heaps with one shared byte array between them. Write a short paragraph that explains what each side owns and what crosses the boundary for build_label and sum_bytes.

- Name what the JavaScript host owns and what the module owns inside its single linear memory.
- Identify the only memory region both sides can touch directly.
- For build_label, say what is copied in and what is copied back out.
- Explain why a u32 result from sum_bytes needs no copy.

ACCEPTANCE CRITERIA

- The answer states that JavaScript holds its own objects while the module holds its own stack and heap inside one linear memory.
- The answer identifies the shared linear memory as the only region both sides can touch.
- It explains that each &str argument to build_label is copied into linear memory and the returned String is copied back into the host.
- It explains that a u32 rides the native interface directly with no allocation and no copy.

HINTS

- The boundary does not disappear under WebAssembly; it becomes an explicit copy you can see and measure.
- Numbers are the only values that travel without an encode-and-copy step.

EXERCISE 2 CODE READING

Trace the C ABI crossing in route_score

OBJECTIVE

Read the C-ABI example and account for every invariant the unsafe block depends on.

STARTER PROMPT

Study the wasm_cpp_ffi_boundary listing where route_score forwards a &str to cpp_route_score as a pointer and length. Trace exactly what crosses the inner C-ABI seam and why the unsafe blocks are sound.

- List what route_score passes to the shim and explain why a slice is not passed directly.
- State the three invariants the SAFETY comments name for the pointer, the length, and the borrow duration.
- Identify which functions are unsafe and which stay in safe Rust.
- Explain what cpp_route_score returns for the input "/orders" and why.

ACCEPTANCE CRITERIA

- The answer notes that route.as_ptr() and route.len() cross because the C ABI has no slice type.
- It names the valid-pointer, correct-length, and read-only-for-the-call invariants from the SAFETY comments.
- It identifies that only the wrapper and the shim are unsafe while the rest stays safe Rust.
- It computes the returned score as bytes.len() as u32 times 10, giving 70 for the 7-byte "/orders".

HINTS

- from_raw_parts reconstructs the slice inside the shim only for the duration of the call.
- Count the characters in "/orders" before multiplying.

EXERCISE 3 IMPLEMENTATION

Add a byte-returning boundary function

OBJECTIVE

Implement an exported-style function that takes a borrowed byte slice and returns an owned buffer copied back to the host.

STARTER PROMPT

Following the sum_bytes pattern, write fn mask_bytes(bytes: &[u8], key: u8) -> Vec<u8> that returns each input byte XORed with key, so the host receives a freshly owned buffer copied out of linear memory.

- Keep the parameter as &[u8] so the input is borrowed and not owned by the module.
- Return an owned Vec<u8> that the host will copy back and own separately.
- Use an iterator or an explicit loop; do not mutate the input slice.
- Note in a comment which direction each value is copied across the boundary.

ACCEPTANCE CRITERIA

- The signature is exactly fn mask_bytes(bytes: &[u8], key: u8) -> Vec<u8>.
- Each output byte equals the corresponding input byte XORed with key.
- The input slice is read but never mutated, and no ownership of the caller's buffer is taken.
- For input [1, 2, 3, 4] with key 1 the function returns [0, 3, 2, 5].

HINTS

- bytes.iter().map(...).collect() builds the owned Vec in one expression.
- The borrowed slice is the copy-in; the returned Vec is the copy-out.

EXERCISE 4 IMPLEMENTATION

Batch crossings instead of crossing per element

OBJECTIVE

Replace a per-element boundary call with a single batched crossing and account for the copy savings.

STARTER PROMPT

A caller currently invokes route_score once per route in a list, paying one boundary crossing per element. Implement fn route_scores(routes: &[&str]) -> Vec<u32> that scores every route in a single call, reusing the route_score logic.

- Accept the whole batch as &[&str] so one crossing covers all routes.
- Reuse the existing route_score logic for each element rather than reimplementing the shim call.
- Return a Vec<u32> aligned by index with the input routes.
- Explain how many boundary crossings this saves compared to calling once per route.

ACCEPTANCE CRITERIA

- The signature is exactly fn route_scores(routes: &[&str]) -> Vec<u32>.
- The result length equals the input length and each entry matches route_score for that route.
- The implementation reuses route_score rather than duplicating the unsafe shim call.
- The answer states the batch makes one crossing instead of one per route.

HINTS

- routes.iter().map(|r| route_score(r)).collect() keeps the batch in one expression.
- The cost you are removing is the per-call boundary overhead, not the arithmetic.

EXERCISE 5 DEBUGGING OR REFACTORING

Fix an unsound boundary wrapper

OBJECTIVE

Diagnose and repair an FFI wrapper whose pointer and length invariants do not hold.

STARTER PROMPT

A teammate wrote fn score_prefix(route: &str, n: usize) -> u32 { unsafe { cpp_route_score(route.as_ptr(), n) } } intending to score only the first n bytes, but it can read past the string. Identify the defect and correct the wrapper.

- Explain why passing n unchecked can violate the correct-length invariant from the SAFETY comment.
- Decide how to clamp or validate n against route.len() before crossing the seam.
- Rewrite the wrapper so the pointer and length always describe a valid read-only region.
- State the SAFETY invariants your corrected version guarantees.

ACCEPTANCE CRITERIA

- The answer identifies that n greater than route.len() lets the shim read beyond the borrowed bytes.
- The fix clamps the length to at most route.len() (for example n.min(route.len())) before the call.
- The corrected wrapper passes a pointer and a length that always describe a valid region for the call.
- A written SAFETY comment names the valid-pointer, correct-length, and read-only invariants.

HINTS

- The C ABI trusts the length you hand it; the wrapper is the only place that can enforce a bound.
- n.min(route.len()) keeps the length inside the borrowed slice.

EXERCISE 6 DESIGN OR PRODUCTION SCENARIO

Choose boundary representations on the cost ladder

OBJECTIVE

Apply the number / bytes / struct / resident-handle cost ladder to a concrete WebAssembly workload.

STARTER PROMPT

The product moves image processing, parsing, and a routing-score calculation into the module. For each of: a per-frame brightness scalar, a parsed config struct sent once at startup, a large image buffer processed every frame, and a parser the host calls thousands of times, choose a boundary representation and justify the cost.

- Map each workload to one rung of the ladder: number, byte array, serialized struct, or resident handle.
- Justify each choice in terms of copies per crossing and encode/decode cost.
- Decide which workload should keep state resident behind a handle instead of copying per call.
- Name one case where reaching for the most convenient type (for example JSON) would make the module slower than the JavaScript it replaced.

ACCEPTANCE CRITERIA

- The brightness scalar is sent as a number with no copy, and the image buffer crosses as a byte array with one copy per crossing.
- The parser called thousands of times is kept resident behind a handle to avoid per-call copies, accepting a managed lifetime.
- Each choice is justified using copies per crossing or encode-plus-decode cost from the ladder.
- The answer names a concrete case (such as a large struct as JSON) where the convenient encoding loses to a compact binary layout.

HINTS

- A number is free, a byte array is one copy, a struct adds encode plus decode, and a handle trades a per-call copy for a lifetime you manage.
- The performance conversation is about the boundary, not the arithmetic inside the module.

Runnable lab

Model the chapter's boundary contracts in plain std Rust: a string-in/string-out crossing, a byte-slice-in/number-out crossing, a pointer-plus-length C-ABI crossing, and a single batched call. Fill in the TODO in route_scores so the whole batch crosses in one call.

Example 29-L. `wasm_boundary_lab.rs` — *Runnable lab · Boundary crossings and the cost ladder*

```rust
// Models the WebAssembly boundary contracts in plain std Rust.
// Two separate heaps with one shared byte array is simulated here by
// passing borrowed slices in and returning owned values out.

fn build_label(service: &str, route: &str) -> String {
    // &str copied in, owned String copied back out.
```

```
    format!("{}::{}", service, route)
}

fn sum_bytes(bytes: &[u8]) -> u32 {
    // borrowed slice copied in, u32 rides back with no copy.
    bytes.iter().map(|&value| value as u32).sum()
}

fn route_score(route: &str) -> u32 {
    // Pointer-plus-length C-ABI crossing, modeled: score is len * 10.
    (route.len() as u32) * 10
}

fn route_scores(routes: &[&str]) -> Vec<u32> {
    // TODO: score every route in a single batched crossing,
    // reusing route_score, and return one u32 per route.
    Vec::new()
}

fn main() {
    let label = build_label("billing", "/ready");
    let total = sum_bytes(&[1_u8, 2, 3, 4]);
    let score = route_score("/orders");
    let scores = route_scores(&["/a", "/bb", "/ccc"]);

    println!("label = {}", label);
    println!("sum = {}", total);
    println!("score = {}", score);
    println!("batch crossings = {}", if scores.is_empty() { 0 } else { 1 });
}
```

OUTPUT

```
label = billing::/ready
sum = 10
score = 70
batch crossings = 1
```

Review questions

- Why does every value richer than a number get copied when it crosses between the JavaScript host and the WebAssembly module?
- What role does wasm-bindgen play after a browser-facing crate is compiled as a cdylib?
- On the cost ladder of boundary representations, how do a number, a byte array, a serialized struct, and a resident handle differ in per-crossing cost?
- How do the outer Rust-to-JavaScript seam and the inner Rust-to-C-shim seam differ in how data and safety are managed?
- Why is the WebAssembly performance discussion almost always about the boundary rather than the arithmetic inside the module?

CHAPTER 30 · AMQP AND MESSAGE BROKERS

AMQP and Message Brokers

Exercises, labs, and review questions for this chapter.

Model an idempotent consumer, add retry and dead-letter handling, and design versioned message contracts

EXERCISE 1 WARM-UP COMPREHENSION

Separate routing-key choice from routing policy

OBJECTIVE

Explain who owns each decision in the publish path so the producer stays decoupled from queue topology.

STARTER PROMPT

The chapter insists that a producer publishes to an exchange with a routing key such as orders.created and never names a queue, while the exchange owns bindings and decides which queues receive a copy. Describe this split in your own words.

- State what the producer controls and what the exchange controls when a message is published.
- Explain why a new team can add a search-indexing queue for orders.created without editing any publisher.
- Describe what happens to a message whose routing key matches no binding, and why that is counted rather than silently dropped.

ACCEPTANCE CRITERIA

- The answer says the producer chooses only the routing key and the exchange owns binding state and routing policy.
- The answer explains that adding a binding for an existing key reaches new queues without changing the producer.
- The answer states that an unbound key produces zero deliveries and is tracked as unrouted, not delivered anywhere.

HINTS

- Reread the line that a producer never names a queue; it publishes to an exchange with a routing key.
- The exchange is the one place routing topology lives, so publishers do not need to be queue-aware.

EXERCISE 2 CODE READING

Trace the idempotent consumer's decision machine

OBJECTIVE

Predict the processed, duplicate, retried, and dead-letter counts by reading the on_delivery state transitions.

STARTER PROMPT

Read the Consumer type from Example 2, whose on_delivery first checks processed_ids, then routes a delivery for order ord-fail through a bounded retry, and finally records successful unique work. Hand-trace the four messages that main feeds it, including the retry-queue drain loop.

- Walk through msg-1 sent twice and say which branch each delivery takes.
- Follow msg-2 with order_id ord-fail through every attempt until the drain loop empties the retry_queue, and say where it lands.
- State the final values of processed, duplicates, retried, and dlq printed by main.

ACCEPTANCE CRITERIA

- The trace identifies the second msg-1 as a duplicate that is counted and returns without redoing the work.
- The trace shows ord-fail being retried at attempts 1 and 2, then moved to the DLQ once attempt reaches 3.
- The predicted output is processed = 2, duplicates = 1, retried = 2, dlq = 1.

HINTS

- A successful unique delivery is the only path that inserts into processed_ids and increments processed.
- The retry branch fires only while attempt is below 3; the third arrival of ord-fail takes the dlq branch.

EXERCISE 3 IMPLEMENTATION

Add a wildcard topic match to the exchange

OBJECTIVE

Extend the routing lookup so a single binding pattern can match a family of routing keys.

STARTER PROMPT

Starting from DirectExchange in Example 1, add a method that supports a single trailing wildcard segment, so binding orders.* reaches a queue for both orders.created and orders.cancelled. Keep the exact-match route method working unchanged.

- Decide how to store wildcard bindings so an exact lookup is still cheap.
- Implement matching by splitting the routing key on '.' and comparing segments, treating '*' as matching exactly one segment.
- Return the union of exact-match queues and wildcard-match queues, with no duplicate queue names.

ACCEPTANCE CRITERIA

- Binding orders.* to a queue makes that queue appear for both orders.created and orders.cancelled.
- An exact binding such as orders.created to billing still resolves through the unchanged route path.
- A routing key matching both an exact binding and a wildcard binding lists each target queue only once.
- A key matching no binding still returns an empty Vec<String>.

HINTS

- Split on '.' and compare segment by segment; '*' covers exactly one segment, not several.
- Collect results into a structure that rejects duplicates, or check membership before pushing.

EXERCISE 4 IMPLEMENTATION

Make the consumer ack only after the durable effect

OBJECTIVE

Encode the commit-then-ack ordering so a crash between the two cannot lose acknowledged work.

STARTER PROMPT

Model a consumer with a durable store (a Vec or HashMap standing in for the database) and an explicit ack list. Implement handle so it writes the durable effect first and only then records the ack, mirroring the chapter's rule that an ack promises the work is safely done.

- Define functions for the durable write and for sending the ack as two separate steps.
- Order them so the durable effect is committed before the ack is recorded.
- Add a duplicate guard so re-handling an already-committed message id acks without writing twice.
- Describe what the broker does to a message that was committed but crashed before the ack.

ACCEPTANCE CRITERIA

- The code commits the durable effect strictly before appending to the ack list.
- A message id already present in the durable store is acked again without a second write.
- The answer explains that a crash after commit but before ack yields a redelivery, which the idempotency guard absorbs.
- No code path records an ack for a message whose durable effect was not committed.

HINTS

- An ack is a promise the work is safely done and the broker may forget the message.
- At-least-once delivery means commit-then-crash is a redelivery, so the handler must tolerate seeing the id again.

EXERCISE 5 DEBUGGING OR REFACTORING

Fix an ack-before-commit consumer

OBJECTIVE

Diagnose and correct a consumer that loses work by acknowledging before its side effect is durable.

STARTER PROMPT

A consumer acks the broker first and then performs the durable write, and it also retries malformed payloads forever. Identify both defects against the chapter's guidance and refactor the handler.

- Explain the data-loss window created when ack happens before the durable commit.
- Reorder the steps so the durable effect is committed before the ack.
- Replace the unbounded retry with a bounded attempt count that sends terminal failures to a DLQ.
- Distinguish which failures deserve a retry and which should go straight to the dead-letter queue.

ACCEPTANCE CRITERIA

- The diagnosis names ack-before-commit as the cause of silently lost work on a mid-handler crash.
- The refactor commits the durable effect before sending the ack.
- Retries become counted and bounded, and a message exceeding the cap is routed to a DLQ.
- A malformed or unsupported payload is sent to the DLQ instead of being retried, since it fails identically every time.

HINTS

- If the consumer acks first and then crashes before the durable effect, the broker has already forgotten the message.
- Requeuing a message that can never succeed just blocks the queue behind it; that is what the DLQ is for.

EXERCISE 6 DESIGN OR PRODUCTION SCENARIO

Design a versioned envelope for mixed deployments

OBJECTIVE

Specify a message contract and consumer policy that survive replay across deploys and version skew.

STARTER PROMPT

The order pipeline must let a message sit in a queue across a deploy, be replayed weeks later, and be consumed by a service running an older version. Design the envelope and the consumer rules that keep this safe, building on the chapter's idempotency and DLQ work.

- List the envelope fields that travel alongside the payload, including a schema version and a stable message id.
- Specify how a consumer handles a version it does not recognize without blocking the queue.
- State how the message id ties into idempotency so replay is harmless.
- Name the queue-health signals you would expose so a pile-up does not stay invisible.

ACCEPTANCE CRITERIA

- The envelope carries an explicit schema version, a stable unique message id, and the payload, rather than a bare struct dump.
- The design keeps the message id as the idempotency key so duplicate or replayed deliveries are deduplicated, not reprocessed.
- An unrecognized version is routed to a DLQ or held for a newer consumer rather than crashing or blocking the queue.
- The answer lists at least queue depth, message age, retry rate, and dead-letter count as the minimum health signals.

HINTS

- A message often outlives the code that produced it, so the version belongs in the envelope, not in tribal knowledge.
- A broker is a place work can pile up invisibly; depth, age, retry rate, and DLQ count are what reveal it.

Runnable lab

Model the chapter's direct exchange: producers publish a routing key, the exchange owns bindings, and a key with no binding is counted as unrouted instead of vanishing. Fill in route so it returns the bound queues for a key and bumps the unrouted counter when nothing matches.

Example 30-L. `direct_exchange_routing_lab.rs` — *Runnable lab · direct exchange routing with an unrouted counter*

```
use std::collections::HashMap;

#[derive(Default)]
struct DirectExchange {
    bindings: HashMap<String, Vec<String>>,
    unrouted: usize,
}

impl DirectExchange {
```

```
    fn bind(&mut self, routing_key: &str, queue: &str) {
        self.bindings
            .entry(routing_key.to_string())
            .or_default()
            .push(queue.to_string());
    }

    fn route(&mut self, routing_key: &str) -> Vec<String> {
        // TODO: return the queues bound to routing_key.
        // If the key has no binding, increment self.unrouted and return an empty Vec.
        Vec::new()
    }
}

fn main() {
    let mut exchange = DirectExchange::default();
    exchange.bind("orders.created", "billing");
    exchange.bind("orders.created", "search");
    exchange.bind("orders.cancelled", "billing");

    let created = exchange.route("orders.created");
    let cancelled = exchange.route("orders.cancelled");
    let _ = exchange.route("orders.refunded");
    let _ = exchange.route("orders.archived");

    println!("orders.created -> {}", created.join(","));
    println!("orders.cancelled -> {}", cancelled.join(","));
    println!("unrouted total = {}", exchange.unrouted);
}
```

OUTPUT

```
orders.created -> billing,search
orders.cancelled -> billing
unrouted total = 2
```

Review questions

- In the publish path, which decisions belong to the producer and which belong to the exchange, and why does that split let teams change routing topology without editing publishers?
- Why must a consumer commit its durable side effect before sending the ack, and what does the broker do with a message that stays unacknowledged?
- Since AMQP delivery is at-least-once, what does it mean for a handler to be idempotent, and how does a recorded message id make replay harmless?
- When should a failed message be retried versus sent to a dead-letter queue, and what goes wrong if a poison message is requeued forever?
- What does backpressure protect against in a broker-based system, and which signals tell you a queue is piling up rather than merely quiet?

CHAPTER 31 · DISTRIBUTED TASK EXECUTION

Distributed Task Execution

Exercises, labs, and review questions for this chapter.

Design a lease-based distributed worker, model at-least-once execution with idempotency, and trace distributed task graphs end to end

EXERCISE 1 WARM-UP COMPREHENSION

Classify delivery semantics and idempotency obligations

OBJECTIVE

Practice choosing at-most-once, at-least-once, or effectively exactly-once side effects from business tolerance rather than slogan.

STARTER PROMPT

Classify four workloads: a thumbnail regeneration job, a billing capture, a cache warmup, and a periodic analytics refresh.

- Which workload can tolerate occasional loss and therefore may accept at-most-once behavior?
- Which workload must tolerate replay and therefore needs idempotency even under at-least-once delivery?
- Which workload can be naturally repeat-safe because it overwrites one deterministic final state?
- Which workload would force you to define a durable dedupe or checkpoint boundary explicitly?

ACCEPTANCE CRITERIA

- You separate delivery semantics from business side-effect semantics clearly.
- You identify at least one task that is naturally repeat-safe and one that needs explicit dedupe state.
- You avoid claiming exactly-once as a queue property by itself.

HINTS

- Ask what happens if the same logical work runs twice.
- A strong answer talks about side effects, not only about transport labels.

EXERCISE 2 CODE READING

Read a lease timeline like an incident reviewer

OBJECTIVE

Explain what happened when one worker timed out, another reclaimed the task, and the first worker later came back.

STARTER PROMPT

Worker A claims task-7, its lease expires, Worker B reclaims task-7, Worker A finally reports success, then Worker B also reports success.

- Which event makes duplicate execution normal instead of surprising?
- Where should idempotent completion state live so only one success is accepted?
- What visibility-timeout or heartbeat change would reduce replay without delaying recovery too much?
- Which metric would tell you that the lease duration is badly matched to real handler time?

ACCEPTANCE CRITERIA

- You explain the duplicate execution path from the lease-expiry timeline concretely.
- You place dedupe or completion recording at the durable side-effect boundary.
- You mention at least one tuning or observability signal such as lease-expiry rate or task age.

HINTS

- A lease expiry is a recovery event, not a correctness bug by itself.
- The bug appears only if the completion path is not idempotent.

EXERCISE 3 IMPLEMENTATION

Build a lease-based worker with idempotent completion

OBJECTIVE

Implement the core mechanics of redelivery after timeout and duplicate-safe completion recording.

STARTER PROMPT

Complete a tiny in-memory queue so timed-out work returns to visibility and a second completion attempt becomes a harmless duplicate.

- Requeue the task only when the current time is past the deadline.
- Record completion in a set keyed by task ID.
- Return a boolean from completion so the caller can tell whether this was the first successful application or a duplicate.
- Keep the queue owner as one struct instead of scattering state globally.

ACCEPTANCE CRITERIA

- The queue requeues timed-out work correctly.
- Completion recording is idempotent.
- The runnable lab prints the expected redelivery, completed count, and duplicate status.

HINTS

- The two key repairs are one requeue condition and one set insert.
- Use the set insert result directly if you want the calmest duplicate signal.

EXERCISE 4 DEBUGGING OR REFACTORING

Centralize distributed retries and poison-task policy

OBJECTIVE

Repair a design that copied retry loops into several handlers and requeues permanent failure forever.

STARTER PROMPT

Three handlers each retry internally with slightly different attempt limits, and every permanent validation error is still requeued.

- Which facts should one shared retry policy own: max attempts, retryable classes, backoff, terminal routing?
- Which failures should be dead-lettered immediately?
- Where should local in-process retry bookkeeping end and queue-level requeue policy begin?
- How would you make the policy testable without a live broker or queue service?

ACCEPTANCE CRITERIA

- You centralize retry classification and visible attempt budgeting.
- You separate transient retryable failure from permanent terminal failure.
- You mention one test strategy such as table-driven classification or DLQ routing tests.

HINTS

- If the same retry logic appears in three places, it already wants one home.
- Poison-task handling is a transport and policy question, not a random handler branch.

EXERCISE 5 DEBUGGING OR REFACTORING

Trace a distributed task graph end to end

OBJECTIVE

Make one task graph observable enough that queue wait, handler work, and fan-in are all attributable.

STARTER PROMPT

A DAG pipeline runs `fetch -> parse -> { enrich, store } -> notify`, but traces only show the inner handler spans and not the time spent waiting in queues.

- Which IDs should every task envelope carry?
- Where should parent-child lineage be recorded for fan-out and fan-in nodes?
- Which span or event should represent queue wait before a worker actually starts the handler?
- What metric would tell you the graph is bottlenecked at aggregation rather than at worker execution?

ACCEPTANCE CRITERIA

- You include a stable task or message ID plus trace lineage fields.
- You represent queue wait as a first-class trace or metric concept.
- You mention at least one graph-specific profiling signal such as frontier size, reducer lag, or critical-path time.

HINTS

- If queue wait is invisible, the trace is incomplete.
- Graph tracing needs lineage, not only one flat request ID.

EXERCISE 6 DESIGN OR PRODUCTION SCENARIO

Choose queue topology, aggregation, tracing, and profiling for a production task service

OBJECTIVE

Map the full distributed-task control plane for a real system with several failure and scaling modes.

STARTER PROMPT

You are designing `upload -> transcode variants -> scan -> persist metadata -> notify`, with bursty uploads, one long-running GPU step, and strict operator requirements around replay and incident forensics.

- Which stages deserve their own queue or shard boundary?
- Where should leases be renewed instead of just lengthened?
- How will final aggregation decide when the whole graph is complete?
- Which tracing fields, queue metrics, and duplicate metrics would you require before rollout?
- Where would you isolate slow GPU retries so they do not stall the rest of the platform?

ACCEPTANCE CRITERIA

- You choose at least one queue or shard split deliberately.
- You define a lease or renewal policy for the long-running step.
- You describe one result-aggregation rule and one failure-isolation rule.
- You mention at least three observability hooks such as queue age, redelivery rate, reducer lag, trace lineage, or duplicate suppression hits.

HINTS

- The cleanest answer gives every expensive stage a visible budget and a visible failure policy.
- A long-running specialist worker is often a stronger argument for leasing discipline than for a longer global timeout.

Runnable lab

Example 31-L. `lease_idempotent_lab.rs` — *Runnable lab · Lease timeout plus idempotent completion*

```rust
use std::collections::{HashSet, VecDeque};

#[derive(Debug, Clone)]
struct Task {
    id: &'static str,
}

struct Queue {
    visible: VecDeque<Task>,
    completed: HashSet<&'static str>,
    now: u64,
}

impl Queue {
```

```
    fn claim(&mut self) -> Option<Task> {
        self.visible.pop_front()
    }

    fn requeue_if_timed_out(&mut self, task: Task, deadline: u64) {
        if self.now < deadline {
            self.visible.push_back(task);
        }
    }

    fn finish_once(&mut self, task: &Task) -> bool {
        false
    }
}

fn main() {
    let mut queue = Queue {
        visible: VecDeque::new(),
        completed: HashSet::new(),
        now: 0,
    };

    queue.visible.push_back(Task { id: "task-1" });

    let first = queue.claim().unwrap();
    queue.now = 6;
    queue.requeue_if_timed_out(first.clone(), 5);

    let redelivery = queue.claim().unwrap();
    let _first_apply = queue.finish_once(&redelivery);
    let duplicate_apply = queue.finish_once(&redelivery);

    println!("redelivered = {}", redelivery.id);
    println!("completed = {}", queue.completed.len());
    println!("duplicate = {}", !duplicate_apply);
}
```

OUTPUT

```
redelivered = task-1
completed = 1
duplicate = true
```

Review questions

- Why is at-least-once delivery the calm default assumption for distributed task systems?
- What is the practical difference between a lease expiry and a processing bug?
- Why should retries and terminal failure paths be centralized instead of copied into each handler?
- What makes queue wait time a first-class profiling signal?
- Why does a large task graph need explicit lineage and aggregation policy rather than one flat success counter?

CHAPTER 32 · MPI AND HIGH-PERFORMANCE COMPUTING

MPI and High-Performance Computing

Exercises, labs, and review questions for this chapter.

Partition matrix rows across ranks, choose collective-friendly layouts, and reason about hybrid MPI plus threads and communication profiling

EXERCISE 1 WARM-UP COMPREHENSION

Choose process partitioning and collective shape from the algorithm

OBJECTIVE

Practice deciding how a dense matrix workload should be split across ranks before any low-level optimization begins.

STARTER PROMPT

You must run a row-wise dense matrix pass on 10,000 rows across 6 ranks, then compute one global convergence scalar every iteration.

- Would you partition by rows, by columns, or by tiles first, and why?
- Which collective is the honest fit for the final convergence scalar: reduce or allreduce?
- What would change if every rank needs the scalar for the next iteration instead of only rank 0?

ACCEPTANCE CRITERIA

- You choose a plausible first partitioning strategy and justify it from access pattern.
- You distinguish reduce from allreduce by who needs the final answer.
- You keep the explanation in terms of layout and synchronization, not only in terms of API names.

HINTS

- Start from how the data is walked in memory.
- Then ask who really needs the aggregate result afterward.

EXERCISE 2 CODE READING

Find the `Vec<Vec<T>>` mistake before the collective boundary

OBJECTIVE

Read a dense MPI preparation path and explain why nested allocation is the wrong default before scatter or gather.

STARTER PROMPT

A teammate stores a dense matrix as `Vec<Vec<f64>>`, then tries to scatter row blocks to ranks and gather results back into the same shape.

- What extra allocations and pointer indirections does the nested shape create?
- Why does one flat `Vec<f64>` plus explicit row counts simplify the collective boundary?
- When would the nested shape still be honest because the data is truly ragged?

ACCEPTANCE CRITERIA

- You explain the dense-layout cost of `Vec<Vec<T>>` precisely.
- You recommend one flat buffer for the dense case with a reason tied to collectives.
- You name one case where nested ownership is still the right shape.

HINTS

- The question is dense versus ragged, not elegant syntax versus ugly syntax.
- Collectives are easiest when the payload is already contiguous.

EXERCISE 3 IMPLEMENTATION

Implement a balanced block partition for matrix rows

OBJECTIVE

Build the row-partition helper every MPI rank needs so remainder rows are distributed predictably.

STARTER PROMPT

Implement `block_range(rows, ranks, rank)` so rows are split as evenly as possible and early ranks absorb the remainder.

- Return a start row and an end row.
- Keep the end exclusive.
- Use the same helper for all ranks instead of special-casing rank 0 and hoping the rest line up.
- Verify the local cell count for rank 2 when `rows = 11`, `ranks = 4`, and `cols = 5`.

ACCEPTANCE CRITERIA

- The helper distributes the remainder rows to the lowest ranks.
- Every range is half-open and non-overlapping.
- The runnable lab prints the expected ranges and rank-2 cell count.

HINTS

- This is the same balanced block distribution many real MPI codes use.
- One good formula is cheaper than many ad hoc branches.

EXERCISE 4 DEBUGGING OR REFACTORING

Choose flat counts and displacements for a collective

OBJECTIVE

Replace row-thinking confusion with the exact units the collective boundary expects.

STARTER PROMPT

You inherit a scatter path that computes row counts correctly but passes row displacements to a collective that expects element displacements.

- Where should the conversion from rows to cells happen?
- How do you compute cumulative displacements for a row-major buffer?
- What test would catch the bug before it hits a full cluster run?

ACCEPTANCE CRITERIA

- You convert counts or offsets into the same units the collective API expects.
- You explain the cumulative-displacement rule clearly.
- You mention at least one focused correctness test such as a tiny uneven matrix with known slices.

HINTS

- Rows are an algorithm concept. Collectives often operate in typed elements or bytes.
- Tiny uneven shapes are usually the best bug traps here.

EXERCISE 5 DEBUGGING OR REFACTORING

Profile communication-heavy HPC work conceptually before rewriting it

OBJECTIVE

Practice building a measurement plan for an MPI program whose slowdown may come from imbalance or collectives rather than from arithmetic.

STARTER PROMPT

A solver slowed down after a mesh change. Every rank still finishes the same kernel code path, but total runtime rose sharply.

- Which timers would you place around local compute, halo exchange, and collective convergence steps?
- Which signals would distinguish load imbalance from too-frequent communication?
- What would you log or trace per rank so rank 0 does not hide the slowest participant?

ACCEPTANCE CRITERIA

- You separate compute, communication, and wait time explicitly.
- You mention at least one rank-local signal and one cross-rank signal.
- You avoid jumping straight to kernel micro-optimization before the communication story is clear.

HINTS

- A slower mesh often changes partition shape before it changes arithmetic intensity.
- The important question is who is waiting for whom, and where.

EXERCISE 6 DESIGN OR PRODUCTION SCENARIO

Choose hybrid MPI plus threads and cluster deployment policy deliberately

OBJECTIVE

Map ranks, local threads, queue budgets, and profiling hooks to one realistic cluster workload.

STARTER PROMPT

You are designing `load block -> local compute -> allreduce convergence -> write checkpoint`, with CPU-heavy kernels, one read-mostly lookup table per rank, and a cluster policy that limits cores per node tightly.

- Would you run more ranks per node, more threads per rank, or a balanced mix?
- Which local work stays inside a Rayon or thread pool within the rank?
- Where would you keep shared immutable data with `Arc<T>` only inside the rank, and where would MPI remain the inter-rank boundary?
- What metrics would you require for node-level oversubscription, imbalance, and collective cost before rollout?

ACCEPTANCE CRITERIA

- You choose a plausible hybrid or non-hybrid deployment shape with a reason tied to topology and kernel behavior.
- You distinguish intra-rank shared immutable state from inter-rank communication clearly.
- You mention at least three observability hooks such as CPU oversubscription, collective wait time, queue depth, or per-rank throughput.

HINTS

- A hybrid design is only good if the total core budget per node stays honest.
- Rust thread rules still matter inside each rank even when MPI solves the cross-process side.

Runnable lab

Example 32-L. `mpi_block_partition_lab.rs` — *Runnable lab · Balanced block partition for matrix rows*

```rust
#[derive(Debug, Clone, Copy)]
struct Partition {
    start_row: usize,
    end_row: usize,
}

fn block_range(rows: usize, ranks: usize, rank: usize) -> Partition {
    let rows_per_rank = rows / ranks;
    let start = rank * rows_per_rank;
    let end = start + rows_per_rank;

    Partition {
        start_row: start,
        end_row: end,
    }
}

fn main() {
    let rows = 11_usize;
    let ranks = 4_usize;
    let cols = 5_usize;

    for rank in 0..ranks {
        let part = block_range(rows, ranks, rank);
        println!("rank {} = {}..{}", rank, part.start_row, part.end_row);
    }

    let part = block_range(rows, ranks, 2);
    println!("cells rank2 = {}", (part.end_row - part.start_row) * cols);
}
```

OUTPUT

```
rank 0 = 0..3
rank 1 = 3..6
rank 2 = 6..9
rank 3 = 9..11
cells rank2 = 15
```

Review questions

- Why is MPI primarily a process model rather than a shared-memory model?
- Why do flat contiguous buffers simplify collective operations so much?
- What is the practical difference between reduce and allreduce in an iterative solver?
- Why is a balanced partition helper worth testing as a first-class component?

- What signals tell you an MPI program is communication-bound instead of compute-bound?
- Why can hybrid MPI plus threads become slower if you ignore total core budgeting on the node?

CHAPTER 33 · PERFORMANCE-ORIENTED RUST

Performance-Oriented Rust

Exercises, labs, and review questions for this chapter.

Write a benchmark plan, refactor for locality, compare loops and iterators responsibly, and make measurement discipline explicit

EXERCISE 1 WARM-UP COMPREHENSION

Separate benchmarking, profiling, tracing, and production observability

OBJECTIVE

Choose the measurement tool that matches the actual performance question.

STARTER PROMPT

You need to compare two parsing functions, explain a latency regression inside one of them, understand where request time is spent across queueing and IO, and confirm whether the regression matters in the live service.

- Which question wants a benchmark?
- Which question wants a profile?
- Which question wants tracing?
- Which question wants production metrics or logs?

ACCEPTANCE CRITERIA

- You map each question to a distinct tool with a workload-based reason.
- You avoid treating a benchmark result as a complete production diagnosis.
- You explain at least one way profiling and tracing answer different questions.

HINTS

- A benchmark compares alternatives under controlled input.
- A trace shows timeline and causality across boundaries.

EXERCISE 2 CODE READING

Find allocation pressure and hidden clones in a hot path

OBJECTIVE

Read a request loop and identify where ownership choices add heap traffic.

STARTER PROMPT

A request filter clones route strings into a temporary vector, formats a label per item, and pushes one record at a time into a dynamically growing output buffer.

- Which values only needed borrowed access?
- Which allocation could be preplanned from a real bound?
- Which clone is semantically real and which only papers over an API-shape problem?
- What would you measure after the refactor to confirm the change helped?

ACCEPTANCE CRITERIA

- You identify at least one unnecessary clone and one avoidable growth pattern.
- You propose a borrowed read path and one preallocation repair.
- You mention at least one follow-up metric such as allocation count, latency, or bytes allocated per request.

HINTS

- Look for helpers that take ownership only to read.
- Look for vectors or strings that grow in obviously bounded loops.

EXERCISE 3 IMPLEMENTATION

Write a benchmark plan for an allocation-heavy function

OBJECTIVE

Design a benchmark that compares alternatives without confusing it with profiling or production tuning.

STARTER PROMPT

You are comparing two versions of a log-enrichment function: one clone-heavy and one borrow-first with preallocation.

- Specify the fixed input shape and size distribution.
- State that the benchmark runs in release mode.
- Define what you will record: wall-clock time, allocations, output count, or all three.
- Describe how you will keep the compiler from optimizing the whole function away when appropriate.

ACCEPTANCE CRITERIA

- Your benchmark plan compares the same logical workload under two implementations.
- You name at least one release-build requirement and one measurement target.
- You distinguish the later profiling step from the benchmark step.
- You avoid claiming a speedup before the plan has been executed.

HINTS

- A good benchmark plan makes the input and stop condition boring.
- If the result is unused, dead-code elimination can fake a win.

EXERCISE 4 DEBUGGING OR REFACTORING

Refactor storage to improve locality

OBJECTIVE

Replace an indirection-heavy dense layout with a flatter one that matches the access pattern.

STARTER PROMPT

You inherit a dense heatmap stored as `Vec<Vec<u64>>`, and the hot loop walks every row in full on every request.

- What does one flat `Vec<u64>` plus `rows` and `cols` remove from the memory-access pattern?
- Which helper methods should expose rows as borrowed slices instead of reconstructing vectors?
- How does this change your reasoning about bounds checks in the inner loop?
- What measurement would you take before and after the refactor?

ACCEPTANCE CRITERIA

- You replace nested ownership with one flat owner for the dense case.
- You mention locality or cache behavior explicitly.
- You explain how row views or chunks make inner-loop bounds reasoning simpler.
- You mention at least one before-and-after measurement target.

HINTS

- The point is not only fewer allocations. It is also more predictable traversal.
- Chunked slice iteration often gives the optimizer a better proof story than ad hoc indexing.

EXERCISE 5 DEBUGGING OR REFACTORING

Compare iterator and loop implementations responsibly

OBJECTIVE

Avoid folklore by designing a fair comparison between two equivalent hot loops.

STARTER PROMPT

You have one implementation written as an iterator chain and one as an explicit `for` loop over the same slice.

- How will you keep the logical work identical between the two versions?
- What would make the comparison unfair, such as extra allocation or a changed branch structure in only one variant?
- Would you inspect generated code or profile data before drawing conclusions from the benchmark alone?
- How would you report the result without claiming one style is globally faster?

ACCEPTANCE CRITERIA

- You keep the workload and output identical between variants.
- You mention at least one unfair comparison trap.
- You include at least one follow-up inspection step such as profiling or generated-code review.
- You report the result as workload-specific rather than as a universal rule.

HINTS

- The style difference should be the variable, not the data shape or allocation pattern.
- Iterator versus loop is a measurement question, not a religion.

EXERCISE 6 DESIGN OR PRODUCTION SCENARIO

Choose release settings and performance review policy for a service

OBJECTIVE

Make build mode, profile settings, and runtime observability part of the performance design.

STARTER PROMPT

You are shipping a latency-sensitive binary with one hot parser, one CPU-heavy enrichment step, and an operator requirement that crash behavior remain explicit.

- Which release-build command should every performance run use?
- Which profile settings would you consider, and what tradeoff does each one carry?
- When might `panic = "abort"` be acceptable, and when would it be the wrong contract?
- Which live signals would you require before trusting the optimized build in production?

ACCEPTANCE CRITERIA

- You name release mode explicitly for measurement.
- You justify at least two profile settings or profile decisions with tradeoffs.
- You explain `panic = "abort"` as a binary contract choice rather than a free speed flag.
- You mention at least two production observability signals such as p99 latency, queue depth, allocation pressure, or error rate.

HINTS

- Compiler flags are workload tools, not trophies.
- A good performance review says what changed in both build behavior and runtime behavior.

Runnable lab

Example 33-L. `hot_routes_lab.rs` — *Runnable lab · Allocation-aware hot-route filter*

```rust
#[derive(Debug)]
struct Request<'a> {
    route: &'a str,
    bytes: usize,
}

fn hot_routes<'a>(requests: &'a [Request<'a>], min_bytes: usize) -> Vec<&'a str> {
    let mut out = Vec::new();

    for request in requests {
        if request.bytes > min_bytes {
            out.push(request.route);
        }
    }
```

```
    out
}

fn main() {
    let requests = [
        Request {
            route: "/health",
            bytes: 128,
        },
        Request {
            route: "/search",
            bytes: 900,
        },
        Request {
            route: "/checkout",
            bytes: 512,
        },
        Request {
            route: "/metrics",
            bytes: 64,
        },
    ];

    let hot = hot_routes(&requests, 512);

    println!("hot = {}", hot.len());
    println!("first = {}", hot.first().copied().unwrap_or("none"));
    println!("capacity ok = {}", hot.capacity() >= requests.len());
}
```

OUTPUT

```
hot = 2
first = /search
capacity ok = true
```

Review questions

- Why is performance work usually clearer after ownership boundaries become honest?
- What is the practical difference between reducing allocations and reducing branch misses?
- Why are slice-based and chunk-based loops often easier to optimize than index-heavy loops?
- When is a clone economically justified even in performance-sensitive code?
- What does release mode change, and what questions does it still not answer by itself?

CHAPTER 34 · MEMORY PROFILING

Memory Profiling

Exercises, labs, and review questions for this chapter.

Find clone pressure, diagnose Rc and Arc leaks, choose profiling tools, and build a memory profiling checklist

EXERCISE 1 WARM-UP COMPREHENSION

Match the symptom to the measurement

OBJECTIVE

Choose whether the next step should be allocation counts, heap snapshots, RSS tracking, queue metrics, or cycle inspection.

STARTER PROMPT

Classify four symptoms: rising RSS after a burst, stable RSS but rising task count, flat heap snapshots with a high allocation rate, and a graph of `Rc` nodes that never seems to disappear.

- Which symptom points first toward fragmentation or allocator retention?
- Which symptom points first toward async backlog rather than a leak?
- Which symptom points first toward churn rather than retained memory?
- Which symptom points first toward logical reachability and cycle inspection?

ACCEPTANCE CRITERIA

- You choose a distinct first measurement for each symptom.
- You distinguish retained memory from allocation churn clearly.
- You avoid calling every memory symptom a leak before evidence exists.

HINTS

- Start by asking what changed: owner count, queue depth, or allocator behavior.
- A useful first measurement is the one that rules out the most wrong theories quickly.

EXERCISE 2 CODE READING

Find clone pressure in a sample workload

OBJECTIVE

Read a request path and identify where deep clones, not Arc clones, are driving memory growth.

STARTER PROMPT

A route filter clones owned strings into an output list, then later formats another owned label per selected item before queueing work.

- Which clone creates another owner of the same allocation and which clone duplicates underlying bytes?
- Which helper only needed borrowed access?
- Where should ownership become explicit if the next stage truly must outlive the input?
- What local counter would you add before reaching for a whole-process heap profiler?

ACCEPTANCE CRITERIA

- You identify at least one deep-clone site correctly.
- You propose at least one borrowed API repair.
- You mention one local measurement such as clone count or cloned-byte count.

HINTS

- Look for `.to_string()`, `.clone()`, and formatting inside a loop.
- The first fix is often API shape, not allocator choice.

EXERCISE 3 IMPLEMENTATION

Instrument clone pressure on purpose

OBJECTIVE

Add a narrow local measurement before moving to a heavier heap profiler.

STARTER PROMPT

Write a small helper that counts how many selected routes are cloned and how many bytes those clones represent.

- Count clones separately from selected outputs.
- Track bytes from the borrowed source length before conversion.
- Keep the instrumentation local to the hot path instead of turning it into a global abstraction.
- Explain when you would delete this instrumentation after the incident is resolved.

ACCEPTANCE CRITERIA

- You count at least clone events and cloned bytes explicitly.
- The measurement stays local and workload-focused.
- You explain why this is a comparative tool rather than a complete memory profile.

HINTS

- A small counter is often enough to prove the next refactor target.
- The important question is what changed across the refactor or incident window.

EXERCISE 4 DEBUGGING OR REFACTORING

Diagnose an Rc cycle leak

OBJECTIVE

Repair a graph whose strong edges make collection impossible even though there is no unsafe code.

STARTER PROMPT

A tree stores parent and child edges as strong `Rc<Node>` references and the root never seems to disappear after local work ends.

- Which edge is observational rather than owning?
- What should become `Weak<Node>`?
- What strong-count signal would you inspect before and after the repair?
- Would this design be calmer with arena IDs instead of shared ownership at all?

ACCEPTANCE CRITERIA

- You explain the logical leak precisely in terms of strong ownership count.
- You replace at least one back-edge with `Weak<T>` in the design.
- You mention one alternative such as arena handles when shared ownership is not semantically real.

HINTS

- The bug is usually not 'Rust leaked.' The bug is 'the graph still has owners.'
- Ask which edge truly owns lifetime and which edge only wants lookup.

EXERCISE 5 DEBUGGING OR REFACTORING

Profile async memory usage before calling it a leak

OBJECTIVE

Read an async pipeline and decide whether the real issue is queue growth, too many tasks, large payloads, or an actual retention bug.

STARTER PROMPT

A Tokio service has an unbounded channel, a `spawn` per input item, and a large owned payload crossing several awaits before the final consumer falls behind.

- Which metrics should you collect first: queue depth, task count, payload size, or all three?
- Which boundary wants a semaphore or bounded queue?
- What should become smaller or more borrowed before the spawned task boundary?
- When would you still suspect a true leak after those backlog fixes?

ACCEPTANCE CRITERIA

- You identify at least one boundedness failure in the design.
- You propose a queue or concurrency cap with a reason.
- You mention at least one payload-shape repair and one metric to confirm the change.

HINTS

- A growing backlog keeps memory alive even if the objects are perfectly reachable by design.
- Measure in-flight work before debugging allocator internals.

EXERCISE 6 DESIGN OR PRODUCTION SCENARIO

Prepare a memory profiling checklist for a real service

OBJECTIVE

Build the review checklist another engineer could use during an incident or a regression investigation.

STARTER PROMPT

Create a checklist for a service that accepts requests, parses them, queues owned work, fans out to async tasks, uses an arena during parsing, and keeps a read-mostly cache.

- Which numbers belong in the first page of the investigation: RSS, live bytes, queue depth, task count, clone count, or all of them?
- Which leak candidates should be named explicitly: Rc or Arc cycles, unbounded queues, non-evicting caches, oversized arenas?
- Which OS-specific tools would you choose on Linux, macOS, and Windows?
- Which acceptance condition tells you the incident is actually resolved instead of merely masked?

ACCEPTANCE CRITERIA

- Your checklist includes allocator or heap signals plus workload signals such as queue depth or task count.
- Your checklist explicitly names at least three likely retention sources.
- Your checklist includes at least one Linux, one macOS, and one Windows tool choice.
- Your checklist defines one concrete success condition after the fix.

HINTS

- A checklist is most useful when it forces the team to rule out the wrong theories quickly.
- Include one stop condition such as RSS after burst, queue age, or cloned bytes per request.

Runnable lab

Example 34-L. `clone_pressure_lab.rs` — *Runnable lab · Remove clone pressure from a hot route filter*

```rust
use std::sync::atomic::{AtomicUsize, Ordering};

static CLONES: AtomicUsize = AtomicUsize::new(0);

fn track_clone(value: &str) -> String {
    CLONES.fetch_add(1, Ordering::Relaxed);
    value.to_string()
}

fn hot_routes(routes: &[&str]) -> Vec<String> {
    let mut out = Vec::with_capacity(routes.len());

    for &route in routes {
        if route.starts_with("/api/") {
            out.push(track_clone(route));
```

```
        }
    }

    out
}

fn main() {
    let routes = ["/api/orders", "/health", "/api/users"];
    CLONES.store(0, Ordering::Relaxed);

    let selected = hot_routes(&routes);

    println!("selected = {}", selected.len());
    println!("clones = {}", CLONES.load(Ordering::Relaxed));
}
```

OUTPUT

```
selected = 2
clones = 0
```

Review questions

- Why is a local clone counter often a good first memory-profiling tool?
- What is the practical difference between allocation churn and retained live data?
- Why can RSS stay high after the logical working set falls?
- Why are unbounded queues and task floods often memory bugs even when no object is unreachable?
- What does `Weak<T>` repair that a strong `Rc<T>` or `Arc<T>` back-edge cannot?

CHAPTER 35 · PERFORMANCE PROFILING

Performance Profiling

Exercises, labs, and review questions for this chapter.

Interpret flame graphs, design Criterion benchmarks, separate CPU and waiting bottlenecks, and profile async, WASM, and FFI boundaries deliberately

EXERCISE 1 WARM-UP COMPREHENSION

Match the question to the instrument

OBJECTIVE

Recall how each profiling question maps to the tool that answers it with the least distortion.

STARTER PROMPT

The chapter argues that the whole toolkit fans out from one decision: what kind of answer do you need? For each question below, name the instrument the chapter would reach for and say why the others would distort the answer.

- Which tool answers "is version B actually faster than version A under a fixed workload?"
- Which tool answers "where is compute time concentrating in this running binary?"
- Why is a CPU sampler the wrong first instrument when an async task takes 200ms of wall time but almost no CPU?

ACCEPTANCE CRITERIA

- A benchmark (Criterion) is named for the version-versus-version comparison, and a CPU sampler is named for finding the hot compute path.
- The answer explains that a sampler only sees code that runs often enough to be caught between samples, so it misses time spent enqueued, waiting on a permit, or parked.
- The answer states that for an async wall-time gap the right signal is tracing spans around each transition, not CPU samples.
- Each choice is justified by the question being asked rather than by which tool happens to be open.

HINTS

- The chapter's rule is to walk the decision diagram from the question, not from the tool you already have open.
- Sampling perturbs the program little but only sees frequently-run code; a benchmark compares, it does not explain why.

EXERCISE 2 CODE READING

Trace the hottest-stage reduction

OBJECTIVE

Read the chapter's hottest_stage listing and explain the single reduction that mirrors what a flame graph shows.

STARTER PROMPT

Study the hottest_stage function over &[StageSample], where each StageSample has a name and a micros count. Hand-trace it on the chapter's sample data (parse=180, serialize=620, lock_wait=40) without running it.

- What does the iter().map(...).sum() line compute, and what is its value for the sample data?
- What does max_by_key(|sample| sample.micros) return, and why is .copied().unwrap() safe here?
- State the three values the function returns as a tuple, in order.
- Explain why this max-over-stages reduction is the same question a flame graph answers visually.

ACCEPTANCE CRITERIA

- The total is correctly computed as 840 microseconds and identified as the sum of all stage micros.
- max_by_key is identified as selecting the StageSample with the largest micros, returning serialize at 620.
- The returned tuple is given as ("serialize", 620, 840) in name, hottest-micros, total order.
- The answer connects "reduce a list of timings to one named winner" with "the flame graph points at the widest stack."

HINTS

- max_by_key on a non-empty slice yields Some, so unwrap cannot panic on this input.
- StageSample derives Copy, which is why .copied() turns &StageSample into an owned value cheaply.

EXERCISE 3 IMPLEMENTATION

Report each stage as a share of wall time

OBJECTIVE

Extend the per-stage decomposition into a percentage breakdown so the widest layer is obvious before any tuning.

STARTER PROMPT

Reuse the StageSample idea from the chapter and implement fn share_of_total(samples: &[StageSample]) -> Vec<(&'static str, u64)>, returning each stage name paired with its integer percentage of the summed micros (truncated, not rounded).

- Compute the total micros once, then map each sample to (name, sample.micros * 100 / total).
- Take samples by shared slice &[StageSample]; do not require ownership of a Vec.
- Use integer arithmetic only and multiply before dividing to avoid losing the fraction.
- Decide what your function returns when the slice is empty and document that choice.

ACCEPTANCE CRITERIA

- The signature is exactly fn share_of_total(samples: &[StageSample]) -> Vec<(&'static str, u64)>.
- For parse=180, serialize=620, lock_wait=40 the result is [("parse", 21), ("serialize", 73), ("lock_wait", 4)].
- Percentages are computed as micros * 100 / total using integer math with multiplication before division.
- The empty-slice case is handled without dividing by zero (for example by returning an empty Vec).

HINTS

- 180 * 100 / 840 truncates to 21; doing the divide first would collapse it to 0.
- A single pass for the total and a second map over the slice is enough; no extra allocation per stage is needed beyond the result Vec.

EXERCISE 4 IMPLEMENTATION

Measure fixed per-crossing overhead

OBJECTIVE

Model the chapter's highest-leverage WASM/FFI experiment: subtract a noop call from a real call to isolate the per-crossing cost.

STARTER PROMPT

The chapter says the single best experiment for a boundary is to compare a noop call against the real call; the difference is your fixed per-crossing overhead. Implement fn crossing_overhead(noop_ns: u64, real_ns: u64, calls: u64) -> (u64, u64) returning (per_call_overhead_ns, total_overhead_ns).

- Define per-call overhead as real_ns saturating-subtract noop_ns so a noisy noop above the real call cannot underflow.
- Compute total overhead as per-call overhead multiplied by calls.
- Return both numbers as a (u64, u64) tuple in the documented order.
- Explain, in one sentence in a comment, why a large per-crossing overhead argues for a batched API.

ACCEPTANCE CRITERIA

- The signature is exactly fn crossing_overhead(noop_ns: u64, real_ns: u64, calls: u64) -> (u64, u64).
- For noop_ns=30, real_ns=110, calls=1_000_000 it returns (80, 80_000_000).
- The subtraction uses saturating_sub (or an equivalent guard) so noop_ns greater than real_ns yields 0 rather than wrapping.
- A comment states that high fixed overhead favors crossing once with a batch over tuning the callee.

HINTS

- u64 subtraction wraps on underflow in release builds unless you guard it; saturating_sub returns 0 instead.
- The noop isolates the marshalling and call cost so the difference is the crossing, not the kernel.

EXERCISE 5 DEBUGGING OR REFACTORING

Fix a lock metric that hides contention

OBJECTIVE

Correct a measurement that conflates hold time with wait time, the two halves of lock cost the chapter insists on separating.

STARTER PROMPT

A teammate reports a single lock_cost_us per lock and concludes a hot lock is fine because its number is small. The number is hold time only; wait time across blocked threads is never recorded. Refactor the metric so the two halves are measured separately.

- Replace the single field with a struct carrying hold_us (time the guard is held doing work) and wait_us (time other threads spend blocked).
- Write fn classify(hold_us: u64, wait_us: u64) -> &'static str returning "contention" when wait_us exceeds hold_us and "hold" otherwise.
- Explain why a lock held briefly but acquired constantly shows low hold_us yet high wait_us.
- Note which redesigns the chapter says to try before reaching for lock-free structures.

ACCEPTANCE CRITERIA

- Hold time and wait time are stored and reported as two separate fields rather than one combined number.
- classify returns "contention" for (hold_us=5, wait_us=900) and "hold" for (hold_us=900, wait_us=5).
- The explanation ties high wait_us with low hold_us to a frequently-acquired short critical section.
- The answer names narrowing shared state, shortening critical sections, or giving a subsystem its own owner before going lock-free.

HINTS

- The two halves fail in opposite ways, so a single averaged number erases exactly the signal you need.
- A hot lock is often an ownership problem hiding underneath, not only a synchronization one.

EXERCISE 6 DESIGN OR PRODUCTION SCENARIO

Resolve the p99 regression argument

OBJECTIVE

Design a decompose-first profiling plan that settles a cross-team latency dispute before anyone rewrites code.

STARTER PROMPT

After a release, p99 latency climbed and three engineers each point at a different signal: a CPU flame graph, growing queue depth, and slightly worse serialization timings. Write the plan that splits one slow request's wall time into its layers before any function is rewritten.

- List the layers a slow request's wall time must be split into and the instrument that measures each.
- State the order of operations: decompose into layers, find the widest, then confirm with a sampler or trace.
- Describe one guard against blaming the first recognizable function (for example serialize sitting above an allocation and copy).
- Define the one-question, one-build, one-recorded-workload discipline you will hold the team to.

ACCEPTANCE CRITERIA

- The plan enumerates CPU compute, queue wait, lock blocking, IO, serialization, and boundary-crossing overhead as the layers, each paired with a fitting instrument.
- The sequence is decompose first, point at the widest layer, then confirm the call stack with a sampler or trace, not optimize-on-sight.
- The plan warns that a wide serialize stack may really be the allocation and copy beneath it, and that an IO path's real cost is often queue time, not CPU samples.
- The plan fixes scope to one bounded question, one release build with usable symbols, and one recorded representative workload.

HINTS

- The chapter's core move is decompose, then point at the widest layer; the widest stage is only the first suspect.
- Release build, symbols good enough for stacks, representative input, one bounded question at a time.

Runnable lab

Mirror the chapter's pipeline decomposition: sum the per-layer micros into wall time, find the layer that owns the most time, and report its integer percentage share. Fill in the dominant-layer reduction where marked.

Example 35-L. `dominant_layer_lab.rs` — *Runnable lab · Decompose wall time, then name the dominant layer*

```
#[derive(Debug)]
struct PipelineStats {
    cpu_us: u64,
```

```
    io_wait_us: u64,
    lock_wait_us: u64,
    serialize_us: u64,
}

fn layers(stats: &PipelineStats) -> [(&'static str, u64); 4] {
    [
        ("cpu", stats.cpu_us),
        ("io", stats.io_wait_us),
        ("lock", stats.lock_wait_us),
        ("serialize", stats.serialize_us),
    ]
}

fn wall_time(stats: &PipelineStats) -> u64 {
    layers(stats).iter().map(|(_, value)| value).sum()
}

fn dominant(stats: &PipelineStats) -> (&'static str, u64) {
    // TODO: return the (name, micros) of the layer that owns the most time.
    // Scan layers(stats) and pick the entry with the largest micros value.
    ("none", 0)
}

fn main() {
    let stats = PipelineStats {
        cpu_us: 410,
        io_wait_us: 120,
        lock_wait_us: 90,
        serialize_us: 180,
    };

    let wall = wall_time(&stats);
    let (name, micros) = dominant(&stats);
    let share = micros * 100 / wall;

    println!("dominant = {}", name);
    println!("wall us = {}", wall);
    println!("dominant share = {}", share);
}
```

OUTPUT

```
dominant = cpu
wall us = 800
dominant share = 51
```

Review questions

- Why does choosing a profiling instrument start from the question you need answered rather than from the tool you already have open?
- What does the x-axis of a flame graph represent, and why is reading it as a timeline a mistake?
- What is the tradeoff between sampling and instrumentation, and when does each one mislead you?
- Why can a CPU profiler tell only part of the story for an async task that takes 200ms of wall time but almost no CPU?
- What are the two halves of lock cost that must be measured separately, and how does each one fail differently?

CHAPTER 36 · DISTRIBUTED TASKS PROFILING

Distributed Tasks Profiling

Exercises, labs, and review questions for this chapter.

Design saturation metrics, trace tail-latency incidents, identify retry storms, and capacity-plan distributed workers

EXERCISE 1 WARM-UP COMPREHENSION

Design metrics for worker saturation on purpose

OBJECTIVE

Choose a small metric set that distinguishes healthy throughput from hidden backlog and retry amplification.

STARTER PROMPT

You run three worker pools: fast CPU tasks, slow GPU tasks, and a reducer pool. Decide which counters, gauges, and histograms each pool should expose.

- Which counters describe admission, completion, retries, and duplicate suppression?
- Which gauges describe visible backlog, in-flight work, and oldest message age?
- Which histograms describe queue wait, run time, and end-to-end latency?
- Which labels would create dangerous cardinality if you added them mechanically?

ACCEPTANCE CRITERIA

- You include at least one counter, one gauge, and one histogram family with a clear reason.
- You distinguish worker saturation from queue saturation explicitly.
- You name at least one label you would forbid, such as task ID or trace ID.

HINTS

- A good metric set tells you whether the pain is admission, execution, or replay.
- Low-cardinality aggregate signals belong in metrics. Per-task identity belongs elsewhere.

EXERCISE 2 CODE READING

Trace a tail-latency incident across logs, metrics, and traces

OBJECTIVE

Practice reconstructing one slow path when mean handler time still looks calm.

STARTER PROMPT

Metrics show queue age rising. Handler p50 is flat. A trace shows one task waited 900 ms before claim and only 70 ms in the handler. Logs show the same task retried twice before success.

- Which system layer is actually dominating end-to-end latency here?
- Which metric should have alerted earlier than handler p50?
- Which log fields or trace fields make the retry path attributable to one task?
- What first mitigation would you test before optimizing handler CPU?

ACCEPTANCE CRITERIA

- You identify queue wait or replay, not handler CPU, as the dominant layer.
- You choose at least one metric and one identity field that make the incident explainable.
- You propose one mitigation tied to the real bottleneck, such as worker budget, retry cap, or queue split.

HINTS

- If queue wait is already dominant, a faster handler may not move the p99 enough to matter.
- The most useful answer names one task identity and one queue identity.

EXERCISE 3 IMPLEMENTATION

Implement a retry-storm window summary

OBJECTIVE

Compute worker saturation, retry rate, and a storm flag from one profiling window.

STARTER PROMPT

Implement helpers over `WindowStats` so one profiling sample can report saturation, retry rate, and whether the window already looks storm-like.

- Use worker saturation as busy workers divided by total workers.
- Use retry rate as retried work divided by claimed work.
- Make the storm rule explicit with one saturation threshold and one retry-rate threshold.
- Keep the helper side-effect free so the lab stays easy to test.

ACCEPTANCE CRITERIA

- The helper computes saturation and retry rate correctly.
- The storm rule is visible in code rather than hidden in prose.
- The runnable lab prints the expected saturation, retry rate, and storm flag.

HINTS

- This is a metric and policy drill, not a transport drill.
- The point is to make the threshold logic reviewable.

EXERCISE 4 DEBUGGING OR REFACTORING

Repair a profiling surface that hides queue latency

OBJECTIVE

Add the missing instrumentation and IDs so traces and logs can explain backlog instead of only handler execution.

STARTER PROMPT

A worker system emits handler spans but never records enqueue time, claim time, or the queue name in any structured field.

- Which timestamps belong in the task envelope or trace events?
- Which log fields should appear on both the enqueue and worker side?
- Would you model queue wait as one explicit span or as derived time between two events?
- How would you keep the extra data low-cost enough for the hot path?

ACCEPTANCE CRITERIA

- You add or describe enqueue and claim visibility explicitly.
- You include at least one stable correlation field such as task ID or trace ID.
- You mention one cost-control tactic, such as aggregate metrics plus sampled detailed traces.

HINTS

- You do not need all detail at full volume. You do need enough detail to explain one incident path.
- If the queue name is invisible, multi-queue tail analysis gets much harder.

EXERCISE 5 DEBUGGING OR REFACTORING

Profile a task graph by critical path instead of average stage time

OBJECTIVE

Replace stage-local averages with graph-aware signals that reveal reducer lag and join bottlenecks.

STARTER PROMPT

A DAG pipeline shows acceptable mean time in every worker stage, yet end-to-end time keeps growing as fan-out width increases.

- Which graph signal should you add: critical-path time, frontier size, reducer backlog, or all three?
- Where does queue wait belong in the graph accounting?
- How would you prove a join stage is the real bottleneck rather than one upstream worker?
- What would you sample or trace to keep the graph story attributable?

ACCEPTANCE CRITERIA

- You include at least one graph-aware signal beyond per-stage averages.
- You keep queue wait inside the graph story rather than outside it.
- You propose one attribution field or trace pattern for fan-out and fan-in nodes.

HINTS

- A graph can be locally healthy and globally slow at the same time.
- The critical path is the first honest simplification.

EXERCISE 6 DESIGN OR PRODUCTION SCENARIO

Capacity-plan a distributed worker fleet with retries and slow-stage isolation

OBJECTIVE

Choose queue splits, worker budgets, and observability targets from arrival rate, service time, and failure profile.

STARTER PROMPT

You are designing `ingest -> parse -> enrich -> store -> notify`, with 180 task submissions per second, one long-running specialist stage, and a requirement that oldest visible age stay under 2 seconds in steady state.

- Which stage deserves its own queue or worker pool instead of sharing one global budget?
- How would you include retry traffic in the worker estimate?
- Which metrics would tell you that the 2-second oldest-age budget is about to fail?
- What drain-time or shutdown goal belongs in the same capacity plan?

ACCEPTANCE CRITERIA

- You define at least one queue or worker split deliberately.
- You include retries in the capacity story instead of treating them as rare noise.
- You mention at least three observability hooks such as oldest visible age, busy workers, retry rate, or reducer backlog.
- You include one shutdown or drain-time target alongside steady-state throughput.

HINTS

- Capacity planning without replay traffic usually underestimates the real budget.
- Oldest visible age is often a better operational target than average queue depth alone.

Runnable lab

Example 36-L. `retry_storm_window_lab.rs` — *Runnable lab · Retry-storm window summary*

```rust
#[derive(Debug, Clone, Copy)]
struct WindowStats {
    claimed: u32,
    retried: u32,
    busy_workers: u32,
    total_workers: u32,
}

fn saturation(_stats: &WindowStats) -> f64 {
    0.0
}

fn retry_rate(_stats: &WindowStats) -> f64 {
    0.0
}

fn retry_storm(_stats: &WindowStats) -> bool {
    false
}

fn main() {
    let stats = WindowStats {
        claimed: 100,
        retried: 40,
        busy_workers: 9,
        total_workers: 10,
    };

    println!("saturation = {:.2}", saturation(&stats));
    println!("retry rate = {:.2}", retry_rate(&stats));
    println!("storm = {}", retry_storm(&stats));
}
```

OUTPUT

```
saturation = 0.90
retry rate = 0.40
storm = true
```

Review questions

- Why is queue latency often a better first incident signal than mean handler time?
- What makes a retry storm a load-amplification problem instead of only an error-rate problem?
- Why should task ID and trace ID stay out of metric labels but stay present in logs and traces?
- What does critical-path time explain that stage-local averages do not explain?
- Why should capacity planning include drain time and retry traffic, not only steady-state throughput?

CHAPTER 37 · CUDA AND GPU ACCELERATION

CUDA and GPU Acceleration

Exercises, labs, and review questions for this chapter.

Estimate transfer cost, sketch a safe kernel wrapper, and choose CPU, Rayon, MPI, or CUDA from workload shape

EXERCISE 1 WARM-UP COMPREHENSION

Choose CPU, Rayon, MPI, or CUDA from the workload

OBJECTIVE

Practice selecting the execution model that matches the dominant cost instead of treating acceleration tools as interchangeable.

STARTER PROMPT

Classify four workloads: a tiny request-local transform over one 4 KB buffer, a one-node dense batch score over host memory, a multi-node domain-decomposed solver, and a dense batched tensor operation with high arithmetic intensity.

- Which workload should stay on CPU because transfer and launch would dominate?
- Which workload wants Rayon because the data already lives on one node?
- Which workload wants MPI because the dominant partition is already process-level?
- Which workload wants CUDA because dense math can amortize the host-device boundary?

ACCEPTANCE CRITERIA

- You choose at least one workload for CPU, one for Rayon, one for MPI, and one for CUDA.
- You justify the GPU case with transfer or launch amortization, not only with the word 'parallel'.
- You keep CPU-bound and waiting-bound questions separate.

HINTS

- Start with arithmetic intensity and batch size.
- Then ask whether the real boundary is host memory, one node, or many nodes.

EXERCISE 2 CODE READING

Estimate transfer cost before discussing the kernel

OBJECTIVE

Translate one matrix job into bytes on the bus before you start tuning arithmetic.

STARTER PROMPT

A service offloads one `f32` matrix multiply with A, B, and C each sized 4096 by 4096, transferring A and B to the device and C back once.

- How many bytes does one 4096 by 4096 `f32` matrix occupy?
- What is the total HtoD plus DtoH traffic for the whole call?
- Why is this boundary less painful for GEMM than for tiny vector add?

ACCEPTANCE CRITERIA

- You compute or approximate the total traffic at about 201,326,592 bytes, or about 192 MiB.
- You explain why dense high-intensity math amortizes the boundary better than tiny low-intensity work.
- You still name launch or synchronization overhead as a separate cost category.

HINTS

- A matrix has `rows * cols` elements and each `f32` is 4 bytes.
- The point is not exact decimal formatting. The point is whether the workload can pay for the transfer.

EXERCISE 3 IMPLEMENTATION

Sketch a safe Rust wrapper around a launch boundary

OBJECTIVE

Build the host-side wrapper that validates shape and launch config before one small unsafe region.

STARTER PROMPT

Implement a small `build_config` and `validate_buffers` pair, then call one `unsafe fn raw_launch(...)` only after those checks succeed.

- Reject mismatched buffer lengths.
- Reject zero-length launches and zero threads per block.
- Return a Rust `Result` from the checked boundary.
- Keep the unsafe region smaller than the validation logic around it.

ACCEPTANCE CRITERIA

- The wrapper checks buffer shape before the unsafe call.
- The wrapper computes block count from length and threads per block.
- The wrapper returns a Rust `Result` rather than a vague boolean.
- The runnable lab prints the expected blocks, threads, and success flag.

HINTS

- This is the same wrapper discipline used around FFI and unsafe elsewhere in the book.
- The GPU boundary is not special here. The invariants still belong in the safe wrapper.

EXERCISE 4 DEBUGGING OR REFACTORING

Repair a tiny-kernel service path that lost to the CPU

OBJECTIVE

Refactor a design that launches too often, copies too much, and synchronizes too eagerly.

STARTER PROMPT

A service copies one small vector to the device, launches one kernel, waits immediately, copies the result back, and repeats that whole cycle per request item.

- Which fix should come first: batch work, fuse kernels, reuse device buffers, or keep the path on CPU?
- What signal would tell you the path is launch-bound rather than kernel-bound?
- When is the best repair to not use the GPU at all for this route?

ACCEPTANCE CRITERIA

- You identify at least one batching or fusion repair and one CPU fallback condition.
- You explain the path in terms of transfer, launch, and synchronization overhead rather than only 'GPU is faster'.
- You mention at least one profiling signal such as launch count, queue wait, or transfer time.

HINTS

- The simplest win is often fewer launches.
- The second simplest win is not offloading work that is too small to amortize the trip.

EXERCISE 5 DEBUGGING OR REFACTORING

Profile a CUDA workload before rewriting the kernel

OBJECTIVE

Separate transfer-bound, launch-bound, sync-bound, and kernel-bound cases clearly.

STARTER PROMPT

A profiling report says kernel time is 2 ms, host-device copies are 11 ms, queue wait is 6 ms, and total wall time is 24 ms for one batch scorer.

- Which category dominates the first diagnosis?
- Which rewrite is probably premature because the report points elsewhere?
- What next measurement would you take inside the dominant category?

ACCEPTANCE CRITERIA

- You identify transfer or admission cost, not kernel math, as the dominant issue.
- You reject at least one likely but wrong optimization target such as low-level kernel tuning first.
- You propose one next measurement and one validation metric after the fix.

HINTS

- If copies and queue wait dominate, shaving kernel arithmetic is not the first win.
- Wall time should drive the next question first.

EXERCISE 6 DESIGN OR PRODUCTION SCENARIO

Integrate a GPU worker into an async and distributed service

OBJECTIVE

Make queue budget, retry policy, and device ownership explicit when the GPU becomes a shared specialist subsystem.

STARTER PROMPT

You are designing `ingest -> parse -> score on GPU -> persist -> notify`, with bursty traffic, one device per host, and a requirement that duplicate replay stay harmless after crash recovery.

- Where should the bounded GPU queue sit, and what does it own?
- When should the async side hand over owned data to the GPU worker?
- How will you separate transient GPU retry from terminal failure or CPU fallback?
- What metrics would you require before rollout?

ACCEPTANCE CRITERIA

- You define one explicit bounded GPU admission point.
- You move owned payloads across the async-to-GPU boundary rather than borrowed request-local views.
- You describe one idempotency or duplicate-safe completion rule for replay after crash or lease expiry.
- You mention at least three observability hooks such as queue age, transfer bytes, launch count, or fallback rate.

HINTS

- Treat the GPU like a scarce worker pool, not like a free helper thread.
- The cleanest design makes queueing, retries, and replay policy visible before the first incident.

Runnable lab

Real CUDA from Rust via cudarc + NVRTC — a GPU matrix-multiply timed against a fair CPU baseline. Build and run it with the GPU Docker project in `public/cuda/`: `docker compose run --rm cuda matmul_challenge`. The harness (device setup, CPU baseline, timing, correctness check) is complete; your job is the kernel's inner dot product. Until you write it the kernel returns zeros and `correct` prints false. Exact times vary by GPU; the speedup and the kernel-vs-end-to-end gap are the point.

Example 37-L. `matmul_challenge.rs` — *Runnable lab · Real CUDA matmul from Rust — measure the GPU speedup*

```
//! Chapter 37 — real CUDA from Rust (CHALLENGE / starter).
//!
//! The whole harness is here — device setup, the CPU baseline, timing, and the
//! correctness check. Two things are left for you:
//!   1. Fill in the kernel's inner dot product (marked TODO below).
//!   2. (optional) Confirm the launch grid covers every output element.
//!
//! As shipped, the kernel writes zeros, so `correct` prints `false`. Implement
//! the dot product and it flips to `true` — and you get the speedup for free.
//!
//! Run:  cargo run --release --bin matmul_challenge [N]      (default N = 1024)

use cudarc::driver::{CudaDevice, LaunchAsync, LaunchConfig};
use cudarc::nvrtc::compile_ptx;
use std::time::Instant;

const KERNEL: &str = r#"
extern "C" __global__ void matmul(const float* A, const float* B, float* C, int N) {
    int row = blockIdx.y * blockDim.y + threadIdx.y;
    int col = blockIdx.x * blockDim.x + threadIdx.x;
    if (row < N && col < N) {
        // TODO: accumulate the dot product of row `row` of A with column `col`
        // of B over k = 0..N, then store it in C[row * N + col].
        // Remember A and B are row-major: A[row][k] = A[row*N + k],
        // B[k][col] = B[k*N + col].
        C[row * N + col] = 0.0f;
    }
}
"#;

fn cpu_matmul(a: &[f32], b: &[f32], n: usize) -> Vec<f32> {
    let mut c = vec![0.0f32; n * n];
    for i in 0..n {
        for k in 0..n {
            let aik = a[i * n + k];
            let brow = &b[k * n..k * n + n];
            let crow = &mut c[i * n..i * n + n];
            for j in 0..n {
                crow[j] += aik * brow[j];
            }
        }
    }
    c
```

```
}

fn main() -> Result<(), Box<dyn std::error::Error>> {
    let n: usize = std::env::args().nth(1).and_then(|s| s.parse().ok()).unwrap_or(1024);
    let a: Vec<f32> = (0..n * n).map(|i| ((i % 13) as f32) * 0.5).collect();
    let b: Vec<f32> = (0..n * n).map(|i| ((i % 7) as f32) * 0.25).collect();

    let t = Instant::now();
    let c_cpu = cpu_matmul(&a, &b, n);
    let cpu_ms = t.elapsed().as_secs_f64() * 1e3;

    let dev = CudaDevice::new(0)?;
    dev.load_ptx(compile_ptx(KERNEL)?, "matmul_mod", &["matmul"])?;
    let func = dev.get_func("matmul_mod", "matmul").unwrap();

    let block = 16u32;
    let grid = (n as u32).div_ceil(block);
    let cfg = LaunchConfig {
        grid_dim: (grid, grid, 1),
        block_dim: (block, block, 1),
        shared_mem_bytes: 0,
    };

    let t_total = Instant::now();
    let a_d = dev.htod_copy(a)?;
    let b_d = dev.htod_copy(b)?;
    let mut c_d = dev.alloc_zeros::<f32>(n * n)?;
    dev.synchronize()?;

    let t_kernel = Instant::now();
    unsafe { func.launch(cfg, (&a_d, &b_d, &mut c_d, n as i32))?; }
    dev.synchronize()?;
    let kernel_ms = t_kernel.elapsed().as_secs_f64() * 1e3;

    let c_gpu = dev.dtoh_sync_copy(&c_d)?;
    let total_ms = t_total.elapsed().as_secs_f64() * 1e3;

    let (mut max_abs, mut max_val) = (0.0f32, 1.0f32);
    for (x, y) in c_cpu.iter().zip(c_gpu.iter()) {
        max_abs = max_abs.max((x - y).abs());
        max_val = max_val.max(x.abs());
    }
    let correct = (max_abs / max_val) < 1e-3;

    println!("device     = {}", dev.name()?);
    println!("matrix     = {n} x {n} (f32)");
    println!("cpu        = {cpu_ms:.1} ms");
    println!("gpu kernel = {kernel_ms:.2} ms");
    println!("gpu total  = {total_ms:.1} ms  (incl. host<->device copies)");
    println!("speedup    = {:.1}x kernel, {:.1}x end-to-end", cpu_ms / kernel_ms, cpu_ms / total_ms);
    println!("correct    = {correct}");
    if !correct {
        println!("note       = kernel still returns zeros — implement the TODO dot product");
    }
    Ok(())
}
```

OUTPUT

```
device     = NVIDIA GeForce RTX 3080 Laptop GPU
matrix     = 1024 x 1024 (f32)
cpu        = 116.3 ms
gpu kernel = 2.35 ms
gpu total  = 12.7 ms  (incl. host<->device copies)
speedup    = 49.4x kernel, 9.1x end-to-end
correct    = true
```

Review questions

- What kinds of workloads actually amortize host-device transfer and launch cost well?
- Why is a safe Rust wrapper around a kernel launch mostly an ownership and invariant story?
- What is the practical difference between host memory ownership and device memory ownership?
- Why are queue wait and launch count often more useful than one isolated kernel timing?
- When is CUDA the wrong choice even when a kernel itself is efficient?

CHAPTER 38 · MERKLE TREE GAMES AND CHALLENGES

Merkle Tree Games and Challenges

Exercises, labs, and review questions for this chapter.

Implement a Merkle tree API, verify inclusion proofs, and design parallel, tamper-evident, and distributed verification systems

EXERCISE 1 WARM-UP COMPREHENSION

State the commitment policy before you state the code

OBJECTIVE

Practice turning Merkle tree folklore into one explicit contract another team can reproduce.

STARTER PROMPT

Write down the policy for one Merkle system over chunked file data: leaf encoding, node encoding, odd-leaf handling, and what the proof envelope must carry.

- Which bytes are hashed at the leaf boundary?
- How are internal nodes domain-separated from leaves?
- What happens when a level has an odd number of hashes?
- Which proof fields must be explicit for another process to verify the path correctly?

ACCEPTANCE CRITERIA

- You define leaf and internal-node encoding separately.
- You pick and justify one odd-leaf policy explicitly.
- You include at least leaf index, leaf count, and sibling orientation in the proof contract.

HINTS

- A Merkle proof is reproducible only if the policy is reproducible.
- Two locally correct implementations can still disagree if the policy is underspecified.

EXERCISE 2 CODE READING

Read a pointer tree and explain why flat levels are calmer

OBJECTIVE

Compare a pointer-rich node tree with a flat level representation from an ownership and operations perspective.

STARTER PROMPT

You inherit one design with `Rc<Node>` parent and child pointers and another design with `Vec<Vec<Hash>>` plus proof steps as value types.

- Which design is easier to serialize or send across a queue?
- Which design is easier to parallelize level by level?
- Which design is easier to make persistent through copy-on-write snapshots?
- Which design carries less incidental ownership machinery for the same proof workload?

ACCEPTANCE CRITERIA

- You explain why Merkle operations usually do not need a pointer graph at all.
- You identify at least one win for flat levels in serialization or parallelism.
- You mention one tradeoff if a system later needs richer navigation than proof generation and verification alone.

HINTS

- The question is not whether pointers can work. It is whether they help this workload.
- Merkle trees are often commitment layers first and navigation trees second.

EXERCISE 3 IMPLEMENTATION

Implement a basic Merkle tree API

OBJECTIVE

Build a small owner that constructs levels from leaves and exposes a root plus proof generation.

STARTER PROMPT

Implement a `MerkleTree` over `&str` leaves with `from_leaves`, `root`, and `proof(index)` using flat per-level storage.

- Keep the owner as flat vectors of hashes.
- Make odd-leaf handling explicit in the build loop.
- Keep proof steps as small value types rather than borrowed references into the tree.
- Do not use `Rc`, `RefCell`, or raw pointers.

ACCEPTANCE CRITERIA

- The tree owns its hashed levels directly.
- Proof generation returns a deterministic sibling path.
- The design stays easy to serialize and easy to move.
- The chapter lab or your local run demonstrates a valid proof for one leaf.

HINTS

- A level builder over `chunks(2)` is enough for the core logic.
- Proof generation is just sibling selection plus parent index movement.

EXERCISE 4 DEBUGGING OR REFACTORING

Repair a proof verifier that forgot orientation

OBJECTIVE

Fix one of the most common practical proof bugs: sibling value without sibling position.

STARTER PROMPT

You inherit `verify_proof` that hashes sibling values in one fixed order and ignores whether the sibling was left or right of the current hash.

- Why is sibling order part of the proof contract?
- Which branch should hash `sibling || acc` and which should hash `acc || sibling`?
- What test vector would catch this bug immediately?
- How would canonical serialization mistakes create the same kind of disagreement?

ACCEPTANCE CRITERIA

- You explain the orientation bug concretely.
- You repair the verifier with an explicit left-or-right branch.
- You mention one small deterministic test case that proves the repair.

HINTS

- A sibling value alone is not enough information.
- The smallest four-leaf tree is usually the best proof bug trap.

EXERCISE 5 DEBUGGING OR REFACTORING

Parallelize the wide levels, not the whole idea blindly

OBJECTIVE

Choose where parallelism helps and where it only adds scheduler cost.

STARTER PROMPT

You want faster tree construction for large batches. Sketch a level-wise parallel build with Rayon or another worker strategy, then decide when the build should fall back to serial work near the root.

- What is the independent unit of work within one level?
- What threshold would stop parallelism on tiny upper levels?
- Would the parallel path still own the same flat data structure?
- What metric would confirm the parallel build actually helps the target workload?

ACCEPTANCE CRITERIA

- You parallelize sibling-pair hashing within a level, not arbitrary cross-level dependencies.
- You define one threshold or condition for switching back to serial work.
- You keep the ownership model compatible with the serial version.
- You mention one measurement such as build latency, CPU utilization, or batch throughput.

HINTS

- Level-by-level parallelism is the calm default because the dependency graph is already staged for you.
- Parallel work that becomes smaller every round usually wants a cutoff.

EXERCISE 6 DESIGN OR PRODUCTION SCENARIO

Choose challenge tracks for content-addressed storage, game integrity, and distributed verification

OBJECTIVE

Turn the chapter topics into three production-shaped challenge designs with explicit budgets and replay policy.

STARTER PROMPT

Design three systems: a chunk store keyed by hash, a tamper-evident game checkpoint feed, and a verifier service that checks remote proofs before downloading full payloads.

- Which system needs proof envelopes versioned across services?
- Which system needs snapshot roots durable enough for replay or dispute resolution?
- Which system wants batch verification or bounded worker pools?
- Which failure and observability signals would you require before rollout?

ACCEPTANCE CRITERIA

- You describe all three challenge systems with distinct goals and boundaries.
- You define one ownership or durability boundary for each system.
- You mention at least one batching, retry, or queue budget where the workload justifies it.
- You include at least one observability hook such as proof-failure rate, oldest queue age, or checkpoint publication lag.

HINTS

- The three challenge tracks are deliberately different so you do not solve them all with one slogan.
- A strong answer keeps commitment policy, storage policy, and execution policy separate.

Runnable lab

Example 38-L. `merkle_proof_verify_lab.rs` — *Runnable lab · Verify one inclusion proof*

```rust
type Hash = u32;

#[derive(Debug, Clone, Copy)]
struct ProofStep {
    sibling: Hash,
    sibling_is_left: bool,
}

fn hash_bytes(tag: u8, bytes: &[u8]) -> Hash {
    let mut hash = 2_166_136_261_u32 ^ tag as u32;

    for &byte in bytes {
        hash ^= byte as u32;
        hash = hash.wrapping_mul(16_777_619);
    }

    hash
}
```

```rust
fn hash_leaf(text: &str) -> Hash {
    hash_bytes(0, text.as_bytes())
}

fn hash_node(left: Hash, right: Hash) -> Hash {
    let mut bytes = [0_u8; 8];
    bytes[..4].copy_from_slice(&left.to_le_bytes());
    bytes[4..].copy_from_slice(&right.to_le_bytes());
    hash_bytes(1, &bytes)
}

fn verify_proof(_leaf: &str, _proof: &[ProofStep], _expected_root: Hash) -> bool {
    false
}

fn main() {
    let alpha = hash_leaf("alpha");
    let beta = hash_leaf("beta");
    let gamma = hash_leaf("gamma");
    let delta = hash_leaf("delta");

    let left = hash_node(alpha, beta);
    let right = hash_node(gamma, delta);
    let root = hash_node(left, right);

    let proof = vec![
        ProofStep {
            sibling: delta,
            sibling_is_left: false,
        },
        ProofStep {
            sibling: left,
            sibling_is_left: true,
        },
    ];

    println!("verified good = {}", verify_proof("gamma", &proof, root));
    println!("verified bad = {}", verify_proof("gxmxa", &proof, root));
}
```

OUTPUT

```
verified good = true
verified bad = false
```

Review questions

- Why is odd-leaf handling a protocol decision rather than a local helper detail?
- Why are flat levels usually calmer than pointer trees for Merkle workloads?
- What makes sibling orientation part of the proof itself?
- When is level-wise parallelism worth the overhead, and when should it stop?
- Why should a proof envelope carry more than only the sibling hashes?

CHAPTER 39 · GRAPH SEARCH GAMES

Graph Search Games

Exercises, labs, and review questions for this chapter.

Implement BFS with stable indices, compare graph representations, and design maze, dependency, and distributed graph-search challenges

EXERCISE 1 WARM-UP COMPREHENSION

Choose adjacency list, matrix, or stable-index arena from the workload

OBJECTIVE

Practice mapping graph workload shape to the representation that keeps ownership and traversal calm.

STARTER PROMPT

Classify four cases: a sparse service dependency graph, a dense all-to-all connectivity matrix for 64 nodes, a maze grid with uniform step cost, and a long-lived mutable route graph that will be serialized and reloaded.

- Which case wants an adjacency list because edges are sparse and iteration dominates?
- Which case justifies a matrix because edge existence lookup is dense and fixed-size?
- Which case wants stable indices because the owner should be one node table rather than many references?
- Which case is better modeled as a graph over grid coordinates rather than as a pointer graph?

ACCEPTANCE CRITERIA

- You choose at least one adjacency-list case and one adjacency-matrix case with a concrete reason.
- You identify stable indices or arena-backed ownership as the calmer default for a mutable serialized graph.
- You explain one representation in terms of memory growth and one in terms of traversal shape.

HINTS

- Ask first whether the graph is sparse or dense.
- Then ask who should own nodes and how long edges need to stay meaningful.

EXERCISE 2 CODE READING

Explain why stable indices beat borrowed neighbors in a growable graph

OBJECTIVE

Read a vector-backed graph design and explain the ownership and reallocation consequences precisely.

STARTER PROMPT

You inherit one graph that stores nodes in `Vec<Node>` and edges as `usize`, and another that stores nodes in `Vec<Node>` but keeps long-lived `&Node` references inside edge lists.

- Which design survives vector growth and later serialization more honestly?
- Why is a long-lived `&Node` inside a growable owner table usually the wrong contract?
- What does the stable-index version give up, and what does it gain?
- Where would a truly shared-owner graph still justify `Rc` or `Arc` instead?

ACCEPTANCE CRITERIA

- You explain why borrowed neighbors into growable storage are the wrong stability model.
- You identify at least one operational win for stable indices, such as mutation calm or serialization ease.
- You name one case where shared ownership could still be semantically real.

HINTS

- The question is not only about memory safety. It is also about which design remains honest after change.
- Stable handles are often easier to queue, trace, and store than references.

EXERCISE 3 IMPLEMENTATION

Implement BFS over an adjacency list with stable indices

OBJECTIVE

Build one small graph owner and traverse it with BFS without pointer-like references between nodes.

STARTER PROMPT

Implement `Graph`, `NodeId`, `bfs_order`, and `shortest_hops` over a `Vec<Node>` plus adjacency lists.

- Keep nodes owned in one vector.
- Use `VecDeque` for the BFS frontier.
- Return stable `NodeId` handles from insertion.
- Do not use `Rc`, `RefCell`, or raw pointers.

ACCEPTANCE CRITERIA

- The graph owner is one `Vec<Node>`.
- Edges store `NodeId` handles rather than references.
- BFS visits nodes in deterministic layer order.
- The runnable lab prints the expected visit order and hop count.

HINTS

- This is the same owner-plus-handle shape used in many ASTs and dependency graphs.
- A visited bitmap or distance vector indexed by `NodeId.0` is usually enough.

EXERCISE 4 DEBUGGING OR REFACTORING

Repair a DFS or cycle-check implementation that over-relies on recursion

OBJECTIVE

Replace a fragile recursive structural walk with a more explicit stack or state-set model when graph depth or cycles make the original risky.

STARTER PROMPT

A service uses recursive DFS for cycle detection on user-supplied dependency graphs and occasionally sees stack depth concerns and confusing re-entry bugs.

- Would an explicit stack improve control over depth and instrumentation?
- Which sets should distinguish 'currently exploring' from 'already finished' if you are detecting cycles?
- How would you report one concrete cycle witness rather than only returning `false`?
- What metric or log line would help operators diagnose a graph that causes unusual depth?

ACCEPTANCE CRITERIA

- You distinguish traversal depth from cycle-state tracking clearly.
- You propose an explicit stack or another bounded-state repair with a reason.
- You include one production-facing diagnostic idea such as cycle witness, depth histogram, or node fan-out summary.

HINTS

- Cycle detection usually needs more than one visited bit.
- An explicit stack is often easier to instrument than deep recursion.

EXERCISE 5 DESIGN OR PRODUCTION SCENARIO

Design a dependency resolver challenge

OBJECTIVE

Turn graph search into a production-shaped dependency solver with explicit cycle handling and stable graph identity.

STARTER PROMPT

You are designing `resolve build units -> produce execution order -> surface one cycle path if no valid order exists`.

- Which graph direction makes the scheduler calm: dependency to dependent or the reverse?
- Which algorithmic pieces belong together: indegree counting, ready frontier, or DFS cycle witness?
- What stable IDs or labels should every node carry for logs and test fixtures?
- What scoring or extension track would you add for version conflicts or optional edges?

ACCEPTANCE CRITERIA

- You define one stable node identity strategy.
- You describe at least one valid ordering path and one cycle-reporting path.
- You include one extension or scoring rule for the challenge track.

HINTS

- A good dependency resolver does more than output a sorted list. It also explains failure.
- Stable IDs make cycle reporting and replay testing much easier.

EXERCISE 6 DESIGN OR PRODUCTION SCENARIO

Design a distributed graph-search challenge with bounded frontier workers

OBJECTIVE

Map graph search to a multi-worker control plane where duplicate suppression, queue age, and trace lineage are first-class.

STARTER PROMPT

You are designing `seed frontier -> expand neighbors -> dedupe visited nodes -> merge frontier -> publish best path so far`, with work split across several worker pools or processes.

- Where does the owned frontier message boundary live?
- How will you stop duplicate discoveries from turning into infinite replay or a retry storm?
- Which queues should be bounded, and what metric should alert first when the frontier gets stuck?
- How would you score Bronze, Silver, and Gold versions of this challenge?

ACCEPTANCE CRITERIA

- You define at least one owned queue boundary and one duplicate-suppression rule.
- You include at least one boundedness rule such as worker concurrency or queue capacity.
- You mention at least two observability hooks such as oldest frontier age, duplicate hit rate, or merge lag.
- You include one scoring or extension-track rule for the challenge.

HINTS

- Distributed graph search is partly an algorithm problem and partly a control-plane problem.
- Frontier size and duplicate rate are often more useful than raw CPU usage at first.

Runnable lab

Example 39-L. `graph_bfs_lab.rs` — *Runnable lab · BFS over an adjacency list with stable indices*

```rust
use std::collections::VecDeque;

#[derive(Clone, Copy, Debug, PartialEq, Eq, Hash)]
struct NodeId(usize);

#[derive(Debug)]
struct Node {
    name: &'static str,
    edges: Vec<NodeId>,
}

#[derive(Default, Debug)]
struct Graph {
    nodes: Vec<Node>,
}

impl Graph {
    fn add_node(&mut self, name: &'static str) -> NodeId {
        let id = NodeId(self.nodes.len());
        self.nodes.push(Node {
            name,
            edges: Vec::new(),
        });
```

```
        id
    }

    fn add_edge(&mut self, from: NodeId, to: NodeId) {
        self.nodes[from.0].edges.push(to);
    }

    fn names(&self, ids: &[NodeId]) -> Vec<&'static str> {
        ids.iter().map(|id| self.nodes[id.0].name).collect()
    }

    fn bfs_order(&self, _start: NodeId) -> Vec<NodeId> {
        Vec::new()
    }

    fn shortest_hops(&self, _start: NodeId, _goal: NodeId) -> Option<usize> {
        None
    }
}

fn main() {
    let mut graph = Graph::default();

    let api = graph.add_node("api");
    let auth = graph.add_node("auth");
    let billing = graph.add_node("billing");
    let search = graph.add_node("search");

    graph.add_edge(api, auth);
    graph.add_edge(api, billing);
    graph.add_edge(auth, search);
    graph.add_edge(billing, search);

    let order = graph.bfs_order(api);
    let hops = graph.shortest_hops(api, search).unwrap_or(0);

    println!("visited = {}", graph.names(&order).join(","));
    println!("hops = {}", hops);
}
```

OUTPUT

```
visited = api,auth,billing,search
hops = 2
```

Review questions

- Why are stable indices usually calmer than borrowed references in graph owners that grow or serialize?
- When is BFS the correct answer even if Dijkstra or A* looks more advanced?
- What is the difference between an admissible and a consistent (monotone) heuristic for A*, and which one does the standard closed-set version need to avoid re-opening nodes?
- Why can frontier parallelism still fail if the visited-set or merge step is one hot lock?
- What extra operational questions appear the moment graph search crosses process or queue boundaries?

CHAPTER 40 · MATRIX OPTIMIZATION GAMES

Matrix Optimization Games

Exercises, labs, and review questions for this chapter.

Implement naive and tiled multiplication variants, choose sparse or dense representations, and design a fair CPU vs GPU benchmark tournament

EXERCISE 1 WARM-UP COMPREHENSION

State the optimization order and why it holds

OBJECTIVE

Recall the chapter's decision pipeline and explain why each stage assumes the earlier one is settled.

STARTER PROMPT

The chapter argues that matrix optimization has a fixed decision order and that the common mistake is reaching for the most exciting lever first. Reconstruct that order and defend it in your own words.

- List the stages of the optimization pipeline from cheapest-to-get-right to most expensive, ending at GPU offload.
- Explain why a clever SIMD kernel or thread pool cannot rescue a bad storage layout or loop order.
- Give one sentence on why the GPU stage is reached last and least often.

ACCEPTANCE CRITERIA

- The stated order begins with storage layout and loop order before SIMD, tiling, parallelism, and GPU offload.
- The answer explains that later stages assume the earlier ones are already honest, so layout and loop order come first.
- The answer notes that GPU offload is rarely needed and should be entered only after the layout question is answered.

HINTS

- The chapter frames this as a left-to-right pipeline where each later stage assumes the earlier ones.
- The repeated mistake is jumping to the accelerator review before the layout review.

EXERCISE 2 CODE READING

Trace the CSR frontier step

OBJECTIVE

Read the CSR data model and hand-execute one sparse matrix-vector frontier expansion.

STARTER PROMPT

Using the chapter's CsrMatrix with indptr = [0, 2, 3, 5, 6, 6], indices = [1, 2, 3, 3, 4, 4], and data all 1.0, hand-trace advance_frontier for the input frontier [1.0, 0.0, 1.0, 0.0, 0.0].

- For each active row, compute the slice indptr[row]..indptr[row + 1] and name the columns it reaches.
- Mark which entries of the output next vector become 1, and explain why inactive rows are skipped entirely.
- State what graph.nnz() and graph.dense_bytes() report and what that contrast is meant to show.

ACCEPTANCE CRITERIA

- Row 0 expands over indptr[0]..indptr[2] reaching columns 1 and 2; row 2 expands over indptr[2]..indptr[4] reaching columns 3 and 4.
- The resulting next frontier is 0,1,1,1,1 and rows 1, 3, 4 are skipped because their frontier value is 0.0.
- nnz is 6 (the length of data) and dense_bytes is rows * cols * 4 = 100, illustrating the metadata-versus-density tradeoff.

HINTS

- Only rows whose frontier value is nonzero contribute; the rest hit the continue.
- indptr[row]..indptr[row + 1] is the half-open range of edge slots for that row.

EXERCISE 3 IMPLEMENTATION

Reorder the multiply loops from i-j-k to i-k-j

OBJECTIVE

Rewrite the inner loop so the innermost index walks a contiguous row of a row-major right-hand matrix.

STARTER PROMPT

Starting from the chapter's matmul_naive triple loop (i, then j, then k), implement matmul_ikj that produces an identical result but nests the loops as i, then k, then j.

- Hoist a.get(i, k) out of the innermost loop into a local before the j loop runs.
- Make the innermost loop iterate j and accumulate into out[i, j] using b.get(k, j).
- Confirm the result equals matmul_naive on the chapter's 3x3 inputs using approx_eq.

ACCEPTANCE CRITERIA

- The loop nest is ordered i, k, j with j innermost, and a.get(i, k) is read once per (i, k) rather than once per (i, j, k).
- The innermost loop indexes b.get(k, j), so it strides one element at a time along a row of B rather than jumping across rows.
- matmul_ikj returns a matrix that approx_eq reports equal to matmul_naive for the same operands.

HINTS

- Same arithmetic, different memory walk: only the loop order and the hoist change.
- With i-k-j the inner loop touches consecutive offsets in row-major storage.

EXERCISE 4 IMPLEMENTATION

Build a CSR matrix from a dense grid

OBJECTIVE

Construct the indptr, indices, and data arrays of a CsrMatrix by scanning a dense row-major buffer.

STARTER PROMPT

Write fn dense_to_csr(rows: usize, cols: usize, dense: &[f32]) -> CsrMatrix that records only the nonzero cells, producing indptr, indices, and data consistent with the chapter's CSR layout.

- Walk the dense buffer row by row using the offset = row * cols + col formula.
- Push each nonzero column index and value, and push a running edge count into indptr after each row.
- Verify that indptr has rows + 1 entries and that the last entry equals data.len().

ACCEPTANCE CRITERIA

- indptr starts with 0, has exactly rows + 1 entries, and its final entry equals indices.len() and data.len().
- indices and data contain exactly the nonzero columns and values in row-major scan order.
- Feeding the result into advance_frontier reproduces the same reachable columns as scanning the dense grid directly.

HINTS

- indptr[row + 1] is just indptr[row] plus the number of nonzeros found in that row.
- A cell is stored only when dense[row * cols + col] != 0.0.

EXERCISE 5 DEBUGGING OR REFACTORING

Fix a tiled multiply that overwrites partial sums

OBJECTIVE

Diagnose and correct a blocking bug where a tile loop assigns instead of accumulating into the output cell.

STARTER PROMPT

A colleague's matmul_tiled mirrors the chapter's ii / kk / jj structure but the inner statement reads out.set(i, j, a_ik * b.get(k, j)) instead of adding to the current value. The result disagrees with matmul_naive whenever a.cols exceeds the tile size.

- Explain why the bug only appears when the k dimension spans more than one tile.
- Restore the accumulation by reading out.get(i, j) and adding a_ik * b.get(k, j) before storing.
- Describe a check that would have caught this regression early.

ACCEPTANCE CRITERIA

- The diagnosis names that successive kk tiles overwrite earlier partial sums because set replaces rather than adds.
- The corrected inner statement is out.set(i, j, out.get(i, j) + a_ik * b.get(k, j)).
- The answer proposes comparing against matmul_naive via approx_eq or a checksum on inputs whose k dimension exceeds the tile.

HINTS

- Each kk tile contributes only part of the dot product for out[i, j].
- The output must start at zero and be added to across all kk tiles, never reset.

EXERCISE 6 DESIGN OR PRODUCTION SCENARIO

Decide the GPU break-even for a scoring service

OBJECTIVE

Apply the chapter's offload cost model to justify a layout-and-placement decision with service evidence.

STARTER PROMPT

A batch scoring service's hot path is a dense matrix multiply. A research spike proposes moving it to a GPU where the kernel measures 2 ms, but the offload wrapper adds device wait, host-to-device copy, launch, and device-to-host copy. Recommend an order of work and a break-even argument.

- List the end-to-end cost components of a GPU offload and which ones the 2 ms kernel figure ignores.
- State which optimizations you would settle on the host first and why, in the chapter's order.
- Explain why the break-even size is a service fact rather than a whiteboard fact.

ACCEPTANCE CRITERIA

- The end-to-end cost is given as device wait plus input copy plus launch plus kernel plus result copy, not just the 2 ms kernel.
- The plan fixes storage layout and loop order, then tiling and CPU parallelism, before considering offload.
- The recommendation ties the break-even to measured queueing, admission, and transfer costs for this specific service rather than a theoretical threshold.

HINTS

- A 2 ms kernel wrapped in copy and queue time can be slower than a tuned host multiply.
- Offload paths can be admission-bound or transfer-bound regardless of kernel speed.

Runnable lab

Complete the innermost loop of matmul_ikj so it accumulates a_ik * b[k, j] into out[i, j]. The loop nest is already i, k, j, so the inner loop walks a contiguous row of B in row-major storage.

Example 40-L. `ikj_matmul_lab.rs` — *Runnable lab · i-k-j matrix multiply over flat row-major storage*

```
struct Matrix {
    rows: usize,
    cols: usize,
    data: Vec<f32>,
}

impl Matrix {
    fn zeros(rows: usize, cols: usize) -> Self {
        Self { rows, cols, data: vec![0.0; rows * cols] }
    }
    fn offset(&self, row: usize, col: usize) -> usize {
        row * self.cols + col
    }
    fn get(&self, row: usize, col: usize) -> f32 {
```

```
        self.data[self.offset(row, col)]
    }
    fn set(&mut self, row: usize, col: usize, value: f32) {
        let index = self.offset(row, col);
        self.data[index] = value;
    }
}

// i-k-j order: the inner loop walks a row of B, not a column.
fn matmul_ikj(a: &Matrix, b: &Matrix) -> Matrix {
    let mut out = Matrix::zeros(a.rows, b.cols);
    for i in 0..a.rows {
        for k in 0..a.cols {
            let a_ik = a.get(i, k);
            for j in 0..b.cols {
                // TODO: accumulate a_ik * b[k, j] into out[i, j].
                // Read out.get(i, j), add a_ik * b.get(k, j), then out.set it back.
                let _ = (a_ik, j);
            }
        }
    }
    out
}

fn checksum(m: &Matrix) -> f32 {
    m.data.iter().copied().sum()
}

fn main() {
    let a = Matrix { rows: 2, cols: 2, data: vec![1.0, 2.0, 3.0, 4.0] };
    let b = Matrix { rows: 2, cols: 2, data: vec![5.0, 6.0, 7.0, 8.0] };
    let c = matmul_ikj(&a, &b);
    println!("c[0,0] = {:.1}", c.get(0, 0));
    println!("c[1,1] = {:.1}", c.get(1, 1));
    println!("checksum = {:.1}", checksum(&c));
}
```

OUTPUT

```
c[0,0] = 19.0
c[1,1] = 50.0
checksum = 134.0
```

Review questions

- In row-major storage, what does the offset = row * cols + col formula encode, and which traversal does it make contiguous?
- Why does reordering the multiply loops from i-j-k to i-k-j change cache behavior without changing the arithmetic or the result?
- What does a CSR matrix store beyond the nonzero values, and what does that metadata buy you over a dense layout?
- What does blocking or tiling improve in a matrix multiply, and what costs does it add?
- What are the components of the end-to-end cost of a GPU offload, and why can a fast kernel still lose the placement decision?

PART VI

Engineering Systems for Production

Error strategy at scale, testing, observability, packaging, and a capstone that ties it together.

41 Error Handling in Large Systems
42 Testing Advanced Rust Systems
43 Observability
44 Packaging and Deployment
45 Capstone: A Distributed Rust System

CHAPTER 41 · ERROR HANDLING IN LARGE SYSTEMS

Error Handling in Large Systems

Exercises, labs, and review questions for this chapter.

Convert panic-based logic into typed errors, separate domain and infrastructure failures, and add context to async propagation

EXERCISE 1 WARM-UP COMPREHENSION

Choose Option, Result, panic, or process abort from the caller's ability to act

OBJECTIVE

Practice separating ordinary absence, recoverable failure, and broken invariants before one error type grows into a junk drawer.

STARTER PROMPT

Classify four events: an optional request header is missing, a database call times out, a user attempts an invalid state transition, and an internal invariant says two states cannot coexist but now both do.

- Which event is local absence best modeled as Option first?
- Which events are recoverable and should become Result with a typed error?
- Which event is an invariant break more honestly modeled as panic or crash policy?
- Where would you translate the typed error if the next boundary is HTTP, AMQP, or FFI?

ACCEPTANCE CRITERIA

- You distinguish Option, Result, and panic in operational terms rather than by style.
- You classify at least one domain failure separately from one infrastructure failure.
- You explain one translation step at a boundary instead of assuming the same contract survives everywhere unchanged.

HINTS

- Ask first what the caller can still do next.
- If the process state is broken, the question may no longer be 'how do I return an error?'

EXERCISE 2 CODE READING

Separate domain and infrastructure errors in one service path

OBJECTIVE

Read one mixed error surface and decide which variants belong to the domain layer, which belong to adapters, and which belong to top-level translation.

STARTER PROMPT

A billing service currently returns `anyhow::Result<()>` from deep inside the domain model and mixes `invalid transition`, `SQL timeout`, and `missing queue channel` into one free-form message chain.

- Which failures belong in a domain error enum and why?
- Which failures belong in infrastructure or adapter errors and why?
- Where should anyhow still remain useful after the separation?
- Which caller actions become easier once the surface is typed again?

ACCEPTANCE CRITERIA

- You move at least one business-rule failure out of the infrastructure bucket.
- You keep anyhow or an equivalent aggregated error layer only where pattern matching has stopped being useful.
- You name at least one caller action that typed separation enables, such as retry, reject, or dead-letter.

HINTS

- A typed contract is most useful where the next layer still branches on it.
- A service shell can still aggregate rich context after lower layers stay typed.

EXERCISE 3 IMPLEMENTATION

Convert panic-based parsing into typed errors

OBJECTIVE

Replace unwrap-driven control flow with a small typed contract that distinguishes missing input from invalid input.

STARTER PROMPT

Refactor `parse_port` so it returns `Result<u16, ConfigError>` with separate `MissingPort` and `InvalidPort` variants instead of panicking.

- Keep the error type small and specific.
- Use `ok_or` or an equivalent explicit missing-input branch.
- Use `map_err` or an equivalent parse-failure branch.
- Return the parsed port directly on success instead of logging inside the helper.

ACCEPTANCE CRITERIA

- The parser no longer panics for routine invalid input.
- Missing and malformed input are distinguishable in the return type.
- The runnable lab prints the expected missing, invalid, and success cases.

HINTS

- This is the same contract repair many real config loaders need first.
- A small enum is often enough to make the whole caller path calmer.

EXERCISE 4 DEBUGGING OR REFACTORING

Add context to async task propagation

OBJECTIVE

Repair a spawned async path so join failure, inner failure, and boundary context stay separate and reviewable.

STARTER PROMPT

You inherit `tokio::spawn(async move { do_work(job).await })` followed by `.await?`, but the resulting error text never says whether the task failed to join or the inner operation failed after joining.

- Where should you add context before the spawn boundary and after the await?
- Why is `JoinHandle<Result<T, E>>` a two-layer error surface instead of one?
- Which fields belong in the context message: job ID, queue name, attempt, resource name?
- When should cancellation be modeled distinctly from ordinary inner failure?

ACCEPTANCE CRITERIA

- You describe the join layer and the inner error layer separately.
- You add at least one useful context message at the operation layer and one at the task layer.
- You explain one case where cancellation or shutdown should not be collapsed into a generic timeout string.

HINTS

- A task panic is not the same incident as a typed adapter or domain failure.
- Context should name the operation, not just repeat the same low-level noun again.

EXERCISE 5 DEBUGGING OR REFACTORING

Translate a Rust error contract across FFI without leaking internals

OBJECTIVE

Replace a foreign-facing Rust-shaped error surface with one the other runtime can actually consume and stabilize.

STARTER PROMPT

A native plugin boundary currently exposes `pub extern "C" fn run(ptr: *const u8, len: usize) -> anyhow::Result<u64>`.

- Which public boundary types should replace anyhow::Result here?
- How will null input, invalid UTF-8, and internal failure be distinguished?
- Which error detail should stay inside the Rust-side log or trace instead of escaping directly over the ABI?
- How would you test the status boundary with a tiny native harness?

ACCEPTANCE CRITERIA

- You translate the boundary into status codes, out parameters, or another explicit ABI-safe contract.
- You distinguish at least two foreign-visible failure classes clearly.
- You keep richer Rust internals behind the wrapper instead of leaking them as the public ABI story.

HINTS

- Another runtime cannot safely consume your internal anyhow chain as if it were a stable public type.
- Boring ABI contracts are a feature, not a failure of imagination.

EXERCISE 6 DESIGN OR PRODUCTION SCENARIO

Design one error contract for HTTP, queue workers, and a native plugin

OBJECTIVE

Make retryability, caller action, boundary translation, and logging discipline explicit across several large-system seams at once.

STARTER PROMPT

You are designing `HTTP request -> domain check -> async worker -> database -> native risk plugin -> publish outcome`, and you want another team to operate the failure modes without reverse-engineering your code.

- Which failures should be visible as domain errors, infrastructure errors, and top-level service errors?
- Which failures are retryable, which are terminal, and where should that policy live?
- Which boundary should log the error, and which boundary should only translate and return it?
- What trace or log fields belong on every failure path to make the incident reconstructable later?

ACCEPTANCE CRITERIA

- You define at least one domain error, one infrastructure error, and one translated service or transport contract.
- You make retryable versus terminal classification explicit instead of relying on message text.
- You identify one boundary that logs and another that only translates to avoid duplicate noise.
- You mention at least two observability fields such as request ID, task ID, attempt, queue, or plugin call name.

HINTS

- A large-system error model is easier to operate when retry policy and logging policy are visible in different places on purpose.
- The same failure should not need five log lines before an operator can tell what actually happened.

Runnable lab

Example 41-L. `typed_error_lab.rs` — *Runnable lab · Replace panic-based parsing with typed errors*

```rust
#[derive(Debug, PartialEq, Eq)]
enum ConfigError {
    MissingPort,
    InvalidPort,
}

fn parse_port(raw: Option<&str>) -> Result<u16, ConfigError> {
    let text = raw.unwrap();
    Ok(text.parse::<u16>().unwrap())
}

fn main() {
    println!("missing = {:?}", parse_port(None));
    println!("bad = {:?}", parse_port(Some("oops")));
    println!("ok = {:?}", parse_port(Some("8080")));
}
```

OUTPUT

```
missing = Err(MissingPort)
bad = Err(InvalidPort)
ok = Ok(8080)
```

Review questions

- When is Option enough, and when should the boundary promote absence into Result?
- Why are domain errors and infrastructure errors worth keeping distinct even if both eventually become one HTTP response or one queue outcome?
- What is the practical split between thiserror and anyhow in a large Rust system?
- Why does a spawned task usually create a two-layer error surface rather than one?
- Why should a boundary usually log once rather than at every layer that propagates the same error upward?

CHAPTER 42 · TESTING ADVANCED RUST SYSTEMS

Testing Advanced Rust Systems

Exercises, labs, and review questions for this chapter.

Choose the right test layer, design property and fuzz strategies, build async integration plans, and create a production-grade testing matrix

EXERCISE 1 WARM-UP COMPREHENSION

Choose unit, integration, property, fuzz, snapshot, or benchmark from the claim

OBJECTIVE

Practice selecting the cheapest test layer that can prove the real behavior instead of defaulting to one broad test shape.

STARTER PROMPT

Classify five needs: a state-machine invariant, a public HTTP contract, a hostile binary parser input space, a large human-reviewable error report, and a latency budget for one hot batch encoder.

- Which need is best served by a unit test first?
- Which need wants a true integration test against a public boundary?
- Which need wants property testing because one invariant should hold across many generated cases?
- Which need wants fuzzing because the input space is adversarial or malformed by nature?
- Which need wants snapshot or golden review, and which one wants a benchmark budget instead?

ACCEPTANCE CRITERIA

- You map each claim to a plausible primary test layer and justify it with cost and confidence.
- You distinguish snapshot or golden review from semantic assertions clearly.
- You keep performance budgets separate from functional correctness tests.

HINTS

- Start with the claim, not with the tool you already know best.
- A good answer minimizes cost while preserving the right kind of evidence.

EXERCISE 2 CODE READING

Find the missing layers in one test suite

OBJECTIVE

Read a realistic but incomplete suite and explain which claims are still unproven.

STARTER PROMPT

A crate has only slow API integration tests. The domain model has no direct tests, the binary parser has no fuzz target, the async worker has sleep-based tests, and snapshot updates are accepted blindly.

- Which invariants should move into unit or property tests?
- Which parser boundary wants fuzzing instead of only end-to-end fixtures?
- Why are sleep-based async tests brittle here?
- Which snapshot practice is too trusting for a production-facing report format?

ACCEPTANCE CRITERIA

- You identify at least one missing unit or property layer, one missing fuzz layer, and one async determinism problem.
- You propose at least one replacement or repair for each gap.
- You explain why the current suite is expensive and still incomplete.

HINTS

- The goal is not 'more tests'. The goal is the right missing tests.
- If the suite is slow and still vague, the layering is probably wrong.

EXERCISE 3 IMPLEMENTATION

Write property tests for a domain invariant

OBJECTIVE

Design a property test around one real state invariant instead of only enumerating a few hand-picked examples.

STARTER PROMPT

Take a small domain type such as a quota window, account ledger, or reservation tracker and write a property asserting the state never exceeds its declared limit after any generated sequence of operations.

- State the invariant before you state the generator.
- Use generated operations, not only a couple of example sequences.
- Keep a deterministic reference rule if that helps explain the expectation.
- Say what should shrink when the invariant fails.

ACCEPTANCE CRITERIA

- You express one real invariant in clear Rust-domain terms.
- You generate a sequence or batch of inputs rather than a single fixed example.
- You explain how a failing case becomes smaller or easier to diagnose under shrinking.

HINTS

- A good property reads like a rule the business or subsystem actually cares about.
- The generator serves the invariant, not the other way around.

EXERCISE 4 DEBUGGING OR REFACTORING

Plan fuzz targets for unsafe parsing code

OBJECTIVE

Choose the right harness boundary and corpus strategy for an unsafe or FFI-adjacent parser.

STARTER PROMPT

You inherit a framed binary parser with one small unsafe fast path that reconstructs headers from bytes and then walks a payload cursor.

- Which function should be the fuzz boundary: raw bytes to parse result, or a much larger transport stack?
- Which seed corpus cases should exist on day one: empty, truncated, exact header, oversized length, duplicated frame, malformed footer?
- How will you compare the unsafe fast path against a safer reference parser or structural invariant?
- What crash or sanitizer signals should fail the fuzz lane immediately?

ACCEPTANCE CRITERIA

- You pick a small direct fuzz boundary and justify it.
- You define at least five useful seed inputs or input classes.
- You mention either a reference implementation or explicit unsafe invariants to check.
- You include at least one tool signal such as sanitizer failure, panic, or UB detection.

HINTS

- The best fuzz target is usually smaller than the whole service.
- Keep the harness close enough to the parser that the crashing bytes still mean something.

EXERCISE 5 DESIGN OR PRODUCTION SCENARIO

Design async integration tests for a service boundary

OBJECTIVE

Make time, retries, task joins, and durable side effects explicit in an async test plan.

STARTER PROMPT

You are testing `HTTP request -> async worker -> database write -> queue publish`, and the retry path must stay idempotent under duplicate delivery or cancellation.

- Which boundaries deserve fake time or a paused runtime?
- Which durable side effects should be asserted before the handler considers the work complete?
- How will the test distinguish join failure, inner failure, and cancellation?
- Which IDs should appear in the assertions so the failure stays attributable later?

ACCEPTANCE CRITERIA

- You include at least one deterministic time-control technique.
- You define one idempotency or duplicate-suppression assertion at the durable boundary.
- You separate task-join behavior from inner operation behavior.
- You mention at least one request, task, or message identity field in the assertions.

HINTS

- If the plan still depends on `sleep`, it probably is not deterministic enough.
- The right assertion is often at the database or outbox boundary, not only at the HTTP status code.

EXERCISE 6 DESIGN OR PRODUCTION SCENARIO

Build a production test matrix for a distributed Rust subsystem

OBJECTIVE

Combine correctness, fuzzing, snapshots, golden files, and performance budgets into one reviewable plan.

STARTER PROMPT

Create a test matrix for a service that parses binary frames, enriches them asynchronously, writes an outbox entry, renders operator-facing reports, and exposes one FFI plugin boundary.

- Which claims stay in unit or property tests?
- Which boundaries want fuzz targets or unsafe invariant tests?
- Which output wants a golden file and which wants a snapshot?
- Which benchmark budget or regression lane belongs in CI, and which one belongs in a separate performance lane?
- What is the minimum end-to-end suite that still proves the transport reality without turning CI into a random-timing lottery?

ACCEPTANCE CRITERIA

- You define at least four distinct layers or lanes in the matrix.
- You include one property or invariant layer, one fuzz or unsafe layer, one output-review layer, and one performance-regression layer.
- You justify why each layer exists and what claim it proves.
- You keep the end-to-end layer narrow enough to stay maintainable.

HINTS

- The cleanest matrix proves different claims at different costs.
- If every claim is left to end-to-end tests, the suite will usually be slow and still incomplete.

Runnable lab

Example 42-L. `property_invariant_lab.rs` — *Runnable lab · Deterministic property-style invariant harness*

```rust
#[derive(Debug, Clone, Copy)]
struct Quota {
    limit: u32,
    used: u32,
}

impl Quota {
    fn reserve(&mut self, qty: u32) -> bool {
        self.used += qty;
        true
    }
}
```

```
fn invariant_holds(limit: u32, ops: &[u32]) -> bool {
    let mut quota = Quota { limit, used: 0 };

    for &qty in ops {
        let _accepted = quota.reserve(qty);
        if quota.used > quota.limit {
            return false;
        }
    }

    true
}

fn main() {
    let scenarios: &[&[u32]] = &[
        &[1_u32, 1, 1],
        &[4_u32, 4],
        &[5_u32, 4],
        &[2_u32, 2, 2, 2],
        &[8_u32],
    ];

    let all_valid = scenarios.iter().all(|ops| invariant_holds(8, ops));
    let violations = scenarios.iter().filter(|ops| !invariant_holds(8, ops)).count();

    println!("cases = {}", scenarios.len());
    println!("all valid = {}", all_valid);
    println!("violations = {}", violations);
}
```

OUTPUT

```
cases = 5
all valid = true
violations = 0
```

Review questions

- Why is a property test often a better fit than several ad hoc example tests for one invariant?
- What does fuzzing prove that a curated integration fixture set usually does not prove?
- Why are sleep-based async tests brittle even when they 'usually pass' locally?
- What is the practical difference between golden files and snapshot tests?
- Why should benchmark regression budgets be looser and more isolated than ordinary correctness assertions?

CHAPTER 43 · OBSERVABILITY

Observability

Exercises, labs, and review questions for this chapter.

Instrument Tokio work with spans, design backpressure and latency metrics, and connect logs, metrics, and traces in incident narratives

EXERCISE 1 WARM-UP COMPREHENSION

Pick logs, metrics, traces, or profiles from the symptom

OBJECTIVE

Practice choosing the observability signal that answers the real incident question instead of defaulting to whichever tool is easiest to add first.

STARTER PROMPT

Classify four symptoms: a queue-backed request path misses p99 while handler p50 stays calm, a sudden spike in rejected payloads appears after one rollout, CPU on one reducer climbs while queue age stays flat, and one service-to-service hop keeps timing out only for one tenant.

- Which symptom wants metrics first?
- Which symptom wants structured logs first?
- Which symptom wants a trace first?
- Which symptom wants a profile first?

ACCEPTANCE CRITERIA

- You map each symptom to a primary signal with a concrete reason.
- You distinguish queue wait from handler CPU clearly.
- You avoid treating raw log volume as a substitute for one well-chosen metric or span.

HINTS

- Start by asking whether the question is aggregate health, one event, one causal path, or one cost hotspot.
- A strong answer uses more than one tool eventually, but still picks a sane first one.

EXERCISE 2 CODE READING

Read a Tokio worker that lost trace context at spawn

OBJECTIVE

Explain why one async service emits traces for local work but still cannot correlate queue claim, handler execution, and downstream publish under one path.

STARTER PROMPT

Review a worker loop that receives a task from `mpsc`, then does `tokio::spawn(handle(task))` without `info_span!`, without `.instrument(...)`, and without putting trace identity in the task envelope.

- Which context is missing from the task boundary?
- What should the envelope carry if the task may cross queue or retry boundaries later?
- Would `#[instrument]` on the handler alone be enough?
- Which one summary line would you want the browser runner or a test harness to prove is now wired correctly?

ACCEPTANCE CRITERIA

- You identify the missing spawn-boundary context precisely.
- You name at least one field such as `trace_id`, `route`, `task_id`, or `attempt` that should cross the boundary explicitly.
- You explain why handler spans alone are incomplete when queue or worker spans are missing.

HINTS

- If the trace stops at `tokio::spawn`, the instrumentation boundary is too small.
- A task envelope is already a contract. Let it carry observability identity too.

EXERCISE 3 IMPLEMENTATION

Design metrics for backpressure and latency

OBJECTIVE

Define a minimal but useful metric surface for a bounded Tokio or broker-backed service.

STARTER PROMPT

Design metrics for `accept -> queue -> worker -> publish`, with occasional retries and one bounded worker pool.

- Which counters should exist for accepted work, completed work, retries, and duplicate suppression?
- Which gauges should exist for queue depth, oldest visible age, and busy workers?
- Which histograms should exist for queue wait, run time, and end-to-end latency?
- Which labels would you explicitly forbid because they create dangerous cardinality?

ACCEPTANCE CRITERIA

- You propose at least one counter, one gauge, and one histogram family.
- You tie each metric to one operational question instead of only listing nouns.
- You forbid at least one high-cardinality label such as `trace_id` or `task_id`.

HINTS

- The best metric set is small enough to operate and rich enough to explain one incident quickly.
- Queue age and queue depth together often tell a better story than throughput alone.

EXERCISE 4 DEBUGGING OR REFACTORING

Repair async tracing and structured logs together

OBJECTIVE

Refactor a worker path so logs and traces share enough identity to explain one retry or dead-letter path without guesswork.

STARTER PROMPT

A service currently logs `retrying task` and `dead-lettered` as plain strings, but the logs do not include `task_id`, `queue`, `attempt`, or `trace_id`, and the retry span is not linked to the original claim span.

- Which fields should become structured log fields immediately?
- Which span relationship should be explicit across claim, retry, and terminal routing?
- How would you keep the extra fields low-cardinality enough for logs while still useful for traces?
- What should the single log-emitting boundary be for a permanent failure?

ACCEPTANCE CRITERIA

- You add at least three structured fields that make correlation materially better.
- You describe one parent-child or retry-lineage tracing repair.
- You identify one boundary that should log the permanent failure once.

HINTS

- The most useful log field is the one that lets you pivot to the trace or durable record immediately.
- A permanent failure path should not produce identical error logs at five layers.

EXERCISE 5 DESIGN OR PRODUCTION SCENARIO

Connect logs, metrics, and traces in one incident narrative

OBJECTIVE

Practice telling one coherent production story from three signal types instead of reading each one in isolation.

STARTER PROMPT

Metrics show oldest visible age rising from 200 ms to 4 s. One trace shows a task waited 3.6 s before claim and then only 80 ms in the handler. Structured logs show the same task retried twice because a downstream store timed out.

- Which layer actually dominates end-to-end latency here?
- Which metric should have paged first?
- Which trace fields or log fields make the retry path attributable?
- Which refactor would you test before touching handler CPU?

ACCEPTANCE CRITERIA

- You identify queue wait and replay pressure, not handler CPU, as the dominant story.
- You cite at least one metric and one identity field that connect the evidence.
- You propose one refactor tied to the real bottleneck, such as retry budgeting, queue split, or more downstream capacity.

HINTS

- This exercise is about causality, not about one favorite dashboard.
- If queue wait dominates already, optimizing 80 ms of handler CPU is probably not first.

EXERCISE 6 DESIGN OR PRODUCTION SCENARIO

Turn observability into SLOs, alerts, and refactoring policy

OBJECTIVE

Design the outer observability contract for a service another team will operate under real failure and capacity pressure.

STARTER PROMPT

You are designing a scoring service with HTTP intake, an internal bounded queue, a GPU-backed worker lane, and a brokered outcome publisher. Operators require end-to-end latency and success SLOs plus a clear story for continuous profiling and OpenTelemetry export.

- Which SLOs would you publish: success rate, oldest visible age, e2e latency, freshness, or a subset?
- Which alerts should fire from burn rate versus absolute queue age or worker saturation?
- Where should OpenTelemetry exporter wiring live relative to domain and transport code?
- Which profiling signals would you sample continuously and which would stay incident-driven?

ACCEPTANCE CRITERIA

- You define at least two SLO-style promises that map to concrete metrics.
- You distinguish burn-style alerting from raw threshold alerting with one reason for each.
- You keep exporter or collector wiring at the platform edge instead of leaking it into domain logic.
- You mention at least one continuous profile signal and one incident-driven deeper measurement.

HINTS

- An SLO is useful only if someone can still act on it.
- Exporter wiring is usually platform configuration, not business behavior.

Runnable lab

Example 43-L. `trace_service_lab.rs` — *Runnable lab · Instrument a Tokio worker with spans*

```rust
use tokio::sync::mpsc;
use tracing::{info_span, instrument, Instrument};

#[derive(Debug, Clone)]
struct Request {
    trace_id: &'static str,
    route: &'static str,
    bytes: usize,
}

async fn handle(request: Request) -> Result<usize, &'static str> {
    if request.bytes == 0 {
        return Err("empty payload");
    }
```

```
    Ok(request.bytes / 10)
}

#[tokio::main]
async fn main() {
    let (tx, mut rx) = mpsc::channel::<Request>(4);

    tx.send(Request {
        trace_id: "req-11",
        route: "/score",
        bytes: 200,
    })
    .await
    .unwrap();

    tx.send(Request {
        trace_id: "req-12",
        route: "/health",
        bytes: 100,
    })
    .await
    .unwrap();

    drop(tx);

    let mut processed = 0_usize;
    let mut last_trace = "none";

    while let Some(request) = rx.recv().await {
        last_trace = request.trace_id;

        match handle(request).await {
            Ok(_) => processed += 1,
            Err(_) => {}
        }
    }

    println!("processed = {}", processed);
    println!("last trace = {}", last_trace);
}
```

OUTPUT

```
instrumented = true
processed = 2
last trace = req-12
```

Review questions

- What does structured logging solve that plain text logs do not solve well?
- Why should queue wait be a first-class latency signal in async and distributed Rust services?
- What is the practical difference between trace identity and metric labels?
- Why is OpenTelemetry usually an export and propagation concern rather than a domain-model concern?
- How does observability-driven refactoring differ from profiling-driven local optimization?

CHAPTER 44 · PACKAGING AND DEPLOYMENT

Packaging and Deployment

Exercises, labs, and review questions for this chapter.

Build a packaging matrix, design a feature-gated release workflow, add supply-chain checks to CI, and write a production-ready release plan

EXERCISE 1 WARM-UP COMPREHENSION

Create a packaging matrix for native, container, wasm, and library outputs

OBJECTIVE

Practice stating which artifact shape each consumer actually needs instead of collapsing every release into one vague 'build'.

STARTER PROMPT

Your team ships one service as a Linux binary, one container image, one browser-facing wasm bundle, and one native library for another product. Write a small packaging matrix for all four.

- Which target triple or crate type belongs to each artifact?
- Which runtime assumption belongs to each artifact: libc, container base, browser glue, or foreign ABI?
- Which artifact should carry feature gates differently from the others?
- Which artifact deserves its own smoke test lane?

ACCEPTANCE CRITERIA

- You define at least three artifact rows with distinct target or crate-type information.
- You name at least one runtime assumption per row.
- You mention at least one artifact-specific smoke test or validation step.

HINTS

- A packaging matrix is most useful when every row answers who will run the artifact and how.
- If two artifacts have different consumers, they probably need different validation too.

EXERCISE 2 CODE READING

Review a Docker multi-stage build and its minimal runtime image

OBJECTIVE

Explain what the builder stage and runtime stage each guarantee, and identify what a too-minimal runtime image may still be missing.

STARTER PROMPT

Review a two-stage Docker build that compiles a Rust binary in one stage and copies it into `scratch` in the second stage.

- What does the builder stage own that should not leak into the runtime stage?
- What does the runtime stage still need besides the executable, such as CA certificates or runtime user setup?
- When would distroless or Alpine be calmer than `scratch`?
- What final smoke test should run against the image rather than only against the binary on the build host?

ACCEPTANCE CRITERIA

- You distinguish builder and runtime responsibilities clearly.
- You name at least one missing runtime dependency that a scratch image may still need.
- You justify one alternative runtime image choice with a workload reason.

HINTS

- Smaller is not always calmer if the process still expects files or runtime metadata that the image does not contain.
- The final image deserves its own boot or health-check test.

EXERCISE 3 IMPLEMENTATION

Implement a packaging matrix for native, container, and wasm builds

OBJECTIVE

Build a tiny release helper that turns one app name and version into three explicit artifact names.

STARTER PROMPT

Implement `artifact_name(app, version, target)` for a native musl binary, a container tag, and a wasm bundle.

- Keep the target set explicit with an enum.
- Name the native output with a target triple suffix.
- Name the container output like a registry tag.
- Name the wasm output like a wasm artifact rather than a native binary.

ACCEPTANCE CRITERIA

- The enum clearly distinguishes native, container, and wasm targets.
- The function returns three different artifact naming shapes.
- The runnable lab prints the expected native, container, and wasm names.

HINTS

- This exercise is about release contracts, not clever formatting.
- The point is that each artifact class has a different naming convention and consumer.

EXERCISE 4 DEBUGGING OR REFACTORING

Design a feature-gated release workflow

OBJECTIVE

Replace one catch-all default build with an explicit feature policy that matches real deployment shapes.

STARTER PROMPT

You inherit a crate where the default feature set includes metrics, admin endpoints, optional native bindings, and browser support, even though most service deployments only need the core server.

- Which features should stay opt-in instead of default?
- Which CI lanes should build `default`, `minimal`, and at least one expanded feature set?
- Which runtime settings should remain runtime configuration instead of compile-time features?
- What artifact-size or attack-surface signal would you compare before and after the change?

ACCEPTANCE CRITERIA

- You move at least one non-core feature out of the default set.
- You define at least two explicit CI feature combinations.
- You distinguish runtime config from compile-time packaging choices clearly.

HINTS

- A good feature matrix makes the smallest honest artifact easy to build and test.
- Not every optional behavior belongs behind a compile-time flag.

EXERCISE 5 DEBUGGING OR REFACTORING

Add supply-chain checks to CI before release day forces the issue

OBJECTIVE

Design a release lane that validates dependency policy and final artifact trust data deliberately.

STARTER PROMPT

Your current pipeline runs tests and publishes artifacts, but it has no checksum step, no SBOM generation, and no dependency-policy or advisory gate.

- Which checks belong before artifact publication and which belong after the artifact is built?
- What should be generated for operators: checksums, signatures, SBOMs, provenance, or all four?
- Which ecosystem checks would you treat as release gates versus advisory signals?
- How will the pipeline surface failure clearly enough that the team knows why publication stopped?

ACCEPTANCE CRITERIA

- You define at least three concrete supply-chain checks or outputs.
- You place at least one gate before publication and one artifact metadata step after build.
- You explain how CI reports failure without leaving the release state ambiguous.

HINTS

- The best release lane tells operators what to trust and tells engineers why a release was blocked.
- Generated metadata is only useful if it follows the final artifact that users actually download.

EXERCISE 6 DESIGN OR PRODUCTION SCENARIO

Write a release-engineering plan for a Rust service with several artifact shapes

OBJECTIVE

Turn packaging into an operable rollout process with smoke tests, health checks, rollback, and artifact inventory.

STARTER PROMPT

You are shipping `gateway`, `gateway-wasm-admin`, and `gateway-risk-lib` from one repository. Operators require canary rollout, rollback instructions, and clear artifact validation on every release.

- Which artifacts should release together and which can version independently?
- Which smoke tests should run on the final binary, the final image, the wasm package, and the native library harness?
- What health or observability checks gate promotion from canary to broad rollout?
- What rollback data must be published with the release so another team can act without rebuilding under pressure?

ACCEPTANCE CRITERIA

- You define at least two artifact classes with distinct release validation.
- You include at least one promotion gate and one rollback requirement.
- You mention at least two observability or health signals that matter during rollout.

HINTS

- A release plan is not complete until rollback is easier to execute than improvisation.
- The final artifact, not only the source commit, is what operators actually roll back to.

Runnable lab

Example 44-L. `packaging_matrix_lab.rs` — *Runnable lab · Packaging matrix*

```rust
#[derive(Debug, Clone, Copy)]
enum Target {
    Native,
    Container,
    Wasm,
}

fn artifact_name(app: &str, version: &str, target: Target) -> String {
    let _ = version;
    let _ = target;
    app.to_string()
}

fn main() {
```

```
    let app = "pricing";
    let version = "1.2.0";

    println!("native = {}", artifact_name(app, version, Target::Native));
    println!("container = {}", artifact_name(app, version, Target::Container));
    println!("wasm = {}", artifact_name(app, version, Target::Wasm));
}
```

OUTPUT

```
native = pricing-x86_64-unknown-linux-musl
container = ghcr.io/acme/pricing:1.2.0
wasm = pricing-wasm32-unknown-unknown.wasm
```

Review questions

- What makes a static binary attractive, and what runtime assumptions can still remain outside the binary itself?
- Why should browser wasm and WASI packaging be treated as different release families?
- When would you choose `cdylib` instead of `staticlib`, and why is the ABI contract part of release engineering?
- Why should feature combinations be tested in CI rather than described only in docs?
- What should be smoke tested: the crate graph, the built binary, the image, or all of the above?

CHAPTER 45 · CAPSTONE: A DISTRIBUTED RUST SYSTEM

Capstone: A Distributed Rust System

Exercises, labs, and review questions for this chapter.

Review the capstone like a staff engineer: architecture, milestones, queue budgets, replay safety, profiling evidence, and rollout gates

EXERCISE 1 WARM-UP COMPREHENSION

Assemble the capstone architecture from earlier chapters

OBJECTIVE

Practice mapping earlier book topics into one distributed system without inventing extra abstraction layers first.

STARTER PROMPT

Sketch `HTTP request -> canonicalize -> compute verification root -> publish task -> claim task -> execute graph or matrix workload -> durable c` as one Rust system.

- Which boundaries are Tokio task boundaries, and which are broker or queue boundaries?
- Where does the task first have to become fully owned?
- Which subsystem should own duplicate suppression and durable finish-once state?
- Where do graph, matrix, and optional accelerator lanes enter the picture?

ACCEPTANCE CRITERIA

- You identify at least one owned message boundary and one async runtime boundary clearly.
- You place duplicate suppression at the durable completion boundary rather than in a fragile local cache only.
- You separate the execution lanes from the transport and verification lanes explicitly.

HINTS

- The queue boundary is usually the first place borrowed request-local state becomes dishonest.
- A good answer names owners, not only components.

EXERCISE 2 CODE READING

Review a flawed capstone delivery path

OBJECTIVE

Spot the production mistakes that appear when a distributed design compiles but its ownership and failure model are still wrong.

STARTER PROMPT

You inherit an API handler that borrows request data into spawned work, publishes to one shared queue for all workloads, acks broker delivery before durable result storage, and records no trace or dedupe key in the envelope.

- Which line or boundary turns borrowed data into an eventual lifetime bug or design lie?
- Why does one shared queue create avoidable tail-latency coupling between fast and slow workloads?
- Why is ack-before-store a correctness bug rather than only a telemetry bug?
- Which missing fields make replay, tracing, or idempotency weaker than they need to be?

ACCEPTANCE CRITERIA

- You identify at least three concrete design failures, not only one vague 'needs refactoring' note.
- You explain one ownership bug, one backpressure or queueing bug, and one correctness or replay bug.
- You propose a repair for each category.

HINTS

- Try to classify each issue as ownership, pacing, correctness, or observability.
- The most important bug is usually the one that loses work or duplicates side effects under crash.

EXERCISE 3 IMPLEMENTATION

Define implementation milestones and acceptance criteria

OBJECTIVE

Break the capstone into reviewable milestones so the repository stays coherent and measurable after each step.

STARTER PROMPT

Propose four milestones for building the capstone in order: a single-process reference implementation, a brokered path, specialized worker lanes, and production rollout hardening.

- What must be working at the end of each milestone?
- Which tests belong to each milestone: unit, integration, soak, or replay?
- Which telemetry should exist before the next milestone begins?
- Which architectural decisions must remain stable across all milestones, such as the task envelope shape?

ACCEPTANCE CRITERIA

- You define at least four milestones with one clear completion condition each.
- You attach at least one test or verification lane to each milestone.
- You preserve one stable task envelope or durable contract across the whole plan.

HINTS

- A milestone is useful only if another engineer can tell when it is done.
- Keep the durable contract stable earlier than the internal executor implementation.

EXERCISE 4 DEBUGGING OR REFACTORING

Repair worker idempotency and queue budgets

OBJECTIVE

Refactor the capstone when duplicate execution and mixed-latency queues are already hurting correctness and p99.

STARTER PROMPT

Metrics show duplicate completions during lease expiry and queue age spikes whenever matrix jobs arrive in bursts. Repair the design before tuning kernels.

- Where should finish-once state live so a replay stays harmless?
- Should graph and matrix work split into separate queues or worker pools?
- Which local queues should be bounded, and where should admission push back instead of buffering forever?
- What regression test would prove the refactor fixed the replay path?

ACCEPTANCE CRITERIA

- You place idempotent completion in one durable owner or store.
- You isolate at least one slow workload class from a fast one.
- You add or propose one bounded queue or concurrency cap explicitly.
- You define one replay or duplicate-suppression test.

HINTS

- Correctness usually gets fixed before throughput.
- A queue split is often a pacing repair as much as a performance repair.

EXERCISE 5 DESIGN OR PRODUCTION SCENARIO

Profile and refactor the capstone from evidence

OBJECTIVE

Turn one latency incident into a measurement-first refactoring plan instead of a guess-driven rewrite.

STARTER PROMPT

One production window shows queue p95 at 800 ms, worker run time at 70 ms, retry rate at 0.25, and one saturated accelerator lane while CPU graph workers remain mostly idle.

- Which layer is the dominant bottleneck right now?
- Which metric should have paged first, and which trace should you inspect next?
- Which refactor should come before any graph or matrix kernel optimization?
- What before-and-after signals would prove the repair moved the real bottleneck?

ACCEPTANCE CRITERIA

- You identify queueing or accelerator admission, not local handler CPU, as the current dominant story.
- You choose at least one metric and one trace or log field for the next diagnostic step.
- You propose one refactor tied directly to the evidence, such as queue split, worker budget change, or retry classification repair.
- You name at least two validation signals after the change.

HINTS

- A 70 ms handler is rarely the first optimization target if queue wait is already 800 ms.
- Retry rate belongs in the capacity story, not only in the error story.

EXERCISE 6 DESIGN OR PRODUCTION SCENARIO

Choose rollout, acceleration, and failure isolation policy

OBJECTIVE

Design the final production posture for the capstone so operators can canary, rollback, and contain slow specialist lanes.

STARTER PROMPT

You are releasing the capstone with optional CPU-only, MPI-assisted, and CUDA-assisted execution paths. Operators require canary rollout, rollback artifacts, and an explicit policy for replay after worker crash.

- Which execution path should be the stable fallback when specialist lanes are saturated or unavailable?
- Which artifact or configuration knobs must be visible during canary so teams know which lane is active?
- How will you isolate retry storms in one specialist lane from the rest of the system?
- What telemetry or health gates must pass before promotion from canary to broad rollout?

ACCEPTANCE CRITERIA

- You define one safe fallback path and one specialist path selection rule.
- You include at least one failure-isolation mechanism such as queue split, worker-pool split, or retry-lane split.
- You include at least two rollout gates tied to concrete telemetry, such as queue age, retry rate, verification failures, or durable completion lag.
- You mention one rollback artifact or operator action that does not depend on rebuilding from memory during the incident.

HINTS

- A capstone is not rollout-ready until the fallback path is boring and explicit.
- Specialist acceleration without isolation is just another outage amplifier.

Review questions

- Where should the capstone first cross from borrowed request data into an owned durable envelope?
- Why should duplicate suppression live near durable completion rather than only in one in-memory worker cache?
- What is the practical difference between profiling queue wait and profiling worker run time in this system?
- Why should specialist graph, matrix, MPI, or CUDA lanes still preserve one stable outer task contract?
- What makes an implementation milestone useful instead of decorative?

PART VII

Networked Services and Secure Boundaries

Build the service layer — REST/OpenAPI, gRPC, WebSockets, TLS, and peer-to-peer — that exposes your system to the world.

46 FastAPI-Style Web Apps, Swagger, and OpenAPI Codegen

47 gRPC Services with Protobuf and Service API Codegen

48 WebSockets and Long-Lived Connections

49 HTTPS, TLS, and Secure Service Boundaries

50 libp2p and Peer-to-Peer Rust Systems

CHAPTER 46 · FASTAPI-STYLE WEB APPS, SWAGGER, AND OPENAPI CODEGEN

FastAPI-Style Web Apps, Swagger, and OpenAPI Codegen

Exercises, labs, and review questions for this chapter.

Translate FastAPI-style ergonomics into Rust seams, choose an OpenAPI workflow, and prevent documentation or codegen drift

EXERCISE 1 WARM-UP COMPREHENSION

Map a FastAPI-style endpoint into Rust transport and service seams

OBJECTIVE

Practice translating a familiar typed endpoint into Rust extractors, typed state, and one application service call without leaking HTTP inward.

STARTER PROMPT

Design one `POST /v1/invoices` endpoint with path or body extractors, typed application state, and an authenticated caller context.

- Which values belong in transport DTOs and which values belong in one application command?
- Which shared resources belong in typed app state: pools, config, telemetry, idempotency store?
- Which checks belong in middleware or auth extraction versus in the handler itself?
- What should the domain service never learn about this HTTP request?

ACCEPTANCE CRITERIA

- You separate transport DTOs from the application command clearly.
- You put at least one real shared dependency into typed app state with a reason.
- You identify at least one concern that belongs in middleware or auth extraction instead of in the domain service.
- You explain why the domain model should not depend on framework request types.

HINTS

- Start from the handler signature you want another engineer to understand in ten seconds.
- Then ask which part of that signature is transport-only versus business-relevant.

EXERCISE 2 CODE READING

Spot transport leakage into the domain layer

OBJECTIVE

Read a handler and service pair and identify where HTTP types, status semantics, or documentation concerns have crossed the wrong boundary.

STARTER PROMPT

A `BillingService::create_invoice` method currently accepts `CreateInvoiceRequest`, returns `Result<InvoiceDto, StatusCode>`, and mentions OpenAPI field examples in comments right above the business rule logic.

- Which parameters or return types are transport-layer concepts rather than domain or application concepts?
- Which type should the service take instead of the HTTP request DTO?
- Which type should the service return so the handler can translate it to HTTP later?
- Where should the documentation examples and status mapping live instead?

ACCEPTANCE CRITERIA

- You identify at least one transport-shaped input and one transport-shaped output that should move out of the service layer.
- You propose a command or query type that is calmer for the service boundary.
- You place HTTP status mapping and documentation concerns back at the transport boundary.
- You explain one concrete testing or refactoring benefit of the repaired seam.

HINTS

- If the service could not run under a message queue or CLI boundary anymore, HTTP has probably leaked too far inward.
- The calm repair usually makes the service boundary easier to unit test as plain Rust.

EXERCISE 3 IMPLEMENTATION

Refactor a handler into a testable service boundary

OBJECTIVE

Implement a small typed handler that converts a request DTO into a command, calls a service, and returns one response shape without leaking transport types inward.

STARTER PROMPT

Take a small create-style endpoint, add an authenticated actor, map the request DTO into a command, call the service, and return a created response.

- Keep the service API free of framework request wrappers.
- Use one typed caller context such as tenant plus permission bit.
- Translate one domain rejection into one API-facing error at the edge.
- Return a status and response payload that another test can assert deterministically.

ACCEPTANCE CRITERIA

- The handler translates request DTO into a separate command type.
- The service boundary does not accept HTTP request DTOs directly.
- The handler performs at least one authorization or edge validation check before the service call.
- The runnable lab prints the expected created status, invoice ID, and tenant.

HINTS

- A create handler should be boring: authorize, map, call, translate.
- If the domain service still takes the request DTO, the transport seam is not finished yet.

EXERCISE 4 DEBUGGING OR REFACTORING

Choose code-first, spec-first, or mixed OpenAPI workflow honestly

OBJECTIVE

Pick the workflow that matches team boundaries and generated-code obligations instead of choosing by fashion.

STARTER PROMPT

You have one Rust server, one TypeScript client, and one partner team that wants the contract reviewed before implementation lands.

- What makes code-first attractive here?
- What makes spec-first attractive here?
- What would a mixed workflow look like if the checked-in spec must still be CI-enforced?
- Which workflow makes generated clients least surprising for the teams involved?

ACCEPTANCE CRITERIA

- You choose one primary workflow and justify it from team and review boundaries.
- You explain at least one tradeoff of the two alternatives.
- You include one CI or artifact rule that keeps the chosen workflow honest.
- You avoid claiming that one style is universally superior in every organization.

HINTS

- The right answer is usually the one that minimizes silent drift between server and clients.
- Think about who reviews the contract and when, not only about who writes the code.

EXERCISE 5 DEBUGGING OR REFACTORING

Prevent documentation drift and generated-code drift

OBJECTIVE

Design a review loop that keeps handlers, OpenAPI artifacts, Swagger UI, and generated clients describing the same API over time.

STARTER PROMPT

A recent rollout changed one handler input field, but the checked-in spec and generated client were not regenerated. The change passed unit tests and broke one downstream team.

- Which CI check should fail first: spec diff, generated client compile, or contract fixture test?
- Which artifacts should be checked into the repository and which can be generated only in CI?
- How would you review Swagger UI or schema output without treating it as the only proof?
- Which fields or operation IDs should be stabilized so downstream diffs stay reviewable?

ACCEPTANCE CRITERIA

- You define at least two drift-prevention checks with different jobs.
- You include at least one checked artifact or snapshot strategy and one generated-client validation strategy.
- You mention stable operation IDs or stable error envelopes explicitly.
- You explain why documentation review still needs semantic contract assertions nearby.

HINTS

- The most useful drift check is the one that fails before a client release is cut from stale types.
- Swagger UI helps humans, but CI still needs machine-checkable contract evidence.

EXERCISE 6 DESIGN OR PRODUCTION SCENARIO

Design one production API surface with versioning, pagination, idempotency, tracing, metrics, and graceful shutdown

OBJECTIVE

Make the non-happy-path parts of a web API explicit before the service ships.

STARTER PROMPT

You are designing a billing API with create, list, and lookup endpoints, one async side-effect path, and an operator requirement that canary rollout and rollback stay safe under load.

- Which routes or envelopes carry version boundaries, and how do they evolve?
- Which list route wants cursor pagination and what limits should be validated at the edge?
- Which create route needs an idempotency key and where does finish-once state live?
- Which trace and metric fields must exist before rollout?
- How does the server stop admitting and drain gracefully during deployment shutdown?

ACCEPTANCE CRITERIA

- You define at least one versioning rule, one pagination rule, and one idempotency rule.
- You include at least two observability hooks such as request ID, trace ID, route, queue age, or publish latency.
- You describe one graceful shutdown sequence that stops admission before exit.
- You keep the explanation tied to transport and application boundaries rather than only to framework settings.

HINTS

- A production API design is still an ownership and pacing design once traffic is real.
- If the service causes duplicate side effects during deploy shutdown, graceful shutdown was not actually graceful.

Review questions

- What does it mean for a domain service to stay HTTP-free in a Rust web application?
- Why are typed extractors and typed state a better long-term seam than passing one raw request object around?
- When is code-first OpenAPI the calm default, and when does spec-first become more honest?
- Why should generated clients stay in the transport layer rather than in the domain layer?
- What CI checks keep Swagger UI and generated SDKs from silently drifting away from handler behavior?

CHAPTER 47 · GRPC SERVICES WITH PROTOBUF AND SERVICE API CODEGEN

gRPC Services with Protobuf and Service API Codegen

Exercises, labs, and review questions for this chapter.

Choose the right RPC shape, contain generated types to the edge, and test schema evolution, deadlines, and streaming backpressure

EXERCISE 1 WARM-UP COMPREHENSION

Choose unary, server streaming, client streaming, or bidirectional streaming from the contract

OBJECTIVE

Practice selecting the RPC shape from data flow and pacing instead of choosing streaming by novelty.

STARTER PROMPT

Classify four endpoints: create one invoice, watch invoice events over time, upload many invoices then get one summary, and a long-lived sync conversation where both peers keep sending updates.

- Which endpoint is a unary call and why?
- Which endpoint wants server streaming because one request opens one subscription?
- Which endpoint wants client streaming because the client owns accumulation into one summary result?
- Which endpoint truly wants bidirectional streaming, and what backpressure question appears immediately after that choice?

ACCEPTANCE CRITERIA

- You map each workload to a plausible gRPC method shape with a transport reason.
- You identify at least one pacing or buffering implication for the streaming cases.
- You avoid using bidirectional streaming as the default when a simpler shape already fits.

HINTS

- Ask who sends once, who sends many times, and who controls pace.
- A streaming method is also a lifecycle and buffer contract.

EXERCISE 2 CODE READING

Read generated transport types without contaminating the domain model

OBJECTIVE

Explain where generated protobuf messages belong and where a hand-written application command should begin.

STARTER PROMPT

A service method currently accepts `CreateInvoiceRequest` directly from generated code and passes it unchanged into a domain aggregate constructor.

- Which fields or types are transport concerns rather than domain concerns?
- Where should tenant, caller identity, or metadata-derived fields be attached?
- Which hand-written command type would make the service boundary calmer?
- What becomes easier to test once the mapping seam exists explicitly?

ACCEPTANCE CRITERIA

- You distinguish generated DTOs from domain or application commands clearly.
- You identify at least one metadata-derived field that should be attached outside generated message code.
- You name at least one testing or refactoring benefit of the separated seam.

HINTS

- Generated messages are transport types with field tags and serialization rules attached.
- The application service usually wants a smaller, more domain-specific command.

EXERCISE 3 IMPLEMENTATION

Map a generated request into a domain command

OBJECTIVE

Build the small adapter that moves one generated transport request into one application command without leaking gRPC into business code.

STARTER PROMPT

Implement `into_command(request, tenant)` so it validates the generated request, moves owned fields into a separate command type, and returns a typed result.

- Reject empty customer IDs or empty line sets explicitly.
- Move owned Strings and Vecs into the command rather than cloning them gratuitously.
- Keep the result type small and transport-facing only at the adapter layer.
- Print tenant, customer, and total cents from the mapped command in the runnable lab.

ACCEPTANCE CRITERIA

- The mapping step returns a separate command type rather than the generated request type itself.
- The adapter validates at least one transport-level shape problem explicitly.
- The runnable lab prints the expected tenant, customer, and total cents.
- The code does not make the downstream service depend on gRPC request wrappers or metadata APIs.

HINTS

- This is the same seam you would unit test heavily even if the live service used tonic.
- Move data once. The generated message already owns its fields.

EXERCISE 4 DEBUGGING OR REFACTORING

Diagnose a deadline, cancellation, or streaming backpressure bug

OBJECTIVE

Repair one of the most common production problems in gRPC systems: work keeps flowing after the caller or the stream has already told you to stop.

STARTER PROMPT

A streaming handler keeps buffering items into a local channel even after the client disconnects, and one unary handler keeps doing CPU-heavy work after the deadline has already expired.

- Where should cancellation or deadline checks happen relative to expensive work?
- Which queue or buffer should be bounded before the stream fan-out gets large?
- Which status code should the server or client surface for deadline expiry or caller cancellation?
- What metric or trace field would prove the repair helped?

ACCEPTANCE CRITERIA

- You place at least one cancellation or deadline check at a meaningful work boundary.
- You propose at least one boundedness repair for stream buffering or worker admission.
- You identify one observability signal such as in-flight stream items, cancelled RPC count, or deadline-exceeded count.

HINTS

- A disconnected client is a pacing signal, not only a transport curiosity.
- If the buffer is unbounded, the stream still has an ownership bug even before it has a cancellation bug.

EXERCISE 5 DEBUGGING OR REFACTORING

Test generated APIs and prevent breaking protobuf changes

OBJECTIVE

Design the CI and test surface that catches schema drift before partner teams regenerate or deploy stale code.

STARTER PROMPT

You added one field, removed another, and changed one method shape from unary to streaming. Now several languages regenerate from the same .proto.

- Which changes are additive and which are breaking?
- Which contract artifact should be diffed or checked in CI?
- Which transport tests should run against generated client and server code?
- Which protobuf tags or names should be reserved after removal?

ACCEPTANCE CRITERIA

- You classify at least one additive change and one breaking change correctly.
- You define at least one schema-drift or breaking-change CI gate.
- You mention one generated-client or generated-server compile or contract test.
- You explicitly reserve removed field tags or names in the policy.

HINTS

- Changing a method shape is usually a bigger break than adding an optional field.
- Broken codegen drift is still contract drift even if the Rust server compiles locally.

EXERCISE 6 DESIGN OR PRODUCTION SCENARIO

Design a polyglot gRPC service lane with auth, tracing, and retries

OBJECTIVE

Turn the chapter topics into one production design another language team can consume safely.

STARTER PROMPT

You are designing a billing gRPC service consumed by Go workers, a C# admin tool, and one C++ plugin process. Operators require auth metadata, tracing, retry guidance, and bounded streaming behavior.

- Which metadata should travel on every call or stream session?
- Which status codes should clients treat as retryable and under what idempotency rule?
- Which routes or methods deserve stream-specific queue budgets or concurrency caps?
- What telemetry should exist before rollout?

ACCEPTANCE CRITERIA

- You define at least one auth or caller-context rule and one trace-propagation rule.
- You classify retryable versus terminal status behavior explicitly.
- You include at least one boundedness or concurrency rule for a streaming path.
- You mention at least three observability hooks such as status code family counts, stream lifetime, cancelled RPCs, or queue age.

HINTS

- The contract is polyglot, so review the .proto and retry guidance as public artifacts.
- A stream without boundedness policy is already under-specified for production.

Runnable lab

Example 47-L. `grpc_transport_mapping_lab.rs` — *Runnable lab · Map a generated request into a domain command*

```rust
#[derive(Debug, Clone)]
struct CreateInvoiceRequest {
    customer_id: String,
    line_totals: Vec<u64>,
}

#[derive(Debug, Clone)]
struct CreateInvoiceCommand {
    tenant: String,
    customer_id: String,
    line_totals: Vec<u64>,
}

fn into_command(
    request: CreateInvoiceRequest,
    tenant: &str,
) -> Result<CreateInvoiceCommand, &'static str> {
    Err("unfinished")
```

```
}

fn main() {
    let request = CreateInvoiceRequest {
        customer_id: String::from("cust-7"),
        line_totals: vec![1_200_u64, 3_000],
    };

    let command = into_command(request, "acme").unwrap();
    let total_cents: u64 = command.line_totals.iter().copied().sum();

    println!("tenant = {}", command.tenant);
    println!("customer = {}", command.customer_id);
    println!("total cents = {}", total_cents);
}
```

OUTPUT

```
tenant = acme
customer = cust-7
total cents = 4200
```

Review questions

- Why should generated protobuf types usually stop at the transport boundary instead of becoming the domain model?
- What makes field tags more durable than field names in protobuf evolution?
- Why are deadlines, cancellation, and backpressure part of the API design rather than only runtime behavior?
- What is the practical difference between a retryable Unavailable and a non-retryable InvalidArgument?
- Why should generated client and server artifacts be rebuilt or checked in CI whenever the .proto changes?

CHAPTER 48 · WEBSOCKETS AND LONG-LIVED CONNECTIONS

WebSockets and Long-Lived Connections

Exercises, labs, and review questions for this chapter.

Select the right live transport, bound slow consumers, design reconnect semantics, and test connection storms and drains

EXERCISE 1 WARM-UP COMPREHENSION

Choose WebSockets, SSE, polling, gRPC streaming, or a broker-backed edge fan-out

OBJECTIVE

Practice selecting the long-lived transport from directionality, latency, browser support, and durability needs instead of defaulting to WebSockets by habit.

STARTER PROMPT

Classify four products: a browser dashboard that only receives updates, a collaborative editor where both sides send commands, an internal polyglot stream between services, and a durable event feed where clients may reconnect after long offline gaps.

- Which case wants SSE because the client only listens?
- Which case really wants full bidirectional state over one connection?
- Which case wants gRPC streaming because service-to-service protobuf contracts matter more than browser support?
- Which case probably wants a broker-backed replay surface rather than direct socket fan-out as the primary durability boundary?

ACCEPTANCE CRITERIA

- You choose at least one transport other than WebSockets on purpose.
- You justify the choice with pacing or durability rather than with framework familiarity.
- You explain one tradeoff involving browser support, replay, or schema tooling.

HINTS

- Two-way interactivity is only one dimension of the choice.
- If long offline replay matters, a broker or durable log may be the real core boundary.

EXERCISE 2 CODE READING

Review explicit read/write ownership in one connection loop

OBJECTIVE

Read two async designs and explain why one writer-task boundary is calmer than several unrelated tasks writing directly through one shared sink.

STARTER PROMPT

Compare a design with one reader task, one application loop, and one writer task against a design that stores the write half behind `Arc<Mutex<...>>` and lets many tasks write whenever they want.

- Which design keeps frame ordering and close behavior easier to reason about?
- Which design makes cancellation and shutdown easier to stage?
- Which design makes slow-consumer backpressure easier to measure?
- When might a shared sink still be acceptable even if it is not the calm default?

ACCEPTANCE CRITERIA

- You explain one real win of the split read/write ownership model.
- You identify at least one real tradeoff or inconvenience of the shared sink design.
- You connect the answer to ordering, shutdown, or observability rather than only to syntax preference.

HINTS

- The main question is not mutex overhead. It is transport ownership and protocol sequencing.
- A close frame is easier to reason about when one writer owns the send path.

EXERCISE 3 IMPLEMENTATION

Implement a small message loop with explicit read/write ownership

OBJECTIVE

Build a small async loop where inbound frames cross one channel into app logic and outbound frames cross another channel into the writer side.

STARTER PROMPT

Implement one reader task, one application task, and one writer task, then react to text, ping, and close frames without sharing the write half directly.

- Keep inbound and outbound message enums separate.
- Use one bounded channel for inbound messages and one for outbound messages.
- Close by signaling shutdown after the close frame is observed.
- Do not pass borrowed request-local data into later spawned work that can outlive the local scope.

ACCEPTANCE CRITERIA

- The design has explicit read and write ownership boundaries.
- Application logic sits between IO and does not write directly to the transport from many places.
- The message loop reacts to ping and close distinctly.
- The code is plausible Rust and the lifecycle is reviewable from the types and channels alone.

HINTS

- A reader-app-writer pipeline is the whole point of the exercise.
- The best answer is easy to explain on a whiteboard without a specific WebSocket crate.

EXERCISE 4 DEBUGGING OR REFACTORING

Replace unbounded broadcast with a bounded slow-consumer policy

OBJECTIVE

Refactor one fan-out hub so backlog becomes visible and one slow client cannot quietly grow memory forever.

STARTER PROMPT

You inherit a room broadcaster that stores every outbound event in an unbounded per-client queue and only notices trouble after RSS climbs.

- Would you disconnect the slow client, drop old events, coalesce to one latest snapshot, or choose several policies by message class?
- Where should the queue bound live: per connection, per room, or both?
- Which metrics should spike first when one client or room falls behind?
- What regression test would prove the new policy is actually enforced?

ACCEPTANCE CRITERIA

- You define one explicit boundedness rule.
- You choose one slow-consumer policy and justify it with product semantics.
- You mention at least one metric and one test for the repair.
- You avoid leaving the queue effectively unbounded under a different name.

HINTS

- There is no neutral slow-consumer policy. Every choice changes product semantics.
- Per-connection bounds are usually the first honest minimum.

EXERCISE 5 DESIGN OR PRODUCTION SCENARIO

Design a production WebSocket protocol with versioning, auth, and shutdown semantics

OBJECTIVE

Make the envelope and lifecycle contract explicit before mixed clients, reconnects, and deploy shutdowns force emergency protocol decisions.

STARTER PROMPT

Design a protocol for `subscribe -> receive events -> ack or resume -> close`, with browser clients, per-tenant auth, and deploy-time drain requirements.

- Which envelope fields are mandatory: protocol version, trace ID, session ID, sequence, or all of them?
- Which messages belong to the transport layer and which belong to the domain layer?
- How will reconnect resume or snapshot catch-up work?
- What should the server send during graceful shutdown before closing the socket?

ACCEPTANCE CRITERIA

- You define one versioned envelope or equivalent contract boundary.
- You separate auth or transport messages from domain events explicitly.
- You describe one reconnect or resume rule and one shutdown rule.
- You include at least one authorization or origin-check decision.

HINTS

- A protocol is easier to evolve when the lifecycle messages are explicit from day one.
- If the server will ever drain, the close reason or terminal event belongs in the design now.

EXERCISE 6 DESIGN OR PRODUCTION SCENARIO

Test disconnect storms, reconnects, and long-lived shutdown behavior

OBJECTIVE

Design the test matrix and observability surface for the failure paths that make or break long-lived connection systems.

STARTER PROMPT

You are testing a multi-tenant live-update service that can reconnect thousands of clients after a load-balancer change and must shut down gracefully during deploys.

- Which parts deserve deterministic unit or integration tests: protocol state, resume rules, bounded queues, graceful close, or all of them?
- Which metrics should prove a disconnect storm is under control: reconnect rate, handshake error rate, queue age, or per-tenant active connections?
- How would you simulate a slow consumer and a drained shutdown in CI without waiting on real wall-clock sleeps?
- Which trace or log fields must exist so one failed reconnect path is attributable later?

ACCEPTANCE CRITERIA

- You include at least one deterministic test for queue or shutdown behavior.
- You mention at least three observability hooks for storm or reconnect diagnosis.
- You include at least one stable ID field for logs or traces.
- You explain how the test plan avoids timing-lottery sleeps where possible.

HINTS

- Storm handling is partly a test problem and partly a metrics problem.
- If the only test is 'the browser reconnected once on my laptop,' the failure matrix is still missing.

Runnable lab

Example 48-L. `bounded_fanout_lab.rs` — *Runnable lab · Refactor unbounded fan-out into a bounded slow-consumer-aware design*

```rust
use std::collections::{HashMap, VecDeque};

#[derive(Debug, Default)]
struct Client {
    pending: VecDeque<String>,
}

#[derive(Debug)]
struct Hub {
    clients: HashMap<&'static str, Client>,
    max_pending: usize,
```

```
    delivered: usize,
    evicted: Vec<&'static str>,
}

impl Hub {
    fn new(max_pending: usize) -> Self {
        Self {
            clients: HashMap::new(),
            max_pending,
            delivered: 0,
            evicted: Vec::new(),
        }
    }

    fn add_client(&mut self, id: &'static str, pending: usize) {
        let mut queue = VecDeque::new();
        for _ in 0..pending {
            queue.push_back(String::from("old"));
        }

        self.clients.insert(id, Client { pending: queue });
    }

    fn broadcast(&mut self, payload: &str) {
        let ids: Vec<_> = self.clients.keys().copied().collect();

        for id in ids {
            if let Some(client) = self.clients.get_mut(id) {
                client.pending.push_back(payload.to_string());
                self.delivered += 1;
            }
        }
    }
}

fn main() {
    let mut hub = Hub::new(2);
    hub.add_client("alpha", 0);
    hub.add_client("beta", 2);
    hub.add_client("gamma", 1);

    hub.broadcast("first");
    hub.broadcast("second");

    println!("active = {}", hub.clients.len());
    println!("evicted = {}", if hub.evicted.is_empty() { "none".to_string() } else { hub.evicted.join(",") });
    println!("delivered = {}", hub.delivered);
}
```

OUTPUT

```
active = 1
evicted = beta,gamma
delivered = 3
```

Review questions

- Why is one writer task usually calmer than many tasks sharing the write half directly?
- What makes queue age or pending depth more actionable than active connection count alone?
- Why are reconnect and resume protocol decisions separate from ping or pong liveness?
- When is SSE a better answer than WebSockets even if the product still wants live updates?
- Why should graceful shutdown for long-lived sockets stop admission before closing active connections?

CHAPTER 49 · HTTPS, TLS, AND SECURE SERVICE BOUNDARIES

HTTPS, TLS, and Secure Service Boundaries

Exercises, labs, and review questions for this chapter.

Choose TLS ownership, harden edge defaults, design cert rotation and mTLS, and debug trust failures without weakening verification

EXERCISE 1 WARM-UP COMPREHENSION

Choose a TLS termination strategy from the real deployment boundary

OBJECTIVE

Practice selecting load balancer, reverse proxy, sidecar, or in-process termination from trust, packaging, and ownership requirements instead of from habit.

STARTER PROMPT

You have one browser-facing Rust API behind a cloud load balancer, one internal admin tool on the same host as a reverse proxy, one service-mesh deployment, and one dedicated edge appliance written in Rust.

- Which deployment naturally wants load-balancer termination?
- Which one fits reverse-proxy termination on the same host?
- Which one already has a sidecar trust boundary and should model identity handoff explicitly?
- Which one most plausibly wants in-process TLS because the Rust service itself owns the full external edge?

ACCEPTANCE CRITERIA

- You choose at least two different termination strategies across the four cases.
- You justify one choice with trust-header policy and one with certificate ownership or packaging policy.
- You avoid pretending one termination strategy is always correct for every service.

HINTS

- Ask who owns public certificates, who trusts proxy headers, and who needs the peer identity directly.
- The calm answer is often the one that keeps trust boundaries smallest and most reviewable.

EXERCISE 2 CODE READING

Identify insecure HTTP, cookie, and header settings in one API edge

OBJECTIVE

Read a sample configuration and explain which values are dangerous in production and which safer defaults belong instead.

STARTER PROMPT

A sample API config sets `redirect_http = false`, `allowed_origin = "*"`, `cookie.secure = false`, `cookie.http_only = false`, `same_site = "None"`, and `hsts_max_age_secs = 0`.

- Which fields weaken browser-side transport or session safety directly?
- Which change is unsafe only in combination, such as wildcard CORS with credentials?
- Which settings should be different for production even if a local demo accepted them?
- What would you test in CI after changing these defaults?

ACCEPTANCE CRITERIA

- You identify at least three concrete insecure settings.
- You propose secure replacements for cookies, redirects, CORS, or HSTS with a reason.
- You mention at least one CI assertion such as secure cookie flags, redirect behavior, or HSTS presence.

HINTS

- The question is not whether the app still works. The question is whether the boundary stays safe under browsers and proxies.
- A secure answer usually names both the bad setting and the operational risk it creates.

EXERCISE 3 IMPLEMENTATION

Harden one API edge configuration deliberately

OBJECTIVE

Translate insecure edge settings into a small typed policy that expresses secure defaults clearly and can be tested deterministically.

STARTER PROMPT

Refactor a tiny config so HTTPS redirects are on, the session cookie becomes hardened, the CORS origin becomes explicit, and HSTS is enabled.

- Keep the config as small plain Rust types.
- Use one exact origin instead of `*`.
- Use a secure session-cookie shape instead of a weak browser default.
- Print one short summary that proves the values changed.

ACCEPTANCE CRITERIA

- The config now expresses secure redirect and HSTS behavior.
- The session-cookie policy is hardened with at least `Secure` and `HttpOnly`.
- The CORS policy is no longer wildcard.
- The runnable lab prints the expected secure summary.

HINTS

- This is a boundary-policy exercise, not a framework exercise.
- If the values are explicit and typed, the next CI or review step gets much easier.

EXERCISE 4 DEBUGGING OR REFACTORING

Debug a TLS failure without weakening verification

OBJECTIVE

Create a repair path for a real certificate or handshake failure while keeping production-safe verification intact.

STARTER PROMPT

A staging client fails with a certificate error. The server certificate recently rotated, traffic now goes through a different proxy layer, and an engineer suggests temporarily disabling verification to confirm the route.

- Which evidence should you check first: hostname or SNI, full chain, trust store, or ALPN mismatch?
- Where could proxy termination or forwarded-scheme trust be the actual bug instead of the certificate itself?
- How would you reproduce the failure locally or in CI with a local CA rather than by skipping checks?
- Which change is acceptable for local diagnostics but unacceptable as a production workaround?

ACCEPTANCE CRITERIA

- You produce a checklist that starts from hostname, chain, trust, and proxy evidence.
- You do not recommend disabling verification as a production fix.
- You include one safe local or CI reproduction strategy using explicit trust material.
- You mention at least one way a proxy or termination change can look like a certificate bug to the app.

HINTS

- Most TLS failures are evidence problems before they are code problems.
- The safe diagnostic path is usually a local CA or explicit trust-bundle path, not a global skip-verify switch.

EXERCISE 5 DESIGN OR PRODUCTION SCENARIO

Design certificate rotation and mTLS boundaries for a service fleet

OBJECTIVE

Make renewal, reload, trust distribution, and peer-identity policy explicit before the first expiry or mixed rollout incident.

STARTER PROMPT

You run one public API, one internal queue consumer, and one service-to-service path that requires mTLS. Certificates rotate every 30 days, and operators want zero-downtime renewals.

- Which hops need only server-authenticated TLS and which need mTLS?
- Where should peer identity be checked: sidecar, reverse proxy, or in-process service?
- How will certificates and trust bundles reload without taking the service offline?
- Which metrics or alerts should warn before expiry or failed reload becomes user-visible?

ACCEPTANCE CRITERIA

- You classify at least one path as server-auth TLS only and one path as mTLS.
- You define where peer identity is enforced and why that owner is appropriate.
- You mention one reload or rollout strategy for rotation.
- You mention at least two observability signals such as cert-expiry horizon, reload failure count, or mTLS rejection rate.

HINTS

- Rotation is both a secret-distribution problem and a runtime-reload problem.
- mTLS is useful only if the peer identity still becomes a policy input somewhere explicit.

EXERCISE 6 DESIGN OR PRODUCTION SCENARIO

Test HTTPS locally and in CI with realistic trust behavior

OBJECTIVE

Build a local and CI test plan that exercises real TLS policy without teaching the team insecure shortcuts.

STARTER PROMPT

You need browser-facing HTTPS tests, one service-to-service mTLS test, and one CI lane that verifies secure cookies, redirects, and HSTS.

- Which SANs or hostnames should local certs cover?
- Which trust material should the browser or client test harness use in CI?
- How would you validate mTLS acceptance and rejection paths explicitly?
- Which assertions belong in the CI smoke test beyond 'port opens over TLS'?

ACCEPTANCE CRITERIA

- You use a local or CI CA and explicit trust bundle rather than an insecure verification bypass.
- You include at least one mTLS acceptance or rejection test.
- You include at least three transport-policy assertions such as HTTPS redirect, secure cookie, HSTS, or origin policy.
- You explain how the plan keeps local and CI trust behavior close enough to production to be meaningful.

HINTS

- A trusted local CA is usually the calmest path for realistic developer HTTPS.
- CI should prove policy, not only reachability.

Runnable lab

Example 49-L. `harden_edge_policy_lab.rs` — *Runnable lab · Harden one API edge policy*

```rust
#[derive(Debug)]
struct SessionCookiePolicy {
    name: &'static str,
    secure: bool,
    http_only: bool,
    same_site: &'static str,
}

#[derive(Debug)]
struct HttpSecurityPolicy {
    redirect_http: bool,
    hsts_max_age_secs: u64,
    allowed_origin: &'static str,
}

impl SessionCookiePolicy {
    fn is_hardened(&self) -> bool {
```

```
        self.secure && self.http_only && self.same_site != "None"
    }
}

impl HttpSecurityPolicy {
    fn cors_mode(&self) -> &'static str {
        if self.allowed_origin == "*" {
            "wildcard"
        } else {
            "locked-down"
        }
    }
}

fn main() {
    let cookie = SessionCookiePolicy {
        name: "session",
        secure: false,
        http_only: false,
        same_site: "None",
    };

    let policy = HttpSecurityPolicy {
        redirect_http: false,
        hsts_max_age_secs: 0,
        allowed_origin: "*",
    };

    println!("redirect = {}", policy.redirect_http);
    println!("cookie hardened = {}", cookie.is_hardened());
    println!("cors = {}", policy.cors_mode());
    println!("hsts = {}", policy.redirect_http && policy.hsts_max_age_secs > 0);
}
```

OUTPUT

```
redirect = true
cookie hardened = true
cors = locked-down
hsts = true
```

Review questions

- What is the practical difference between server-authenticated TLS and mTLS?
- Why should proxy-header trust be tied to a known network boundary instead of accepted from arbitrary callers?
- When is HSTS a safe default, and when is it premature?
- Why are local CA and CI trust bundles safer than skip-verification flags for HTTPS testing?
- What makes certificate rotation a deployment problem as much as a cryptography problem?

CHAPTER 50 · LIBP2P AND PEER-TO-PEER RUST SYSTEMS

libp2p and Peer-to-Peer Rust Systems

Exercises, labs, and review questions for this chapter.

Model swarm loops, bound discovery, plan NAT/relay fallback, and decide when P2P beats brokers or service APIs

EXERCISE 1 WARM-UP COMPREHENSION

Model one tiny P2P protocol as events and state transitions

OBJECTIVE

Practice designing a peer protocol as owned events and explicit state transitions before any real sockets or crates appear.

STARTER PROMPT

Design a minimal sync protocol where a node discovers a peer, opens a connection, sends one state request, receives one state summary, and later observes one gossip update.

- Which events belong to the network edge and which commands belong to local application logic?
- Which peer or request IDs must be carried through the state machine?
- Which state transitions add pending work and which clear it?
- What should the node record so another engineer can replay the same sequence in a deterministic test?

ACCEPTANCE CRITERIA

- You separate local commands from inbound network events clearly.
- You include at least one pending-request or connected-peer state transition.
- You identify at least one stable correlation field such as peer ID or request ID.
- You explain the design in terms of owned events rather than shared mutable transport access.

HINTS

- A protocol loop is usually easier to test when every event is a plain value.
- If the state change is important in production, it should have a name in the model.

EXERCISE 2 CODE READING

Identify ownership and async-task boundaries in a swarm loop

OBJECTIVE

Read a libp2p-style event loop and explain who owns the swarm, who owns application state, and what moves through channels or tasks.

STARTER PROMPT

You inherit a node that stores the swarm behind a broad shared lock while several unrelated tasks mutate protocol state and write directly to transport handles.

- Which part of the system should own the swarm loop directly?
- Which data should cross boundaries as owned commands or events instead of as shared references?
- Which path becomes easier to reason about once one writer owns outbound transport effects?
- What cancellation or shutdown bug becomes more likely when many tasks all think they own the connection state?

ACCEPTANCE CRITERIA

- You identify at least one owner for the swarm and one owner for application state explicitly.
- You propose owned messages or commands at one async or channel boundary.
- You connect the answer to shutdown, ordering, or duplicate-state risk rather than only to syntax taste.

HINTS

- The broad shared lock is often compensating for a missing ownership boundary.
- One event loop with owned inputs is usually calmer than many tasks with partial transport authority.

EXERCISE 3 IMPLEMENTATION

Build a small libp2p-style swarm loop over owned enums

OBJECTIVE

Implement one small event loop that records connected peers, tracks one pending request map, and clears state on response.

STARTER PROMPT

Create `PeerId`, `Event`, and `SwarmState`, then handle a connect, request, and response sequence with no real networking required.

- Keep the owner as one plain Rust struct.
- Use one map keyed by request ID for pending work.
- Use one small log or event queue for observability.
- Do not use `Rc`, `RefCell`, or raw pointers for the main design.

ACCEPTANCE CRITERIA

- The event loop keeps one central owner for protocol state.
- Requests create pending entries and responses clear them.
- The runnable lab prints the expected connected count, pending count, and final log line.
- The code is plausible Rust and easy to test without a live network.

HINTS

- This is the same ownership story many real libp2p nodes want internally.
- A pending-request map is often the simplest way to make response handling honest.

EXERCISE 4 DEBUGGING OR REFACTORING

Repair peer discovery with abuse controls and observability

OBJECTIVE

Turn an unbounded peer-discovery feature into one that has explicit limits, dedupe, and telemetry before it melts under hostile or noisy inputs.

STARTER PROMPT

A discovery subsystem accepts every advertised peer, queues unlimited dials, and emits no metric for dial failure, relay fallback, or duplicate discovery.

- Where should dedupe live: known peer set, pending dial set, or both?
- Which queue or dial-attempt budgets should be explicit?
- Which peer records should be rejected before expensive work happens?
- What metrics or logs should exist before the feature rolls to production?

ACCEPTANCE CRITERIA

- You add or propose at least one dedupe boundary and one explicit budget.
- You identify one cheap validation step that should run before expensive dial or allocation work.
- You mention at least two observability hooks such as dial failure count, relay usage, or oldest pending dial age.
- You explain one test or simulation case for the repair.

HINTS

- Discovery is both a correctness problem and a resource-exhaustion problem.
- If you do not measure relay fallback, you may not know the network is degraded until users tell you.

EXERCISE 5 DESIGN OR PRODUCTION SCENARIO

Design NAT traversal, relay fallback, and connectivity telemetry

OBJECTIVE

Choose a realistic connectivity policy for peers that do not all live on public routable networks.

STARTER PROMPT

You are deploying a peer mesh across laptops, cloud VMs, and branch-office devices. Some peers can accept inbound dials, some cannot, and relay capacity is limited.

- Which peers should try direct dial first, and when should they fall back to a relay?
- Which metrics should prove whether relay usage is becoming the default path accidentally?
- How would you decide the lease or retry policy for repeated failed direct dials?
- What operator dashboard fields would you require before rollout?

ACCEPTANCE CRITERIA

- You define one direct-dial path and one relay fallback path explicitly.
- You mention at least one retry or budget rule for failed connectivity attempts.
- You include at least three observability hooks such as hole-punch success, relay usage, dial failure rate, or oldest queued outbound connection.
- You keep the explanation grounded in product and network shape rather than generic P2P enthusiasm.

HINTS

- A relay is a useful escape hatch and a dangerous silent default.
- Connectivity budgets matter before application throughput budgets do.

EXERCISE 6 DESIGN OR PRODUCTION SCENARIO

Choose P2P vs brokered messaging, gRPC, or HTTP for one product

OBJECTIVE

Defend whether a given product boundary really benefits from P2P instead of from a more centralized transport.

STARTER PROMPT

Your team is building a collaborative edge cache, a public billing API, and an internal durable task pipeline. Choose the primary communication model for each and justify the choice.

- Which path truly benefits from direct peer identity and partial decentralization?
- Which path wants durable replay and queue semantics instead of mesh connectivity?
- Which path wants typed service RPC or plain HTTP rather than overlay routing?
- Would any of the systems benefit from a hybrid model, and where would the boundary split?

ACCEPTANCE CRITERIA

- You choose at least one case for P2P and at least one case against it with a clear reason.
- You include at least one hybrid design where control plane and data plane use different transports.
- You explain the choice in terms of durability, topology, or operational cost rather than only in terms of implementation style.

HINTS

- P2P is not the advanced answer to every network problem.
- Hybrid is often the calm senior answer when topology and durability pull in different directions.

Runnable lab

Example 50-L. `swarm_loop_lab.rs` — *Runnable lab · Swarm loop with owned events*

```rust
use std::collections::{HashMap, VecDeque};

#[derive(Debug, Clone, Copy, PartialEq, Eq, Hash)]
struct PeerId(&'static str);

#[derive(Debug, Clone)]
enum Event {
    Connected(PeerId),
    Request {
        peer: PeerId,
        id: u64,
    },
    Response {
        peer: PeerId,
```

```
        id: u64,
        body: String,
    },
}

#[derive(Debug, Default)]
struct SwarmState {
    connected: Vec<PeerId>,
    pending: HashMap<u64, PeerId>,
    log: VecDeque<String>,
}

impl SwarmState {
    fn on_event(&mut self, event: Event) {
        match event {
            Event::Connected(peer) => {
                self.connected.push(peer);
            }
            Event::Request { peer, id } => {
                self.pending.insert(id, peer);
                self.log.push_back(format!("request {}", id));
            }
            Event::Response { peer, id, body } => {
                // repair this branch
            }
        }
    }
}

fn main() {
    let mut state = SwarmState::default();
    let peer = PeerId("peer-b");

    state.on_event(Event::Connected(peer));
    state.on_event(Event::Request { peer, id: 7 });
    state.on_event(Event::Response {
        peer,
        id: 7,
        body: String::from("state-v3"),
    });

    println!("connected = {}", state.connected.len());
    println!("pending = {}", state.pending.len());
    println!("last = {}", state.log.back().map(String::as_str).unwrap_or("none"));
}
```

OUTPUT

```
connected = 1
pending = 0
last = response peer-b state-v3
```

Review questions

- Why do owned events and commands make a swarm loop easier to test than shared transport state does?
- What is the practical difference between peer identity, channel security, and application-level authorization?
- Why are NAT traversal and relay fallback product concerns instead of networking trivia?
- When is a deterministic scalar version plus tie-break rule enough for sync, and when is it too weak?
- What signs tell you a product really wants P2P instead of a broker or service API?

PART VIII

Frontier Specializations

Dip in on demand: zero-knowledge proofs, verifiable and portable ML inference, on-chain smart contracts, and no-std for constrained targets.

51 Zero-Knowledge Proofs for Rust Engineers

52 ZoKrates Workflows and Ethereum Verifiers

53 EZKL, Verifiable LLM Inference, and GPU-Aware ZKML

54 ONNX Model Inference in Rust

55 Smart Contracts in Rust: Solana, Sei, and Beyond

56 no-std Rust and Constrained Runtime Derivatives

CHAPTER 51 · ZERO-KNOWLEDGE PROOFS FOR RUST ENGINEERS

Zero-Knowledge Proofs for Rust Engineers

Exercises, labs, and review questions for this chapter.

Keep public and private material distinct, isolate proving from verification, and threat-model proof-backed APIs

EXERCISE 1 WARM-UP COMPREHENSION

Separate public inputs, private witnesses, and proofs

OBJECTIVE

Practice saying which data the verifier may know, which data must stay private, and which artifact is only the proof.

STARTER PROMPT

Classify one proof-backed billing flow with a public credit limit, a public committed digest, private line items, and one proof sent to a verifier API.

- Which fields are public inputs the verifier is expected to see?
- Which fields are witness-only and should never cross the verification boundary?
- Which artifact is the proof, and which artifacts are proving or verification keys?
- Which fields would still be safe to log at the verifier boundary, and which would not?

ACCEPTANCE CRITERIA

- You distinguish public statement data from witness data clearly.
- You name the proof as a separate artifact instead of treating it like the witness itself.
- You identify at least one logging or transport consequence of that separation.

HINTS

- Ask what the verifier is supposed to learn by design.
- If the verifier sees raw private inputs, the system may still be correct but it is no longer zero-knowledge in the intended sense.

EXERCISE 2 CODE READING

Refactor an API workflow so proving and verification are isolated

OBJECTIVE

Read one service path and explain why the prover boundary should not look like the verifier boundary.

STARTER PROMPT

A request handler currently derives the witness, generates the proof inline, verifies it again locally, and stores both proof and witness in the same outbound record.

- Which steps belong in a proving worker or background lane rather than in the request handler?
- Which boundary should receive only statement plus proof artifact?
- Which artifact custody mistake makes private witness data too easy to leak or over-retain?
- What metric would tell you the inline proving path is already hurting the API edge?

ACCEPTANCE CRITERIA

- You move at least one heavy step out of the request handler.
- You separate verifier input from witness input explicitly.
- You identify one privacy or retention problem in the original workflow.
- You mention at least one latency or queue signal that the repaired design should expose.

HINTS

- The verifier should not need the witness.
- The fact that the server can verify locally does not mean the same process should always do both jobs inline.

EXERCISE 3 IMPLEMENTATION

Define typed proving and verification requests

OBJECTIVE

Design Rust types that force the prover and verifier boundaries to stay honest.

STARTER PROMPT

Create one `ProofRequest`, one `VerificationRequest`, and one `ProofArtifact` shape for a service that proves a public total is below a public limit without revealing the hidden addends.

- Keep witness fields only in the proving request.
- Keep verifier-facing types free of witness material.
- Carry a circuit ID or protocol version explicitly.
- Add one place for public statement fields such as total and limit.

ACCEPTANCE CRITERIA

- The proving and verification structs are distinct.
- The verification path does not require witness-only fields.
- You include one version or circuit-identity field.
- The design is small enough that another engineer could review it quickly.

HINTS

- If one struct is doing every job, the boundary is probably too wide.
- Versioning belongs on artifacts and requests early, not only after the first migration pain.

EXERCISE 4 DEBUGGING OR REFACTORING

Repair transcript and serialization bugs before proofs go cross-service

OBJECTIVE

Fix the integration mistakes that let a prover and verifier disagree even though both local components 'work.'

STARTER PROMPT

One service serializes public inputs through a map with unstable key order, another service changed the transcript domain string during a refactor, and verification now fails intermittently across deployments.

- Which part of the system needs canonical ordering or stable field layout?
- Why is a changed transcript domain string a protocol break rather than a cosmetic edit?
- Which CI or contract test should fail before rollout next time?
- What artifact should carry the protocol or circuit version explicitly?

ACCEPTANCE CRITERIA

- You identify at least one canonical serialization repair and one domain-separation repair.
- You explain why the issue is a protocol-compatibility bug rather than only a runtime bug.
- You propose at least one drift-prevention check in CI or contract tests.

HINTS

- A prover and verifier that hash the same fields in different order are not interoperable.
- Domain strings are part of the transcript contract, not a free rename surface.

EXERCISE 5 DESIGN OR PRODUCTION SCENARIO

Threat-model a proof-backed API or game mechanic

OBJECTIVE

Make privacy, integrity, denial-of-service, and operator risks explicit before the system ships.

STARTER PROMPT

Design a threat model for one proof-backed API or game mechanic where clients submit proofs to unlock rewards or verify hidden state transitions.

- What must remain private, and what can still leak through public inputs or metadata?
- Which attack path is resource exhaustion: giant artifacts, repeated invalid proofs, or expensive witness requests?
- Which trust assumption would you call out explicitly: setup artifacts, circuit correctness, key distribution, or transcript policy?
- What telemetry would let operators see an attack or misconfiguration early?

ACCEPTANCE CRITERIA

- You describe at least one privacy risk, one integrity risk, and one denial-of-service risk.
- You separate cryptographic assumptions from Rust implementation or deployment assumptions.
- You mention at least two observability hooks such as verification failure rate, artifact size, or proving queue age.

HINTS

- A proof-backed design can still leak metadata or fall over under load.
- Threat models are strongest when they include both adversarial and operator-failure cases.

EXERCISE 6 DESIGN OR PRODUCTION SCENARIO

Plan witness generation, artifact custody, and hardware lanes for a proof workflow

OBJECTIVE

Turn one abstract proving story into an operable Rust system with explicit artifacts and specialized execution boundaries.

STARTER PROMPT

You are integrating a proof workflow that may involve an external circuit DSL or ML proof toolchain, witness files, proving parameters, GPU acceleration, and a lightweight verifier service.

- Where does circuit or model compilation end and witness generation begin?
- Which artifacts should be cached, versioned, or content-addressed: circuit ID, proving key, verification key, witness schema, proof blob?
- Which stage might justify GPU acceleration, and how would you isolate that boundary from the API edge?
- Which rollout or testing rule would you require before several teams depend on the verifier output?

ACCEPTANCE CRITERIA

- You distinguish compiled circuit or model artifacts from witness generation and proof artifacts.
- You name at least three explicit artifacts and one versioning or custody rule.
- You isolate the heavy proving lane from the lightweight verifier lane.
- You mention at least one testing or rollout gate for mixed deployments.

HINTS

- This is where external ecosystems such as ZoKrates or EZKL still need ordinary systems design.
- A good answer makes artifact ownership boring enough that operations and incident response stay possible.

Runnable lab

Example 51-L. `proof_boundary_lab.rs` — *Runnable lab · Keep verification separate from the witness*

```rust
#[derive(Debug, Clone, Copy)]
struct Statement {
    public_total: u64,
    public_limit: u64,
}

#[derive(Debug, Clone, Copy)]
struct Witness {
    left: u64,
    right: u64,
}

#[derive(Debug, Clone, Copy)]
struct ProofArtifact {
```

```
    public_total: u64,
    public_limit: u64,
    proof_bytes_len: usize,
}

fn prove(statement: Statement, witness: Witness) -> Result<ProofArtifact, &'static str> {
    if witness.left + witness.right != statement.public_total {
        return Err("sum constraint failed");
    }

    if statement.public_total > statement.public_limit {
        return Err("limit violated");
    }

    Ok(ProofArtifact {
        public_total: statement.public_total,
        public_limit: statement.public_limit,
        proof_bytes_len: 96,
    })
}

fn verify(_statement: Statement, _proof: ProofArtifact) -> bool {
    false
}

fn main() {
    let statement = Statement {
        public_total: 45,
        public_limit: 50,
    };
    let witness = Witness { left: 20, right: 25 };
    let proof = prove(statement, witness).unwrap();

    println!("verified = {}", verify(statement, proof));
    println!("public total = {}", statement.public_total);
    println!("proof bytes = {}", proof.proof_bytes_len);
}
```

OUTPUT

```
verified = true
public total = 45
proof bytes = 96
```

Review questions

- Why should witness generation and verification usually live at different service boundaries?
- What makes transcript domain separation a protocol rule instead of one local implementation detail?
- Why is a proof artifact not the same thing as a witness or proving key?
- What does zero knowledge fail to protect if public inputs or metadata are already too revealing?
- Why should large proof systems still be reviewed for ordinary Rust concerns such as queues, logging, unsafe wrappers, and resource limits?

CHAPTER 52 · ZOKRATES WORKFLOWS AND ETHEREUM VERIFIERS

ZoKrates Workflows and Ethereum Verifiers

Exercises, labs, and review questions for this chapter.

Split build/release/runtime responsibilities, version artifacts, keep verifier requests witness-free, and plan audits and rollouts

EXERCISE 1 WARM-UP COMPREHENSION

Split the ZoKrates workflow into build-time, release-time, and runtime responsibilities

OBJECTIVE

Practice assigning each ZoKrates step to the right operational lane instead of letting the whole pipeline collapse into one opaque script.

STARTER PROMPT

Classify `compile`, `setup`, `compute-witness`, `generate-proof`, `export-verifier`, and `verify` across build-time, release-time, and runtime boundaries for one proof-backed service.

- Which steps happen once per circuit or release rather than once per request?
- Which steps belong in a proving worker instead of a request handler?
- Which steps can happen in CI or a controlled admin CLI lane rather than in build.rs?
- Which step should stay verifier-side and witness-free?

ACCEPTANCE CRITERIA

- You place compile, setup, witness generation, proof generation, export, and verification in distinct operational lanes.
- You identify at least one step that should not run per request.
- You explain why verifier input should exclude witness material.

HINTS

- Start by asking which artifact is expensive and which one is durable.
- A good answer treats proof generation like a worker job, not like a tiny helper function.

EXERCISE 2 CODE READING

Review a proof workflow for proof-friendly input modeling

OBJECTIVE

Spot the places where transport-shaped data is being pushed into the ZoKrates boundary without enough normalization or commitment discipline.

STARTER PROMPT

A Rust API currently forwards raw JSON strings, variable-length lists, and unbounded notes fields directly into a proof step that is supposed to justify only one public total and one public limit.

- Which parts should become canonical numeric inputs before witness generation?
- Which large or irregular fields should become commitments or stay witness-only?
- Which fields should never become public verifier inputs just because they already existed in the HTTP payload?
- Which tests would prove the normalization contract is stable across versions?

ACCEPTANCE CRITERIA

- You identify at least one field that should be normalized in Rust before the ZoKrates step.
- You identify at least one field that should remain witness-only or become a commitment.
- You mention at least one reproducibility or contract test for the mapping.

HINTS

- Transport convenience and proof convenience are rarely the same thing.
- If the verifier does not need a field, it probably should not become a public input by accident.

EXERCISE 3 IMPLEMENTATION

Design a versioned artifact manifest for proof generation and verification

OBJECTIVE

Build a small Rust-facing manifest that keeps circuit, keys, proof blobs, and verifier contract metadata attributable.

STARTER PROMPT

Design one `ArtifactManifest` or equivalent set of Rust types that names a circuit ID, proving key ID, verification key ID, proof path, and verifier contract path or address.

- Keep proving-side and verifying-side metadata distinguishable.
- Include one protocol or circuit version field.
- Decide which artifact fields are safe to expose to verifier callers and which are proving-only.
- Think about retention policy and auditability while naming the fields.

ACCEPTANCE CRITERIA

- Your design names at least four distinct artifacts explicitly.
- You include one version or circuit-identity field.
- You separate at least one proving-only field from a verifier-facing field.
- The resulting type set is small enough for another engineer to review quickly.

HINTS

- If one struct does every job, the custody model is probably too vague.
- Versioning belongs on artifacts early, not after the first migration failure.

EXERCISE 4 DEBUGGING OR REFACTORING

Repair a verifier integration that leaks witness or mixes versions

OBJECTIVE

Fix the most common production mistakes around verifier requests: witness exposure, wrong key pairing, or wrong contract version.

STARTER PROMPT

You inherit a verifier service that stores witness paths next to verifier request DTOs, chooses the verifier contract from a mutable config string at runtime, and sometimes pairs a new proof blob with an older verification key.

- Which fields should disappear from verifier-facing request types immediately?
- How should contract address or path selection become versioned and explicit?
- Which key or circuit mismatch should fail before any external verifier call is attempted?
- Which log or trace fields belong on that failure path?

ACCEPTANCE CRITERIA

- You remove at least one witness-related field from the verifier boundary.
- You define one explicit version or key-matching rule.
- You identify one preflight validation that should fail before a verifier call or transaction submit happens.
- You mention at least one structured field such as circuit ID, key ID, or contract version.

HINTS

- If the verifier can see the witness path, the boundary is already too wide.
- Version mismatch is a contract failure, not a 'maybe verify and see' runtime strategy.

EXERCISE 5 DEBUGGING OR REFACTORING

Design reproducibility and negative tests for the proof pipeline

OBJECTIVE

Turn proof generation and verification into a testable workflow rather than a one-off manual command sequence.

STARTER PROMPT

You want CI confidence that proof generation still works after refactors, and that wrong inputs, wrong versions, or wrong keys fail clearly rather than failing as one generic false result.

- Which artifact versions should be pinned in CI fixtures?
- Which negative cases must exist: wrong public inputs, wrong verification key, wrong contract version, or wrong witness?
- Where should you use native verification versus Ethereum-oriented verifier tests?
- How would you keep the fixtures reproducible and reviewable over time?

ACCEPTANCE CRITERIA

- You define at least two positive and two negative workflow tests.
- You separate native verify tests from contract or deploy-path tests.
- You mention at least one artifact pinning or manifest rule that improves reproducibility.
- You explain how failing cases become attributable rather than generic.

HINTS

- A fast native verify path is a good first CI gate, but it is not the same as a deploy-path test.
- The best negative test is the one that tells you which contract or artifact drifted.

EXERCISE 6 DESIGN OR PRODUCTION SCENARIO

Plan audits, upgrades, and rollout for an Ethereum verifier integration

OBJECTIVE

Make trusted setup, verifier deployment, dual-version rollout, and operational ownership explicit before the first circuit upgrade.

STARTER PROMPT

You are shipping a proof-backed service whose verifier contract lives in an Ethereum-oriented environment. A circuit update is coming, setup provenance matters, and operators want canary rollout plus rollback.

- Which audit surfaces exist separately: proving scheme assumptions, circuit logic, Rust artifact custody, and Solidity verifier deployment?
- How will old and new verifier versions coexist during rollout?
- Which artifact or address mapping must be published for operators and downstream consumers?
- Which telemetry or failure rates gate promotion or trigger rollback?

ACCEPTANCE CRITERIA

- You name at least three distinct audit or review surfaces.
- You describe one mixed-version rollout or dual-verifier strategy.
- You include at least one published artifact mapping such as circuit ID to contract address.
- You mention at least two rollout signals such as verification failure rate, proving queue age, or contract-submit failure rate.

HINTS

- An upgrade is rarely just one code deploy. It is usually an artifact and contract migration too.
- Keep the rollback story boring enough that another engineer can execute it under pressure.

Runnable lab

Example 52-L. `verifier_request_boundary.rs` — *Runnable lab · Build a verifier request without leaking witness material*

```rust
#[derive(Debug, Clone)]
struct ProofBundle {
    verifier_contract: String,
    public_inputs: Vec<String>,
    proof_path: String,
    witness_path: String,
}

#[derive(Debug, Clone)]
struct VerificationRequest {
    contract: String,
    public_inputs_len: usize,
    proof_path: String,
    witness_included: bool,
}
```

```
fn to_verification_request(bundle: &ProofBundle) -> VerificationRequest {
    VerificationRequest {
        contract: String::new(),
        public_inputs_len: 0,
        proof_path: bundle.witness_path.clone(),
        witness_included: true,
    }
}

fn main() {
    let bundle = ProofBundle {
        verifier_contract: String::from("contracts/AgeCheckVerifier.sol"),
        public_inputs: vec![String::from("45"), String::from("50")],
        proof_path: String::from("artifacts/proof.json"),
        witness_path: String::from("artifacts/witness"),
    };

    let request = to_verification_request(&bundle);

    println!("contract = {}", request.contract);
    println!("public inputs = {}", request.public_inputs_len);
    println!("witness included = {}", request.witness_included);
}
```

OUTPUT

```
contract = contracts/AgeCheckVerifier.sol
public inputs = 2
witness included = false
```

Review questions

- Why should compile and setup usually live in different operational lanes from witness generation and proof generation?
- What makes a proof request different from a verifier request in Rust terms?
- Why is contract or key version drift often a bigger operational risk than one single failed proof?
- What does native verification prove, and what does it not prove about the Ethereum deployment path?
- Why should generated verifier contracts still go through normal Solidity review and release discipline?

CHAPTER 53 · EZKL, VERIFIABLE LLM INFERENCE, AND GPU-AWARE ZKML

EZKL, Verifiable LLM Inference, and GPU-Aware ZKML

Exercises, labs, and review questions for this chapter.

Classify ZKML artifacts, separate proving from verification, profile bottlenecks honestly, and plan versioned rollout

EXERCISE 1 WARM-UP COMPREHENSION

Classify which parts of an LLM inference workflow are public, private, deterministic, or artifact-bound

OBJECTIVE

Practice separating verifier-visible claims from witness-only material and from versioned artifacts before the first proving job is queued.

STARTER PROMPT

Classify one LLM scoring workflow with a tokenizer version, prompt token IDs, attention mask, quantization bits, output commitment, proof blob, verifier key ID, and a public model hash.

- Which fields are public inputs the verifier is expected to see?
- Which fields are witness-only and should stay on the proving side?
- Which fields are artifacts that must be versioned even if they are not secret?
- Which parts must be deterministic for the workflow to stay reproducible across prover and verifier deployments?

ACCEPTANCE CRITERIA

- You separate public claims from witness material clearly.
- You identify at least two artifact-bound fields such as model hash, verifier key ID, or tokenizer version.
- You name at least one reproducibility consequence of getting the classification wrong.

HINTS

- Ask what the verifier is supposed to learn, what the prover must know, and what both sides must agree on.
- Artifact-bound does not automatically mean secret; it often means versioned and attributable.

EXERCISE 2 CODE READING

Design a Rust service boundary around an EZKL-style proving job

OBJECTIVE

Read one service workflow and decide where HTTP or gRPC transport ends, where the proving queue begins, and where the verifier lane becomes witness-free.

STARTER PROMPT

A service currently accepts an API request, tokenizes input inline, generates a witness inline, proves inline, verifies inline, and stores prompt text together with the proof blob in one response record.

- Which steps belong in the submitter edge versus a proving worker lane?
- Which boundary should first own an artifact manifest instead of raw request-local state?
- Which data should never appear in the verifier-facing request type?
- Which metric would tell you the inline design is already hurting the API edge?

ACCEPTANCE CRITERIA

- You move at least one heavyweight step out of the request handler.
- You identify at least one verifier boundary that should be witness-free.
- You place artifact versioning or manifest lookup at one explicit boundary.
- You mention at least one latency or queue signal that the repaired design should expose.

HINTS

- The proving lane should look like a worker boundary, not like a convenience helper.
- If the verifier sees prompt tokens or witness paths, the boundary is already too wide.

EXERCISE 3 IMPLEMENTATION

Implement separate proving and verification request types

OBJECTIVE

Build small Rust types that force the proving request and verification request to stay distinct.

STARTER PROMPT

Design one `ProvingJob`, one `ProofArtifact`, and one `VerificationJob` for a workflow that proves a public output commitment was produced from one model artifact without revealing the raw prompt.

- Keep witness fields only on the proving side.
- Keep verifier fields limited to public claims and proof artifact references.
- Add one circuit or model version field.
- Name at least one artifact ID explicitly instead of hiding it in a path string only.

ACCEPTANCE CRITERIA

- The proving and verification types are structurally distinct.
- The verifier-facing type does not require witness-only data.
- You include one version or artifact-identity field explicitly.
- The design is small enough that another engineer could review it quickly.

HINTS

- If one struct does every job, the boundary is probably still vague.
- Versioning belongs in the type shape early, not after the first migration pain.

EXERCISE 4 DEBUGGING OR REFACTORING

Repair numerical reproducibility drift before the prover and verifier split across services

OBJECTIVE

Fix the class of bugs where prover and verifier disagree because preprocessing or transcript policy drifted, not because the proof backend is broken.

STARTER PROMPT

One deployment changed quantization bits, another changed tokenizer special-token handling, and a third reordered public inputs in one manifest serializer. Cross-service verification now fails intermittently.

- Which parts of the workflow need canonical ordering or stable field layout?
- Which fields belong in the artifact manifest so a mismatch is visible before prove or verify happens?
- Why is quantization or tokenization drift a protocol break rather than just a model-tuning difference?
- Which CI or contract test should fail before rollout next time?

ACCEPTANCE CRITERIA

- You identify at least one canonical serialization repair and one preprocessing-version repair.
- You explain why the issue is a protocol-compatibility bug, not only a runtime bug.
- You propose at least one CI or contract gate for drift detection.

HINTS

- If both sides compute a different graph or public input ordering, the proof boundary changed.
- A deterministic artifact manifest is often the simplest first repair.

EXERCISE 5 DEBUGGING OR REFACTORING

Profile a hypothetical GPU-backed ZKML pipeline and identify the real bottleneck

OBJECTIVE

Separate transfer-bound, witness-bound, prove-bound, verify-bound, and queue-bound cases clearly before changing hardware or code.

STARTER PROMPT

A window summary shows queue wait 700 ms, witness generation 260 ms, proof generation 1100 ms, verification 35 ms, and GPU transfer 220 ms for one batch path.

- Which category dominates wall time right now?
- Which rewrite is probably premature because the bottleneck is elsewhere?
- What next measurement would you take inside the dominant category?
- Which signals would prove the repair moved the real bottleneck instead of only changing one micro-metric?

ACCEPTANCE CRITERIA

- You identify the current dominant latency layer correctly.
- You reject at least one likely but wrong optimization target.
- You propose one deeper measurement and at least one before-and-after validation metric.
- You treat GPU usage as workload-dependent rather than automatically correct.

HINTS

- If queue wait is already high, faster kernels may not move the user-visible story enough.
- Keep witness, prove, verify, and transfer as separate buckets.

EXERCISE 6 DESIGN OR PRODUCTION SCENARIO

Productize ZKML with APIs, job orchestration, artifact versioning, and observability

OBJECTIVE

Turn one proof-backed inference prototype into an operable service design with bounded queues and reviewable artifact custody.

STARTER PROMPT

You are shipping a service with submit, status, and verify endpoints; CPU and GPU proving lanes; one artifact registry; and one downstream audit sink. Mixed model versions and canary rollout are expected.

- Which artifacts should be versioned and content-addressed: model hash, circuit ID, quantization manifest, verifier key ID, proof blob, tokenizer manifest?
- Which queues or worker pools deserve separate budgets?
- Where should fallback from GPU to CPU happen, and how should it be exposed to operators?
- Which metrics, traces, and alerts would you require before the service is called production-ready?

ACCEPTANCE CRITERIA

- You name at least three explicit artifacts and one custody or version rule for each class.
- You define at least one separate queue or worker-lane budget.
- You include one fallback or failure-isolation rule around the GPU lane.
- You mention at least three observability hooks such as queue age, proof bytes, verification failures, or fallback rate.

HINTS

- A good answer makes proving heavy, verification light, and artifacts attributable.
- If mixed model versions are expected, treat version lookup as a first-class service concern.

Runnable lab

Example 53-L. `gpu_queue_boundary_lab.rs` — *Runnable lab · Build a verifier request without leaking witness material*

```rust
#[derive(Debug, Clone)]
struct ProvingJob {
    model_id: String,
    gpu_requested: bool,
    public_outputs: Vec<String>,
    witness_path: String,
    proof_path: String,
}

#[derive(Debug, Clone)]
struct VerificationJob {
    proof_path: String,
    public_output_count: usize,
    witness_included: bool,
```

```
}

fn build_verification_job(job: &ProvingJob) -> VerificationJob {
    VerificationJob {
        proof_path: job.witness_path.clone(),
        public_output_count: 0,
        witness_included: true,
    }
}

fn main() {
    let job = ProvingJob {
        model_id: String::from("llm-int8:v3"),
        gpu_requested: true,
        public_outputs: vec![String::from("token_hash"), String::from("score_hash")],
        witness_path: String::from("artifacts/witness.json"),
        proof_path: String::from("artifacts/proof.pf"),
    };

    let verify = build_verification_job(&job);
    let prove_queue = if job.gpu_requested { "zkml.gpu" } else { "zkml.cpu" };

    println!("prove queue = {}", prove_queue);
    println!("verify payload = {}", verify.public_output_count);
    println!("witness leaked = {}", verify.witness_included);
}
```

OUTPUT

```
prove queue = zkml.gpu
verify payload = 2
witness leaked = false
```

Review questions

- Why should verification requests usually stay witness-free?
- What makes tokenization, quantization, and public-input ordering protocol concerns instead of private implementation details?
- Why is a proving lane usually a queue and capacity problem as much as a cryptography problem?
- What does a verifiable inference system prove, and what does it not prove?
- Why should GPU-backed proving still be treated as an optional execution lane rather than as the whole architecture?

CHAPTER 54 · ONNX MODEL INFERENCE IN RUST

ONNX Model Inference in Rust

Exercises, labs, and review questions for this chapter.

Load and run a session, shape and batch tensors, choose execution providers, and own pre- and post-processing at the model boundary

EXERCISE 1 WARM-UP COMPREHENSION

Name what the session owns and what you own

OBJECTIVE

Restate the three-stage inference pipeline and identify which stage the ONNX runtime is responsible for.

STARTER PROMPT

The chapter frames inference as a three-stage pipeline with the session in the middle: raw input becomes a tensor, the session turns that tensor into output tensors, and post-processing turns those into a domain decision. In your own words, describe each stage and assign ownership of correctness.

- List the three stages in order and say what data crosses each boundary.
- State which single stage the runtime owns and which two stages are ordinary Rust code you write.
- Explain why the chapter says the runtime makes the math correct but does nothing to make shapes, dtypes, and label maps correct.
- Point to where forward and argmax from the chapter's listings fall in this pipeline.

ACCEPTANCE CRITERIA

- The answer names pre-processing (build the input tensor), the session run (tensor to output tensors), and post-processing (output tensors to a domain decision) in that order.
- The answer states that the runtime owns only the middle box and that the two ends are caller-owned Rust code.
- The answer maps forward to the session's arithmetic and argmax to caller-owned post-processing.
- The answer explains that shape, dtype, and label-map errors are the caller's responsibility, not the runtime's.

HINTS

- The session owns the math; you own the contract that feeds and reads it.
- Re-read the mental-model paragraph: the runtime owns the middle, the two ends are yours.

EXERCISE 2 CODE READING

Trace one input through the dense layer and argmax

OBJECTIVE

Read the chapter's forward and argmax listing closely enough to predict its exact output by hand.

STARTER PROMPT

Using the chapter's WEIGHTS, BIAS, and the fixed input_tensor [0.5, -1.0, 2.0, 0.25], compute the three logits by hand and determine which class argmax selects, without running the program.

- Write out logits[o] = bias[o] + sum_c weights[o][c] * input[c] for each of the three output classes.
- Compute all three logit values to four decimal places.
- Apply argmax: which index holds the largest logit, and what score does it print?
- Confirm that this is exactly what a one-layer ONNX Gemm operator plus your argmax would produce.

ACCEPTANCE CRITERIA

- The three logits are computed as approximately [-1.5750, 1.0500, 0.9750].
- argmax selects class 1 because logit[1] is the largest.
- The reported score equals the value of logit[1], approximately 1.0500.
- The answer identifies the matmul-plus-bias as the Gemm the runtime would run and argmax as caller-owned post-processing.

HINTS

- Each logit is one dot product of an input vector with one weight row, plus that row's bias.
- argmax never normalizes; it only compares raw logits, so the largest raw value wins.

EXERCISE 3 IMPLEMENTATION

Add a softmax post-processing stage

OBJECTIVE

Implement a numerically stable softmax as caller-owned post-processing that turns logits into a probability distribution.

STARTER PROMPT

Extend the chapter's one-layer model with fn softmax(logits: &[f32; OUT]) -> [f32; OUT] so the three logits become probabilities that sum to 1.0, then print the confidence of the predicted class.

- Keep the signature fn softmax(logits: &[f32; OUT]) -> [f32; OUT].
- Subtract the maximum logit before calling exp so large values do not overflow.
- Divide each exponentiated value by the total so the output sums to 1.0.
- Print the probability vector and the confidence of the argmax class.

ACCEPTANCE CRITERIA

- softmax borrows the logits and returns a new [f32; OUT] without mutating its input.
- The implementation subtracts the maximum logit before exponentiating for numerical stability.
- The returned probabilities sum to 1.0 within floating-point rounding.
- For the chapter's fixed input the confidence printed for class 1 is approximately 0.5000.

HINTS

- softmax is post-processing you own, just like argmax; the runtime does not do it for you.
- exp() lives on f32 in std, so no external crate is needed.

EXERCISE 4 IMPLEMENTATION

Extend the batch and verify per-row argmax

OBJECTIVE

Work with a row-major flat tensor by adding a third row and recovering each row by stride without copying the buffer.

STARTER PROMPT

Starting from the chapter's batched listing, add a third input row to the flat Vec<f32> and confirm that the per-row loop slicing with r * IN..(r + 1) * IN still recovers each row correctly and prints the right argmax per row.

- Append a third feature row to build_batch so the flat buffer holds three rows of IN values.
- Confirm rows is computed as batch.len() / IN and now equals 3.
- Verify the slice &batch[r * IN..(r + 1) * IN] borrows each row without allocating.
- Run forward_row and argmax on the new row and state its predicted class and score.

ACCEPTANCE CRITERIA

- The flat Vec<f32> length is a multiple of IN and rows evaluates to 3.
- Each row is obtained by slicing the existing buffer, with no per-row allocation or copy.
- The loop prints one argmax and score line per row, including the new third row.
- The answer explains that one session run over N rows amortizes session overhead better than N single-row runs.

HINTS

- The batch is one contiguous buffer laid out row-major, exactly how a runtime stores a [rows, features] tensor.
- Slicing returns a borrowed &[f32]; the underlying Vec keeps owning the bytes.

EXERCISE 5 DEBUGGING OR REFACTORING

Diagnose a shape and dtype mismatch

OBJECTIVE

Reason about why a tensor whose shape or dtype disagrees with the graph either errors or silently returns wrong answers.

STARTER PROMPT

A colleague's session run sometimes errors and sometimes returns confident nonsense after they changed the input builder. The graph declares a named input of shape [4] with dtype f32, but their code now passes a length-3 vector in one path and an i64 buffer in another. Explain each failure and fix the contract.

- Explain what happens when the input length is 3 but the graph fixed the input dimension at 4.
- Explain why passing an i64 buffer where the graph expects f32 is a different class of bug.
- Identify which of these the runtime can catch at run time and which can run and return wrong logits.
- Describe the smallest fix that restores the declared shape and dtype contract.

ACCEPTANCE CRITERIA

- The answer identifies a length-3 input against a fixed [4] input as a shape mismatch that the runtime should reject.
- The answer identifies the f32-versus-i64 dtype mismatch as a separate failure that can run and produce confident nonsense.
- The answer states that the most common inference bug is a shape or dtype mismatch, not a wrong model.
- The fix restores the input to a length-4 f32 tensor matching the named graph input.

HINTS

- The graph names each input and fixes its shape and dtype; your tensor must match exactly.
- A wrong dtype can be reinterpreted byte-for-byte and still produce numbers, which is why it is dangerous.

EXERCISE 6 DESIGN OR PRODUCTION SCENARIO

Choose ort or tract and a session lifecycle

OBJECTIVE

Make and justify a runtime and execution-provider decision under a stated deployment constraint, and place session construction correctly.

STARTER PROMPT

Your team must serve the exported classifier inside an existing Rust service. Constraint A: the service ships as a single binary that cross-compiles cleanly to several targets, no GPU, no native dependencies. Constraint B: the service runs on a CUDA host and latency at the 99th percentile matters. Decide ort versus tract for each, and place session construction in the request lifecycle.

- For Constraint A, choose between the native ort runtime and the pure-Rust tract path and justify it from build and deploy concerns.
- For Constraint B, choose a runtime and describe registering execution providers in priority order with CPU fallback.
- State where in the service lifecycle the session should be built, and why it must not be rebuilt per request.
- Name one tradeoff your chosen execution-provider ordering introduces beyond raw speed.

ACCEPTANCE CRITERIA

- Constraint A selects tract or a pure-Rust path and justifies it by clean cross-compilation and no native or GPU dependency.
- Constraint B selects ort and describes registering providers such as CUDA or TensorRT in priority order with CPU fallback.
- The answer places session construction once at startup and reuses it across requests, because building a session is the expensive step.
- The answer names a concrete provider tradeoff, such as per-operator placement, fallback to CPU, or added native dependency and image size.

HINTS

- The decision is rarely about raw speed on one model; it is about what your build and deploy story can accept.
- Build the session once and run it many times; loading parses, optimizes, and allocates, while running is comparatively cheap.

Runnable lab

Complete the softmax function so the one-layer model reports calibrated probabilities. The forward pass and argmax mirror the chapter's dense layer; softmax is post-processing you own, exactly like argmax.

Example 54-L. `softmax_post_processing_lab.rs` — *Runnable lab · softmax over a one-layer model*

```
// Companion lab: add a softmax stage so the one-layer model reports
// calibrated probabilities, not just raw logits. The forward pass and
// argmax mirror the chapter's dense layer; softmax is post-processing
```

```
// that you own, exactly like argmax.

const IN: usize = 4;
const OUT: usize = 3;

// Weights stored row-major: one row of IN values per output class.
const WEIGHTS: [[f32; IN]; OUT] = [
    [0.2, 0.8, -0.5, 0.1],
    [-0.3, 0.5, 0.9, 0.4],
    [0.6, -0.2, 0.3, -0.7],
];
const BIAS: [f32; OUT] = [0.1, -0.2, 0.05];

// One fixed input "tensor" of shape [IN].
fn input_tensor() -> [f32; IN] {
    [0.5, -1.0, 2.0, 0.25]
}

// Dense layer: logits[o] = bias[o] + sum_c weights[o][c] * input[c].
fn forward(input: &[f32; IN]) -> [f32; OUT] {
    let mut logits = [0.0f32; OUT];
    for o in 0..OUT {
        let mut sum = BIAS[o];
        for c in 0..IN {
            sum += WEIGHTS[o][c] * input[c];
        }
        logits[o] = sum;
    }
    logits
}

// Post-processing you own: largest logit wins.
fn argmax(logits: &[f32; OUT]) -> usize {
    let mut best = 0;
    for i in 1..OUT {
        if logits[i] > logits[best] {
            best = i;
        }
    }
    best
}

// Numerically stable softmax: subtract the max logit before exponentiating.
fn softmax(logits: &[f32; OUT]) -> [f32; OUT] {
    // TODO: replace this uniform placeholder with a real softmax.
    // 1. Find the maximum logit.
    // 2. Set probs[i] = (logits[i] - max).exp().
    // 3. Divide every entry by the sum so the vector sums to 1.0.
    let _ = logits;
    [1.0 / OUT as f32; OUT]
}

fn main() {
    let input = input_tensor();
    let logits = forward(&input);
    let probs = softmax(&logits);
    let class = argmax(&logits);

    println!("logits = [{:.4}, {:.4}, {:.4}]", logits[0], logits[1], logits[2]);
    println!("probs = [{:.4}, {:.4}, {:.4}]", probs[0], probs[1], probs[2]);
    println!("probs sum = {:.4}", probs[0] + probs[1] + probs[2]);
    println!("predicted class = {}", class);
    println!("confidence = {:.4}", probs[class]);
}
```

OUTPUT

```
logits = [-1.5750, 1.0500, 0.9750]
probs = [0.0362, 0.5000, 0.4638]
probs sum = 1.0000
predicted class = 1
confidence = 0.5000
```

Review questions

- Why is ONNX described as a contract, and what does standardizing operators and tensor declarations let a producer and consumer do independently?
- Why is building a session the expensive step, and what operational habit does that justify in a request-serving service?
- What role does ndarray play for ONNX inference in Rust, and what does a tensor consist of beyond its raw buffer?
- What does an execution provider decide, and why is registering providers in priority order a deployment and latency tradeoff rather than a free speedup?
- When a tensor borrows a Vec for a session run, what does the borrow checker enforce, and what is the calm pattern for owning inputs and outputs around the run?

CHAPTER 55 · SMART CONTRACTS IN RUST: SOLANA, SEI, AND BEYOND

Smart Contracts in Rust: Solana, Sei, and Beyond

Exercises, labs, and review questions for this chapter.

Model an account-based program, contrast wasm-actor and account execution, and reason about serialization, gas, and deterministic execution boundaries

EXERCISE 1 WARM-UP COMPREHENSION

Name the four questions every chain answers

OBJECTIVE

Recall the chapter's four-question framework and the shared shell-around-a-pure-core model so the platform differences become a checklist rather than trivia.

STARTER PROMPT

The chapter argues that Solana, Sei (CosmWasm), and the EVM are three answers to the same four questions, and that the Rust you write to satisfy each is more alike than the platforms suggest. Restate that framing in your own words without copying the comparison table.

- List the four questions the chapter asks of any chain (state location, execution target, serialization, entry shape).
- Describe the 'thin shell around a pure core' shape: what enters as untrusted bytes, what the handler operates on, and what is persisted.
- Explain which part of a contract is chain-specific and which part is ordinary Rust.
- Give one sentence on why the chapter calls the boundary format the contract's 'real public API'.

ACCEPTANCE CRITERIA

- The four questions are stated as: where state lives, what the code runs as, how data is serialized across the host boundary, and what shape the entry point has.
- The answer identifies the handler as pure, deterministic Rust over owned domain types, with all chain-specific concerns confined to the shell.
- The answer states that untrusted bytes are decoded into owned types, processed, then re-encoded into bytes the runtime persists.
- The answer connects the boundary format to Chapter 19's lesson that a wire format is a versioned public contract.

HINTS

- The four facts are state model, execution target, serialization, and entry shape.
- Everything chain-specific lives in the shell; the handler is the part you could lift unchanged to another platform.

EXERCISE 2 CODE READING

Trace the counter through the Solana account model

OBJECTIVE

Read the chapter's Solana counter listing closely enough to explain why the program holds no state and how the account buffer carries it instead.

STARTER PROMPT

Study the smart_contract_solana_counter listing: an Account owns a [u8; 8] data buffer, and process_instruction borrows that buffer mutably, decodes a little-endian u64, applies an Instruction, and writes it back. Walk one full call through the code.

- Identify exactly where the counter value is stored across calls, and confirm process_instruction owns no persistent state of its own.
- Explain what read_counter and write_counter stand in for, and why from_le_bytes / to_le_bytes appear instead of a real codec.
- Trace the values for SetTo(41) followed by Increment and state what the final counter is.
- Explain why process_instruction takes data: &mut [u8] rather than taking the Account by value.

ACCEPTANCE CRITERIA

- The answer states that state lives in account.data and that the program is stateless code the runtime calls with a mutable view of that buffer.
- The answer identifies from_le_bytes / to_le_bytes as a hand-rolled, Borsh-style little-endian codec standing in for a real serializer.
- The trace yields after_set = 41, after_increment = 42, and final counter = 42.
- The answer explains that borrowing the buffer mutably lets the program mutate persisted state without owning it, matching Solana's external-account model.

HINTS

- Follow the bytes: the account buffer is the only thing that survives between two calls to process_instruction.
- A mutable borrow of the buffer is how the runtime lends the program write access to account data it does not own.

EXERCISE 3 IMPLEMENTATION

Add a Decrement instruction with a floor at zero

OBJECTIVE

Extend the chapter's instruction set while keeping the stateless decode-apply-encode entry point and avoiding an unsigned underflow.

STARTER PROMPT

Starting from the smart_contract_solana_counter model, add an Instruction::Decrement variant. Decrementing a counter already at 0 must not panic or wrap to u64::MAX; it must saturate at 0.

- Add the Decrement variant to the Instruction enum.
- Handle it in process_instruction so the decode, apply, and write-back structure is unchanged.
- Use a saturating or checked operation so subtracting below zero floors at 0 rather than wrapping.
- Demonstrate it: SetTo(1), then two Decrement calls, and confirm the counter reads 0.

ACCEPTANCE CRITERIA

- The Instruction enum gains a Decrement variant and process_instruction matches it without changing the read_counter / write_counter calls.
- Decrementing at 0 produces 0 rather than panicking or wrapping to a huge value.
- After SetTo(1) and two Decrement calls the final counter read from the buffer is 0.
- The entry point still returns Result<u64, ProgramError> and the buffer is the only persisted state.

HINTS

- u64::saturating_sub floors subtraction at 0 without a panic.
- The new arm slots into the same match in process_instruction; nothing about read/write needs to change.

EXERCISE 4 IMPLEMENTATION

Encode the platform comparison as queryable data

OBJECTIVE

Turn the four-question comparison table into runnable data and add a lookup that answers a single question across all three platforms.

STARTER PROMPT

Using the smart_contract_platform_compare model, where each Platform maps to a PlatformModel with four fields, write fn serialization_of(platform: Platform) -> &'static str and a loop that prints the serialization format for every platform.

- Reuse describe to obtain the PlatformModel rather than duplicating the field data.
- Implement serialization_of so it returns the serialization field for the given platform.
- Iterate the three Platform variants and print one line per platform naming its serialization format.
- Confirm the output reports Borsh for Solana, JSON/serde for Sei/CosmWasm, and ABI for EVM/Solidity.

ACCEPTANCE CRITERIA

- serialization_of delegates to describe and returns the model's serialization field, with no second copy of the platform facts.
- The loop covers all three variants: Solana, SeiCosmWasm, and EvmSolidity.
- Output pairs each platform with the correct format: Borsh, JSON/serde, and ABI respectively.
- Platform stays a Copy enum so it can be passed by value into the helper without cloning.

HINTS

- describe already centralizes the four facts; the helper just reads one field off its result.
- Because Platform derives Copy, you can pass it into serialization_of and still use it afterward.

EXERCISE 5 DEBUGGING OR REFACTORING

Harden the decode boundary against a hostile increment

OBJECTIVE

Find and fix the place where an undecoded or unbounded value reaches state, applying the chapter's borrow-parse-validate-then-mutate discipline.

STARTER PROMPT

The counter program uses current + 1 for Increment. A counter sitting at u64::MAX is valid input from a hostile caller, and in release builds that addition wraps to 0 instead of failing. Make the increment path reject overflow instead of silently wrapping.

- Explain why current + 1 is a boundary bug, not just a theoretical edge case, given that input bytes come from an untrusted counterparty.
- Add a ProgramError::Overflow variant and switch the Increment (and any SetTo/Add) path to checked_add.
- Return Err(ProgramError::Overflow) before write_counter runs, so no invalid value reaches the buffer.
- Show that incrementing a buffer initialized to u64::MAX returns an error and leaves the buffer unchanged.

ACCEPTANCE CRITERIA

- The answer names the bug as an unvalidated value reaching state: a release-mode wrapping add on attacker-controlled bytes.
- process_instruction uses checked_add and returns ProgramError::Overflow on overflow rather than wrapping or panicking.
- The error is returned before write_counter, so the account buffer is never mutated on the failing path.
- Incrementing a u64::MAX buffer yields an Err and a subsequent read still shows u64::MAX.

HINTS

- checked_add returns Option<u64>; map None to ProgramError::Overflow with ok_or.
- Validate fully before you mutate: the write should never run on the error path.

EXERCISE 6 DESIGN OR PRODUCTION SCENARIO

Plan the Solana-to-Sei port

OBJECTIVE

Reason about which parts of a working contract change and which stay constant when moving from Solana to a Sei (CosmWasm) deployment.

STARTER PROMPT

A team ships the Solana counter program and is asked to deploy comparable logic to a Sei (CosmWasm) chain. Using the four-question framework, write a short porting plan that separates the shell that must change from the pure core that can stay.

- Map each of the four questions to its Solana answer and its Sei/CosmWasm answer (state model, execution target, serialization, entry shape).
- Identify the concrete code that must change: the entry shape (process_instruction versus instantiate/execute/query), the state model (external account versus contract-owned kv store), and the serialization (Borsh versus JSON/serde).
- Identify what stays: the deterministic handler that computes the next counter from the current one over owned domain types.
- State the boundary discipline that must hold on both platforms regardless of format.

ACCEPTANCE CRITERIA

- The plan correctly contrasts external accounts plus Borsh plus process_instruction (Solana) against contract-owned kv plus JSON/serde plus instantiate/execute/query (Sei).
- The plan confines the changes to the shell: deserialization, storage access, and entry-point dispatch.
- The plan keeps the pure handler logic unchanged and explains why it can move across platforms.
- The plan reaffirms decode-validate-then-mutate and a versioned boundary format as constant requirements on both chains.

HINTS

- Only the shell is chain-specific; the next-counter computation is ordinary deterministic Rust.
- Sei swaps a single entry point for instantiate/execute/query and a contract-owned key-value store, but the no-undecoded-value-reaches-state rule is identical.

Runnable lab

Model a stateless Solana-style program over an external account buffer. Fill in process_instruction so it decodes the little-endian counter, applies the instruction with checked_add, and rejects overflow before any value is written back to state.

Example 55-L. `solana_counter_boundary_lab.rs` — *Runnable lab · harden the Solana counter's decode-validate boundary*

```rust
// A crate-free model of a Solana-style program over an external account buffer.
// State lives in the account's byte buffer; the program is stateless code that
// borrows the buffer, decodes a little-endian u64 by hand (as Borsh would),
// applies an instruction, validates, and writes the bytes back.

#[derive(Debug)]
enum Instruction {
    Increment,
    Add(u64),
}

#[derive(Debug)]
enum ProgramError {
    DataTooSmall,
    Overflow,
}

fn read_counter(data: &[u8]) -> Result<u64, ProgramError> {
    if data.len() < 8 {
        return Err(ProgramError::DataTooSmall);
    }
    let mut bytes = [0u8; 8];
    bytes.copy_from_slice(&data[..8]);
    Ok(u64::from_le_bytes(bytes))
}

fn write_counter(data: &mut [u8], value: u64) -> Result<(), ProgramError> {
    if data.len() < 8 {
        return Err(ProgramError::DataTooSmall);
    }
    data[..8].copy_from_slice(&value.to_le_bytes());
    Ok(())
}

// Stateless entry point: decode, apply, validate against overflow, re-encode.
fn process_instruction(data: &mut [u8], instruction: &Instruction) -> Result<u64, ProgramError> {
    let current = read_counter(data)?;
    // TODO: compute `next` from `current` and `instruction` using checked_add,
    // returning ProgramError::Overflow instead of wrapping. For now this stub
    // ignores the instruction so the program still compiles.
    let next = current;
    write_counter(data, next)?;
    Ok(next)
}

fn main() {
    let mut data = [0u8; 8];

    let a = process_instruction(&mut data, &Instruction::Add(40)).unwrap();
    let b = process_instruction(&mut data, &Instruction::Increment).unwrap();

    // A hostile instruction that would overflow must be rejected, not wrap.
    let mut maxed = u64::MAX.to_le_bytes();
    let bad = process_instruction(&mut maxed, &Instruction::Increment);

    println!("after add = {}", a);
    println!("after increment = {}", b);
    println!("counter = {}", read_counter(&data).unwrap());
    println!("overflow rejected = {}", bad.is_err());
}
```

OUTPUT

```
after add = 40
after increment = 41
counter = 41
overflow rejected = true
```

Review questions

- What are the four questions the chapter says every chain answers differently, and what are Solana's answers to each?
- In the Solana model, where does contract state live, and why is the program itself described as stateless?
- What do from_le_bytes and to_le_bytes stand in for in the counter listing, and why does the chapter hand-roll them instead of using a crate?
- What does 'never let an undecoded or unvalidated value reach state' mean in practice, and in what order should you borrow, parse, validate, and mutate?
- Which three facts about a contract's runtime (no ambient I/O, metered execution, re-execution by every validator) constrain the ordinary Rust you can write, and why?

CHAPTER 56 · NO-STD RUST AND CONSTRAINED RUNTIME DERIVATIVES

no-std Rust and Constrained Runtime Derivatives

Exercises, labs, and review questions for this chapter.

Choose the right runtime surface, design fixed-capacity paths, audit constrained unsafe code, and keep portable logic testable from the host

EXERCISE 1 WARM-UP COMPREHENSION

Place each function at its lowest ring

OBJECTIVE

Restate the three-ring capability model (core, alloc, std) and the decision of which ring a given function belongs to.

STARTER PROMPT

The chapter describes the standard library as three stacked layers and frames no_std as opting out of the upper ones. Using the crate-root listing, explain why checksum lives in core, encode_frame lives behind the alloc feature, and write_diagnostic lives behind the std feature.

- State what capability each ring requires of the target: core, alloc, std.
- For each of the three functions, name the single concrete need that pins it to its ring.
- Explain why no_std and no-heap are described as two separate decisions, not one.
- Say what the target triple decides about which ring a build can reach.

ACCEPTANCE CRITERIA

- core is described as always available, alloc as requiring a heap, and std as requiring an OS.
- checksum is pinned to core because it takes only a slice and returns an integer; encode_frame to alloc because it returns an owned Vec<u8>; write_diagnostic to std because it produces a String and is meant for host diagnostics.
- The answer states that a crate can be no_std yet still use alloc, so dropping std does not by itself remove the heap.
- The target triple is identified as what determines the highest ring a given build is allowed to use.

HINTS

- Look at the return types: a borrowed slice needs nothing, an owned Vec needs a heap, a host String for diagnostics needs the OS.
- no_std removes the std layer; the heap belongs to the alloc layer below it.

EXERCISE 2 CODE READING

Trace the fixed-capacity Pool

OBJECTIVE

Read the fixed-capacity Pool and predict its observable outputs without running it.

STARTER PROMPT

Read Example 2, the Pool<const SLOTS, const BYTES> type with alloc_copy, as_slice, release, and in_use. Predict the three printed values for a Pool::<2, 8> after allocating b"abc" and b"rust", attempting b"more", and then releasing the first slot.

- Walk alloc_copy and state when it returns None versus Some(SlotId).
- Determine why the third allocation of b"more" fails for a pool of two slots.
- Compute in_use after the first slot is released, and state what sent_bytes (the length from as_slice) is.
- Explain what release resets so the slot can be reused.

ACCEPTANCE CRITERIA

- alloc_copy returns None when bytes.len() exceeds BYTES, and otherwise None only when every slot is already used.
- The third allocation fails because both of the two slots are occupied, not because of byte capacity.
- in_use is 1 after release, overflow is true, and sent_bytes is 3 (the length of b"abc").
- release is identified as clearing the used flag and zeroing the recorded length for that slot.

HINTS

- SLOTS bounds how many buffers exist; BYTES bounds the size of each one.
- as_slice reads lens[id], which release sets back to 0.

EXERCISE 3 IMPLEMENTATION

Keep checksum slice-based across every ring

OBJECTIVE

Write the baseline portable API as a function over a borrowed slice so it compiles at the core ring.

STARTER PROMPT

Implement fn checksum(bytes: &[u8]) -> u32 so it works identically whether the caller holds a fixed array on a sensor board, a wasm guest buffer, or a Vec<u8> on a server. The function must not require alloc or std.

- Keep the signature exactly fn checksum(bytes: &[u8]) -> u32.
- Sum each byte widened to u32 and avoid allocating any intermediate collection.
- Confirm in prose that nothing in the body would force the alloc or std ring.
- Show one call from an array literal and one from a Vec<u8> to demonstrate the same function serves both.

ACCEPTANCE CRITERIA

- The signature stays fn checksum(bytes: &[u8]) -> u32 and takes a borrowed slice, not an owned Vec.
- The body widens each byte to u32 and sums without constructing a new Vec or String.
- The function compiles with no reference to alloc or std types, so it belongs at the core ring.
- checksum(b"abc") returns 294 and the same function accepts &vec[..] from a Vec<u8>.

HINTS

- Arrays, vectors, and subslices all coerce to &[u8], which is the whole reason to take a slice.
- A reduction over a slice is one iterator chain: map each byte to u32, then sum.

EXERCISE 4 IMPLEMENTATION

Transfer slot ownership to a DMA handoff

OBJECTIVE

Use move semantics to encode that a peripheral, not the caller, owns a buffer while a transfer is in flight.

STARTER PROMPT

Following the chapter's submit_to_dma idea, write fn submit_to_dma(slot: SlotId) -> InFlight that takes the SlotId by value and returns a handle, plus fn complete(token: InFlight) -> SlotId that returns ownership once the transfer finishes. The caller must be unable to read the buffer between submit and complete.

- Define SlotId and a separate InFlight type that wraps the moved SlotId.
- Make submit_to_dma consume the SlotId by value so the caller no longer holds it.
- Make complete consume the InFlight token and hand the SlotId back.
- Explain in prose why a use of the slot between submit and complete is a compile error, not a runtime race.

ACCEPTANCE CRITERIA

- submit_to_dma takes slot: SlotId by value and returns an InFlight handle that owns it.
- complete takes the InFlight token by value and returns the original SlotId.
- After calling submit_to_dma, any attempt to use the moved SlotId fails to compile.
- The write-up names move semantics, not a runtime check or atomic, as what enforces exclusive access during the transfer.

HINTS

- Taking a parameter by value moves it; the previous binding is no longer usable.
- The peripheral conceptually owns the memory until complete returns the SlotId.

EXERCISE 5 DEBUGGING OR REFACTORING

Replace an infallible push with try_reserve

OBJECTIVE

Convert an allocation that assumes success into a fallible path that returns the out-of-memory case to the caller.

STARTER PROMPT

A frame encoder behind the alloc feature calls out.extend_from_slice(payload) directly, so an allocation failure aborts the whole firmware image. Refactor it so it reserves capacity fallibly first and returns an error to the caller instead of aborting.

- Change the return type so the failure is visible, for example Result<Vec<u8>, TryReserveError>.
- Call try_reserve for the needed capacity before pushing or extending any bytes.
- Only grow the buffer once the capacity is secured, so a failure leaves the buffer untouched.
- State why aborting on allocation failure is unacceptable on a constrained target but tolerated on a hosted one.

ACCEPTANCE CRITERIA

- The function returns a Result whose error path corresponds to a failed reservation rather than aborting.
- try_reserve (or try_reserve_exact) is called and its error is propagated before any push or extend_from_slice.
- No bytes are written into the buffer on the failure path.
- The explanation notes that on constrained targets allocation can genuinely fail, so out-of-memory must be a handled value, not a process abort.

HINTS

- try_reserve returns Result<(), TryReserveError>; the ? operator turns its error into your function's error.
- Secure capacity first, then extend, so the function is left in a clean state when reservation fails.

EXERCISE 6 DESIGN OR PRODUCTION SCENARIO

Ship one framing library across four homes

OBJECTIVE

Design the feature-gated surface and runtime contract of a single crate that must run on firmware, a hypervisor, a wasm guest, and a server.

STARTER PROMPT

Your team owns a packet-framing and checksum library that must ship to four targets from the opening scenario: bare-metal firmware, kernel-adjacent hypervisor startup code, a sandboxed wasm guest behind a narrow ABI, and a normal server. Decide which functions live in core, which sit behind alloc, and which sit behind std, and state the runtime contract each target imposes.

- Assign the checksum, frame-encode, and host-diagnostic surfaces to the core, alloc, and std rings respectively, and justify each placement.
- Name what each target must supply that std would otherwise provide: panic handler, allocator, startup glue, linker script.
- Decide the memory strategy for the firmware path: fixed-capacity slots, fallible allocation, or both, and why.
- Describe how host-side std tests validate the portable core logic separately from slower target-specific harnesses.

ACCEPTANCE CRITERIA

- The smallest truthful API is slice-based and in core; owned helpers are alloc-gated; host diagnostics are std-gated, matching the shape the chapter recommends.
- The freestanding targets are shown to require an explicit panic handler, allocator choice, startup, and linker script as part of the release contract.
- The firmware path prefers fixed-capacity or fallible-memory handling rather than assuming infallible allocation.
- Portable core logic is tested on the host with std while target-specific behavior is validated separately, keeping the fast tests fast.

HINTS

- Design the smallest truthful API first and add owned conveniences only where the product boundary genuinely benefits.
- The common failure is an unspoken assumption: that an allocator, logs, or unwind exist on every target.

Runnable lab

Model the chapter's fixed-capacity Pool in plain std: a frame pool with a bounded slot count where allocation can fail. Fill in alloc_copy so it rejects oversized inputs and reuses the first free slot.

Example 56-L. `fixed_capacity_pool_lab.rs` — *Runnable lab · fixed-capacity frame pool*

```
struct FramePool {
    used: Vec<bool>,
    frames: Vec<Vec<u8>>,
    capacity: usize,
}

impl FramePool {
    fn new(slots: usize, capacity: usize) -> Self {
        FramePool {
            used: vec![false; slots],
            frames: vec![Vec::new(); slots],
            capacity,
        }
    }

    // TODO: Reject inputs longer than `self.capacity` by returning None.
    // Otherwise find the first free slot, mark it used, copy the bytes into
    // it, and return Some(index). Return None if every slot is already used.
    fn alloc_copy(&mut self, bytes: &[u8]) -> Option<usize> {
        let _ = bytes;
        None
    }

    fn release(&mut self, id: usize) {
        self.used[id] = false;
        self.frames[id].clear();
    }

    fn in_use(&self) -> usize {
        self.used.iter().filter(|&&u| u).count()
    }
}

fn checksum(bytes: &[u8]) -> u32 {
    bytes.iter().map(|&b| b as u32).sum()
}

fn main() {
    let mut pool = FramePool::new(2, 8);

    let first = pool.alloc_copy(b"abc").unwrap_or(0);
    let _second = pool.alloc_copy(b"rust");
    let overflow = pool.alloc_copy(b"overflowing").is_none();
    let first_sum = checksum(pool.frames[first].as_slice());

    pool.release(first);

    println!("checksum = {}", first_sum);
    println!("overflow = {}", overflow);
    println!("in_use = {}", pool.in_use());
}
```

OUTPUT

```
checksum = 294
overflow = true
in_use = 1
```

Review questions

- What capability does each of the three rings require of a target, and which ring is always available regardless of target triple?
- Why are no_std and no-heap two independent decisions rather than one, and how does the alloc feature relate to that distinction?
- On a freestanding target, why does the compiler require you to supply a #[panic_handler], and what does std normally provide in its place?
- How does taking a slot by value in a submit_to_dma-style API turn a potential data race with a DMA engine into a compile-time error?
- What does try_reserve change about a buffer-growing function compared with calling push or extend_from_slice directly, and why does that matter on a constrained target?

END OF BOOK

Advanced Rust

O. Iakushkin

Set in Georgia and Helvetica Neue; code in Consolas.
Syntax highlighting by Pygments. Rendered with headless Chrome and PyMuPDF.

Thank you for reading.